

Canonical Diophantine Certificates

Witness-faithful polynomial representations of bounded computation: trace classes, accelerators, polynomial trajectories, memory logs, queues and rewriting

Research report prepared for Vladimir Reshetnikov. Manuscripts dated September 30, 2026; merged in the batch-60/61 write phase.

Abstract

[write] An existential Diophantine representation says that a computation exists; it need not preserve which computation occurred, how many computations occurred, or how many arithmetic witnesses describe one of them. This report collects seven manuscripts that construct *witness-faithful* (single-fold or parsimonious) polynomial representations of bounded computation and locate the boundary beyond which such representations are as hard as the open single-fold and finite-fold problems.

For Petri nets and interval-guarded translations, an explicit convex quadratic integer polynomial has its natural zeros in bijection with the commutation classes of enabled executions of exact length T ; its nonnegative real zero set is a bounded rational polytope whose lattice points are exactly those zeros (Part II, manuscript 05). Quartic second routes (manuscripts 04 and 06), a Cartier–Foata variant for bounded causal height (manuscript 01), exact all-marking commutation tests, an exponential automaton-state lower bound and $\#P$ -completeness of counting accompany it (Parts I and II). Single-fold quartics of fixed arity describe multisets, fixed-word powers and nested repetitions of counter actions independently of the repetition counts (Part III); the relation “ $p(i) \geq 0$ for all $i \leq T$ ” with $O((d+1)^3)$ witnesses independent of T (Part IV); random-access memory logs with $24L$ witnesses (Part V); cyclic-tag, deletion-tag and fixed-read FIFO executions with one coordinate per consumed symbol and one guard slack, together with a proof that faithful polynomial word memory needs two natural coordinates (Part VI); and FRACTRAN runs and scheduled SKI reductions (Part VII). Part VIII collects further substrates.

Part IX proves, with proofs from four manuscripts and a sketch in a fifth, that a single-fold (finite-fold) polynomial for universal halting exists exactly when every computably enumerable set has a single-fold (finite-fold) Diophantine representation, which is an open problem that this report does not solve.

Status. [write] A merge of seven AI-assisted, unrefereed manuscripts, six delivered in batch 60 and one in batch 61 of ProveIt’s incoming reports (Appendix K). The theorems carry ordinary mathematical proofs; the accompanying Python programs perform finite exact checks and are not proofs. No statement of this report is formalized in Lean or Rocq, although the report sits beside ProveIt’s formal Hilbert’s-tenth-problem project; the project declarations it cites are named in the README. Global priority is not claimed for any construction, and no resolution of the general single-fold or finite-fold problem is claimed. Every manuscript’s own status statement is printed in Section 3.

Contents

1	The research problem and its relation to ProveIt	6
2	Conventions	7
3	The seven manuscripts	16
I	Exact commutation and resource algebra	27
4	An exact commutation test for Petri transitions [05]	28
5	Guarded translations and exact commutation [01]	31
6	The exact resource envelope of a multiset [01]	33
7	The exact resource-summary algebra [06]	34
8	A first-failure product for resource legality [07]	37
II	Canonical trace classes	39
9	Canonical schedules with linear Boolean memory [05]	39
10	A parsimonious quadratic encoding of raw executions [05]	41
11	Canonical traces: compact quartic and convex quadratic forms [05]	43
12	Costs, exact counts, and a closed-form example [05]	46
13	Counting hardness survives maximal commutation [05]	48
14	A symbolic normal-form monitor for concurrent traces [04]	49
15	Quartic certificates for Petri-net trace classes [04]	51
16	Counting, weights, and the limits of low degree [04]	53
17	A compact signature for lexicographic trace normality [06]	55
18	A bijection between bounded trace classes and polynomial roots [06]	57
19	Causal traces and canonical layers [01]	59
20	An explicit quartic for causal histories [01]	60

III	Compressed repetition and accelerators	63
21	A fixed-arity single-fold quartic accelerator [01]	64
22	Fixed block programs with unbounded repetition [01]	65
23	Worked examples [01]	67
24	Saturation: nested normality only sees zero, one, and two [06]	69
25	A single-fold quartic compiler [06]	70
26	Expressiveness and an exact nesting boundary [06]	73
IV	Polynomial trajectories	76
27	The problem addressed and the precise restricted result [02]	77
28	The finite-difference sign certificate [02]	78
29	The single-fold quartic theorem and its cost [02]	80
30	Boolean guards and empty execution prefixes [02]	82
31	Accelerating polynomial trajectories [02]	83
32	Computational substrates covered by the theorem [02]	84
33	Canonical nontermination certificates and input bounds [02]	87
34	Worked examples [02]	89
35	Manuscript 02: Conclusion [02]	90
V	Memory logs, RAM and graph evaluation	90
36	Canonical arithmetic with quadratic residuals [03]	91
37	Deterministic sorting as a canonical polynomial circuit [03]	93
38	The canonical random-access memory theorem [03]	94
39	Size, height, and the cost of canonical routing [03]	97
40	A linear-size alternative: exact large-base products [03]	98

41 From memory logs to complete RAM executions [03]	101
42 Applications to computational substrates [03]	103
43 Exact counting and first-halting probabilities [03]	105
44 Manuscript 03: Conclusion [03]	107
 VI Queues: tag systems and FIFO networks	 107
45 Little-endian words and faithful concatenation [07]	108
46 Cyclic tag systems without intermediate queues [07]	109
47 Two examples: a real computation and a self-starting fiction [07]	111
48 Variable count, degree, and witness height [07]	112
49 General deletion-tag systems [07]	114
50 The exact dimension of polynomial word memory [07]	117
51 A quartic certificate from two-coordinate memory [07]	119
52 FIFO networks and switched arithmetic dynamics [07]	121
53 Manuscript 07: Conclusion [07]	123
 VII FRACTRAN, SKI and term rewriting	 124
54 A linear-size, unique-witness FRACTRAN priority certificate [04]	124
55 Context-local Diophantine compilation of SKI rewriting [04]	127
56 Local FRACTRAN and combinatory-logic interfaces, and bounded rewriting [02, 01, 05]	130
 VIII Other substrates	 132
57 Other computational substrates [05]	132
58 Other computational substrates through the same interface [04]	134
59 Other computational substrates and safe adapters [01]	136
60 Other substrates and the limit of the construction [02]	136

61 Other computational substrates: transfers and limits [06]	137
IX The boundary of fixed-arity compression	138
62 Fixed-dimensional compression and the finite-fold boundary [05]	139
63 The boundary of unbounded canonicalization [01]	142
64 The exponentiation boundary and universal first halting [02]	144
65 The exact boundary of uniform single-fold compression [03]	145
66 Unbounded computation and the fixed-arity frontier [04]	147
67 The exact finite-fold obstruction [06]	149
68 The fixed-arity and finite-fold boundary [07]	150
69 Implementation and validation	151
70 Formalization targets	163
71 Research questions	171
72 Conclusions of manuscripts 05, 01, 04 and 06	183
Appendices	184
A Manuscript 05: A minimal end-to-end example	184
B Manuscript 05: Edge cases and specification checklist	185
C Manuscript 05: Source and verification provenance	186
D Manuscript 01: Reproducibility and source-audit notes	186
E Manuscript 02: The finite checking formula in one place	187
F Manuscript 02: Reproduction and the exported polynomial format	187
G Manuscript 03: Additional validation of the linear backend	188
H Manuscript 03: Claim and dependency ledger	188
I Manuscript 04: Notation and resource-count reference	189
J Manuscript 06: A completely displayed local power polynomial	190

K Provenance of this report	191
References	193

1 The research problem and its relation to ProveIt

1.1 Beyond an existence-only translation

A conventional application of the Matiyasevich–Robinson–Davis–Putnam theorem turns an effectively enumerable reachability predicate into a polynomial solvability problem. It does not, by itself, identify polynomial solutions with executions. One execution can acquire many auxiliary encodings; a nondeterministic execution may have many schedules; and a scheduling quotient can introduce another layer of multiplicity.

For a compiler intended for proofs, counting, or optimization, the natural stronger specification is a bijection

$$\{\text{computational objects of interest}\} \longleftrightarrow \{z \in \mathbb{N}^n : P(z) = 0\}. \quad (1)$$

The entire right-hand tuple must be determined, not merely its projection to a few distinguished coordinates. Otherwise even the number of solutions of the equation can be unrelated to the number of computations.

Here the main computational objects are executions modulo specified swaps of adjacent transitions. We resolve four concrete design questions. First, which swaps preserve enabledness at *every* marking? Second, how can one enforce one representative per commutation class without expanding an exponentially large automaton? Third, can all arithmetic auxiliaries be unique while keeping the polynomial degree small? Fourth, what does this genuinely bounded construction imply—and not imply—about a single polynomial representing arbitrarily long universal computations?

1.2 Repository baseline and the extension developed here

The inspected ProveIt snapshot has commit

e8bb0931d67f80d9fce87a8cddb0f661ff19f956.

Its Hilbert-tenth-problem development contains theorems `boundedForall_dioph`, `exactIter_dioph`, and `existsExactIter_dioph`; the last two provide Diophantine exact iteration and finite reachability under appropriate Diophantine substitutions [3]. These are the correct existing interfaces to reuse for existence-level consequences.

The present article adds an explicit *semantic fibre* specification. We keep the transition word, its states, the normal-form recognizer, and every auxiliary variable visible, prove unique extension, and only then collect the constraints into one polynomial. The bounded construction is elementary and does not need MRDP. Its relation to the repository is therefore complementary: it supplies a structured front end and a stronger correctness theorem, rather than another proof of existential trace closure.

The repository’s status register distinguishes completed results from pending formalization [2]. No repository build was performed for this article. Statements about its interfaces are based on the inspected sources, not on a new independent kernel audit. No result below relies on an unverified numerical optimization in the repository.

1.3 Theorems, classical ingredients, and novelty

The exact encoding, its size accounting, its unique-extension proof, the graph realization, the memory lower bound, and the particular counting reduction are all supplied in full. They form a concrete extension suitable for review and implementation. We do not claim that an exhaustive literature search establishes priority for each individual observation.

The trace-monoid forbidden-factor characterization of lexicographic normal forms is classical; an explicit published form appears in Barthélemy [18]. Sum-of-squares conjunction and arithmetic encodings of Boolean constraints are also standard ideas. Register-machine Diophantine representations have a long history, including Jones–Matiyasevich [8]. The contribution pursued here is the *combined guarantee*: exact semantic commutation, canonical representative selection, no free witnesses, bounded real geometry, and explicit separation from fixed-dimensional finite-fold compression.

The distinction between polynomial representability and finite-fold or single-fold representability is essential. Matiyasevich discusses this distinction and the associated conjectures [9]; Cantone, Cuzziol, and Omodeo discuss recent single-fold specification methods and the danger of introducing uncontrolled witnesses [10]. Our boundary theorem concerns these stronger specifications. It is not presented as their solution.

1.4 How this report is organized

[write] This report merges seven manuscripts written independently for the same purpose, witness-faithful Diophantine compilation of bounded computation. Six (numbered 01–06 here) were delivered in batch 60 of Provet’s incoming reports and pin the repository at e8bb0931d; the seventh (07) was delivered in batch 61, pins 725d2ebb6, and names manuscripts 04 and 06 as its companions. Manuscript 05, whose introduction the preceding subsections reproduce, is the base: its trace-class theorem has the weakest hypothesis and the strongest conclusion among the four manuscripts that prove one (Part II). The seven titles, abstracts and status statements, and every manuscript’s own introduction, are printed in Section 3; Appendix K records their archives, pins and commits.

After the conventions (Section 2), the material is arranged by subject in nine Parts:

Part	Subject	Manuscripts
I	Exact commutation and resource algebra (Part I)	05, 01, 06, 07
II	Canonical trace classes (Part II)	05; 04, 06 (second routes); 01
III	Compressed repetition and accelerators (Part III)	01, 06
IV	Polynomial trajectories (Part IV)	02
V	Memory logs, RAM and graph evaluation (Part V)	03
VI	Queues: tag systems and FIFO networks (Part VI)	07
VII	FRACTRAN, SKI and term rewriting (Part VII)	04, 02, 01, 05
VIII	Other substrates (Part VIII)	05, 04, 01, 02, 06
IX	The boundary of fixed-arity compression (Part IX)	all seven

The back matter collects each manuscript’s implementation and validation record (Section 69), its formalization proposals (Section 70), the merged research questions (Section 71), the remaining conclusions, the manuscripts’ appendices, the provenance record and one merged bibliography.

A result proved by several manuscripts is printed once, at the place named in Appendix K, and credits every manuscript that proves it; a genuinely different proof or construction of the same result is kept as a marked second route. Two such mergers change the shape of the text: the universal-halting equivalence is one theorem, Theorem 62.1, and the absence of a computable witness-height bound is one proposition, Proposition 62.2. At the places where the other manuscripts stated them, a note points to the merged statement and the manuscript’s own proof follows.

2 Conventions

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every witness ranges over $\mathbb{N}$; every polynomial has integer coefficients, and subtraction in a displayed polynomial is subtraction in $\mathbb{Z}$, never truncated subtraction. A *residual*

system is a finite list of integer polynomials required to vanish; its single equation is the sum of their squares, of degree at most four when every residual is quadratic and at most two when every residual is affine. A node of an arithmetic circuit names a polynomial subexpression and is not a witness; only an added coordinate is. Parameters are natural except in Part IV, whose coefficient parameters are signed integers. In Part VI the program, the input and terminal words and the horizon are data of the construction, not unknowns.

A representation is *single-fold* if each parameter tuple has at most one witness, *finite-fold* if it has finitely many, and *parsimonious* for a set of semantic objects if its witnesses are in a specified bijection with them. “Canonical certificate” (Part V) means a single-fold representation.

Words used in two senses.

- *Certificate*. Here, a natural witness tuple of a polynomial. In ProveIt’s Hilbert’s-tenth-problem project a “straight-line certificate” of the 1980 Theorem 5 is a program whose arithmetic operations are counted; the project’s 75-operation universal certificate is of that kind. No result of this report concerns operation counts.
- *Height*. Part II’s *causal height* H (from manuscript 01) is the number of Cartier–Foata layers of a trace. In Part VI, H is an upper bound on queue length. Elsewhere a *witness height* bounds the magnitude of witness coordinates.
- *Length* T is the number of steps of a bounded execution in Parts II, VI, VII and VIII; in Part IV T is the horizon of $\{0, \dots, T\}$.
- *Memory*. Part V certifies logs of a random-access memory. Part VI’s *word memory* is a numerical encoding of a finite word; its dimension is the number of natural coordinates of the encoding.
- *Universal*. The Jones articles’ *universal pair* (ν, δ) bounds unknowns and degree of an existence representation; Theorem 62.1 concerns a universal polynomial that is moreover single-fold or finite-fold, which is equivalent to an open problem.

[write] Three further collisions need the same care. The letter H has two more meanings: in Part VII it is the total context depth of an SKI schedule (manuscript 04), and in Part II $H_{i,a}$ (manuscript 05) and $H_{a,i}$ (manuscript 06) are linear forms in the selectors of one step. In Part II, “memory” (Theorems 9.5 and 14.3) counts the states or bits of a deterministic finite-state recognizer of normal forms, which is neither Part V’s nor Part VI’s notion. Finally, “canonical” qualifies three different objects: a canonical *representative* (the lexicographically least word of a trace class, or the Cartier–Foata normal form), a canonical *witness* (the unique auxiliary tuple of a single-fold representation, Part V’s “canonical certificate”), and canonical *padding* (a uniquely forced way of filling a fixed-length record). The tempting false reading of a “canonical certificate for a trace” is that the trace has one witness among all the traces with the same endpoints; the true reading, throughout Part II, is that each trace class has exactly one witness, so the number of zeros at fixed endpoints is the number of classes.

Letters that change meaning between Parts. [write] Each Part keeps the letters of the manuscript it prints. The dictionary below lists the letters that change meaning; the tempting false reading is to carry a letter across a Part boundary.

Symbol	Meaning by manuscript (and Part)
places, transitions	05: d places, m transitions (Parts I, II, VIII); 01: p coordinates, m actions, alphabet A (Parts I–III); 04: p places, r transitions, and in FRACTRAN r fractions (Parts II, VII); 06: p resources, q transition labels (Parts I–III); 07: p is the period of a cyclic-tag program, and also the number of resources in the first-failure product and of queues in a FIFO network (Parts I, VI).
m	01: number of actions; 05: number of transitions; 06: an endpoint marking ($m \xrightarrow{w} n$); 04: marking variables $m_{t,i}$; 07: appendant lengths m_i .
u, v	01: upper guard u_a and displacement v_a ; 05: upper guard u_a (interval guards), production v_a , and the auxiliaries $u_{i,a}$; 06: least input u_a and its output v_a ; 07: guard slack u ; 04: production matrix V .
D	01: dependence relation $D = (A \times A) \setminus I$; 05: sequential demand D_{ab} ; 06: linear forms $D_{a,i}$; 07: total number of consumed symbols D and prefix factors D_i .
d	05: number of places; 01: $d = \sum_a D(a) $; 02: nominal degree bound; 07: deletion number.
N	05: final marking; 01: witness counts N_{acc} , N_H ; 03: number of padded records; 06: number of syntax nodes; 07: $N_{B,d}$, the number of halted words.
B	01, 04: witness-height bounds; 04: stack base; 05: coordinate bounds B_p ; 07: alphabet size.
K	01: universal halting $K(e, x)$; 05: label-additive cost $K(w)$; 07: $K = (B - 1)!$.
C	03: number of comparators; 05: linear forms $C_{i,a}$; 07: the largest number of symbols read in one event.
L	01: loss totals L_j (and $L = n_a + 2n_b$ in an example); 03: log length; 05: lower box L_{ab} ; 07: length residual.
G	01: gain totals G_j ; 07: the first-failure guard.
I	a symmetric irreflexive independence relation in every manuscript that uses one; 05 and 06 name maximal ones $I_{\max}$ and I_* .

Reading the merged text. [write] Text printed from a manuscript keeps that manuscript’s wording. In it, “this article”, “this paper”, “the present report” and “below” refer to that manuscript; theorem, section and equation numbers are this report’s. Where a manuscript names one of its files, the shipped name in code/ or data/ is printed (the README maps every delivered name to its shipped name), except inside a displayed command listing, which keeps the delivered layout because the programs import one another by their delivered names. A manuscript’s repository citations are to the pinned commit it names. Renamed macros and symbols: manuscript 02’s `\code` (small typewriter) and manuscript 05’s `\code` (a non-breaking URL) print as manuscript 01’s `\code`; the pin macros of manuscripts 01, 02, 04 and 07 (01’s and 07’s share the name `\repoSHA` but hold different commits) are printed as the literal commit identifiers; manuscript 06 defined a `\word` macro it never used, so manuscript 07’s is the only one. No mathematical symbol was renamed. The rest of this section prints each manuscript’s own conventions.

2.1 Manuscript 05: Conventions and witness-preserving arithmetic

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. All unknowns are nonnegative integers unless a real relaxation is explicitly being considered. Polynomial equalities, including their subtractions, are interpreted in $\mathbb{Z}$ or $\mathbb{R}$, not using truncated subtraction on $\mathbb{N}$. A transition label identifies a transition, not merely its visible action name; distinct transitions with identical effects remain distinct labels.

Definition 2.1 (Representations and fibres). A polynomial $P(x, z) \in \mathbb{Z}[x, z]$ represents a relation R if

$$R(x) \iff \exists z \in \mathbb{N}^n P(x, z) = 0.$$

It is *finite-fold* if for each fixed x there are only finitely many such z , and *single-fold* if there is at most one. A representation of a set of computational objects is *parsimonious* here when an explicitly defined decoding map is a bijection from its entire natural zero set to those objects.

A finite number n of witness *coordinates* says nothing about the cardinality of a fibre. Even a one-variable equation such as $0z = 0$ has infinitely many natural solutions. Conversely, the polynomials in the main theorem generally have many solutions at a fixed pair of endpoints: one for each execution class. Their unique-extension property is conditional on the selected canonical word.

Lemma 2.2 (Unique slack and scalarization). *Suppose $L_1(z), \dots, L_s(z)$ are integer polynomials. Then*

$$\sum_{j=1}^s L_j(z)^2 = 0 \iff L_j(z) = 0 \text{ for every } j$$

over $\mathbb{Z}$ and over $\mathbb{R}$. For integer-valued expressions A, B , the inequality $A \leq B$ is equivalent to the existence of a unique $s \in \mathbb{N}$ satisfying $A + s = B$.

Proof. Every square is nonnegative. A sum of nonnegative reals is zero exactly when every summand is zero. The slack is necessarily $B - A$, which is a natural number exactly under the stated inequality. $\square$

Lemma 2.3 (Affine Boolean conjunction). *For fixed $p, q \in \{0, 1\}$, the natural solutions of*

$$p = u + s_1, \quad q = u + s_2, \quad 1 + u = p + q + s_3 \quad (2)$$

consist of exactly one tuple (u, s_1, s_2, s_3) , with $u = pq$. For real $p, q \in [0, 1]$, the same equations imply

$$\max(0, p + q - 1) \leq u \leq \min(p, q).$$

Proof. Nonnegativity gives $u \leq p$, $u \leq q$, and $u \geq p + q - 1$. For Boolean inputs these bounds determine u : it is zero when either input is zero and one when both are one. Each slack then has a prescribed value. The displayed real bounds are precisely the same inequalities. $\square$

This small gadget is responsible for the quadratic, rather than quartic, final polynomial. We do not use a nonunique decomposition into sums of squares to enforce nonnegativity: natural-variable domains provide it directly. Replacing all natural variables by arbitrary integer variables would invalidate the later one-hot and slack arguments. Replacing a natural variable by a sum of four squares preserves existence but not the claimed bijection.

2.2 Manuscript 01: Three different meanings of an exact representation

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. For an integer polynomial $P(x, w)$ and fixed parameter tuple x , let

$$Z_P(x) = \{w \in \mathbb{N}^k : P(x, w) = 0\}.$$

We distinguish three contracts.

Definition 2.4. An *existence representation* of a relation R satisfies $R(x) \iff Z_P(x) \neq \emptyset$. It is *single-fold* if $|Z_P(x)| \leq 1$ for every x , and *finite-fold* if $|Z_P(x)| < \infty$ for every x . Given a set $C(x)$ of semantic objects, a *parsimonious representation* of C is a specified bijection $Z_P(x) \simeq C(x)$.

A parsimonious history representation need not be single-fold: there can be several genuinely different histories between the same endpoints. The goal is not to identify those histories, but to avoid multiplying each of them by irrelevant choices. Conversely, a single-fold decision representation need not display a recognizable computation at all.

The universal single-fold and finite-fold questions are not solved by ordinary MRDP. Matiyasevich distinguishes ordinary polynomial representability from single-fold *exponential* representability and asks whether the two improvements can be combined [9]. We use this distinction as a precise boundary, rather than claiming that deterministic execution automatically yields a unique polynomial witness.

2.3 Manuscript 02: Single-fold arithmetic without hidden choices

Definition 2.5 (Single-fold representation). For a relation $R(\mathbf{a})$, a polynomial representation

$$R(\mathbf{a}) \iff \exists \mathbf{w} \in \mathbb{N}^m P(\mathbf{a}, \mathbf{w}) = 0$$

is *single-fold* if for every fixed input $\mathbf{a}$ there is at most one $\mathbf{w}$ satisfying the equation. Thus a true instance has exactly one witness and a false instance has none. It is *finite-fold* if every input has only finitely many witnesses.

Our convention is $0 \in \mathbb{N}$. Free parameters may range over $\mathbb{Z}$, while existential coordinates always range over $\mathbb{N}$. To obtain an entirely natural-input relation, replace each signed free parameter a by $a^+ - a^-$, with $a^+, a^- \in \mathbb{N}$. These are *free inputs*; their possible redundancy does not create extra existential witnesses for a fixed input pair. When a signed quantity itself is existentially quantified, its pair must be normalized as below.

2.3.1 Canonical signed coordinates

Lemma 2.6 (Signed normal form). *Every $v \in \mathbb{Z}$ has a unique representation*

$$v = v^+ - v^-, \quad v^+, v^- \in \mathbb{N}, \quad v^+ v^- = 0.$$

It is $v^+ = \max(v, 0)$ and $v^- = \max(-v, 0)$.

Proof. The product condition implies at least one coordinate is zero. If $v > 0$, then v^- cannot be positive or v^+ zero, so $(v^+, v^-) = (v, 0)$. The case $v < 0$ is symmetric, and $v = 0$ forces both coordinates to vanish. $\square$

Without the product condition, the pairs $(v^+ + k, v^- + k)$ would give infinitely many encodings. This elementary issue is not cosmetic: it is exactly the sort of witness multiplicity that the construction must avoid.

Lemma 2.7 (A total single-fold sign test). *For a given integer v , the following equations have exactly one solution $(b_-, b_0, b_+, s) \in \mathbb{N}^4$:*

$$\begin{aligned} b_\epsilon (b_\epsilon - 1) &= 0 \quad (\epsilon \in \{-, 0, +\}), \\ b_- + b_0 + b_+ &= 1, \\ v &= (b_+ - b_-)(s + 1), \\ b_0 s &= 0. \end{aligned} \tag{3}$$

The three bits respectively indicate $v < 0$, $v = 0$, and $v > 0$.

Proof. Exactly one of the three bits is 1. If $b_+ = 1$, the magnitude equation says $v = s + 1 > 0$ and determines $s = v - 1$. If $b_- = 1$, it says $v = -(s + 1) < 0$ and determines $s = -v - 1$. If $b_0 = 1$, it forces $v = 0$; the last equation then forces $s = 0$. The indicated assignment exists in each case, and the cases are mutually exclusive. $\square$

All equations in (3) are quadratic or linear when v is a linear expression in existing wire coordinates. In particular, the sign of a negative integer needs neither a square-root witness nor a choice of squares representing an inequality.

2.3.2 Total circuits and the quartic conversion

Lemma 2.8 (Canonical circuit lemma). *Let $\Phi(\mathbf{a})$ be a fixed quantifier-free Boolean combination of integer polynomial equalities and inequalities. Given an arithmetic and Boolean circuit of size S for Φ , one can effectively construct $O(S)$ natural auxiliary coordinates and quadratic integer residuals $Q_1, \dots, Q_N$ such that*

$$\Phi(\mathbf{a}) \iff \exists \mathbf{u} \in \mathbb{N}^{O(S)} \sum_{j=1}^N Q_j(\mathbf{a}, \mathbf{u})^2 = 0.$$

The auxiliary tuple is unique whenever it exists. The resulting polynomial has degree at most four in all its free and existential variables.

Proof. Evaluate the entire circuit in a fixed topological order. Represent every signed arithmetic output by its canonical pair from Lemma 2.6. For an addition gate, impose

$$z^+ - z^- = (x^+ - x^-) + (y^+ - y^-), \quad z^+ z^- = 0.$$

For multiplication, replace the right-hand side by $(x^+ - x^-)(y^+ - y^-)$. These equations have degree at most two, and the output pair is uniquely determined by the previously determined inputs. Fixed integer constants and signed free inputs are allowed in the same formulas. Affine expressions may be retained as expressions rather than stored as extra wires.

For an atomic comparison, first evaluate its polynomial difference and then use Lemma 2.7. Equality is the zero bit; strict inequality is one of the other bits. Non-strict inequality is the sum of the appropriate two bits. The Boolean operations are represented by

$$\neg b = 1 - b, \quad b \wedge c = bc, \quad b \vee c = b + c - bc.$$

Each output is uniquely determined and is automatically a bit when its inputs are bits. Store outputs as additional wires as necessary to keep every defining residual quadratic. Finally require the truth-output bit to be 1.

All internal computations, including those belonging to a disjunct that is not needed to make Φ true, are still evaluated. Consequently they have unique values and cannot introduce unconstrained witnesses. Induction along the circuit proves that all internal coordinates are uniquely determined by $\mathbf{a}$ before the final truth assertion is made. That assertion holds precisely when Φ is true.

A sum of squares of integers is zero exactly when each summand is zero. Squaring the quadratic residuals and summing them yields the advertised single equation and the degree bound. The number of residuals and variables per gate is bounded by an absolute constant. $\square$

Remark 2.9 (What is and is not being eliminated). The lemma removes Boolean structure and comparisons from a *fixed finite* formula. It does not eliminate an arbitrary bounded universal quantifier, nor a variable-length circuit. The next sections provide a special finite formula for one such quantifier, using polynomial degree.

Example 2.10 (An unsafe disjunction). The relation $a = 0 \vee b > 0$ can be expressed by $\exists s \in \mathbb{N} \ a(b - s - 1) = 0$. But when $a = 0$, every s is allowed. A product representing a disjunction preserves existence but not uniqueness. A total sign circuit has no such freedom. Canonical padding of inactive certificate components will play the same role below.

2.4 Manuscript 03: Conventions

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every polynomial has coefficients in $\mathbb{Z}$, but all input and witness coordinates range over $\mathbb{N}$. Subtraction in a displayed polynomial is subtraction in $\mathbb{Z}$, *not* truncated natural subtraction. These conventions are essential to the comparison gadget.

A *residual system* is a finite list $F_1, \dots, F_R$ of integer polynomials, required to vanish. Its associated polynomial is

$$P = \sum_{i=1}^R F_i^2. \quad (4)$$

At natural, integer, or real arguments, $P = 0$ is equivalent to simultaneous vanishing of all residuals. In particular, quadratic residuals produce a polynomial of degree at most four. We keep both representations: the residual list exposes the semantics, while (4) supplies the single equation.

Definition 2.11 (Canonical certificate). For a relation $R(x, y)$, a certificate $F(x, y, z) = 0$ is *canonical* when $R(x, y)$ holds exactly when there exists a witness z , and for each fixed (x, y) there is at most one such z . If R is the graph of a total function, there is exactly one pair (y, z) for each x .

The word *parsimonious* will mean that projection from polynomial solutions to the specified semantic executions is a bijection. Distinct labelled nondeterministic choices count as distinct executions, even when they later merge into the same machine state.

2.5 Manuscript 04: Arithmetic interfaces and canonical auxiliary variables

2.5.1 Domains, residuals, and witnesses

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every witness variable ranges over $\mathbb{N}$, but all polynomial expressions are evaluated in $\mathbb{Z}$. In particular, subtraction is ordinary integer subtraction, not truncated natural subtraction. A *residual system* is a finite list

$$F_1(\mathbf{x}) = 0, \dots, F_E(\mathbf{x}) = 0, \quad F_i \in \mathbb{Z}[\mathbf{x}]. \quad (5)$$

The single associated polynomial is

$$P(\mathbf{x}) = \sum_{i=1}^E F_i(\mathbf{x})^2. \quad (6)$$

Since an integer square is nonnegative, $P = 0$ if and only if every residual vanishes. If $\deg F_i \leq 2$, then $\deg P \leq 4$. This conversion introduces no variables and preserves the entire solution set, not just solvability.

A displayed sum of squares is already an explicit polynomial. Expanding it into a list of monomials can enlarge its description. Our exact equation counts count the residuals; they are not claims about a minimum number of operations or about the number of monomials in a fully expanded quartic.

Definition 2.12 (Witness-faithfulness). For fixed external data d , a residual system is witness-faithful for a set C_d if it comes with mutually inverse maps between C_d and all its natural solutions. If the objects are classes under an equivalence relation, the maps must be defined on classes, rather than on arbitrary representatives.

2.5.2 Parsimonious quadratization

The following familiar circuit principle will also prevent several common multiplicity errors.

Lemma 2.13 (Unique-extension quadratization). *Given a finite system of integer polynomial equations in natural variables $\mathbf{x}$, one can effectively construct a system of equations of degree at most two in $(\mathbf{x}, \mathbf{w})$ such that every original solution has exactly one extension $\mathbf{w}$, and no other solutions exist. Consequently the construction preserves finite-fold and single-fold fibers with respect to any distinguished input coordinates.*

Proof. Write each polynomial as $F^+ - F^-$, where F^+ and F^- have nonnegative integer coefficients. Construct acyclic arithmetic circuits for these two polynomials using natural constants, addition, and multiplication. For each gate introduce a fresh natural variable w . An addition gate gives $w - u - v = 0$ and a multiplication gate gives $w - uv = 0$; a constant gate gives $w - c = 0$. Finally equate the two output variables.

For any assignment to $\mathbf{x}$, evaluating the circuits gives an extension satisfying all gate equations. Conversely, topological induction through the circuit forces each gate variable to equal that value. The final output equality is precisely $F = 0$. Applying the construction to all residuals proves the assertion. Combining the quadratic residuals by (6) preserves this unique extension. $\square$

An arbitrary integer variable should not instead be replaced by $u - v$ with unrestricted $u, v \in \mathbb{N}$: the replacements $(u + k, v + k)$ immediately give infinitely many encodings of the same integer. The lemma avoids this problem by computing the positive and negative polynomial parts as *functions* of the original variables.

Lemma 2.14 (Canonical branch activation). *Suppose each labeled branch has a natural-witness certificate with at most one auxiliary assignment for fixed input and output. A finite branch choice can be compiled into quadratic residuals so that the resulting solutions correspond exactly to valid labeled choices, with no multiplicity from inactive branches.*

Proof. Use one-hot natural selectors x_a with $x_a(x_a - 1) = 0$ and $\sum_a x_a = 1$. For every branch-local witness $w_{a,j}$ impose $(1 - x_a)w_{a,j} = 0$. Thus all inactive branch witnesses are zero. Compute the positive and negative parts of each branch residual by the always-active circuits of Lemma 2.13; their outputs u, v are uniquely determined even when that branch is inactive. Gate the final equality by $x_a(u - v) = 0$, a quadratic equation. In the selected branch the original certificate holds and has its assumed unique extension; elsewhere the witnesses are fixed at zero and the circuit wires are forced. This proves the bijection. $\square$

The branch *label* is part of the computational object in this lemma. If two labels describe the same semantic event, a further canonicalization is needed. Also, directly multiplying a selector by a quadratic residual gives a cubic: computing its two polynomial parts first is what retains quadratic residual degree.

2.6 Manuscript 06: Domains, schedules, and three kinds of uniqueness

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every unknown in a Diophantine equation ranges over $\mathbb{N}$, while coefficients and polynomial subtraction belong to $\mathbb{Z}$. Thus $x - y$ in an equation is ordinary integer subtraction, never truncated natural subtraction. Vector inequalities, minima, and products explicitly called coordinatewise are taken coordinate by coordinate.

Definition 2.15 (Token transition). Fix $p \geq 1$. A transition a is a pair

$$u_a, v_a \in \mathbb{N}^p.$$

It is enabled at marking m when $m \geq u_a$, and its result is $m - u_a + v_a$. A finite ordered alphabet A labels the transitions; distinct labels may have identical numerical behavior. Write $q = |A| \geq 1$.

The model is exactly the firing semantics of a weighted Petri net once labels are assigned to transitions. It also describes multiset rewriting with a fixed finite species set. There is no test for the absence of tokens, no reset, copying operation, or multiplication of the current marking. An explicit input requirement permits a transition to consume and restore a catalyst. Such a transition cannot in general be replaced by its net displacement.

For a word $w = a_1 \cdots a_L$, write $m \xrightarrow{w} n$ when every successive transition is enabled and the final marking is n . The empty word ε is executable at every marking and leaves it unchanged.

Definition 2.16 (Parametrized repetition expression). Fix parameter names $k_1, \dots, k_r$. An expression has grammar

$$E ::= \varepsilon \mid a \mid (EF) \mid E^{k_j}.$$

An assignment $\mathbf{k} \in \mathbb{N}^r$ determines a finite word $\llbracket E \rrbracket_{\mathbf{k}}$ by concatenation and ordinary word powers. In particular, E^0 expands to ε , whether or not the expansion of E could execute from the given marking. The same parameter may occur at many nodes. The repetition depth is the maximum number of repetition nodes on a root-to-leaf path.

We count each empty expression, letter, concatenation, and repetition as a syntax node. Unless stated otherwise, the syntax is a tree. The template is fixed before the free numerical parameters are supplied. This is not a program that chooses a different syntactic body in response to its current counter values. Parameter-dependent choices will occur only inside the *certificate compiler*, and their values will be uniquely constrained.

Definition 2.17 (Single-fold auxiliary representation). A polynomial $Q(x, z) \in \mathbb{Z}[x, z]$ represents a relation $R(x)$ single-fold when

$$\#\{z \in \mathbb{N}^s : Q(x, z) = 0\} = \begin{cases} 1, & R(x), \\ 0, & \neg R(x). \end{cases}$$

A representation is finite-fold when this set is finite for every x and nonempty exactly on R .

The declaration of the free tuple x is part of the theorem. In our compressed compiler it includes the starting marking, ending marking, and *all* repetition parameters. If the parameters are subsequently quantified, the number of solutions can increase. A second uniqueness notion concerns the word represented by a parameter tuple. A third concerns the word selected inside a commutation class. These three notions will be kept separate.

Example 2.18 (An infinite multiplicity hidden by projection). For $E = (a^{k_1})^{k_2}$, every tuple with $k_1 = 0$ or $k_2 = 0$ expands to the empty word. Our compiler has exactly one auxiliary solution for each such tuple and equal endpoints. If (k_1, k_2) become witnesses too, the empty execution has infinitely many certificates. No local uniqueness gadget can remove this multiplicity without changing the parameterization.

2.7 Manuscript 07: Arithmetic and word interfaces

2.7.1 Natural witnesses, integer evaluation

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Witness coordinates lie in $\mathbb{N}$, but polynomial coefficients and evaluation lie in $\mathbb{Z}$. In particular, every subtraction in this article is signed subtraction. Formal queue lengths after an illegal event may be negative; silently replacing them by truncated natural subtraction would invalidate several arguments.

Definition 2.19. A *residual system* is a finite list $R_1, \dots, R_s \in \mathbb{Z}[X_1, \dots, X_k]$. Its associated polynomial is

$$P = \sum_{j=1}^s R_j^2.$$

Its natural zero set is the set of $\mathbf{x} \in \mathbb{N}^k$ satisfying every residual equation $R_j(\mathbf{x}) = 0$.

The equivalence follows because each square is a nonnegative integer. A quadratic residual system therefore gives a polynomial of degree at most four. Introducing an extra coordinate for a circuit gate changes the witness count; simply naming a polynomial subexpression does not.

2.7.2 Arithmetic circuits are polynomial syntax

A directed acyclic arithmetic circuit with integer constants and addition, subtraction, and multiplication denotes one ordinary integer polynomial. Reusing a node is a way of writing a polynomial compactly. It is not existential quantification over the node's value. Our JSON artifacts make this distinction explicit: only the listed input variables are witnesses; internal nodes are evaluated deterministically.

Consequently an $O(T)$ circuit need not expand to $O(T)$ monomials. For example, a product of T affine factors may have exponentially many monomials when expanded. Circuit operations on large integers are also not unit-cost bit operations. We report degree, number of witness coordinates, arithmetic size, and witness bit height separately.

The horizon T , the input word, and the program are parameters of the *construction*, not additional unknowns. A symbol such as 2^T is an integer coefficient computed before solving the equation. Products indexed by $i < T$ are finite syntactic products for that particular polynomial.

3 The seven manuscripts

[write] Each subsection below gives one manuscript's title, author line and date as delivered, its abstract and status statement verbatim, and then its own introduction (except manuscript 05's, which is Section 1). The introductions describe each manuscript's results in its own terms and numbering-free words; where they name a theorem, the reference points to where this report prints it.

3.1 Manuscript 05 (base): Canonical Trace Polytopes

[write] *Subtitle: "Witness-Preserving Diophantine Representations of Computational Substrates". "Research memorandum prepared for Vladimir Reshetnikov", September 30, 2026; pinned at e8bb0931d. Batch 60, manuscript 05 (canonical_trace_polytopes.zip). Printed in Section 1 (introduction), Section 2 (conventions), Parts I, II, VII, VIII and IX, and the back matter.*

Abstract. An existential Diophantine representation says that a computation exists; it need not preserve which computation occurred, how many computations occurred, or how many arithmetic witnesses describe it. We construct a stronger, bounded-horizon representation. For a Petri net with d places, m distinctly labelled transitions, and horizon T , an explicit convex quadratic integer polynomial has its nonnegative integer zeros in bijection with the commutation classes of enabled length- T executions between prescribed endpoints. The construction uses

$$(2T + 1)d + (7T + 1)m$$

unknowns. Its nonnegative real zero set is a bounded rational polytope, and every slack variable is uniquely determined by the canonical execution. A smaller quartic presentation is also given.

We derive an exact all-marking criterion for permissible commutations, realize arbitrary independence graphs by unit-weight nets, and prove an exponential automaton-state lower bound that explains the use of Boolean memory rather than an explicitly expanded automaton. Counting classes remains #P-complete even for unit-weight, 1-safe nets and their maximal static commutation relation. The same witness-preserving arithmetic methods apply to bounded counter programs, Boolean circuits, cellular automata, and explicitly size-bounded rewriting.

Finally, we prove a precise boundary theorem: a single finite-fold polynomial representation of the decidable verifier for canonical first-halting universal computations is equivalent to the general finite-fold Diophantine conjecture; the analogous statement holds for single-fold representations. The

bounded constructions do not cross this boundary. Complete mathematical proofs, an exact-arithmetic reference implementation, reproducible finite checks, and a formalization and research agenda are included.

Status. This is a proved theorem package and executable research prototype, not a claimed resolution of the general finite-fold or single-fold conjecture. Classical ingredients are identified in the text. Global priority for the combined results has not been established, and the new proofs have not been checked in Lean.

[write] *Manuscript 05's introduction is Section 1.*

3.2 Manuscript 01: Canonical Diophantine Certificates for Causal Computation

[write] *Subtitle: “Exact commuting-block acceleration, quartic trace quotients, and the boundary of single-foldness”. “Research report prepared for the ProveIt project”, 30 September 2026; pinned at e8bb0931d. Batch 60, manuscript 01 (Diophantine_Causal_Computation.zip). Printed in Parts I, II, III, VII, VIII and IX and the back matter.*

Abstract. Ordinary Diophantine representability records whether a computation exists; it need not preserve how many computational objects exist. We develop explicit ordinary polynomial representations which control the entire witness fibre for a substantial class of computational substrates. An action is an integer translation restricted to a coordinatewise lower-and-upper guard. This includes weighted Petri transitions and finite-control counter instructions with zero tests.

Our first result gives the exact initial-resource box in which *every* ordering of a prescribed multiset of actions can execute. When the active actions commute as partial maps, the same box characterizes *some* ordering. For Petri transitions it reduces to total resource loss plus the largest retained resource requirement. The resulting accelerator is a single-fold polynomial of degree at most four, with witness arity independent of the multiplicities. A concrete three-action example uses eleven witnesses even for multiplicities 10^{100} , 10^{80} , 10^{90} .

Our second construction combines these exact resource tests with the classical Cartier–Foata normal form. For each fixed causal-height bound, the nonnegative roots of an explicit quartic are in bijection with executable trace classes, not with their interleaved schedules. All intermediate markings, guard slacks, and normal-form auxiliaries are uniquely determined. We give complete proofs, exact variable and equation counts, block-composition extensions, an executable compiler, and differential tests. Finally, we prove that a single fixed-arity single-fold representation for universal halting would imply the full single-fold MRDP principle. The bounded-height family and commuting-block accelerator do not make that unproved leap.

Status. This is an unrefereed mathematical development with reproducible exact-integer tests, not a Lean-checked extension of ProveIt. The particular resource formulas and compiler assemblies are derived and proved here; their priority in the wider literature is not established. Classical arithmetization, trace normal forms, and the finite-fold conjecture are explicitly distinguished from these constructions.

3.2.1 The representation problem beyond MRDP

What is already available in ProveIt The starting point is the ProveIt repository at the pinned revision

e8bb0931d67f80d9fce87a8cddb0f661ff19f956.

Its Hilbert-tenth development already supplies MRDP, finite-witness integer polynomials, and Diophantine closure under exact finite iteration. In particular, the inspected source `Diophantine/Common/DiophantineTrace.lean` contains the interfaces `boundedForall_dioph`, `exactIter_dioph`, and `existsExactIter_dioph`. These are existence statements about Diophantine representations; they do not assert uniqueness of auxiliary witnesses [1, 3].

This is an important distinction in selecting a research target. Merely observing that finite executions of an effective substrate form a computably enumerable relation, and then applying MRDP, would add little to that development. Nor would another general register-machine-to-Diophantine proof be new: the repository includes the Jones–Matiyasevich 1984 route, and independent mechanized work uses Minsky machines and FRACTRAN [1, 8, 14].

The present target is *fibre control*: eliminate witnesses that encode nothing more than a choice of interleaving, an oversized coding base, an unconstrained inactive branch, or a noncanonical padding length. This is useful both mathematically, when counting solutions, and architecturally, when designing verified compilers that should preserve a specified semantic object rather than only existential truth.

Questions answered in this report The concrete questions are as follows. Can one characterize the exact resources needed to serialize a multiset without inspecting all its permutations? Can this become an ordinary low-degree polynomial with one complete witness tuple? Can a polynomial count causal traces rather than scheduling permutations? And which step would be needed to turn these constructions into a universal single-fold theorem?

The answers are given by the envelope theorem (Theorem 6.1), the fixed-arity accelerator (Theorem 21.1), the causal-height compiler (Theorem 20.1), and the universal-halting equivalence (Theorem 62.1). These are scoped representation results, not a claim to have settled a previously named major open conjecture.

Component	Role and provenance
MRDP and exact iteration	Existing repository infrastructure; used as context, not as the proof of the explicit compilers.
Cartier–Foata layers	Classical normal form; a self-contained proof of the version used here is supplied.
Resource envelope	Direct extremal-prefix derivation for arbitrary repeated box-guarded translations.
Quartic compilers	Explicit constructions with proofs of all auxiliary-witness uniqueness claims.
Global compression	Reduced to the single-fold or finite-fold MRDP principle; not claimed to follow from the bounded construction.
Formal verification	Proposed integration work. The attached Python tests are not proof-assistant verification.

3.3 Manuscript 02: Canonical Diophantine Certificates for Polynomial Trajectories

[write] Subtitle: “Single-fold acceleration, finite-difference sign tables, and the boundary of general computation”. “Research manuscript for the ProveIt project. Prepared with ChatGPT”, September 30, 2026; pinned at `e8bb0931d`. Batch 60, manuscript 02 (`canonical_diophantine_certificates.zip`). Printed in Section 2 (its Section 2) and Parts IV, VII, VIII and IX and the back matter; the first two subsections of its introduction open Part IV (Section 27).

Abstract. The existence of a Diophantine representation of finite computation does not by itself control the number of representing witnesses. We develop a canonical certificate construction for a substantial restricted class of computations: trajectories polynomial in the iteration counter, with arbitrary fixed Boolean combinations of polynomial guards.

The main construction eliminates a bounded universal quantifier over one integer variable without exponentiation and without an existentially chosen Chinese-remainder modulus. For every fixed degree bound d , the relation $\forall i \in \{0, \dots, T\} p(i) \geq 0$ has a single-fold representation by one integer polynomial of degree at most four, with $O((d+1)^3)$ natural witnesses, independently of T . The geometric part consists of exactly $(d+1)(2d+1)$ coordinates. These record the unique maximal sign intervals of $p, \Delta p, \dots, \Delta^d p$, with canonical padding. Monotonicity between successive difference levels reduces verification to endpoint tests.

We prove single-fold representations for exact guarded execution and first exit of polynomially iterable maps, including unitriangular nonlinear updates, unipotent affine maps, and a certified eventually quasi-polynomial affine extension. A canonical root bound also gives single-fold certificates for nontermination from a specified input. We provide a reproducible integer-only certificate generator, checker, and compiler to explicit quadratic residuals whose squared sum is the promised quartic.

The contribution is a constructive restricted result, not a solution of the general finite-fold or single-fold representation problem. The proofs are given here; historical priority of this particular construction has not been established, and no proof-assistant verification is claimed.

3.3.1 The problem addressed and the contribution

What is inherited from ProveIt We inspected the ProveIt repository at commit e8bb0931d67f80d9fce87a8cddb0f661ff19f956. Its Computability/HilbertTenthProblem development contains an MRDP interface and a finite-trace interface. In particular, DiophantineTrace.lean provides ordinary Diophantine closure for bounded universal quantification, exact iteration, and finite reachability [3]. These interfaces do not state uniqueness of the auxiliary witnesses. The project also maintains corrected editions of the Jones articles, including the Jones–Matiyasevich register-machine treatment of exponential Diophantine representation [1].

We use this as a precise project-level motivation: ordinary finite reachability is already available, so another invocation of MRDP would not constitute the intended advance. The new target is a restricted *multiplicity-preserving acceleration interface*. None of our proofs requires assuming an unproved repository theorem. Conversely, source inspection is not a fresh compilation or transitive axiom audit of the repository, and we make no such claim.

Relation to prior work and novelty status There are two relevant bodies of prior work. Single-fold specifications require care with arithmetic auxiliary variables and disjunction; the recent treatment by Cantone, Cuzziol, and Omodeo discusses these issues and the role of exponentiation [10]. Our total-circuit normalization is an explicit implementation-oriented formulation of elementary ingredients, not a claim that canonical arithmetic gadgets are new.

Polynomial and triangular loop analysis also has substantial prior literature. Hark, Frohn, and Giesl study triangular weakly nonlinear loops with polynomial Boolean guards and reductions for termination [25]. Lommen, Meyer, and Giesl use closed forms and stabilization thresholds in modular complexity analysis [26]. We therefore do not claim to introduce polynomial loop acceleration, polynomial closed forms, or input-dependent termination bounds.

The result developed here is the particular *canonical finite-difference certificate*, its explicit cubic witness bound, and its use in single-fold ordinary polynomial representations of execution. The cited papers were checked for the surrounding landscape, not exhaustively mined to prove historical priority.

Accordingly, this manuscript offers complete proofs of a specific constructive theorem, while leaving priority and publication novelty to further expert comparison. Its usefulness does not depend on labeling a standard consequence as a breakthrough.

3.4 Manuscript 03: Linear-Size Canonical Diophantine Certificates for Random-Access and Graph Computation

[write] *Subtitle: “Quartic equations, unique witnesses, and exact path weights”. “Research note prepared for Vladimir Reshetnikov”, September 30, 2026; pinned at e8bb0931d. Batch 60, manuscript 03 (ProveIt_Linear_Size_Diophantine_Certificates.zip). Printed in Section 2 and Parts V and IX and the back matter.*

Abstract. MRDP settles the existence of Diophantine descriptions of effective computation, but does not automatically preserve individual executions or control artificial witness multiplicity. We give two explicit canonical compilers for finite random-access memory logs over the natural numbers. Every valid log has exactly one auxiliary witness, and every invalid log has none.

The stronger scalar-size result uses exactly $24L$ witnesses and $22L + 1$ quadratic residuals for a length- L log. Their sum of squares is a sparse polynomial of degree at most four. A proposed sorted list is certified by packing complete records in a uniquely fixed radix and comparing two products at a base large enough to make coefficient carries impossible. The product identity is deterministic and exact, not a probabilistic fingerprint. All large powers are computed by quadratic chains. If the input coordinates have b bits, the largest witness has $O(L^2(b + \log(L + 2)))$ bits.

A complementary low-height compiler uses a canonical two-equation comparison gadget and a sorting network. Its exact counts are $10C + 2L + 3(N - 1)$ witnesses and $10C + 3L + 4(N - 1) + 1$ residuals for N padded records and C comparators. It gives explicit $O(L \log^2(L + 2))$ size, or $O(L \log(L + 2))$ using optimal networks, while keeping every witness to $b + O(\log(L + 2))$ bits.

We prove parsimonious composition with RAM instruction certificates, apply it to counter, cellular, stack, and immutable combinator-graph computation, and derive exact execution-count and first-halting probability formulas. We also prove the boundary: fixed-variable single-fold compression of one universal halting substrate is equivalent to the general single-fold conjecture. Our constructions are horizon-indexed families, not that compression. Both memory backends emit actual sparse quartics and pass reproducible exhaustive and adversarial tests. The proofs are ordinary mathematical proofs, not newly Lean-verified results; literature-wide priority is not claimed.

Keywords. Diophantine representation; random-access memory; parsimonious compilation; sorting networks; combinatory logic; finite traces.

3.4.1 The research target and its relation to ProveIt

What is already available The repository snapshot used for this investigation is

e8bb0931d67f80d9fce87a8cddb0f661ff19f956.

Its Hilbert-tenth documentation reports MRDP in both directions, a register-machine encoding, and reusable interfaces for bounded quantification, exact iteration, and finite reachability. Its combinatory-logic documentation also carefully distinguishes positive-step simulations from operational reflection and full abstraction. These are the relevant starting points [1, 2, 4]. We inspected these documents at the pinned revision; we did not rebuild the repository or independently audit its entire formal development.

The classical register-machine route is represented by Jones and Matiyasevich [8]. Carneiro’s Lean development supplies relevant historical context for machine-checked Diophantine exponentiation

[15]. Neither the classical theorem nor an existential library interface automatically gives a one-to-one correspondence between program executions and polynomial witnesses. That missing contract is the target here.

A statement that a Turing-complete substrate is Diophantine, obtained only by composing its interpreter with MRDP, would not be a new mathematical result. Instead we ask a quantitative and intensional question:

Can arbitrary finite memory access be represented by a sparse quartic equation, with exactly one auxiliary witness for each valid access log, without expanding the entire memory address space or introducing a factor proportional to address bit width in the number of arithmetic constraints?

The answer is yes. Sections 36–40 give two complete constructions: a low-height sorting-network compiler and a linear-size large-base-product compiler, both with exact counts and executable artifacts.

Prior art and the actual contribution Sorted memory transcripts and routing-network checks are established techniques. In particular, the RAM arithmetization in *SNARKs for C* checks local transitions and a transcript ordered by address and timestamp, using routing constraints [27]. We do not claim to invent that architecture or quasilinear verification. The distinction pursued here is an exact natural-number polynomial with a fully specified, unique auxiliary fiber, including the comparison and routing data. A deterministic comparison network makes this contract transparent. This is a different guarantee from merely exhibiting some valid routing.

Batcher’s sorting construction and the asymptotically optimal sorting-network theorem are also established results [28, 29]. Our proofs identify the precise arithmetic interface they need and quantify the complete resulting Diophantine certificate. The explicit bitonic version is proved and implemented below. Its improved network bound uses the external optimal-network theorem; no AKS implementation is supplied. The separate linear-size construction in Section 40 does not use that theorem or any sorting network. It uses an elementary carry-free integer evaluation argument to certify a unique sorted record list, paying for fewer coordinates with greater height.

Status	Content
Established background	MRDP; sorting-based RAM verification; Batcher and AKS sorting networks.
Proved in this note	Two exact canonical memory compilers; linear scalar size; witness and residual counts; size–height tradeoffs; composition and weighted-path consequences.
Executable artifact	A general symbolic memory-log compiler, sparse quartic export, canonical witness generation, and deterministic regression tests.
Not established here	Literature-wide novelty of every formulation; fixed-variable single-fold representation of arbitrary halting; a formal verification of this new compiler; full abstraction of ProveIt’s existing lambda-to-combinator translations.

This status distinction matters. The result is a concrete research extension with complete proofs, not a declaration that a famous open conjecture has been solved. Its potential value to ProveIt is a stronger reusable specification: existence, uniqueness, decoding, and quantitative bounds travel together.

3.5 Manuscript 04: Witness-Faithful Diophantine Compilation

[write] *Subtitle: “Concurrent traces, priority arithmetic, and context-local rewriting”. “Research article prepared for Vladimir Reshetnikov”, September 30, 2026; pinned at e8bb0931d. Batch 60, manuscript 04*

(*Witness_Faithful_Diophantine_Compilation.zip*). Printed in Section 2 and Parts II, VII, VIII and IX and the back matter.

Abstract. Existential Diophantine representability is already available for every computably enumerable computational semantics. A more discriminating question is whether arithmetic solutions faithfully represent the intended computational objects, without spurious choices of auxiliaries or repeated encodings of the same concurrent execution. We develop three explicit constructions addressing this question. For a Petri net with p places and r labeled transitions, exact T -step executions between fixed endpoints, modulo a specified disjoint-support commutation relation, are in bijection with the natural zeros of a polynomial of degree at most four in $(p + r)(2T + 1)$ variables. The construction uses a quadratic, r -bit monitor for lexicographic trace normal forms; a matching linear-in-alphabet memory lower bound follows from an explicit family requiring exponentially many deterministic automaton states. For an ordered r -fraction FRACTRAN program, an exact run is represented uniquely using $T(7r + 2) + 1$ variables and $T(8r + 3) + 2$ quadratic residuals. For SKI rewriting along a fixed schedule, a context-local encoding uses whole-subterm integer codes, with size linear in the schedule and its total context depth, rather than in the number of tree nodes traversed by a full tableau.

All constructions include two-way proofs, exact resource counts, and executable exact-integer certificates. We distinguish these bounded, witness-faithful constructions from fixed-arity single-fold and finite-fold representations of unbounded computation. The latter are not obtained by determinism alone. The article identifies the precise compression frontier, gives a witness-height obstruction, and proposes a research program linking these constructions to the existing computability developments in ProveIt. Global priority for the particular constructions is not established; no resolution of the general finite-fold or single-fold problem is claimed.

Keywords. Diophantine representation; arithmetic certificates; Petri nets; trace monoids; FRACTRAN; combinatory logic; single-fold representations.

3.5.1 The problem beyond existential representability

What is already settled The Matiyasevich–Robinson–Davis–Putnam theorem, abbreviated MRDP, identifies computably enumerable subsets of the natural numbers with Diophantine sets [7]. Thus a computational substrate with an effective finite-execution semantics generally presents no new *existence* problem: its finite reachability relation is computably enumerable and hence Diophantine. The constructive route through register machines has a substantial history; Jones and Matiyasevich give an exponential-Diophantine register-machine construction, and Larchey-Wendling and Forster’s full mechanization uses FRACTRAN as an intermediate language [8, 13]. Re-proving that FRACTRAN, SKI reduction, or Petri-net reachability is merely Diophantine would therefore miss the interesting issue.

The inspected ProveIt snapshot already contains MRDP interfaces, bounded universal quantification, exact iteration of a Diophantine relation, and finite reachability [1, 3]. Its combinatory-logic development supplies concrete operational semantics and compiler simulations [4]. These are appropriate foundations, but ordinary existential equivalence says nothing about how many arithmetic witnesses encode one execution.

We instead ask for an explicit map

$$\{\text{intended computational objects}\} \longleftrightarrow \{\text{natural solutions of an arithmetic certificate}\}$$

that is a *bijection*. Depending on the substrate, the intended object is a labeled sequential run, a commutation class of concurrent runs, or a reduction along a specified schedule. The distinctions matter. Two different firing words may describe the same concurrent trace, while two distinct traces may have the same endpoint and the same transition counts.

The contributions and their scope The principal construction is a direct arithmetic quotient of Petri-net executions by independent swaps. It does not first enumerate a potentially exponential-state normal-form automaton. A bit for each transition records exactly the information needed to prohibit a nonminimal linearization. The resulting polynomial has one solution per concurrent trace class, not one solution per scheduling permutation.

The second construction handles *priority*. FRACTRAN is deterministic only after the least applicable fraction is selected. A polynomial that merely allows any applicable fraction models a different language. We build the least-index choice into a cumulative prefix circuit and force every division and inactive slack to be canonical.

The third construction handles *locality*. A single integer represents a whole SKI subtree. A prescribed contextual contraction need decompose only the path to the redex and the finite pattern at its root. The result is a compact certificate in variable count, but not necessarily in bit length. This is a useful separation of two notions of succinctness, not a claim that integer coding removes all costs of large terms.

Construction	Object represented	Natural-solution multiplicity
Petri net, fixed T	A class of enabled words under the chosen independent swaps	Exactly one per class
FRACTRAN, fixed T	The exact priority-respecting run	Zero or one
SKI, fixed schedule	The scheduled contextual reduction	Zero or one
Unbounded reachability	Existence of some finite execution	MRDP supplies existence, not these multiplicity guarantees

The exact compiler theorems, their joint interpretation, and the resource accounting are the results developed in this article. The normal-form criterion, circuit encodings, and general MRDP implications are established background or elementary consequences, and are identified as such. The constructions have not undergone independent peer review or a new proof-assistant check. The accompanying tests are reproducibility evidence, not substitutes for the proofs.

Related work and a necessary distinction Lexicographic trace normal forms have a classical forbidden-factor characterization, often attributed to Anisimov and Knuth; a modern use appears in Lohrey, Stober, and Weiß [20]. Counting trace classes is also an active subject. De Colnet, Meel, and Mathur study counting and sampling the classes intersected by a regular language, presented by a finite automaton, and obtain complexity and approximation results [21]. Our purpose is different: a succinct operational model is compiled into an exact polynomial with controlled fibers. No new general trace-counting principle is asserted.

A subtlety is that a class intersecting an arbitrary language need not have its lexicographically least member in that language. Our Petri-net endpoint languages are *trace-closed* for the specified independence relation. This closure, proved below, is what makes normal-form filtering preserve the desired set of classes.

3.6 Manuscript 06: Saturation and Single-Fold Diophantine Compilation of Concurrent Counter Programs

[write] *Subtitle: “Resource summaries, canonical traces, and nested repetition schemes”. “Research article. Prepared for Vladimir Reshetnikov”, September 30, 2026; pinned at e8bb0931d. Batch 60, manuscript 06 (diophantine_counter_programs.zip). Printed in Section 2 and Parts I, II, III, VIII and IX and the back matter.*

Abstract. We give an explicit, witness-preserving route from structured concurrent counter executions to ordinary polynomial equations over the nonnegative integers. A firing word is summarized by the least initial resources it needs and the resources it leaves at that least input. This classical partial-translation algebra admits exact composition and arbitrary-power formulas with unique quadratic witnesses. Independently, we construct a Boolean transfer signature for lexicographic trace normality and prove that every signature satisfies $S^2 = S^3$. Consequently, in an arbitrarily nested repetition expression, normality depends on each repetition parameter only through the three cases 0, 1, and at least 2.

Combining the two summaries gives a polynomial of degree at most four with $O((p+q)N)$ auxiliary variables for a schedule with N syntax nodes, p resources, and q transition labels. For fixed endpoints and repetition parameters, there is exactly one auxiliary solution precisely when the specified execution is valid and, optionally, already canonical. The expanded execution length does not occur in this variable bound. A separate bounded-word construction gives a genuine bijection between polynomial solutions and executable trace classes.

We also locate a precise expressiveness boundary. Flat repetition schemes have Presburger-definable execution relations, whereas separated increment/decrement schemes of nesting depth two represent arbitrary systems of quadratic equations. For one counter, separated depth- d schemes correspond exactly to degree- d polynomial equations. Finite-fold and single-fold Diophantine representability are equivalent to the corresponding bounded-multiplicity parameterizations by separated one-counter schemes of depth four. These equivalences identify the remaining obstruction; they do not resolve the finite-fold or single-fold conjectures. An executable compiler and 49,722 finite checks accompany the proofs.

Status. The article contains mathematical proofs and tested prototype code, not a new kernel-checked formalization. Classical ingredients are identified explicitly. Historical priority for the combined constructions and formulations is not established by this study.

3.6.1 Purpose, context, and the main results

A general existence theorem can hide exactly the information needed by a compiler. The Matiyasevich–Robinson–Davis–Putnam theorem says that every computably enumerable relation has a Diophantine representation [7]. It does not, by itself, say how to preserve a particular execution, how many witnesses that execution acquires, how large those witnesses are, or whether two witnesses encode the same concurrent behavior. These distinctions become essential when Diophantine representation is used as a computational substrate rather than only as an undecidability theorem.

The motivating repository, *ProveIt*, already documents MRDP in both directions, finite-trace and bounded-quantifier interfaces, register-machine encodings, and universal Diophantine equations [6]. Its relevant status register also distinguishes completed formal results from remaining obligations [6]. Repeating that a computable interpreter has a Diophantine graph would add little to that program. We instead develop a direct compiler with a precise contract: a specified structured execution receives *one and only one* auxiliary certificate, with a degree and size bound visible from its syntax.

There are two logically different sources of multiplicity. An arithmetic encoding can create many witnesses for one computation. Independently, a concurrent computation can have many interleavings, or a repetition expression can have many parameter tuples that expand to the same word. We eliminate the first multiplicity. For all words of a fixed length we can also eliminate the interleaving multiplicity. We do not silently identify either achievement with eliminating multiplicity from arbitrary existential repetition parameters.

The three principal constructions For a finite collection of token-consuming and token-producing transitions, a word induces a partial translation of $\mathbb{N}^P$. Its domain is a principal upper orthant. The least

input and its output provide an exact resource summary. Composition uses coordinatewise cancellation; repetition has a closed formula including the exceptional zero-power case. All local choices can be expressed by uniquely solvable quadratic systems.

The second construction concerns a different object: the lexicographically smallest word in a trace class. The familiar normal-form language is regular, but constructing a full transition table on all subsets of the alphabet would be wasteful. We describe its block action with $3q + 1$ Boolean coordinates, subject to a disjointness invariant. This description admits linear-size composition circuits. More importantly, its powers stabilize after the second power. The stabilization remains valid inside arbitrary nested repetitions.

The third construction combines these branches without conflating them. The resource summary is invariant under the permitted commutations. The normal-form signature is intentionally *not* invariant: it distinguishes the chosen representative from the other words in the same trace. Their quadratic constraints can nevertheless be joined in a single sum of squares.

Theorem 3.1 (Summary of the compilation results). *Let E be a fixed parametrized repetition expression with N syntax nodes over q transitions acting on p nonnegative counters. Let I be a fixed symmetric, irreflexive relation of globally commuting transition labels.*

- (i) *Whether the expansion of E is lexicographically least in its I -trace class is unchanged when every repetition parameter k is replaced by $\min(k, 2)$.*
- (ii) *One can construct an integer polynomial of degree at most four with $O((p + q)N)$ auxiliary variables whose natural solutions, for fixed endpoints and repetition parameters, form a singleton exactly when the specified execution is valid and already lexicographically least. Without the normality requirement the auxiliary bound is $O(pN)$.*
- (iii) *For fixed length $T \geq 1$, one can instead construct a quartic polynomial with*

$$q(2T + 1) + p(7T - 4)$$

witness variables whose solutions are in bijection with the executable I -trace classes of length T between the specified endpoints.

Part (i) is proved in Section 24, part (ii) in Section 25, and part (iii) in Section 18. A second group of results, in Sections 26 and 67, characterizes exactly where unbounded parameter search restores the full difficulty of Diophantine solvability.

What is classical, and what is proved here The algebra of one partially blind counter is the bicyclic monoid; products of such storage components are standard in the study of valence automata [23]. Lexicographic trace normal forms go back to Anisimov and Knuth [19]. The conversion of arithmetic circuits into quadratic systems, followed by a sum of squares, is a standard degree-reduction method. Single-fold representation and its closure issues have an extensive literature [9, 10]. None of these ingredients is presented as a new discovery.

The contributions established in this article are the explicit synchronized construction, its local-to-global uniqueness proof, the compact normality signature and its cap-at-two consequence for nested schemes, the stated witness and trace-counting bounds, and the solution-preserving substrate normal forms. We give full proofs rather than relying on novelty labels. A targeted literature review did not establish historical priority for every equivalent formulation. In particular, a useful new implementation or packaging of known algebra should not be confused with a first theorem about that algebra.

All statements below are independent of any unverified theorem elsewhere in ProveIt. The repository is the motivation and a prospective integration target. The source audit used its tree at commit e8bb0931d67f80d9fce87a8cddb0f661ff19f956; it was not a rebuild or an independent audit of the repository's proof kernel dependencies.

3.7 Manuscript 07: Causality Without Tableaux: Compact Diophantine Certificates for Queue Universality

[write] Subtitle: “Product guards, tag systems, and optimal polynomial word-memory dimension”. “Research article. Prepared for Vladimir Reshetnikov”, September 30, 2026; pinned at 725d2ebb6. Batch 61, its only manuscript (*Causal_Diophantine_Queue_Compilation.zip*); numbered 07 in this report. Printed in Section 2 and Parts I, VI and IX and the back matter.

Abstract. We construct ordinary integer polynomials whose natural zeros faithfully encode finite computations of cyclic tag systems and general deletion-tag systems. The main cyclic-tag construction uses exactly $T + 1$ variables for a T -step execution, has degree at most $\max\{4, 2T\}$, and admits a linear-size arithmetic circuit. Its variables are the consumed bits and one uniquely determined slack; no intermediate queue is quantified. The key is a causal reconstruction theorem: one global word identity becomes sufficient when accompanied by a product guard. A first-failure lemma proves that this guard is positive for legal executions and exactly zero for every illegal candidate, even if later formal resource values become negative.

Two alternatives expose the cost tradeoff. A bounded-root construction removes the slack at the expense of a larger circuit. A local radix construction yields a quartic polynomial in $3T - 2$ variables. The general deletion- d construction uses $dT + 1$ variables, including all consumed symbols, with an explicit degree bound and explicit denominator clearing. Fixed-read FIFO networks admit a corresponding consumed-symbol-plus-one bound.

A separate obstruction explains the radix architecture. Finite words admit a faithful two-coordinate natural-number encoding with affine insertion at either end. No faithful one-coordinate natural-number encoding can make both binary prepends and even one fixed append polynomial. An elementary positive-leading-coefficient centralizer argument proves this dimension lower bound.

The article gives complete mathematical proofs, executable compilers, an independent exact-integer verifier, finite exhaustive checks, and a proposed formalization and research program. These are horizon-indexed, witness-faithful constructions, not a fixed-arity finite-fold representation of unbounded halting. Historical priority for the particular constructions is not established; no general finite-fold conjecture is claimed solved.

Verification status. The proofs below are mathematical proofs and the supplied Python artifacts have been tested. This package does not contain a new kernel-checked Lean or Rocq formalization. Repository audit reports are distinguished from checks performed for this article.

3.7.1 The question beyond representability

Why another Diophantine construction? The Matiyasevich–Robinson–Davis–Putnam theorem identifies computably enumerable sets of natural numbers with Diophantine sets. Thus, once a computational substrate has an effective finite-execution semantics, the mere existence of a Diophantine representation of its halting relation is not a new universality theorem. The interesting questions concern the structure of the representing polynomial: its variables, degree, coefficient size, circuit size, and the relationship between its solutions and actual computational histories. The formal developments of Carneiro and of Larchey-Wendling and Forster provide important precedents for making such reductions precise [15, 13].

ProveIt’s Hilbert-tenth-problem development already supplies the forward and reverse MRDP interfaces, finite polynomial representations, and trace infrastructure [5, 1]. We therefore do not repackage “this machine is effective, hence MRDP applies” as a new result. Instead we ask three concrete questions.

Can a certificate for a variable-length queue avoid storing every intermediate queue? Can a global equality between the words consumed and produced certify an execution without permitting a symbol to create itself after an illegal read? Finally, can a natural-number memory with polynomial operations at both ends really be reduced from two coordinates to one? Theorems 46.1, 8.1, and 50.4 answer these questions in explicit models.

Cyclic tag systems are a useful test case: their binary queue semantics is very small, yet the class is computationally universal. Cook uses them as an intermediate substrate in his universality proof for Rule 110 [31]. Tag-system simulation also has a quantitative literature; Woods and Neary establish polynomial-time simulation results rather than merely abstract universality [32]. We use these existing universality results as context. Our proofs concern arithmetic certificates for the precisely defined queue dynamics below.

The results at a glance For a fixed cyclic-tag program, input word, terminal word, and positive horizon T , the main construction represents the exact T -step execution relation. For empty terminal word it represents first halting at exactly T . Every successful instance has exactly one natural solution.

Construction	Variables	Degree bound	Residuals
Causal word identity	$T + 1$	$\max\{4, 2T\}$	$T + 3$
Bounded-root, no slack	T	$2 \max\{2, T, H\}$	$2T + 2$
Local radix, empty endpoint	$3T - 2$	4	$3T$

Here H is an explicit upper bound on queue length, defined in Section 48. The first and third rows have $O(T)$ arithmetic-circuit size for a fixed program; the second has $O(T(1 + H))$ size. All three denote ordinary polynomials, not expressions with variable exponents or an unbounded product operator.

For deletion- d tag systems over an alphabet of size B , we give $dT + 1$ variables and degree at most

$$2 \max\{B, T(B - 1), d(T - 1)(B - 1), 1\}.$$

The separate memory theorem is an actual lower bound, but in a carefully specified architecture: faithful word encodings with polynomial insertion maps. No lower bound on the number of variables of arbitrary Diophantine representations is asserted.

Relation to earlier work and scope of novelty [write] Manuscripts 04 and 06 of this report treat concurrent resource traces, priority arithmetic in FRACTRAN, context-local SKI rewriting and compressed counter schedules (Parts II, III and VII). This article instead treats FIFO word content, temporal availability of produced symbols, and the dimension of polynomial word memory. The companion articles are not proof dependencies.

Radix encodings, sums of squares, finite-table interpolation, and polynomial centralizers are established mathematical tools. Our contribution is the explicit construction and proof of the displayed resource guarantees, the causal obstruction and its repair, and the sharply stated word-memory consequence. The centralizer proof is self-contained; the broader theory has a substantial literature, for example Atela and Hu [33]. A targeted literature check does not establish historical priority for every formulation here. These results should be evaluated as proved constructions and a proved architectural obstruction, not as a claim that a named longstanding conjecture has been settled.

Part I

Exact commutation and resource algebra

Source and scope. [write] This Part prints four manuscripts' treatments of one question: when does a sequence of guarded resource updates execute, and when may two adjacent updates be exchanged at every state? Manuscript 05 gives the exact all-marking commutation criterion for Petri transitions, realizes every independence graph, and, in a subsection moved here from its section on other substrates, extends the criterion to interval-guarded translations (Theorems 4.1, 4.4 and 4.5). Manuscript 01 proves the same box criterion for its guarded translations (Proposition 5.1) and its scalar Petri classification, and then the exact resource envelope of all serializations of a multiset (Theorem 6.1). Manuscript 06 develops the resource-summary algebra of consume/produce words: principal partial translations, composition by cancellation, exact powers, and its own global commutation test (Theorem 7.7). Manuscript 07 closes the Part with its first-failure product (Theorem 8.1), used by the queue certificates of Part VI.

Duplicates. [write] The commutation criterion is proved three times. Manuscript 05's Theorems 4.1 and 4.5 are printed as the main statements, since they name the maximal relation and cover interval guards; manuscript 01's Proposition 5.1 has the same box formulas (12)–(13) as Theorem 4.5 and is kept, with its own proof, as a second route; manuscript 06's Theorem 7.7 states the criterion through least inputs, $\min(u_b, v_a) = \min(u_a, v_b)$, which is the counter form of $D_{ab} = D_{ba}$. Manuscript 01's envelope and manuscript 07's first-failure product are different theorems, proved by the same induction on formal prefix states (as is manuscript 01's Proposition 22.2 in Part III); manuscript 06 exponentiates one word exactly (Theorem 7.5), and manuscript 01 does so in Proposition 22.2.

Setting and hypotheses. [write] Each manuscript keeps its own model. Manuscript 05: a Petri net with d places and m transitions, consumption c_a and production v_a ; interval guards $l_a \leq x \leq u_a$ with translation updates in Theorem 4.5. Manuscript 01: box-guarded translations $F_a(x) = x + v_a$ on $\ell_a \leq x \leq u_a$ in $\mathbb{N}^p$, with lower and upper guards, m actions. Manuscript 06: token-consuming and token-producing transitions on p counters with q labels, and no upper guards. Manuscript 07: candidate event sequences on p resources whose formal integer histories are allowed to become negative after an illegal step.

4 An exact commutation test for Petri transitions [05]

4.1 Model and sequential demands

Let $\mathcal{N}$ have $d \geq 0$ places and a finite ordered alphabet $\mathcal{A} = \{1, \dots, m\}$ of transitions, where $m \geq 1$. Transition a has consumption and production vectors

$$c_a, v_a \in \mathbb{N}^d.$$

At a marking $x \in \mathbb{N}^d$ it is enabled exactly when $x \geq c_a$ coordinatewise, and its result is

$$a(x) = x - c_a + v_a.$$

Thus every label acts as a partial function. In a word ab , transition a is performed first.

For a vector $y \in \mathbb{Z}^d$, write y_+ for its coordinatewise positive part. Define the *sequential demand*

$$D_{ab} = c_a + (c_b - v_a)_+ = \max(c_a, c_a + c_b - v_a). \quad (7)$$

Theorem 4.1 (Exact all-marking commutation criterion). *The two-step word ab is enabled at x if and only if $x \geq D_{ab}$. Transitions a, b commute as partial functions if and only if*

$$D_{ab} = D_{ba}. \quad (8)$$

Consequently

$$I_{\max} = \{(a, b) : a \neq b, D_{ab} = D_{ba}\}$$

is the maximal symmetric irreflexive, context-independent relation whose adjacent swaps preserve enabledness and final markings for all starting markings. It can be computed using $O(dm^2)$ coordinate comparisons and arithmetic operations.

Proof. The two enabling conditions are $x \geq c_a$ and $x - c_a + v_a \geq c_b$. Their conjunction is exactly $x \geq \max(c_a, c_a + c_b - v_a) = D_{ab}$. Whenever both orders are enabled they have the same result,

$$x - c_a + v_a - c_b + v_b.$$

They are therefore the same partial function exactly when their domains agree. Two principal coordinate orthants $\{x : x \geq D\}$ and $\{x : x \geq E\}$ agree exactly when $D = E$: inclusion in one direction forces the corresponding coordinate inequality by testing the corner of the smaller orthant. This proves necessity as well as sufficiency.

A swap inside a longer word has a common prefix marking. Equality of the two partial maps preserves whether the two-step fragment exists and, if it exists, the marking at the beginning of the unchanged suffix. Hence every pair in the displayed relation is sound in all contexts. Conversely, a relation sound in every context is sound for the empty prefix and suffix at every marking, so every one of its pairs satisfies (8). Computing both demand vectors for each pair gives the operation bound. $\square$

The operation bound is not a claim of constant-cost arithmetic on unbounded integers; the bit complexity includes the arc-weight bit lengths. Only additions, comparisons, and positive parts of input-size integers are needed.

Example 4.2 (Why effect vectors are insufficient). On one place let a produce a token and let b consume a token:

$$(c_a, v_a) = (0, 1), \quad (c_b, v_b) = (1, 0).$$

The net changes $+1$ and -1 commute algebraically, but ab is enabled at marking zero and ba is not. Indeed $D_{ab} = 0$ while $D_{ba} = 1$.

Remark 4.3 (Sequential commutation is not simultaneous independence). The relation in Theorem 4.1 means equality of sequential partial maps. It is not a claim that the two transitions can fire simultaneously, nor that their token occurrences are causally independent in an unfolding. Two pure consumers sharing a place satisfy $D_{ab} = c_a + c_b = D_{ba}$, although they compete for the same resource. The quotient in this article is explicitly the sequential commutation quotient.

4.2 There is no hidden graph restriction

Theorem 4.4 (Realizing arbitrary independence graphs). *Every symmetric irreflexive relation I on m labels is exactly $I_{\max}$ for a Petri net with unit arc weights and at most $\binom{m}{2}$ places.*

Proof. For each unordered nonedge $\{a, b\}$, with $a < b$, introduce a private place p_{ab} . Transition a produces one token there, transition b consumes one token there, and all other transitions leave it untouched. At this coordinate $D_{ab} = 0$ and $D_{ba} = 1$, so the nonedge is not in $I_{\max}$.

If $\{a, b\}$ is an edge of I , no private place is assigned to this pair. At every existing place, at most one of these two transitions acts. A transition and an identity action have equal demands in either order. Thus all coordinates satisfy $D_{ab} = D_{ba}$, and the pair belongs to $I_{\max}$. The arcs have weights zero or one, and there is one place per nonedge. $\square$

For a complete graph, the construction has zero places and all transitions are identity actions. An unused dummy place can be added when a convention requires a nonempty place set; this has no semantic effect. The theorem concerns commutation at all markings, not boundedness or safety from a particular initial marking.

4.3 Exact commutation with interval guards

[write] *This subsection is moved from manuscript 05's section on other substrates (Section 57, Part VIII), where it follows the treatment of counter and register programs with zero tests; its corollary for guarded programs is in Part II (Corollary 11.5). Manuscript 01's Proposition 5.1 below proves the same box formulas.*

The Petri demand formula cannot simply be reused after adding zero tests. Domains now contain equalities as well as lower bounds, and a previously commuting update may destroy a guard. The needed stronger test is still explicit for interval-guarded translations.

Theorem 4.5 (Exact commutation with interval guards). *Suppose transition a has domain*

$$D_a = \{x \in \mathbb{N}^d : l_a \leq x \leq u_a\}$$

and update $x \mapsto x + \delta_a$, where $l_a \in \mathbb{N}^d$, $u_a \in (\mathbb{N} \cup \{\infty\})^d$, $l_a \leq u_a$, and $l_a + \delta_a \in \mathbb{N}^d$. Define

$$L_{ab} = \max(l_a, l_b - \delta_a),$$

$$U_{ab} = \min(u_a, u_b - \delta_a),$$

using $\infty - t = \infty$. The domain of ab is the box $[L_{ab}, U_{ab}] \cap \mathbb{N}^d$, declared empty if any lower endpoint exceeds its upper endpoint. Two transitions commute as partial maps exactly when either both composed boxes are empty, or both are nonempty and

$$L_{ab} = L_{ba}, \quad U_{ab} = U_{ba}.$$

The maximal such relation is computable with $O(dm^2)$ coordinate operations.

Proof. The two domain conditions are $l_a \leq x \leq u_a$ and $l_b \leq x + \delta_a \leq u_b$. Intersecting their coordinate intervals gives exactly the stated box. The final update, when defined, is $x + \delta_a + \delta_b$ in either order. Empty compositions agree as partial functions. For nonempty integer boxes, equality of domains is equivalent to equality of all lower and upper endpoints: each finite extreme is attained, and an unbounded coordinate is distinguished from a bounded one by an arbitrarily large coordinate value. This proves the criterion and the direct pairwise algorithm. $\square$

Lower guards express positivity and consumption; upper guard zero expresses a zero test. Finite control can itself be a coordinate with lower and upper endpoints both equal to the source state, and update equal to target minus source. Thus the interval model includes the usual zero-testing counter programs, not just ordinary Petri nets. Reset and copy updates are not translations and are not covered by this theorem.

5 Guarded translations and exact commutation [01]

5.1 A substrate with natural-number states

Fix a dimension $p \geq 1$ and a finite nonempty action alphabet A of size m . Each action a has a lower guard $\ell_a \in \mathbb{N}^p$, a displacement $v_a \in \mathbb{Z}^p$, and an upper guard $u_a \in (\mathbb{N} \cup \{\infty\})^p$. We require, coordinatewise,

$$\ell_{aj} \geq \max(0, -v_{aj}), \quad \ell_{aj} \leq u_{aj}. \quad (9)$$

The action is the partial map

$$F_a(x) = x + v_a \quad \text{defined precisely when } \ell_a \leq x \leq u_a. \quad (10)$$

Condition (9) guarantees natural outputs. An infinite upper guard imposes no upper restriction.

For a word $w = a_1 \cdots a_t$, execution means that every successive partial map is defined. Its endpoint, when it exists, is $x + \sum_{i=1}^t v_{a_i}$. The endpoint equation alone is insufficient: intermediate guards can fail even if the final vector is nonnegative.

A weighted Petri transition with consumption $c_a \in \mathbb{N}^p$ and production $d_a \in \mathbb{N}^p$ is obtained by taking

$$\ell_a = c_a, \quad v_a = d_a - c_a, \quad u_a = \infty.$$

Upper guards also permit exact-value tests, in particular a zero test $\ell_{aj} = u_{aj} = 0$. This elementary model is sufficiently broad to expose the distinction between resource feasibility and mere net displacement.

5.2 An exact and computable independence relation

We call distinct actions independent when they commute *as partial maps*, including equality of domains:

$$F_b \circ F_a = F_a \circ F_b. \quad (11)$$

This is stronger than equality of final translations on the common domain. All translations have commuting displacements; their guards need not commute.

Proposition 5.1 (Exact box test; manuscript 01, second route to Theorem 4.5). *The domain of “first a , then b ” is the box with bounds*

$$L_{ab,j} = \max(\ell_{aj}, \ell_{bj} - v_{aj}), \quad (12)$$

$$U_{ab,j} = \min(u_{aj}, u_{bj} - v_{aj}). \quad (13)$$

It is empty if $L_{ab,j} > U_{ab,j}$ for some j . Two actions commute if and only if both composition boxes are empty, or both are nonempty and their lower and upper bounds agree.

Proof. The two enabling conditions are $\ell_a \leq x \leq u_a$ and $\ell_b \leq x + v_a \leq u_b$. Intersecting them gives the displayed box. Whenever both compositions are defined, their outputs are the same vector $x + v_a + v_b$. Thus equality of partial maps is exactly equality of their domains. Nonempty boxes in $\mathbb{N}^p$ of this form have uniquely determined finite lower endpoints and finite or infinite upper endpoints, so equality is detected coordinatewise. Empty boxes must be compared as empty sets, not by their possibly different displayed bounds. $\square$

Testing one pair requires $O(p)$ integer comparisons and additions. Testing all unordered pairs gives the largest static irreflexive independence relation in $O(pm^2)$ such operations. Any symmetric irreflexive subrelation of this relation can be used below, at the cost of retaining more distinct traces.

Remark 5.2 (Not physical simultaneous firing). Our independence concerns interchangeable *sequential* schedules. It is not the Petri-net step convention that reserves all input tokens simultaneously. For example, two named transitions each consuming one token and producing none commute as partial maps: either order is defined exactly at markings at least two. They are both individually enabled at marking one, but their two-event word is not executable there. The resource theorem below retains this distinction.

5.3 A closed classification for one Petri coordinate

The box test has a useful explicit specialization. Let (c, d) and (e, f) be the consumption–production pairs of two actions on one coordinate, and put $v = d - c$, $w = f - e$.

Proposition 5.3 (Scalar Petri commutation; the one-coordinate case of Theorem 4.1). *The two lower-guarded translations commute precisely in the cases in Table 2. For vector-valued Petri transitions the criterion must hold in every coordinate.*

Displacement signs	Necessary and sufficient condition
$v = w = 0$	Always
$v > 0, w > 0$	$c = e$
$v < 0, w < 0$	$d = f$
$v = 0, w \neq 0$	$c \leq \min(e, f)$
$w = 0, v \neq 0$	$e \leq \min(c, d)$
$vw < 0$	Never

Table 2: Commutation as equality of partial sequential maps, not concurrent token reservation.

Proof. There is no upper bound, so the criterion is

$$\max(c, e - v) = \max(e, c - w). \quad (14)$$

If $v, w > 0$ and $c > e$, the left side equals c , while both arguments on the right are less than c . The case $e > c$ is symmetric; $c = e$ works.

If $v = -r < 0$ and $w = -s < 0$, substitute $c = d + r$ and $e = f + s$. Both sides of (14), after subtracting $r + s$, become respectively $\max(d - s, f)$ and $\max(f - r, d)$. If $d > f$, the second is d and the first is strictly smaller; the reverse inequality is symmetric. Equality of d, f is sufficient.

For $v = 0$, (14) reads $\max(c, e) = \max(e, c - w)$. With $w > 0$ it holds exactly when $c \leq e$; with $w < 0$ it holds exactly when $c \leq e + w = f$. The other zero-displacement case is symmetric. If $v > 0$ and $w = -s < 0$, the right side is $\max(e, c + s)$, strictly greater than both c and $e - v$, so equality fails. Finally, two zero displacements give the identical threshold $\max(c, e)$.

For vectors, all composition domains are nonempty upward boxes. Equality of their coordinate bounds is necessary and sufficient by Proposition 5.1. $\square$

This classification recognizes shared-resource commutation that a disjoint-coordinate test would miss. A read-only transition can commute with a consuming transition when its read threshold is no greater than the resource retained after the latter fires.

6 The exact resource envelope of a multiset [01]

Write

$$v_{aj}^+ = \max(v_{aj}, 0), \quad v_{aj}^- = \max(-v_{aj}, 0).$$

For multiplicities $n = (n_a)_{a \in A} \in \mathbb{N}^m$, let

$$S_j^- = \sum_a n_a v_{aj}^-, \quad S_j^+ = \sum_a n_a v_{aj}^+, \quad A_n = \{a : n_a > 0\}. \quad (15)$$

For each action-coordinate pair define the fixed constants

$$r_{aj} = \ell_{aj} - v_{aj}^- \geq 0, \quad s_{aj} = u_{aj} + v_{aj}^+ \quad (u_{aj} < \infty). \quad (16)$$

The extrema below are coordinatewise. The maximum of an empty set of nonnegative lower margins is zero; the minimum of an empty set of upper margins is infinity.

Theorem 6.1 (Universal-serialization envelope). *For arbitrary actions satisfying (9), the following are equivalent:*

- (i) Every word with multiplicity vector n is executable from $x \in \mathbb{N}^p$.
- (ii) For every coordinate j ,

$$x_j \geq S_j^- + \max_{a \in A_n} r_{aj}, \quad (17)$$

$$x_j \leq \min_{\substack{a \in A_n \\ u_{aj} < \infty}} s_{aj} - S_j^+. \quad (18)$$

When these conditions hold, every such word ends at

$$y = x - S^- + S^+. \quad (19)$$

No pairwise commutation assumption is needed for this theorem.

Proof. Fix a coordinate j and distinguish one occurrence of an active action a . Just before that occurrence, let δ be the sum of displacements contributed by the preceding occurrences in this coordinate. Its guard is

$$x_j \geq \ell_{aj} - \delta, \quad x_j \leq u_{aj} - \delta \quad (u_{aj} < \infty).$$

Among all permutations and all choices of this distinguished occurrence, the smallest possible preceding displacement is

$$\delta_{\min} = -S_j^- + v_{aj}^-.$$

Indeed, put all other negative-displacement occurrences before it and all positive-displacement occurrences after it; zero-displacement occurrences are immaterial. No subset of the other occurrences has smaller total displacement. The strongest lower requirement due to a is consequently

$$\ell_{aj} - \delta_{\min} = S_j^- + \ell_{aj} - v_{aj}^- = S_j^- + r_{aj}.$$

Similarly the largest possible preceding displacement is $\delta_{\max} = S_j^+ - v_{aj}^+$, obtained by putting all other positive occurrences first. The strongest finite upper requirement due to a is

$$u_{aj} - \delta_{\max} = s_{aj} - S_j^+.$$

Taking the maximum or minimum over active a proves that the displayed box is the intersection of every guard constraint at every position of every ordering.

For sufficiency, fix any ordering. Each of its prefix displacements lies between the two extremal values just used for its current action. The assumed inequalities therefore imply that every formal intermediate state satisfies its next guard. Induction on the word length then makes these formal states an actual execution in $\mathbb{N}^P$; condition (9) ensures the next output is natural.

For necessity, if all orderings execute, in particular the extremal ordering for each action and coordinate executes. Thus each of its extremal guard inequalities holds. Different coordinates may use different extremal permutations: this is legitimate because the hypothesis quantifies over *all* permutations. If $n = 0$, there are no occurrences, the empty word is always executable, and both formulas give the identity domain and endpoint. Summing displacements gives (19). $\square$

Corollary 6.2 (Exact acceleration under commutation). *If the actions in A_n commute pairwise as partial maps, then some ordering of the multiset is executable from x if and only if the envelope conditions hold. Thus some ordering, every ordering, and the explicit resource box are equivalent.*

Proof. Any two multiset words are connected by adjacent transpositions. Swapping identical letters changes nothing; swapping distinct active letters replaces two equal composite partial maps. Pre- and post-composition preserve that equality. All complete words therefore denote the same partial map, with the same domain. Apply Theorem 6.1. $\square$

Corollary 6.3 (Petri resource formula). *For Petri transitions, every ordering of multiplicities n executes from x exactly when, for every j ,*

$$x_j \geq \sum_a n_a (c_{aj} - d_{aj})_+ + \max_{a \in A_n} \min(c_{aj}, d_{aj}). \quad (20)$$

Under pairwise commutation of the active transitions, the same condition characterizes existence of an ordering. The endpoint is $x + \sum_a n_a (d_a - c_a)$.

Proof. Here $v^- = (c - d)_+$ and $c - (c - d)_+ = \min(c, d)$; there are no upper guards. Substitute into Theorem 6.1 and Corollary 6.2. $\square$

The two terms in (20) have different meanings. The sum measures total irreversible loss over the block. The maximum measures the largest resource that must remain available while a transition fires. Shared read-only requirements are maximized, not summed.

Example 6.4 (Why existential and universal feasibility differ). Take $a : x \mapsto x + 1$ with lower guard zero and $b : x \mapsto x - 1$ with lower guard one. From zero, ab executes and ba does not. The envelope for one occurrence of each requires $x \geq 1$. This is the correct universal condition and an incorrect existential condition without commutation. The two partial maps fail the box test.

Example 6.5 (A repeated read versus repeated loss). For a read $x \mapsto x$ guarded by $x \geq r$, any positive number of repetitions requires only $x \geq r$. For a unit loss guarded by $x \geq 1$, n repetitions require $x \geq n$. A test that reserves r fresh tokens for each repeated read overestimates requirements; a test that only checks individual enabling underestimates repeated loss.

7 The exact resource-summary algebra [06]

7.1 Every word is a principal partial translation

For $U, V \in \mathbb{N}^P$, let $[U, V]$ denote the partial map

$$U + r \mapsto V + r \quad (r \in \mathbb{N}^P). \quad (21)$$

Its domain is $U + \mathbb{N}^P$. We compose such maps in execution order: the left factor executes first.

Lemma 7.1 (Faithful summaries). *Each word w has a unique pair $(U_w, V_w) \in \mathbb{N}^P \times \mathbb{N}^P$ such that*

$$m \xrightarrow{w} n \iff \exists r \in \mathbb{N}^P (m = U_w + r \wedge n = V_w + r). \quad (22)$$

For fixed m, n , the slack r , if it exists, is unique.

Proof. The empty word has summary $(0, 0)$ and a letter a has summary (u_a, v_a) . Closure under concatenation is proved in Lemma 7.2, so these base cases give existence by induction on length. For uniqueness, the domain of $[U, V]$ has the unique least element U , and the value at that element is V . Thus equality of the partial maps forces equality of both coordinates. Finally, $m = U_w + r$ determines r uniquely. $\square$

An independent closed description makes the enabling information explicit. Set $\Delta_i = v_{a_i} - u_{a_i} \in \mathbb{Z}^P$. For each coordinate j ,

$$(U_w)_j = \max \left(0, \max_{1 \leq i \leq L} \left((u_{a_i})_j - \sum_{h < i} (\Delta_h)_j \right) \right), \quad V_w = U_w + \sum_{i=1}^L \Delta_i. \quad (23)$$

The inner maximum is absent when $L = 0$. Indeed, the inequalities saying that all steps are enabled are exactly the inequalities whose largest right-hand side occurs in (23). Nonnegativity of V_w follows from executability at U_w . In particular, every finite word is executable from *some* sufficiently large marking, although not necessarily from the marking of interest.

7.2 Composition by cancellation

Lemma 7.2 (Resource product). *For $x = (u, v)$ and $y = (u', v')$, set $c = \min(v, u')$. Then*

$$x \star y = (u + u' - c, v + v' - c) \quad (24)$$

is the summary of executing x followed by y .

Proof. It suffices to work in one coordinate. Write $v = c + s$ and $u' = c + t$, where $s, t \geq 0$ and at least one of s, t is zero. Starting at $u + r$, the first partial map produces $v + r$. The second is enabled precisely when $c + s + r \geq c + t$. Since $st = 0$, this is equivalent to $r \geq t$: if $t = 0$ it is automatic, and if $t > 0$ then $s = 0$. Writing $r = t + r'$ therefore gives the composite action

$$u + t + r' \mapsto v' + s + r'.$$

Here $u + t = u + u' - c$ and $v' + s = v + v' - c$, which proves the formula. $\square$

Corollary 7.3. *The operation $\star$ is associative with identity $(0, 0)$. For p counters its algebra is the direct product of p copies of the one-counter partial-translation algebra.*

Proof. Composition of partial maps is associative. The summary map is faithful by Lemma 7.1, so the corresponding pairs are equal. The identity and coordinatewise product assertions follow directly from the formula. $\square$

In one coordinate this is a concrete realization of the bicyclic monoid: first decrement u times, then increment v times. An increment followed by a decrement cancels on all inputs, whereas a decrement followed by an increment remains undefined at zero. We use the explicit pairs rather than an abstract presentation because they display exactly which natural-number witnesses a compiler needs. The broader connection between bicyclic storage, partially blind counters, and graph monoids is classical [23].

Lemma 7.4 (Unique minimum gadget). *For fixed $x, y \in \mathbb{N}$, the equations*

$$c + s = x, \quad c + t = y, \quad st = 0 \quad (25)$$

have the unique solution $c = \min(x, y)$, $s = x - c$, $t = y - c$ in $\mathbb{N}^3$.

Proof. The displayed values satisfy the equations. Conversely, $c \leq x, y$. Since $st = 0$, either $s = 0$ and $c = x \leq y$, or $t = 0$ and $c = y \leq x$. These cases agree when $x = y$ and determine all three variables. $\square$

For composition we apply (25) to $x = v$, $y = u'$ and set the new coordinates to $u + t$ and $v' + s$. Every defining residual has degree at most two, and every auxiliary coordinate is forced. This is stronger than merely asserting that a minimum is Diophantine.

7.3 Exact powers, including zero repetitions

Theorem 7.5 (Power formula). *Let $x = (u, v)$, and put $c = \min(u, v)$, $\alpha = u - c$, $\beta = v - c$ coordinatewise. Then*

$$x^{\star k} = \begin{cases} (0, 0), & k = 0, \\ (c + k\alpha, c + k\beta), & k \geq 1. \end{cases} \quad (26)$$

In every coordinate $\alpha\beta = 0$.

Proof. In one coordinate, either $u \geq v$, so $\beta = 0$, or $v \geq u$, so $\alpha = 0$. For $k = 1$ the formula recovers (u, v) . Suppose $k \geq 1$. If $\beta = 0$, the output after k copies is c and its cancellation against the next input $c + \alpha$ is c . The composite is $(c + (k + 1)\alpha, c)$. If $\alpha = 0$, the input stays c and the output becomes $c + (k + 1)\beta$. This proves the positive-power formula by induction. The zero-power formula is the identity of the monoid. $\square$

Example 7.6 (A catalyst cannot be retained at the zeroth power). The command decrement-then-increment on one counter has summary $(1, 1)$. Every positive power still requires one token, but its zeroth power is the empty word and is enabled at zero. Substituting $k = 0$ into the positive-power line of (26) would incorrectly retain the requirement $(1, 1)$.

A two-way zero test can itself have unique polynomial witnesses. Let $e, \ell \in \mathbb{N}$ satisfy

$$e(e - 1) = 0, \quad k = e + \ell, \quad (1 - e)\ell = 0. \quad (27)$$

If $e = 0$ these equations force $k = \ell = 0$; if $e = 1$ they force $k \geq 1$ and $\ell = k - 1$. Thus $e = \mathbf{1}_{k > 0}$ uniquely. The power coordinates are then

$$U' = ec + k\alpha, \quad V' = ec + k\beta. \quad (28)$$

Together with (25), all residuals remain quadratic. Notice that the child summary and its minimum witnesses are still constrained when $k = 0$. Disabling their equations would create free witnesses and destroy single-foldness.

7.4 A sharp global commutation test

Disjoint resource footprints provide one sufficient commutation relation, but are not necessary. Exact summaries give the largest relation appropriate for exchanging adjacent labels on *all* markings.

Theorem 7.7 (Global adjacent-swap criterion; the counter form of Theorem 4.1). *For two transition labels a, b , the following conditions are equivalent:*

- (i) *The partial actions of ab and ba are equal, including their domains.*
- (ii) *Their minimum input vectors are equal.*
- (iii) *Coordinatewise,*

$$\min(u_b, v_a) = \min(u_a, v_b). \quad (29)$$

Consequently, the relation I_ consisting of the distinct pairs satisfying (29) is the largest symmetric, irreflexive label relation for which every permitted adjacent swap preserves executability and result at every starting marking.*

Proof. By (24), the minimum inputs of ab and ba are respectively

$$u_a + u_b - \min(v_a, u_b), \quad u_a + u_b - \min(v_b, u_a).$$

Equality is exactly (29). Both words have the same total increment $v_a - u_a + v_b - u_b$, so equality of minimum inputs also implies equality of minimum outputs. Faithfulness proves the equivalence. The last assertion follows by applying the equivalence to each pair separately. $\square$

Two pure consumers of the same resource satisfy the criterion, as do two pure producers. Pure increment and pure decrement of one counter do not. The criterion is about equality of the *two-step partial maps*; it is not the usual stronger assertion that two individually enabled transitions can always both execute. Two consumers may compete for a marking that enables each separately but not their composition. This distinction is important when relating the construction to other notions of concurrency.

Fix henceforth any symmetric, irreflexive $I \subseteq I_*$. Define $\equiv_I$ as the congruence on A^* generated by $ab \equiv_I ba$ for $(a, b) \in I$. Every class is finite, since swaps preserve length and letter multiplicities. Equality of partial actions is preserved under composition, so

$$w \equiv_I w' \implies \text{res}(w) = \text{res}(w'). \quad (30)$$

The converse need not hold: distinct traces can have the same partial action. Our later counting theorem counts these trace classes, not all semantic partial maps.

8 A first-failure product for resource legality [07]

8.1 Formal histories may continue after failure

Fix p resources, an initial vector $r_0 \in \mathbb{N}^p$, and a candidate sequence of T events. Event i consumes $c_i \in \mathbb{N}^p$ and produces $a_i \in \mathbb{N}^p$. Define its *formal history* over $\mathbb{Z}^p$ by

$$r_{i+1} = r_i - c_i + a_i \quad (0 \leq i < T),$$

regardless of whether event i is enabled. The sequence is legal when

$$r_{ij} \geq c_{ij} \quad \text{for every } i < T \text{ and } j < p.$$

Let $(x)_{\underline{d}} = \prod_{h=0}^{d-1} (x - h)$, with $(x)_{\underline{0}} = 1$.

Theorem 8.1 (First-failure product). *The integer*

$$G = \prod_{i=0}^{T-1} \prod_{j=0}^{p-1} (r_{ij})_{\underline{c_{ij}}}$$

is positive for a legal event sequence and is exactly zero for an illegal one. Hence legality is equivalent to

$$\exists u \in \mathbb{N} \quad G = u + 1,$$

and the witness u is unique whenever it exists.

Proof. If the sequence is legal, every factor in a nonempty falling factorial satisfies $r_{ij} - h \geq c_{ij} - (c_{ij} - 1) = 1$. Empty products equal one, so $G \geq 1$.

Suppose instead that the sequence is illegal and choose its first disabled event i . Every earlier event is legal. Induction on the earlier events gives $r_i \in \mathbb{N}^p$: subtracting an enabled consumption and adding a nonnegative production cannot create a negative resource. There is a coordinate j with

$$0 \leq r_{ij} < c_{ij}.$$

The falling factorial for this coordinate includes the factor with $h = r_{ij}$, which is zero. Thus the entire product is zero. Formal resource values at later events can be negative, but they cannot undo this zero factor.

The final assertion follows because the positive integers are exactly the numbers $u + 1$ with $u \in \mathbb{N}$. Such a u must equal $G - 1$. $\square$

8.2 Why this is not a generic positivity trick

Replacing an arbitrary collection of strict inequalities by positivity of their product is unsound: two negative factors have positive product. The theorem avoids this problem for a structural reason. The first disabled state is still nonnegative, and the falling factorial vanishes at every forbidden consumption level $0, \dots, c_{ij} - 1$. A negative factor arising later is always preceded by a zero factor.

For a queue deleting one symbol per event, put $p = 1$ and $c_i = 1$. If event i appends $q_i \geq 0$ symbols, then

$$\ell_i = n - i + \sum_{h < i} q_h, \quad G = \prod_{i < T} \ell_i.$$

Legality is exactly $G \geq 1$. For deletion of d symbols, use $\prod_{i < T} (\ell_i)_d$ instead.

Remark 8.2 (Polynomial scope). The theorem is stated for candidate *integer* consumptions and productions. It becomes a fixed polynomial formula directly when the falling-factorial orders are fixed, as in the queue constructions below. An unrestricted variable consumption would make the displayed product length variable; that notation alone would not be an ordinary polynomial. Finite choices of consumption can be treated by separate finite-table constructions, but are not silently included in the degree claims here.

8.3 Separating content from resources

The product sees only resource availability. It does not check which symbol is read, whether an instruction is selected correctly, or whether a produced word is the prescribed one. Its role is complementary: a content equation will identify the candidate symbols, and the resource product will guarantee that these symbols are available in the correct temporal order. This separation is the mechanism that permits us to eliminate intermediate queues.

Part II

Canonical trace classes

Source and scope. [write] Four manuscripts prove that the natural zeros of an explicit polynomial are in bijection with commutation classes of bounded executions. Manuscript 05 is the base: its Sections 4–8 (normal forms with linear Boolean memory, raw runs, the quartic and the convex quadratic certificates, costs and a ballot formula, and counting hardness) are printed first, together with its corollary for interval-guarded programs, moved here from its section on other substrates (Corollary 11.5). Manuscript 04’s Sections 3–5 (normal-form monitor, Petri quartic, counting and Parikh refinements), manuscript 06’s Sections 4 and 7 (transfer signatures for trace normality and the bounded trace-class bijection) and manuscript 01’s Sections 6–7 (Cartier–Foata layers and the causal-history quartic) follow.

The shared theorem and its second routes. [write] For exact length T the bijection is Theorem 11.1 (degree four) and Theorem 11.2 (degree two, convex, with a rational polytope as real zero set), from manuscript 05. It needs only a sound independence relation $I \subseteq I_{\max}$ and extends to interval guards and zero tests (Corollary 11.5); manuscript 04’s Theorem 15.2 needs independence by *disjoint supports* (60), and its variable count $(p + r)(2T + 1)$ is the count $(2T + 1)(d + m)$ of Theorem 11.1 in manuscript 04’s letters. Manuscript 06’s Theorem 18.1 (quartic, $q(2T + 1) + p(7T - 4)$ witnesses, resource summaries on a binary concatenation tree) is a third construction. Both are kept, with their proofs, as marked second routes. Manuscript 01’s Theorem 20.1 is a different theorem: it bounds the Cartier–Foata height H of a trace rather than its length, and its box-guarded actions carry upper guards.

Other duplicates. [write] Manuscript 04’s monitor (Theorem 14.2) is the flag recurrence of Proposition 9.4, and its forbidden-factor lemma, like manuscript 06’s Lemma 17.1, is Lemma 9.2; all are kept with their proofs. The exponential automaton lower bound is proved twice with different families: Theorem 9.5 uses $2k$ labels, and Theorem 14.3 uses $2k + 1$ letters and realizes the relation by disjoint supports in a 1-safe net. $\#P$ -completeness of class counting is proved for the maximal relation with unit weights (Theorem 13.1) and for empty independence (Theorem 16.2); these are different quotients and both are kept. The Parikh and additive-cost refinement is printed in manuscript 05’s generating-function form, Corollary 12.1; manuscripts 01 (Corollary 20.2), 04 (Corollary 16.1) and 06 (the corollary after Theorem 18.1) state it for their own certificates.

Setting and hypotheses. [write] Manuscript 05: a Petri net with d places and m distinctly labelled transitions, any symmetric irreflexive $I \subseteq I_{\max}$, optional interval guards. Manuscript 04: p places, r transitions, I restricted to disjoint supports. Manuscript 06: consume/produce transitions on p counters, q labels, I a relation of globally commuting labels, no upper guards. Manuscript 01: box-guarded translations with lower and upper guards, I a valid relation of commuting pairs, a fixed height bound H .

9 Canonical schedules with linear Boolean memory [05]

Fix any symmetric irreflexive $I \subseteq I_{\max}$. On words over $\mathcal{A}$, let $\equiv_I$ be the equivalence relation generated by adjacent swaps $ab \leftrightarrow ba$ for $(a, b) \in I$. Equivalent words have equal length and equal multiplicities of all labels. Every class of words of length T is finite and has a unique lexicographically least word, denoted its normal form.

Lemma 9.1 (Execution invariance). *If $w \equiv_I w'$, then w is enabled from M if and only if w' is, and their final markings agree whenever they are enabled.*

Proof. Apply Theorem 4.1 to each generating swap and compose the resulting partial-function equalities. $\square$

9.1 The forbidden-factor test

The next characterization is a classical trace-language fact [18]; we record a proof to make its use in the compiler explicit.

Lemma 9.2 (Lexicographic forbidden factors). *A word is not lexicographically least in its I -class if and only if it has a factor*

$$bua, \quad a < b, \quad (a, b) \in I, \quad (a, c) \in I \text{ for every letter } c \text{ of } u. \quad (31)$$

Proof. Such a factor allows the final a to move left across u and b , making the word smaller. Conversely, order the occurrences of a word by the transitive closure of $i < j$ whenever their labels do not commute. Equivalent words are precisely the labelled linear extensions of this occurrence order: adjacent incomparable swaps preserve it, and moving the first occurrence of a target linear extension to the front, then inducting, connects any two extensions.

If the given extension is not lexicographically least, consider its first nongreedy choice. A later occurrence with smaller label a was already available. It is incomparable with every occurrence skipped before reaching it, since a predecessor would contradict availability and a successor cannot occur earlier in the given extension. Hence all these skipped labels commute with a . The first skipped label is some $b > a$, giving (31). $\square$

Example 9.3 (Adjacent inversion checks miss normal-form violations). Let $a < b < c$, with aIb and bIc but not aIc . The word cab has no adjacent decreasing independent pair: ca is dependent and ab is increasing. Nevertheless

$$cab \equiv_I cba \equiv_I bca,$$

and bca is lexicographically smaller. Normalization must remember more than the immediately preceding label.

9.2 A forbidden-set recurrence

For each candidate next label a , keep one flag F_a . After a prefix w , set $F_a(w) = 1$ exactly when w has a suffix bu with

$$b > a, \quad aIb, \quad \text{every letter of } u \text{ independent of } a.$$

The empty suffix u is permitted. All flags initially vanish. Appending a is forbidden precisely when $F_a = 1$.

Proposition 9.4 (Exact flag update). *On appending a label b , the flags change by*

$$F'_a = \mathbf{1}_{aIb} (\mathbf{1}_{a < b} \vee F_a). \quad (32)$$

A word is a normal form if and only if no step appends a label whose old flag is one.

Proof. If a and b are dependent, any suffix whose letters must commute with a is broken, and b itself cannot start a new forbidden suffix. If aIb and $a < b$, the one-letter suffix b starts such a suffix regardless of the old flag. If aIb and $b < a$, the new letter can only prolong a previously existing suffix, not start one. These are exactly the three cases in (32). The last assertion is Lemma 9.2 applied at the right endpoint of every forbidden factor. $\square$

For arithmetic use, at step i let $e_{i,b}$ be one-hot selectors. Write

$$C_{i,a} = \sum_{b:aIb} e_{i,b}, \quad H_{i,a} = \sum_{b>a:aIb} e_{i,b}, \quad W_{i,a} = C_{i,a} - H_{i,a}. \quad (33)$$

On a natural one-hot assignment, C, H, W are Boolean, $H \leq C$, and H and W cannot both equal one. The recurrence is

$$F_{i+1,a} = H_{i,a} + W_{i,a}F_{i,a}. \quad (34)$$

It uses only m bits of state. All indices range over $i = 0, \dots, T - 1$ unless indicated otherwise.

9.3 Why an exponentially large automaton is avoidable but real

Theorem 9.5 (Exponential state lower bound). *For every $k \geq 1$ there is an independence relation on $2k$ ordered labels whose language of lexicographic normal forms requires at least 2^k nonrejecting states in any deterministic finite automaton. Thus any deterministic finite-state implementation requires at least k bits of memory in the worst case. The m flags of Proposition 9.4 are optimal in order of growth, up to a constant factor.*

Proof. Order the alphabet as

$$a_1 < b_1 < a_2 < b_2 < \dots < a_k < b_k.$$

Let $a_i I b_j$ exactly when $j \leq i$, impose symmetry, and allow no independence between two a 's or between two b 's. For $S \subseteq \{1, \dots, k\}$, let w_S consist of the letters b_j with $j \in S$, in decreasing index order. Each w_S is normal, since all b -letters are dependent.

After this prefix, $F_{a_i} = 1$ exactly when $i \in S$. Any selected b_j with $j > i$ is dependent with a_i and occurs before all smaller indices; it cannot leave a final flag. The letter b_i , if present, sets the flag. Every later b_j has $j < i$, commutes with a_i , and is smaller than a_i , so it preserves but does not set the flag. Therefore $w_S a_i$ is normal exactly when $i \notin S$.

For distinct S, S' , choose i in their symmetric difference. The continuation a_i is accepted after exactly one of $w_S, w_{S'}$. A deterministic automaton must assign distinct states to these 2^k prefixes. They are all accepted prefixes, so these are nonrejecting states. A device with b bits has at most 2^b states, giving $b \geq k$. $\square$

By Theorem 4.4, these difficult relations occur as exact semantic commutation graphs of unit-weight Petri nets. This lower bound is not a lower bound on the number of unbounded arithmetic variables: one integer can store many bits. It specifically explains why expanding the recognizer into one variable per automaton state would be the wrong compiler architecture.

10 A parsimonious quadratic encoding of raw executions [05]

Fix a net, natural endpoints $M, N \in \mathbb{N}^d$, and an exact horizon $T \in \mathbb{N}$. Introduce the variables

$$x_{i,p} \quad (0 \leq i \leq T), \quad r_{i,p} \quad (0 \leq i < T), \quad e_{i,a} \quad (0 \leq i < T),$$

where $1 \leq p \leq d$ and $1 \leq a \leq m$. They all range over $\mathbb{N}$. The equations are

$$x_{0,p} = M_p, \quad x_{T,p} = N_p, \quad (35)$$

$$\sum_{a=1}^m e_{i,a} = 1, \quad (36)$$

$$x_{i,p} = \sum_a c_{a,p} e_{i,a} + r_{i,p}, \quad (37)$$

$$x_{i+1,p} = r_{i,p} + \sum_a v_{a,p} e_{i,a}. \quad (38)$$

Let P_{raw} be the sum of squares of the left-minus-right residuals of these equations.

Theorem 10.1 (Exact raw-run fibres). *The natural zeros of P_{raw} are in explicit bijection with the enabled length- T transition words taking M to N . The polynomial has degree at most two. The displayed, unoptimized presentation has*

$$V_{\text{raw}} = (2T + 1)d + Tm,$$

$$R_{\text{raw}} = 2(T + 1)d + T$$

variables and squared residuals, respectively.

Proof. A natural solution of (36) has exactly one coordinate equal to one and all others zero. Let a_i be its selected label. Equations (37) and the nonnegativity of r give $x_i \geq c_{a_i}$, so the step is enabled. They also force

$$r_i = x_i - c_{a_i}, \quad x_{i+1} = x_i - c_{a_i} + v_{a_i}.$$

Together with (35), this decodes a valid execution.

Conversely, a valid word fixes the selectors. Starting from $x_0 = M$, it uniquely determines each marking x_i and then each remainder r_i . Enabledness guarantees that these are natural numbers. They satisfy every equation, and there are no other variables. This proves both existence and uniqueness of extension. All residuals are affine, so their squared sum has degree at most two. Counting the three variable arrays and the equations gives the formulas. $\square$

Lemma 10.2 (Uniform coordinate bounds). *Every natural solution of (35)–(38), and every nonnegative real solution of these same equations, satisfies*

$$0 \leq x_{i,p}, r_{i,p} \leq B_p := M_p + T \max_a v_{a,p}, \quad 0 \leq e_{i,a} \leq 1. \quad (39)$$

Proof. The selector sum bounds each nonnegative selector by one. From (37), $0 \leq r_{i,p} \leq x_{i,p}$, and from (38),

$$x_{i+1,p} \leq x_{i,p} + \max_a v_{a,p}.$$

Induction from M_p proves the sharper bound with i in place of T for x_i , and hence the stated common bound. $\square$

The raw-run encoding already avoids a frequent error: gating a separate transition equation by a selector while leaving inactive-branch auxiliaries free. There is only one enabling remainder per place and time, and it belongs to the selected step.

11 Canonical traces: compact quartic and convex quadratic forms [05]

11.1 A small quartic presentation

Add the flag variables $f_{i,a} \in \mathbb{N}$ for $0 \leq i \leq T$. Besides the raw-run equations, impose

$$f_{0,a} = 0, \quad (40)$$

$$f_{i+1,a} = H_{i,a} + W_{i,a}f_{i,a}, \quad (41)$$

$$\sum_a e_{i,a}f_{i,a} = 0. \quad (42)$$

The linear forms H, W are those of (33). Let P_4 be the sum of the squares of all displayed residuals.

Theorem 11.1 (Compact canonical certificate). *Natural zeros of P_4 are in bijection with the I -commutation classes of enabled length- T words from M to N . It has degree at most four, with*

$$V_4 = (2T + 1)(d + m), \quad (43)$$

$$R_4 = 2(T + 1)d + (T + 1)m + 2T. \quad (44)$$

At every zero all selectors and flags are Boolean; no separate equations $f(f - 1) = 0$ are required.

Proof. Decode a raw run using Theorem 10.1. Since each selector row is one-hot, the recurrence (41), starting from zero, forces f_i to be exactly the Boolean flags of Proposition 9.4. In (42), every summand is nonnegative and only the selected label can contribute. Thus the selected old flag vanishes at every step, so the word is normal.

Conversely, each normal enabled word uniquely determines its raw witness and all flag rows. It satisfies the rejection equations. By Lemma 9.1, every enabled commutation class has its normal representative among the executions with the same endpoints. Uniqueness of this representative and of all its auxiliaries proves the bijection. The extra flag array contributes $(T + 1)m$ variables; its initial and recurrence equations contribute $(T + 1)m$ residuals, and rejection contributes T . Recurrence and rejection are at most quadratic, so the squared sum is at most quartic. $\square$

11.2 Eliminating products without introducing multiplicity

We next replace each recurrence and rejection test by affine constraints. At each (i, a) introduce five variables

$$u_{i,a}, \quad s_{i,a}^1, \quad s_{i,a}^2, \quad s_{i,a}^3, \quad s_{i,a}^4 \in \mathbb{N}.$$

Replace (41)–(42) by

$$f_{i,a} = u_{i,a} + s_{i,a}^1, \quad (45)$$

$$W_{i,a} = u_{i,a} + s_{i,a}^2, \quad (46)$$

$$1 + u_{i,a} = f_{i,a} + W_{i,a} + s_{i,a}^3, \quad (47)$$

$$f_{i+1,a} = H_{i,a} + u_{i,a}, \quad (48)$$

$$1 = e_{i,a} + f_{i,a} + s_{i,a}^4. \quad (49)$$

Keep the raw-run equations and (40). Let $L_1, \dots, L_{R_2}$ be all their affine residuals, including (45)–(49), and define

$$P_2(z) = \sum_{j=1}^{R_2} L_j(z)^2. \quad (50)$$

This is an explicit polynomial specified without an expensive expansion into monomials. The implementation also supports expansion for small examples.

Theorem 11.2 (Canonical trace polytope theorem). *For every N, M, N, T and symmetric irreflexive $I \subseteq I_{\max}$ as above, the polynomial (50) has all the following properties.*

- (i) *It belongs to $\mathbb{Z}[z_1, \dots, z_{V_2}]$, has degree at most two, and is convex on $\mathbb{R}^{V_2}$.*
- (ii) *Its natural zero set is in explicitly computable bijection with the I -commutation classes of enabled length- T executions from M to N .*
- (iii) *Its nonnegative real zero set is a bounded rational polytope. Its lattice points are exactly the natural zeros in (ii).*
- (iv) *The unoptimized presentation has*

$$V_2 = (2T + 1)d + (7T + 1)m, \quad (51)$$

$$R_2 = 2(T + 1)d + (5T + 1)m + T. \quad (52)$$

- (v) *At every natural zero, each selector, flag, u , and s^j is at most one, and all markings and enabling remainders satisfy (39). Thus the polynomial has finitely many natural zeros, with an explicit coordinate bound.*

Proof. We separate arithmetic soundness, uniqueness, and real geometry.

Natural soundness. The raw constraints decode an enabled run. Each $W_{i,a}$ and $H_{i,a}$ is Boolean and they are disjoint. Start with the forced Boolean row $f_0 = 0$. If f_i is Boolean, Lemma 2.3 applied to (45)–(47) forces

$$u_{i,a} = f_{i,a} W_{i,a}.$$

Then (48) gives the correct next flag. It is Boolean because $u \leq W$ and $H + W = C \leq 1$. Induction proves the Boolean flag interpretation at every layer. Equation (49) is exactly $e_{i,a} + f_{i,a} \leq 1$. It rules out selecting a forbidden label. The decoded run is consequently normal.

Existence and uniqueness. Starting from a canonical enabled word, set the raw variables and flags as before. Each u and each slack must be

$$\begin{aligned} u &= fW, & s^1 &= f - u, & s^2 &= W - u, \\ s^3 &= 1 + u - f - W, & s^4 &= 1 - e - f. \end{aligned}$$

All these values are zero or one. Nonnegativity of the last expression is precisely the no-forbidden-selection property. They satisfy every affine equation. Conversely, the equations and the preceding induction force exactly these values. Thus the extension is unique, and the class bijection follows as in Theorem 11.1.

Real geometry. Write the affine system as $Az = b$ with integer A, b . Scalarization gives

$$\{z \in \mathbb{R}_{\geq 0}^{V_2} : P_2(z) = 0\} = C := \{z \in \mathbb{R}_{\geq 0}^{V_2} : Az = b\}.$$

This is a rational polyhedron, possibly empty. The selectors lie in simplices, so $0 \leq H, W$ and $H + W = C \leq 1$. Inductively assume $0 \leq f_i \leq 1$. Equations (45)–(47) give $0 \leq u \leq f_i, W$. Hence

$$0 \leq f_{i+1} = H + u \leq H + W = C \leq 1.$$

The first two slacks are at most one. The third is $1 + u - f_i - W \leq 1$ because $u \leq f_i$ and $W \geq 0$; the fourth is $1 - e - f_i \leq 1$. All are nonnegative by the variable domains. The raw variables obey Lemma 10.2. Every coordinate of C is therefore bounded. A bounded rational polyhedron is a rational polytope, with the empty case allowed.

Finally, $P_2 = \|Az - b\|_2^2$ has Hessian $2A^\top A$, which is positive semidefinite. The counts are the raw counts, plus $(T + 1)m$ flag variables and $5Tm$ local auxiliaries; the added residuals are $m + 5Tm$. This gives (51) and (52) and completes all assertions. $\square$

Remark 11.3 (What is fixed and what is a parameter). The net, the alphabet order, the horizon, and the relation I are fixed when this polynomial is formed. The endpoints can alternatively remain free polynomial parameters; they enter only affinely, so the joint degree is still at most two. Treating arbitrary arc weights as additional variables would create products with selectors and would change this degree accounting. Treating T as a variable in one fixed polynomial is a different problem altogether.

The number of variables and equations is linear in $T(d + m)$. In a sparse residual representation, the coefficient occurrences require at most order $T(dm + |I| + d + m)$ space, up to the endpoint terms and bit lengths. This is not a claim that a fully expanded square has the same sparsity: squaring a dense affine residual introduces cross terms. Nor are the displayed constants claimed optimal; obvious deterministic or redundant arrays can be eliminated in particular instances.

11.3 The real relaxation is not automatically integral

Example 11.4 (A nonempty polytope with no lattice points). Take two places, two transitions, and

$$c_a = (2, 0), \quad c_b = (0, 2), \quad v_a = v_b = (0, 0), \quad M = (1, 1), \quad N = (0, 0), \quad T = 1.$$

The two pure consumers commute by Theorem 4.1. Neither is enabled at M , so there are no integer solutions. Nevertheless set

$$e_{0,a} = e_{0,b} = \frac{1}{2}, \quad x_0 = (1, 1), \quad x_1 = r_0 = (0, 0).$$

These satisfy the raw affine constraints. In the order $a < b$, take $f_0 = 0$, $u = 0$, and $f_{1,a} = 1/2$, $f_{1,b} = 0$. For each label set

$$s^1 = 0, \quad s^2 = W, \quad s^3 = 1 - W, \quad s^4 = 1 - e.$$

All are nonnegative and satisfy (45)–(49). Thus C is nonempty but $C \cap \mathbb{Z}^{V_2}$ is empty.

Convexity of P_2 is not an algorithm for its natural zeros. Linear programming can find the fractional point in this example and still decide nothing affirmative about a genuine execution.

11.4 A concrete scheduling example

Let a produce one token, let b be a no-op, and let c consume one token, on one place. In the order $a < b < c$, the maximal commutation relation has precisely the pairs aIb and bIc and their reverses. The word cab is enabled from marking one but not canonical, as in Example 9.3. Its normal representative is bca ; its marking trace is

$$1 \xrightarrow{b} 1 \xrightarrow{c} 0 \xrightarrow{a} 1.$$

After the first b , the flag for a is one. The subsequent c , which is dependent with a , clears it before the final a is selected. The construction records that flag history uniquely; it does not merely impose a global sorting of all labels.

For this net, horizon six and endpoints $M = N = 2$, there are 140 enabled words and 28 commutation classes. The quadratic certificate has 142 variables and 113 residuals; the quartic one has 52 variables and 47 residuals. These counts are reproduced by the included program, not inferred from a floating-point solver.

11.5 Canonical quadratic certificates for guarded programs

[write] *This corollary and the paragraph after it are moved from manuscript 05's section on other substrates (Part VIII), where they follow Theorem 4.5 (printed in Part I, Section 4.3); the bound $B_p = M_p + T \max_a v_{a,p}$ is that of (240) there.*

Corollary 11.5 (Canonical quadratic certificates for guarded programs). *Fix an interval-guarded translation system as in Theorem 4.5, numerical endpoints, and horizon T . Let g be the number of finite upper-guard entries among all transition coordinates. Its executions modulo the maximal commutation relation in Theorem 4.5 have a parsimonious convex quadratic certificate with*

$$V_2 + Tg \text{ variables, } R_2 + Tg \text{ residuals,}$$

where V_2, R_2 are given by (51)–(52). Its nonnegative real zero set is bounded.

Proof. Use the underlying Petri updates $c_a = l_a$ and $v_a = l_a + \delta_a$, which are nonnegative by assumption. Add, for every finite $u_{a,p}$ and each time step, a unique natural slack $g_{i,a,p}$ with

$$x_{i,p} + g_{i,a,p} = u_{a,p} e_{i,a} + B_p(1 - e_{i,a}), \quad (53)$$

where $B_p = M_p + T \max_a v_{a,p}$. At a natural one-hot selector, this enforces the selected upper guard; each inactive guard enforces only the already established bound $x_{i,p} \leq B_p$. Hence each enabled guarded word has exactly one raw extension, including all upper-guard slacks, and no other words have extensions.

Use the sound relation from Theorem 4.5 in the same flag constraints. The proof of the class bijection needs only partial-map commutation, now supplied by that theorem, so it applies without change. In the real relaxation the new right-hand side is between B_p and $u_{a,p}$, and the added slack is bounded by their maximum. The old coordinate bounds remain valid. There is one new variable and one affine residual per finite guard and time step. $\square$

The distinction between the two maximal relations matters. A pair can commute vacuously because both guarded orders have empty domains, even if the underlying unguarded Petri transitions do not commute. It is the *guarded* execution set that is then invariant. The reference compiler implements these optional upper guards and recomputes their exact commutation relation; it does not silently reuse the unguarded relation.

12 Costs, exact counts, and a closed-form example [05]

12.1 Which observations survive the quotient?

For label costs $\kappa_a \in \mathbb{N}^q$, define

$$K(w) = \sum_{i=0}^{T-1} \kappa_{a_i}.$$

This quantity is constant on commutation classes because adjacent swaps preserve the multiplicity of each label. It can be retained by adding the affine equations

$$y_j = \sum_{i,a} \kappa_{a,j} e_{i,a} \quad (1 \leq j \leq q). \quad (54)$$

The new output variables y_j are unique. Prescribing a target value of y , or imposing $y_j + s_j = B_j$ for a budget, retains unique extension. These additions do not increase the polynomial degree.

Corollary 12.1 (Generating functions are preserved). *Let $\mathcal{R}_T(M, N)/I$ be the finite set of commutation classes represented by P_2 . Then, with commuting indeterminates $t_1, \dots, t_q$,*

$$\sum_{[w] \in \mathcal{R}_T(M, N)/I} t^{K(w)} = \sum_{z \in \text{Sol}_{\mathbb{N}}(P_2)} t^{\sum_{i,a} \kappa_a e_{i,a}(z)}.$$

Here $t^v = \prod_j t_j^{v_j}$. In particular, the number of natural zeros is the number of classes, and taking one cost coordinate per label retains the entire Parikh-vector generating polynomial.

Proof. Apply the bijection of Theorem 11.2; each matched pair has the same displayed cost. Both sums are finite by the explicit bounds. $\square$

Neither state-dependent costs nor maximum intermediate resource usage are automatically preserved. For the one-place producer a and no-op b , the orders ab and ba commute, yet a cost charged to b equal to the current marking has different values. More generally, any observation depending on the prefix state must be included in the partial-map semantics before commutation is certified. A label-additive cost is a sufficient condition, not a universal description of observable equivalence.

A stochastic interpretation also requires care. Counting one representative per class is not the same as summing the probability of all schedules in that class. The latter requires a separate class-weight calculation, potentially including the number and probabilities of its linear extensions.

12.2 A ballot formula for the producer-idle-consumer net

The small example from Section 11 admits an exact family of counts. This gives an independent combinatorial check on both the semantics and the enumerator.

Theorem 12.2 (Class and schedule counts for one resource). *For the one-place net with producer a , idle transition b , and consumer c , fix $M, N, T \in \mathbb{N}$. Let*

$$\mathcal{L} = \{\ell : 0 \leq \ell \leq T, \quad r_\ell = (\ell + N - M)/2 \in \{0, 1, \dots, \ell\}\}.$$

Use $\binom{n}{k} = 0$ when k is an integer outside $0 \leq k \leq n$. Set

$$A_\ell = \binom{\ell}{r_\ell} - \binom{\ell}{r_\ell + M + 1}.$$

Then the number of maximal-commutation classes and the number of raw schedules are respectively

$$\#(\mathcal{R}_T(M, N)/I_{\max}) = \sum_{\ell \in \mathcal{L}} A_\ell, \quad \#\mathcal{R}_T(M, N) = \sum_{\ell \in \mathcal{L}} \binom{T}{\ell} A_\ell. \quad (55)$$

The class Parikh generating polynomial is

$$\sum_{\ell \in \mathcal{L}} A_\ell U^{r_\ell} V^{\ell - r_\ell} B^{T - \ell},$$

where U, V, B mark producer, consumer, and idle occurrences.

Proof. Deleting all idle letters leaves a word over a, c , and adjacent allowed swaps change neither this reduced word nor the number of idle letters. Conversely, all placements of a fixed number of idle letters around the same reduced word are equivalent, since b commutes with both other labels. Thus a class is exactly an enabled reduced word of some length ℓ , together with $T - \ell$ idle letters.

A reduced word ending at N has r_ℓ up-steps and $\ell - r_\ell$ down-steps. Of the $\binom{\ell}{r_\ell}$ unrestricted words, the ones that go below zero are counted by $\binom{\ell}{r_\ell + M + 1}$. To see this, reflect the part of the path from its starting height M through its first visit to -1 . This maps it bijectively to an unrestricted path from $-M - 2$ to N of length ℓ . Such a path has $r_\ell + M + 1$ up-steps; reflecting through its first crossing of -1 reverses the construction. Hence A_ℓ is exactly the enabled reduced-word count.

There are $\binom{T}{\ell}$ choices of positions for a fixed reduced word in a raw schedule. Idle steps cannot alter enabledness. Summing proves both counts and recording the three multiplicities proves the generating polynomial. $\square$

For $M = N = 2, T = 6$, the nonzero terms A_ℓ for $\ell = 0, 2, 4, 6$ are 1, 2, 6, 19. Thus the class count is 28, whereas the raw count is

$$1 + 15 \cdot 2 + 15 \cdot 6 + 19 = 140.$$

The formula also explains why a canonical class count cannot generally be obtained by dividing the raw count by one uniform symmetry factor: the factors $\binom{T}{\ell}$ differ among classes.

13 Counting hardness survives maximal commutation [05]

We now let the net and endpoints vary as finite input data. Arc weights and markings are encoded in binary; the exact horizon T is encoded in unary. The latter convention is essential for a polynomial-size unrolling. The complexity class $\#P$ counts accepting branches of polynomial-time nondeterministic computations; Valiant's work is a foundational source for this counting framework [22].

Theorem 13.1 (Parsimonious counting completeness). *Counting maximal-static-commutation classes of enabled length- T Petri-net executions from M to N is $\#P$ -complete under parsimonious polynomial-time reductions. Hardness holds even for nets with only unit arc weights that are 1-safe from the specified initial marking. Consequently, counting natural zeros of the quadratic certificates produced by Theorem 11.2 is $\#P$ -complete on this promised family of certificates.*

Proof. Membership. Compute $I_{\max}$ by Theorem 4.1. Guess a word of length T using a fixed-width binary encoding of each label, rejecting unused binary codes. Every word has exactly one guessing branch. Simulate it, check the endpoints, and run the flag recognizer. Exactly the canonical enabled words accept. Since T is unary, the word length is polynomial in the input size. By Lemma 10.2, intermediate marking bit lengths are polynomial as well. The arithmetic and flag checks run in polynomial time. Hence each commutation class contributes exactly one accepting branch.

Hardness construction. Start with a Boolean circuit on n input wires, with g gates of fan-in at most two and designated output wire o . Gates are topologically ordered. Write $w = n + g$. We construct a net that guesses the input, evaluates every gate deterministically, tests that the output is one, and finally erases the stored values. The erasure is important: all accepting inputs must lead to the same final marking.

For each wire j , create two value places V_j^0, V_j^1 , initially empty. Create control places $Q_0, \dots, Q_{2w+1}$ with a single token initially at Q_0 . Every transition moves the control token from one stage to the next. The data operations at the stages are as follows.

1. At input stage $j < n$, two distinct transitions write a token to V_j^0 or V_j^1 .
2. At gate stage j , include one transition for each assignment to its distinct predecessor wires. There are at most four. The transition reads the corresponding predecessor value places catalytically: a read place has both a unit input and a unit output arc. It writes one token to the correct value place of the new output wire. For repeated predecessors, consolidate duplicate reads rather than consuming two tokens from one place.
3. At stage w , the only transition requires a token at V_o^1 , retains it, and advances the control. No transition accepts an output zero.
4. At the next w stages, two possible transitions for each wire erase its zero or one token. Exactly one is enabled. At the end all value places are empty.

The common final marking has its only token at Q_{2w+1} , and the horizon is

$$T = 2w + 1.$$

Every reachable marking is 1-safe: each wire is written once, read without change, and then erased once; only one control token exists. All arcs have weight zero or one. After the input choices, exactly one gate or erasure transition is enabled at each stage. Thus satisfying input assignments are in bijection with accepting runs of this common length and common endpoint. The construction is polynomial in the circuit size.

The quotient does not merge assignments. Distinct input assignments select different labelled input transitions. Their Parikh vectors are different, and all commutations preserve Parikh vectors. Thus no two accepting runs belonging to different assignments can become equivalent, even when the relation is the maximal one computed from the constructed net. There is already only one run for each assignment. Soundness of the relation also ensures that any word equivalent to an accepting run is an accepting run. Consequently each such class has exactly one run.

For completeness, counting satisfying assignments to circuits is itself a parsimonious universal #P problem: write a polynomial-time accepting predicate as a Boolean circuit on a fixed-length certificate, using deterministic padding to make the certificate representation unique. Its satisfying input strings are exactly the accepting certificates. Applying the construction above proves hardness. Finally, Theorem 11.2 preserves the count when passing to the quadratic certificate. $\square$

This theorem pinpoints the limitation of the geometric representation. A compact convex quadratic polynomial can encode a hard *integer counting* problem because its real zero polytope need not be integral. It does not imply hardness for solving the corresponding unconstrained real least-squares problem. The theorem also does not say that every family with a large commutation relation is hard; graph and net structure can still support useful parameterized algorithms.

14 A symbolic normal-form monitor for concurrent traces [04]

14.1 Words, independence, and the forbidden-factor criterion

Let $\mathcal{A} = \{0, \dots, r-1\}$ be a totally ordered alphabet and let $I \subseteq \mathcal{A}^2$ be symmetric and irreflexive. Letters in I are called independent. Write $u \equiv_I v$ if u and v are related by finitely many swaps of adjacent independent letters. This is a length-preserving congruence; each class is finite and therefore has a unique lexicographically least word, denoted $\text{NF}_I(u)$.

Lemma 14.1 (Classical normal-form criterion). *A word is lexicographically least in its I -class if and only if it has no factor bva such that $a < b$ and a is independent of b and of every letter of v .*

Proof. Such a factor is an immediate obstruction: swap its last a successively left through v and b . The resulting equivalent word is lexicographically smaller.

For the converse, distinguish repeated occurrences by their order among occurrences of the same letter. Equivalent words have the same occurrences, and the relative order of every pair of dependent occurrences is unchanged by an allowed swap. Suppose u' is an equivalent, smaller word. Delete the common prefix, matching the distinguished occurrences there. Let b be the next letter of u , and $a < b$ the next letter of u' . The occurrence of a used by u' occurs later in u , so u has a factor bva at that position. Every occurrence in bv is still unmatched. If one were dependent on a , its order before a would persist in u' , contradicting that a is the next unmatched occurrence of u' . Thus a is independent of every letter of bv , giving the forbidden factor. $\square$

This proof uses only the invariance of dependent-occurrence order; it does not require a construction of a full automaton. In particular, checking only adjacent inversions would be inadequate. The middle word v can contain letters that prevent a local sorting rule from exposing the obstruction immediately.

14.2 Flags with a direct arithmetic update

For a prefix u , define $f_a(u) \in \{0, 1\}$ to record whether there exists an occurrence of some $b > a$ in u such that b is independent of a and every letter after that occurrence in u is independent of a . Initially all flags are zero. On appending c , the update is

$$f_a(uc) = \begin{cases} 0, & (c, a) \notin I, \\ 1, & (c, a) \in I \text{ and } c > a, \\ f_a(u), & (c, a) \in I \text{ and } c < a. \end{cases} \quad (56)$$

The equality case belongs to the first line because I is irreflexive. An appended letter c creates a forbidden factor precisely when $f_c(u) = 1$.

Encode the letter at time t by one-hot variables $x_{t,c}$. The case distinction becomes the quadratic equation

$$f_{t+1,a} = f_{t,a} \sum_{\substack{c < a \\ (c,a) \in I}} x_{t,c} + \sum_{\substack{c > a \\ (c,a) \in I}} x_{t,c}. \quad (57)$$

Normality is imposed by the single equation per time step

$$\sum_{a \in \mathcal{A}} x_{t,a} f_{t,a} = 0. \quad (58)$$

Theorem 14.2 (Quadratic normal-form monitor; the flag recurrence of Proposition 9.4). *For a fixed word of length T , equations $f_{0,a} = 0$ and (57) force a unique flag tableau. Its entries lie in $\{0, 1\}$ without separate Boolean equations. Adding (58) at every time accepts exactly the lexicographically least words of their trace classes.*

Proof. Because each row of selectors is one-hot, exactly one case of (56) applies. Induction on t proves both uniqueness and Booleanity, and proves that the computed flags have the defining suffix-obstruction meaning. Equation (58) is exactly $f_{t,c} = 0$ for the selected c . Therefore all those equations hold exactly when no forbidden factor ever appears. Apply Lemma 14.1. $\square$

The monitor uses r bits of working memory, plus a rejection state in an ordinary automaton implementation. The displayed arithmetic description uses $O(T(r + |I|))$ terms, where $|I|$ counts ordered independent pairs. The number of equations is $O(Tr)$, even for dense I ; this must not be confused with an $O(Tr)$ bound on a fully expanded description for every graph.

14.3 Why a symbolic representation matters

Theorem 14.3 (Exponential-state lower-bound family; a variant of Theorem 9.5). *For every $k \geq 1$ there is an independence relation on $2k + 1$ ordered letters such that every deterministic finite automaton recognizing its lexicographic trace normal forms has at least 2^k states. Thus, in the worst case, a deterministic streaming recognizer needs $\Omega(r)$ bits of memory. This remains true for independence relations realized by disjoint supports in an always-enabled, 1-safe Petri net.*

Proof. Order the alphabet as

$$r_1 < \dots < r_k < a_1 < \dots < a_k < b.$$

Let the independent pairs, together with their reverses, be (b, a_i) for all i and (r_i, a_j) for $i \neq j$. There are no other independent pairs.

For $S \subseteq \{1, \dots, k\}$ let p_S be b followed by the r_i with $i \in S$, in increasing order. Each p_S is a normal form. After reading b , every flag f_{a_i} is one. Reading r_i clears f_{a_i} , while each other r_j preserves it. Consequently

$$p_S a_i \text{ is a normal form} \iff i \in S. \quad (59)$$

If a deterministic automaton reached the same state after distinct prefixes p_S and $p_{S'}$, choose i in their symmetric difference. Appending a_i would have to produce the same acceptance outcome, contradicting (59). Hence all 2^k prefixes require distinct states.

For the Petri-net realization, create one place for every unordered pair of distinct dependent letters. Put one token in every such place. Each transition has a consume-one/produce-one self-loop at precisely the places corresponding to pairs containing that transition. Every transition is always enabled and every marking remains unchanged and 1-safe. Two distinct transition supports overlap exactly when their letters are dependent. Thus disjoint support realizes the prescribed I . $\square$

The lower bound concerns deterministic recognition of normal forms. It is *not* a lower bound on all Diophantine encodings, nondeterministic certificates, or the number of variables in a polynomial. Moreover, the net in the proof uses structural resource independence; transitions with the same overall effect are not automatically identified.

15 Quartic certificates for Petri-net trace classes [04]

15.1 Operational semantics and trace closure

A Petri net here has $p \geq 1$ places and $r \geq 1$ distinctly labeled transitions. Let $U_{ia}, V_{ia} \in \mathbb{N}$ be the numbers of tokens consumed and produced by transition a at place i . At marking $m \in \mathbb{N}^p$, transition a is enabled if $m_i \geq U_{ia}$ for every i ; its successor is

$$m'_i = m_i - U_{ia} + V_{ia}.$$

Its support is $\text{supp}(a) = \{i : U_{ia} + V_{ia} > 0\}$. Fix a symmetric irreflexive I satisfying

$$(a, b) \in I \implies \text{supp}(a) \cap \text{supp}(b) = \emptyset. \quad (60)$$

One may use all disjoint-support pairs, or any symmetric subset.

Lemma 15.1 (Enabled-swap property). *If a firing word contains adjacent independent transitions ab , swapping them gives another enabled firing word with the same final marking. Hence the set of firing words of length T from s to z is a union of complete I -classes.*

Proof. Suppose a fires at m and b fires after a . Since a changes no place in the support of b , all enabling inequalities for b already hold at m . After b , the enabling inequalities for a still hold. The effects add coordinatewise and have disjoint supports, so both orders reach $m - U_a + V_a - U_b + V_b$. The rest of the firing word is unchanged. Iterating this argument along any finite sequence of allowed swaps proves closure. $\square$

This quotient is by *the specified transition commutations*. It does not identify arbitrary executions with equal endpoints, equal Parikh vectors, indistinguishable token identities, or isomorphic occurrence processes. These are different semantic quotients.

15.2 The complete residual system

Fix a horizon $T \in \mathbb{N}$ and endpoint markings $s, z \in \mathbb{N}^p$. Introduce the following natural variables:

$$\begin{aligned} m_{t,i} & \quad (0 \leq t \leq T, 1 \leq i \leq p), \\ x_{t,a} & \quad (0 \leq t < T, 0 \leq a < r), \\ e_{t,i} & \quad (0 \leq t < T, 1 \leq i \leq p), \\ f_{t,a} & \quad (0 \leq t \leq T, 0 \leq a < r). \end{aligned}$$

The m variables are markings, x selects a transition, e is the enabling slack, and f is the monitor. The system consists of

$$m_{0,i} = s_i, \quad m_{T,i} = z_i, \quad (61)$$

$$x_{t,a}(x_{t,a} - 1) = 0, \quad \sum_a x_{t,a} = 1, \quad (62)$$

$$m_{t,i} = \sum_a U_{ia}x_{t,a} + e_{t,i}, \quad (63)$$

$$m_{t+1,i} + \sum_a U_{ia}x_{t,a} = m_{t,i} + \sum_a V_{ia}x_{t,a}, \quad (64)$$

$$f_{0,a} = 0, \quad (65)$$

$$f_{t+1,a} = f_{t,a} \sum_{\substack{c < a \\ (c,a) \in I}} x_{t,c} + \sum_{\substack{c > a \\ (c,a) \in I}} x_{t,c}, \quad (66)$$

$$\sum_a x_{t,a} f_{t,a} = 0. \quad (67)$$

Unspecified indices range over all values for which their variables are defined. Every residual has degree at most two. Let $P_{N,I,T,s,z}$ be the sum of their squares.

Theorem 15.2 (Witness-faithful concurrent compilation; manuscript 04, second route to Theorem 11.1 under disjoint supports). *The natural zeros of $P_{N,I,T,s,z}$ are in canonical bijection with the I -classes of enabled length- T firing words from s to z . The polynomial has degree at most four. The displayed construction uses exactly*

$$V_{\text{Petri}} = (p + r)(2T + 1), \quad (68)$$

$$E_{\text{Petri}} = 2p + r + T(2p + 2r + 2) \quad (69)$$

variables and quadratic-or-linear residuals, respectively. Every solution coordinate is at most

$$B = \max \left\{ 1, \max_i s_i + T \max_{i,a} V_{ia} \right\}. \quad (70)$$

If all markings reachable from s are 1-safe, every solution coordinate is Boolean.

Proof. From a class to a solution. By Lemma 15.1, the unique least word $w = a_0 \cdots a_{T-1}$ in the class is still an enabled run from s to z . Let m_t be its successive markings, set $x_{t,a} = 1$ exactly when $a = a_t$, and put $e_{t,i} = m_{t,i} - U_{i,a_t}$. Enabling makes every $e_{t,i}$ natural. Use the unique monitor tableau of Theorem 14.2. All equations hold, including the normality tests.

From a solution to a class. Because the domain is $\mathbb{N}$, the equations in (62) select exactly one transition a_t at each step. Equation (63) proves its enabling inequalities. Equation (64) is precisely its successor marking. The endpoint equations therefore produce a length- T run from s to z . The flag equations and Theorem 14.2 force this word to be least in its class.

Uniqueness. For a fixed word, the initial marking and update equations force all markings; the enabling equations force all slacks; and the monitor equations force all flags. The selectors are the word itself. Thus there is exactly one solution per normal word, and hence per class.

Counts. The four variable families contribute $(T+1)p$, Tr , Tp , and $(T+1)r$, giving (68). Endpoints contribute $2p$ residuals and initial flags contribute r . Each step contributes r Boolean equations, one selector sum, p enabling equations, p updates, r monitor updates, and one normality equation, giving (69).

Height. At each step, any place increases by at most $\max_{i,a} V_{ia}$. Hence every marking coordinate satisfies (70). Each enabling slack is between zero and the corresponding marking; all other coordinates are bits. If every reachable marking is 1-safe, this also makes every slack a bit. The sum-of-squares conversion proves the degree statement. $\square$

For $T = 0$, the same formulas apply: there is exactly one solution if $s = z$ and none otherwise. Some equations then refer to the same marking variables. They are still counted as displayed residuals. Counts throughout this paper are construction counts, not minimality claims.

15.3 Why token-balance equations are insufficient

An incidence equation

$$z = s + (V - U)v$$

records only a transition-count vector $v \in \mathbb{N}^r$. It does not certify enabling or an order in which transitions can fire. For example, take two places and transitions moving a token in opposite directions. From the zero marking, neither transition is enabled, but $v = (1, 1)$ satisfies the zero-to-zero incidence equation. Equations (63) rule out this spurious object. The normal-form monitor then removes a different kind of redundancy: multiple valid orders of independent transitions.

Example 15.3 (A fork and a join). Use five places and three transitions $a < b < c$:

	Consumed vector	Produced vector
a	$(1, 0, 0, 0, 0)$	$(0, 1, 0, 0, 0)$
b	$(0, 0, 1, 0, 0)$	$(0, 0, 0, 1, 0)$
c	$(0, 1, 0, 1, 0)$	$(0, 0, 0, 0, 1)$.

From $s = (1, 0, 1, 0, 0)$ to $z = (0, 0, 0, 0, 1)$ in three steps, the enabled words are abc and bac . Only a and b are independent, so there is one trace class, represented by abc . The certificate has 56 variables and 67 residuals, with a unique Boolean solution. The accompanying file `data/04-witness-faithful-petri-fork-join.json` contains the complete residuals and that solution.

16 Counting, weights, and the limits of low degree [04]

16.1 Exact counting and Parikh refinements

Write $C_{T,s,z}$ for the class set in Theorem 15.2. Then

$$\#C_{T,s,z} = \#\{\mathbf{y} \in \mathbb{N}^{V_{\text{Petri}}} : P_{N,I,T,s,z}(\mathbf{y}) = 0\}. \quad (71)$$

Both sides are finite, and the height bound supplies a finite search box. The count is at most r^T . This is a parsimonious representation: no correction factor for permutations, unused branches, or arithmetic coding choices is needed.

Corollary 16.1 (Parikh and additive-cost refinement; compare Corollary 12.1). *Adjoin variables n_a and equations $n_a = \sum_{t < T} x_{t,a}$. The extension remains witness-faithful, and n_a is the number of occurrences of transition a in the represented class. For fixed natural transition costs c_a , a further equation $C = \sum_a c_a n_a$ gives a unique cost coordinate. Thus coefficient counts in*

$$G_{T,s,z}(\mathbf{u}) = \sum_{[w] \in C_{T,s,z}} \prod_a u_a^{\text{Par}_a(w)}$$

are exact solution counts of quartic certificates with the corresponding linear constraints.

Proof. The new variables are uniquely determined by the selectors. Independent swaps preserve each transition count and hence every additive cost. Imposing prescribed values filters precisely the desired classes. The new residuals are linear, so degree at most four is retained after summing squares. $\square$

This does not imply that equal Parikh vectors determine equal trace classes. The polynomial G can have coefficients larger than one for exactly that reason.

16.2 A complexity consequence for succinct nets

The following consequence explains why the small degree should not be interpreted as an efficient generic solution method. It is a standard counting-reduction phenomenon instantiated by our explicit compiler, rather than a new general complexity classification of traces.

Theorem 16.2 (Counting hardness for the bounded-net family; empty independence, compare Theorem 13.1). *Counting the classes represented in Theorem 15.2, with the net and a unary horizon as input, is in #P. It is #P-hard even for 1-safe nets in which all transitions share a self-loop place, so that disjoint-support independence is empty. Consequently the corresponding structured family of quartic zero-counting problems is #P-complete, even on this Boolean-witness subclass.*

Proof. For membership, nondeterministically choose a length- T word using a fixed binary encoding of each letter, rejecting unused encodings. There is exactly one choice string per word. Simulate the net, check the endpoint, and run the monitor. Marking bit lengths are bounded by the input bit length plus $O(\log(T + 1))$ using (70). With T unary, these operations take polynomial time. Exactly one word from each class accepts.

For hardness, start with a Boolean circuit over fan-in-two AND and unary NOT gates. Counting its satisfying assignments is the defining standard circuit form of #P. Let it have n input bits and g gates in topological order. Build a net with one control token moving through successive stages and a dual-rail pair of places for each input and gate value. At an input stage, two transitions choose the bit and put a token on its zero or one rail. At each gate stage, transitions for its possible input valuations read the relevant rails by self-loops, put one token on the fresh correct output rail, and advance control. If an input wire occurs twice, its rail is read only once in the enabling pattern; this avoids an unintended demand for two tokens. Exactly one gate transition is enabled for each assigned input valuation.

An additional stage advances only when the designated circuit output is true. Finally, erase the $n + g$ stored bits in a fixed order, with a zero and a one transition at each stage. Precisely one erasure transition is enabled. The target is the final control token with all value rails empty. Add a common consume-one/produce-one lock place to every transition. It makes disjoint-support independence empty without changing the executions.

All stages are visited once, all value rails receive at most one token, and the net is 1-safe. Every satisfying assignment determines exactly one accepting word, including its deterministic gate evaluations and erasures. No other accepting words exist. Every such word has

$$T = (n + g) + 1 + (n + g) = 2(n + g) + 1$$

steps and the same target marking. The construction is polynomial in circuit size, and because I is empty, words are classes. Theorem 15.2 then gives the same count as natural quartic zeros, all of whose coordinates are Boolean. $\square$

The theorem concerns a succinct operational representation, not an explicitly enumerated reachability automaton. This matters when comparing it with algorithms whose input size is the number of automaton states. It also shows that exact uniqueness of the auxiliary variables removes encoding redundancy, not the intrinsic combinatorial difficulty of choosing a computation.

17 A compact signature for lexicographic trace normality [06]

Let $<$ be the fixed total order on A . A word is *normal* when it is the lexicographically least word in its I -trace class. Equal-length words are being compared, so no convention about comparing a proper prefix with a longer word affects the definition. We first recall the classical local obstruction to normality and then derive a small block signature.

17.1 The forbidden-factor criterion

Lemma 17.1 (Lexicographic obstruction). *A word w is nonnormal if and only if it has a factor bua such that $a < b$ and every letter in bu is independent of a .*

Proof. If such a factor occurs, the last a can be commuted left across all of bu . This replaces bua by abu , producing a lexicographically smaller equivalent word.

For the converse, distinguish occurrences even when their labels agree. On the occurrences of w , place an order relation from position i to position j whenever $i < j$ and their labels are not independent, and take transitive closure. This is a partial order. The words in the trace class are exactly the labelings of its linear extensions: an adjacent independent swap preserves all these precedence constraints, and any two linear extensions can be connected by swapping adjacent incomparable occurrences. The latter claim follows by moving the first occurrence of a target linear extension leftward, then applying induction to the remaining occurrences. Every occurrence passed in this move must be incomparable.

The smallest labeling of a linear extension is obtained greedily by choosing the least label among the available minimal occurrences. Equal-labeled occurrences are dependent and retain their order, so ties do not introduce a different labeled word. If w is not the smallest labeling, let its first greedy failure be at a position labeled b . An available occurrence with a smaller label a appears later in the displayed word. Availability means that none of the intervening remaining occurrences is a predecessor of that a ; in particular, each intervening label, including b , is independent of a . They form a factor bua as asserted. $\square$

The normal form and the underlying sorting interpretation are classical [19]. We include the proof because the exact obstruction, not just regularity of the normal-form language, determines the compiler.

17.2 One bit of memory per possible smaller letter

For each $a \in A$, maintain a bit F_a . After reading a prefix, the meaning of $F_a = 1$ is that the prefix contains an occurrence of some $b > a$ such that this occurrence and every subsequent letter are independent of a . Thus reading a next would complete a forbidden factor. Initially $F_a = 0$.

When the next letter is b , reject if $F_b = 1$. The memory update is

$$F'_a = d_{ab}F_a + h_{ab}(1 - F_a), \quad d_{ab} = \mathbf{1}_{(a,b) \in I}, \quad h_{ab} = \mathbf{1}_{(a,b) \in I \wedge b > a}. \quad (72)$$

If a is dependent on b , every old candidate is blocked and the new bit is zero. If $b > a$ and a is independent of b , the letter b itself is a new candidate and the bit becomes one. In the remaining independent case the bit is unchanged. These alternatives prove the memory invariant by induction. Together with Lemma 17.1, they prove that this deterministic recognizer accepts exactly the normal words.

It is convenient to continue applying the memory update even after a rejection. Acceptance and memory evolution are separate: a block may have a well-defined final memory even though some earlier position caused failure.

Definition 17.2 (Block signature). A signature is a tuple

$$S = (C, R, \alpha, \beta), \quad C \in \{0, 1\}, \quad R, \alpha, \beta \in \{0, 1\}^q, \quad \alpha_a \beta_a = 0.$$

It describes a block w when, for every incoming bit vector F ,

$$f_w(F)_a = \alpha_a F_a + \beta_a, \quad (73)$$

$$w \text{ causes no rejection from } F \iff C = 1 \text{ and } F_a R_a = 0 \text{ for all } a. \quad (74)$$

Here f_w is the memory transformation obtained by processing the whole block.

The disjointness condition means that each coordinate transformation is exactly one of the identity map, the constant-zero map, and the constant-one map. The bit C records acceptance from the all-zero incoming memory. The vector R records which incoming bits can trigger a rejection. When $C = 0$, the acceptance predicate is already false and different R vectors can describe the same predicate. We do not require a minimal semantic signature. We use the particular signature determined by the following recursive formulas; that choice is fully deterministic.

Lemma 17.3 (Empty and one-letter signatures). *The empty block has signature*

$$\mathbf{1}_S = (1, \mathbf{0}, \mathbf{1}, \mathbf{0}).$$

For the one-letter block b , a signature is

$$C = 1, \quad R_a = \mathbf{1}_{a=b}, \quad \alpha_a = \mathbf{1}_{(a,b) \in I \wedge b < a}, \quad \beta_a = \mathbf{1}_{(a,b) \in I \wedge b > a}. \quad (75)$$

Proof. The empty word does nothing and never rejects. For one letter b , rejection is precisely the incoming condition $F_b = 1$. Formula (72) is constant one in the independent $b > a$ case, identity in the independent $b < a$ case, and constant zero otherwise. Irreflexivity of I excludes a fourth case with $a = b$ independent of itself. $\square$

17.3 Composition without enumerating subsets

For bits x, y , write $x \vee y = x + y - xy$. All vector operations below are coordinatewise except for the displayed scalar product.

Theorem 17.4 (Signature product). *Suppose $S = (C, R, \alpha, \beta)$ describes x and $T = (D, S_0, \gamma, \delta)$ describes y . Then the following signature $S \odot T = (C', R', \alpha', \beta')$ describes xy :*

$$\begin{aligned} \alpha'_a &= \gamma_a \alpha_a, & \beta'_a &= \gamma_a \beta_a + \delta_a, \\ R'_a &= R_a \vee ((S_0)_a \alpha_a), & C' &= CD \prod_{a \in A} (1 - (S_0)_a \beta_a). \end{aligned} \quad (76)$$

All resulting coordinates are bits and $\alpha'_a \beta'_a = 0$.

Proof. Substitution in the final-memory formula gives

$$f_y(f_x(F))_a = \gamma_a(\alpha_a F_a + \beta_a) + \delta_a = (\gamma_a \alpha_a) F_a + (\gamma_a \beta_a + \delta_a).$$

The summands in the last constant term cannot both be one because $\gamma_a \delta_a = 0$. If $\gamma_a \alpha_a = 1$, then $\gamma_a = \alpha_a = 1$, forcing $\beta_a = \delta_a = 0$ and hence disjointness.

For acceptance, both blocks must accept. Their conditions are

$$C = D = 1, \quad F_a R_a = 0, \quad (\alpha_a F_a + \beta_a)(S_0)_a = 0 \quad (a \in A).$$

All terms in the last product are nonnegative. It vanishes precisely when $\beta_a(S_0)_a = 0$ and $F_a \alpha_a(S_0)_a = 0$. The conditions independent of F give C' , and the remaining conditions forbid an incoming one whenever R_a or $\alpha_a(S_0)_a$ is one. This is the stated R'_a . $\square$

These formulas can also be viewed as an associative product on all such signatures, not merely as a rule on signatures arising from words. To verify associativity, the memory coefficients associate by composition of affine maps. For three factors, the forbidden incoming bits are the coordinatewise union

$$R \vee (\alpha S_0) \vee (\alpha \gamma T_0),$$

independent of bracketing. The conditions on constant terms say, in either bracketing, that none of the constant-one contributions at a later rejection test is present. Distributivity of Boolean conjunction and disjunction makes the scalar expressions equal. The empty signature is a two-sided identity. Thus recursive multiplication agrees with reading letters one at a time.

There are at most $2 \cdot 6^q$ possible signatures: 2 choices for C , 2^q for R , and 3^q for the disjoint pairs (α_a, β_a) . A compiler need not list them. Each product requires only $O(q)$ Boolean gates. This is the useful distinction between a small *description of a transformation* and an explicit transition table on all 2^q incoming memory states.

18 A bijection between bounded trace classes and polynomial roots [06]

The compressed compiler fixes a schedule template and its parameters. For a stronger statement about concurrent *behaviors*, we now allow every word of a fixed length T . The witness tuple will include the chosen word itself, and normality will remove exactly the redundant commutation representatives.

18.1 One-hot word selection and deterministic memory

Fix $T \geq 1$. For each position $0 \leq i < T$ and label $a \in A$, introduce $z_{i,a} \in \mathbb{N}$ with

$$z_{i,a}(z_{i,a} - 1) = 0, \quad \sum_{a \in A} z_{i,a} = 1. \quad (77)$$

These variables select exactly one word w of length T . Its i th letter summary has coordinates

$$U_i = \sum_a u_a z_{i,a}, \quad V_i = \sum_a v_a z_{i,a}. \quad (78)$$

Use any full binary concatenation tree on these T leaves, with the resource minimum gadget at each internal node. Impose the root endpoint equations (112).

For canonicity, introduce memory variables $F_{a,i}$ for $a \in A$ and $0 \leq i \leq T$, beginning with $F_{a,0} = 0$. Define the following *linear expressions*, without additional variables:

$$D_{a,i} = \sum_{b:(a,b) \in I} z_{i,b}, \quad H_{a,i} = \sum_{b:(a,b) \in I, b > a} z_{i,b}.$$

Impose the quadratic residuals

$$F_{a,i+1} = D_{a,i}F_{a,i} + H_{a,i}(1 - F_{a,i}), \quad (79)$$

$$\sum_{a \in A} z_{i,a} F_{a,i} = 0 \quad (0 \leq i < T). \quad (80)$$

For a selected word, the recurrence forces the unique memory evolution of (72). Its values are bits by induction, so separate Boolean constraints on these variables are unnecessary. Equation (80) is exactly the rejection test for the selected letter. Even without using its one-hot interpretation, all its summands are nonnegative and therefore cannot cancel.

Let $P_T(m, n, z, \eta)$ be the sum of the squares of all residuals in this construction, where z denotes the word selectors and η all other witnesses.

Theorem 18.1 (Bounded trace-class root bijection; manuscript 06, second route to Theorem 11.2). *For fixed $m, n \in \mathbb{N}^p$ and $T \geq 1$, the natural solutions of $P_T = 0$ are in bijection with the I -trace classes of words w satisfying*

$$|w| = T, \quad m \xrightarrow{w} n.$$

The polynomial has degree at most four and exactly

$$q(2T + 1) + p(7T - 4) \quad (81)$$

witness variables in the allocation described above. All witnesses are at most $\max(1, WT, \|m\|_\infty)$.

Proof. A root determines a unique word by (77). The resource equations force its exact summaries and endpoint slack, and the memory equations force its exact recognition run. Therefore the selected word is executable with the required endpoints and is normal. Conversely, any such normal word gives one selector assignment and, by the uniqueness of the resource and memory computations, exactly one full root.

Every executable trace class has a unique normal word. Adjacent swaps in I preserve the entire partial action by (30); hence that normal word is executable from m to n , whether or not it was the original representative used to specify the class. Thus normal executable words are in bijection with precisely the stated trace classes.

For the variable count, selectors contribute qT . A full binary tree on T leaves has $2T - 1$ nodes, each with $2p$ summary coordinates, and $T - 1$ internal nodes, each with $3p$ cancellation witnesses. The endpoint contributes p slacks and memory contributes $q(T + 1)$. The total is

$$qT + 2p(2T - 1) + 3p(T - 1) + p + q(T + 1) = q(2T + 1) + p(7T - 4).$$

Every residual is quadratic or linear, proving the quartic bound. The resource height proof with subwords of length at most T , together with Boolean selectors and memory, gives the height assertion. $\square$

For $T = 0$, no word-choice or internal witnesses are needed. The polynomial $\sum_{j=1}^p (m_j - n_j)^2$ has the single empty witness tuple when $m = n$ and none otherwise, exactly matching the empty trace.

Corollary 18.2 (Prescribing the Parikh vector; compare Corollary 12.1). *Given $v \in \mathbb{N}^q$, add the linear equations*

$$v_a = \sum_{i=0}^{T-1} z_{i,a} \quad (a \in A).$$

The same construction, without additional witnesses, counts exactly the executable trace classes with that Parikh vector.

A Parikh vector is generally coarser than a trace: dependent labels retain order information. Conversely, two different traces may induce the same resource summary. The three quotients—letter counts, trace classes, and partial actions—should not be substituted for one another in a counting claim.

18.2 Why the length parameter is not free here

The polynomials P_T form a family. Their number of variables grows with T . There is no assertion that one fixed polynomial receives T as a free input and has exactly one root per trace for all T . Passing from the family to a single unbounded trace representation would require an additional coding construction. Ordinary Diophantineness of an encoded trace does not guarantee that the additional coding witnesses are unique or finite in number.

For each fixed T the root set is finite and has at most q^T members. The height bound gives a finite search box, although enumerating that entire box is not proposed as an efficient algorithm. The value of the theorem is its exact combinatorial meaning, not a speedup over direct word enumeration. The compressed compiler addresses a different efficiency question: avoiding expansion of one highly structured schedule.

19 Causal traces and canonical layers [01]

19.1 The quotient to be represented

Fix a symmetric irreflexive relation $I \subseteq A \times A$ consisting of commuting action pairs. Put $D = (A \times A) \setminus I$; in particular $(a, a) \in D$. Two words are trace-equivalent when one can be obtained from the other by swapping adjacent I -independent letters. Write $\text{Tr}(A, I)$ for this quotient monoid.

By equality of partial maps, equivalent words have the same execution domain and endpoint. Accordingly, a trace is executable from x when any, equivalently every, representative is executable. This equivalence is why we required equality of domains in the independence test.

A trace is not the same thing as an endpoint. Distinct labels with identical effects remain distinct labels, and dependent orders remain distinct traces even when their particular endpoints happen to coincide. The quotient removes precisely the swaps specified by I , no more.

19.2 A self-contained normal-form lemma

The underlying normal form is classical Cartier–Foata theory [16, 17]. We prove the exact version needed by the polynomial compiler to make the padding and predecessor conditions explicit.

For a word with individually distinguished occurrences, generate a partial order by putting occurrence i below occurrence j whenever $i < j$ in the word and their labels are D -dependent. Take the transitive closure. The resulting labelled dependence poset has an intrinsic order on repeated occurrences of each letter because self-dependence is included.

Lemma 19.1 (Posets and swaps). *Two words represent the same trace exactly when their labelled dependence posets agree, with the k th occurrence of each label matched to the k th occurrence. The representatives of a trace are exactly the label words of the linear extensions of its dependence poset.*

Proof. Swapping adjacent independent occurrences changes the order of no dependent pair and so preserves all generating comparisons and their transitive closure. Conversely, fix a dependence poset and two linear extensions. Locate the first occurrence in the second extension within the first extension. All occurrences before it in the first extension are incomparable with it: a predecessor would also have to precede it in the second extension, while a successor could not precede it in the first. Incomparable occurrences must have independent labels, since every dependent pair was made comparable. Move the selected occurrence left by adjacent allowed swaps. Remove this common first occurrence and repeat inductively. Thus any two linear extensions are connected by allowed swaps.

The original word is a linear extension. A linear extension orders every dependent pair in the prescribed direction, so reconstructs the same dependence poset. This proves both assertions. $\square$

Give each occurrence the level

$$h(e) = 1 + \max\{h(f) : f \text{ is an earlier occurrence with } (\lambda(f), \lambda(e)) \in D\}, \quad (82)$$

with the maximum of an empty set equal to zero. Let C_i be the set of labels of level i .

Theorem 19.2 (Cartier–Foata layer characterization). *Every trace has a unique sequence of nonempty layers $C_1, \dots, C_h$ satisfying:*

- (a) *each C_i is a clique for I , so distinct members commute and no label is repeated;*
- (b) *every $a \in C_{i+1}$ has a predecessor $b \in C_i$ with $(b, a) \in D$.*

Concatenating the layers, in any order within each layer, represents the trace. The number h is the length of a longest chain in the dependence poset.

Proof. The level of an occurrence is its longest-chain rank in the dependence poset. Indeed, every generating predecessor has a lower rank, and any longest chain can be refined to use generating comparisons between successive levels; the recurrence (82) computes exactly these ranks in a topological order. Thus the levels are intrinsic to the trace by Lemma 19.1.

Two dependent occurrences cannot have the same level, because the earlier is a generating predecessor of the later. Hence each layer has distinct pairwise independent labels. If an occurrence has level $i + 1$, the finite maximum in (82) is attained by a dependent earlier occurrence of level i . This is condition (b). Sorting occurrences by level respects the poset, so concatenating the layers yields a linear extension and hence the same trace.

Conversely, start from a sequence satisfying (a) and (b), and concatenate its layers in arbitrary internal orders. Induct on i . All letters of the first layer have level one, since no two of them are dependent. A letter in layer $i + 1$ is independent of its same-layer predecessors, has no predecessor from a later layer, and has a dependent predecessor in layer i . By induction the maximum predecessor level is exactly i , so its computed level is $i + 1$. Thus the proposed layers are precisely the intrinsic levels of the resulting trace. This also proves uniqueness. Finally, the maximum rank equals the longest-chain length by the rank recurrence. $\square$

19.3 Unique trailing padding

For a fixed bound H , append empty layers to a normal form until it has H layers. The local conditions above characterize these padded forms if empty layers are allowed: if $C_i = \emptyset$, condition (b) forces $C_{i+1} = \emptyset$, and hence all later layers are empty. No choice of a padding length needs to be encoded. The all-empty sequence represents the empty trace.

This small observation is important for fibre preservation. A separate “finished” flag with unconstrained transitions, or arbitrary idle letters, can create several certificates for the same finite history. Trailing empty layers with the predecessor condition do not.

20 An explicit quartic for causal histories [01]

20.1 Parameters and witnesses

Fix the action alphabet A , a valid independence relation I , and a natural causal-height bound H . Only $x, y \in \mathbb{N}^P$ are endpoint parameters. For $H \geq 1$ introduce Boolean-intended variables z_{ha} for $1 \leq h \leq H$ and $a \in A$; they indicate membership in the h th layer.

Let e be the number of unordered *distinct* dependent pairs and set

$$D(a) = \{b : (b, a) \in D\}, \quad d = \sum_a |D(a)| = m + 2e.$$

Fix an ordering $b_{a1}, \dots, b_{a,d_a}$ of each $D(a)$, where $d_a = |D(a)| \geq 1$.

The witnesses are the layer bits, internal states $M_h \in \mathbb{N}^P$ for $1 \leq h < H$, totals L_{hj}, G_{hj} , lower slacks α_{haj} , finite-upper slacks β_{haj} , and blocker products U_{hak} for $1 \leq h < H$, $1 \leq k \leq d_a$. Put $M_0 = x$, $M_H = y$, and use $U_{ha0} = 1$ as a constant, not a witness.

20.2 The complete residual catalogue

Use the following residuals, for all indices in their specified ranges:

$$z_{ha}(z_{ha} - 1), \tag{83}$$

$$z_{ha}z_{hb} \quad \text{for each unordered distinct dependent pair } \{a, b\}, \tag{84}$$

$$L_{hj} - \sum_a z_{ha}v_{aj}^-, \quad G_{hj} - \sum_a z_{ha}v_{aj}^+, \tag{85}$$

$$M_{hj} + L_{hj} - M_{h-1,j} - G_{hj}, \tag{86}$$

$$z_{ha}(M_{h-1,j} - L_{hj} - r_{aj} - \alpha_{haj}), \quad (1 - z_{ha})\alpha_{haj}, \tag{87}$$

$$z_{ha}(s_{aj} - G_{hj} - M_{h-1,j} - \beta_{haj}), \quad (1 - z_{ha})\beta_{haj} \quad (u_{aj} < \infty), \tag{88}$$

$$U_{hak} - U_{ha,k-1}(1 - z_{h,b_{ak}}) \quad (h < H), \tag{89}$$

$$z_{h+1,a}U_{ha,d_a} \quad (h < H). \tag{90}$$

Let $Q_{A,I,H}$ be the sum of their squares. All residuals have degree at most two, including the blocker recurrences; no high-degree conjunction is left unlifted.

Theorem 20.1 (Parsimonious causal-history compiler; bounded causal height, not length). *For fixed $A, I, H \geq 1$, the nonnegative roots in all witness coordinates of*

$$Q_{A,I,H}(x, y; w) = 0$$

are in an explicit bijection with the executable I -traces from x to y whose Cartier–Foata height is at most H . The polynomial has degree at most four, with

$$N_H = H(m(1 + p) + f + 2p) + (H - 1)(d + p), \tag{91}$$

$$E_H = Hm + He + 3Hp + 2H(pm + f) + (H - 1)(d + m) \tag{92}$$

witnesses and squared residuals, respectively. In particular, every such trace gives exactly one complete natural solution, including all auxiliary coordinates.

Proof. Extraction. A zero of Q makes all residuals vanish. Equation (83) makes each z_{ha} Boolean. The corresponding subsets $C_h = \{a : z_{ha} = 1\}$ are cliques by (84).

Induction on k in (89) gives

$$U_{hak} = \prod_{t=1}^k (1 - z_{h,b_{at}}) \in \{0, 1\}. \tag{93}$$

Therefore (90) says that an action present in layer $h + 1$ has at least one dependent action in layer h . Section 19 implies that these are the uniquely padded layers of a trace of height at most H .

The totals (85) are the losses and gains for the layer multiplicities z_h . The guard slacks (87)–(88) assert its envelope at M_{h-1} . Within a layer all actions are pairwise independent and hence commute, so Corollary 6.2 makes every internal ordering executable. Equation (86) sets the endpoint to M_h . Concatenating all layers yields an execution from x to y . Its trace is independent of the chosen internal order.

Construction. Take an executable trace of height at most H . Its unique padded normal form fixes z . Any representative has the same partial map, so the layer-concatenated representative is executable. Start at $M_0 = x$ and set the boundary states by the cumulative displacements. Set L, G by their aggregate formulas. Set each active slack to its exact nonnegative guard margin and every inactive slack to zero. Define U by the product (93). The resource theorem and normal-form characterization verify every residual.

Uniqueness. The trace fixes its padded normal form, hence all z . From x and z , the displacement recurrence fixes all states; the aggregate equations fix all totals. The complementary active/inactive equations fix every slack, and the blocker recurrences fix every U . Thus there is exactly one full root over that trace. Extraction and construction are inverse because they use these same formulas.

Counts. There are Hm bits, $(H - 1)p$ internal-state entries, $2Hp$ totals, Hpm lower slacks, Hf upper slacks, and $(H - 1)d$ blockers. Summing gives (91). The residual families contribute, in order, Hm , He , $2Hp$, Hp , $2Hpm$, $2Hf$, $(H - 1)d$, and $(H - 1)m$, which gives (92). Squaring residuals of degree at most two proves the degree bound. $\square$

For $H = 0$ there are no witnesses and the separate polynomial

$$Q_{A,I,0}(x, y) = \sum_{j=1}^p (y_j - x_j)^2$$

represents the single empty trace when $x = y$, and no trace otherwise.

20.3 Counting, costs, and finite fibres

Corollary 20.2 (Exact trace counts; compare Corollary 12.1). *For each fixed x, y, H , the number of natural roots of $Q_{A,I,H}$ is the number of executable trace classes between these endpoints with height at most H , and is at most 2^{Hm} . If prescribed letter counts n_a are desired, adjoining the squared linear residuals*

$$\sum_{h=1}^H z_{ha} - n_a$$

preserves degree at most four and selects precisely those traces with these counts. No new witness is required.

Proof. Use the bijection in Theorem 20.1. There are at most 2^{Hm} Boolean layer matrices, each with at most one completion. The additional residuals record exactly the number of occurrences of each action in that matrix. $\square$

The same construction records exact total length or any fixed additive integer action cost by one further linear equation. It does not encode a scheduling-dependent cost that varies between equivalent words; such a cost is not a function on the chosen quotient and would require a different semantic contract.

Proposition 20.3 (An explicit bound on all history witnesses). *Suppose $W \geq 1$ bounds every finite guard constant and every $|v_{aj}|$, and $X = \max_j x_j$. Every root of $Q_{A,I,H}$ has all witness coordinates at most*

$$B = X + (H + 1)mW + 2W + 1. \quad (94)$$

Proof. A layer contains at most m actions, so every executable boundary state is at most $X + HmW$ in each coordinate. Each loss or gain total is at most mW . An active lower slack is $M_{h-1,j} - L_{hj} - r_{aj}$, which is nonnegative and no greater than its state coordinate because $r_{aj} \geq 0$. An active upper slack is at most $s_{aj} \leq 2W$. Inactive slacks are zero; all bits and blockers are zero or one. These bounds are each no greater than (94). $\square$

Consequently the bounded-height construction permits complete finite enumeration in principle, with witness bit length logarithmic in B . This is a representation and size statement, not a claim that enumerating B^{NH} points is a practical solver.

20.4 A parameter that must not be hidden

The bound H determines the number of variables and equations. Thus $Q_{A,I,H}$ is an effectively generated family of ordinary polynomials, not one fixed-arity polynomial with H as an input variable. Taking the union over all H yields an effective reachability relation, and MRDP can represent that union existentially. Nothing in that observation preserves the preceding root bijections or finite-foldness.

Part III

Compressed repetition and accelerators

Source and scope. [write] This Part collects the fixed-arity constructions whose witness count does not grow with repetition counts. Manuscript 01's Sections 4–5 give the single-fold quartic accelerator for a multiset of box-guarded actions (Theorem 21.1), fixed block programs, exact powers of a fixed word (Proposition 22.2) and rank and progress criteria for endpoint-level single- and finite-foldness; its Section 8 works the examples (eleven witnesses for multiplicities 10^{100} , 10^{80} , 10^{90}). Manuscript 06's Sections 5–6 prove that nested normality only sees repetition counts 0, 1 and at least 2 (Theorems 24.1 and 24.2) and build the single-fold quartic compiler for parametrized nested repetition expressions (Theorem 25.1); its Section 8 locates the expressiveness boundary (flat schemes are Presburger-definable, separated depth-two schemes represent arbitrary quadratic systems).

Relation between the two. [write] Neither construction contains the other. Manuscript 06's nested expressions generalize manuscript 01's single-level word power to arbitrarily nested repetitions and add the normality test; manuscript 01's actions carry upper guards, and its accelerator characterizes *every* serialization of a multiset, and *some* serialization under commutation. The single-word case appears in both forms, Proposition 22.2 and Theorem 7.5 (Part I), and both are credited.

Setting and hypotheses. [write] Manuscript 01: a fixed finite alphabet of box-guarded translations on $\mathbb{N}^p$; multiplicities, endpoints and phase counts are parameters of the polynomial, not witnesses, unless a corollary says otherwise. Manuscript 06: consume/produce transitions on p counters with lower guards only, q labels, and a fixed repetition expression with N syntax nodes whose repetition parameters are inputs.

21 A fixed-arity single-fold quartic accelerator [01]

21.1 The complete polynomial system

Fix the action descriptions once and for all. Treat $x, y \in \mathbb{N}^p$ and $n \in \mathbb{N}^m$ as *parameters*. Let

$$f = \#\{(a, j) : u_{aj} < \infty\}.$$

Introduce natural witnesses z_a, k_a , two totals L_j, G_j , lower slacks α_{aj} for all a, j , and upper slacks β_{aj} exactly for the finite upper guards. Define the following residuals:

$$z_a(z_a - 1), \quad n_a - z_a(1 + k_a), \quad (1 - z_a)k_a; \quad (95)$$

$$L_j - \sum_a n_a v_{aj}^-, \quad G_j - \sum_a n_a v_{aj}^+, \quad y_j + L_j - x_j - G_j; \quad (96)$$

$$z_a(x_j - L_j - r_{aj} - \alpha_{aj}), \quad (1 - z_a)\alpha_{aj}; \quad (97)$$

$$z_a(s_{aj} - G_j - x_j - \beta_{aj}), \quad (1 - z_a)\beta_{aj} \quad (u_{aj} < \infty). \quad (98)$$

Here and throughout the residuals are *integer* expressions evaluated on natural variables. Subtraction is not truncated. Let P_A be the sum of the squares of all listed residuals.

Theorem 21.1 (Single-fold quartic accelerator). *For every fixed action alphabet A , P_A has total degree at most four and*

$$N_{\text{acc}} = 2m + 2p + pm + f \quad \text{witnesses}, \quad (99)$$

$$E_{\text{acc}} = 3m + 3p + 2pm + 2f \quad \text{squared residuals}. \quad (100)$$

For every fixed (x, y, n) , its natural zero set has exactly one point if every serialization of n executes from x to y , and is empty otherwise. If the active actions commute, “every” can be replaced by “some.” Neither arity nor degree depends on the magnitudes of the multiplicities.

Proof. A sum of integer squares vanishes exactly when every residual vanishes. The first residual in (95) forces $z_a \in \{0, 1\}$. If $n_a = 0$, the second forces $z_a = 0$, and the third then forces $k_a = 0$. If $n_a > 0$, the second forces $z_a = 1$ and $k_a = n_a - 1$, after which the third is automatic. Thus the support encoding has one and only one solution for every natural n_a .

Equations (96) uniquely set $L = S^-$ and $G = S^+$. If $z_a = 1$, the lower residual forces $\alpha_{aj} = x_j - S_j^- - r_{aj}$; the naturalness of this uniquely determined slack is exactly the relevant lower bound. If $z_a = 0$, the inactive equation uniquely sets $\alpha_{aj} = 0$, and no guard is imposed. The same argument for the finite upper guard gives $\beta_{aj} = s_{aj} - S_j^+ - x_j$ when active and zero otherwise.

The existence of all these natural slacks is therefore exactly the envelope box. The remaining endpoint residual is (19). Theorem 6.1 gives soundness and completeness; Corollary 6.2 gives the existential interpretation under commutation. Every witness was uniquely determined in the preceding argument, proving single-foldness.

There are $2m$ support witnesses, $2p$ totals, pm lower slacks, and f upper slacks. The three support residuals per action, three total/endpoint residuals per coordinate, and two residuals per slack give the stated counts. Every residual has degree at most two, including degrees in the parameter variables, so P_A has degree at most four. $\square$

21.2 Why the inactive equations are indispensable

The equation $z\alpha = 0$ by itself is not a canonical conditional constraint: when $z = 0$ it permits every natural α . In our construction the active residual imposes the inequality and the complementary residual

$(1 - z)\alpha = 0$ fixes unused storage to zero. A single omitted inactive equation can turn a singleton fibre into an infinite one.

Similarly, $n = z(1 + k)$ does not determine k when $n = z = 0$. The third residual in (95) is not an optimization detail; it is part of the uniqueness proof. These are small examples of a general rule: a semantic proof that ignores inactive branches need not be a fibre-preserving proof.

21.3 What is compressed and what is not

The construction compresses arbitrarily many repetitions into fixed-arity arithmetic. It does not promise small *numbers*: the witness $k_a = n_a - 1$ has essentially the same bit length as n_a , as it should. The totals are computed by linear forms in the multiplicities. No powers, factorials, prime coding, or CRT witnesses occur.

The sparse residual presentation has $O(pm + f)$ terms for a fixed action description, counting the terms in all aggregate sums. Expanding the sum of squares can produce $O(pm^2)$ terms from those sums, before cancellation. Thus a short constraint list, a short arithmetic circuit, and a small expanded monomial list are different size measures. The attachment exposes both representations for the worked examples, rather than silently equating them.

Remark 21.2 (Not a new universal variable bound). The arity in (99) is a bound for the specific envelope relation of a fixed finite alphabet. It is not a universal Diophantine-equation bound, and it does not improve ProveIt's independently reported universal certificate operation count [1].

22 Fixed block programs with unbounded repetition [01]

A single commuting block is not the only unbounded relation obtained without sequence coding. Fix an ordered list of $b \geq 1$ phases. In phase s , a fixed alphabet A_s of m_s actions is pairwise commuting. Multiplicities $n^{(s)}$ may be arbitrarily large. Only the number and order of the phases are fixed.

Theorem 22.1 (Single-fold composition with explicit counts). *The relation asserting that the successive phases with multiplicities $n^{(1)}, \dots, n^{(b)}$ carry x to y has a single-fold quartic representation. It has*

$$(b - 1)p + \sum_{s=1}^b (2m_s + 2p + pm_s + f_s) \quad (101)$$

witnesses, in addition to the endpoint and multiplicity parameters. Its arity is independent of every repetition count.

Proof. Introduce natural boundary states $X_1, \dots, X_{b-1}$, set $X_0 = x$ and $X_b = y$, and use the residual system of Section 21 for each phase between X_{s-1} and X_s , with disjoint local witnesses. The sum of all residual squares is still a quartic.

For fixed input and all phase counts, the endpoint of each phase is forced by its net displacement. Consequently every intermediate state is unique. Once those states are fixed, Theorem 21.1 gives unique local witnesses whenever the phase executes. Conversely, a zero provides a legal execution of each phase; their concatenation is a legal block program. Counting the boundary states and local witnesses gives (101). $\square$

The phase counts are semantic data in this theorem. If the same label can occur in adjacent phases, one untagged trace could have more than one decomposition into phase counts. The theorem does not claim to eliminate that ambiguity. One may retain phase tags as part of the semantic object, require counts as input, or establish a separate unique-decomposition result.

22.1 Fixed-word macros: noncommuting loop bodies

The same model also compresses repetition of a fixed word whose individual actions need not commute. This is a closure consequence, not a new universality claim.

Proposition 22.2 (Exact powers of a fixed word). *Let $w = a_1 \cdots a_t$ be fixed, put $\delta_i = \sum_{k=1}^i v_{a_k}$ with $\delta_0 = 0$, and set $\Delta = \delta_t$. Its partial map has domain given by*

$$\ell_{w,j} = \max(0, \max_{1 \leq i \leq t} (\ell_{a_i j} - \delta_{i-1,j})), \quad (102)$$

$$u_{w,j} = \min_{1 \leq i \leq t} (u_{a_i j} - \delta_{i-1,j}). \quad (103)$$

If this box is nonempty, it defines a valid guarded-translation macro with displacement Δ . For $n > 0$, the word w^n executes precisely when

$$x_j \geq \ell_{w,j} + (n-1)\Delta_j^-, \quad x_j \leq u_{w,j} - (n-1)\Delta_j^+ \quad (104)$$

for every coordinate, with infinite upper bounds omitted. Its endpoint is $y = x + n\Delta$. This relation, including $n = 0$, has a single-fold quartic representation with $2 + 3p + f_w$ witnesses, where f_w counts the finite macro upper bounds. The arity is independent of both n and the length of the already compiled fixed word.

Proof. Before letter a_i , the formal state is $x + \delta_{i-1}$. Pulling every guard back to x gives (102)–(103). The same induction on formal prefix states as in Theorem 6.1 proves that their intersection is exactly the execution domain. If nonempty, its lower corner itself lies in the domain. Its final state is natural, so $\ell_{w,j} + \Delta_j \geq 0$, proving the required validity condition for the macro.

Apply Theorem 6.1 to this one macro with multiplicity n . Its lower envelope is $n\Delta_j^- + \ell_{w,j} - \Delta_j^-$, and its upper envelope is $u_{w,j} + \Delta_j^+ - n\Delta_j^+$, giving (104). Theorem 21.1 with $m = 1$ supplies the stated witness count, includes the zero-repetition case, and proves unique completion.

If the macro domain is empty, only $n = 0$ can execute; that special case is instead represented without witnesses by $n^2 + \sum_j (y_j - x_j)^2$. For the empty word, use the identity macro with lower bound zero and no upper bound. $\square$

This extends the fixed-phase construction to expressions such as $w_1^{n_1} \cdots w_b^{n_b}$ for fixed words w_s , even if the letters within a loop body are highly dependent. The macro description is computed in time linear in the total word length times the state dimension; its guard and displacement constants can grow with that length. Thus constant witness arity does not mean that the source word has disappeared from the coefficient description. An empty-domain macro simply forces its repetition count to zero. Multiplicities remain specified parameters unless a separate argument such as the next two corollaries controls their fibres.

22.2 Endpoint-level single-foldness by displacement rigidity

There is a useful sufficient criterion for making the counts existential without creating extra roots. List all phase-labelled action occurrences as $q = \sum_s m_s$ action types. Let $V \in \mathbb{Z}^{p \times q}$ have their displacements as columns.

Corollary 22.3 (Rigid endpoint certificates). *If V has full column rank over $\mathbb{Q}$, then the endpoint relation*

$$R(x, y) \iff \text{some phase multiplicities carry } x \text{ to } y$$

has a single-fold quartic representation obtained from Theorem 22.1 by turning the q count parameters into witnesses.

Proof. Every root satisfies $y - x = Vn$. Full column rank makes n unique over $\mathbb{Q}$, hence at most one natural count vector can occur. For that vector, Theorem 22.1 makes every remaining witness unique. Existence is unchanged by treating the counts existentially. $\square$

Full column rank is sufficient, not necessary. It is deliberately easy to verify and identifies the source of uniqueness: the endpoint recovers the semantic counts, while the compiler recovers their auxiliary certificate.

22.3 Endpoint-level finite-foldness from progress

Corollary 22.4 (Progress-bounded fibres). *Suppose there is a vector $w \in \mathbb{Q}^P$ and $\varepsilon > 0$ such that $w \cdot v_i \geq \varepsilon$ for every phase-labelled action type. The same existential-count quartic is finite-fold at each fixed pair (x, y) .*

Proof. Every root obeys

$$w \cdot (y - x) = \sum_i n_i (w \cdot v_i) \geq \varepsilon \sum_i n_i.$$

If the left side is negative there are no roots. Otherwise all count vectors lie in the finite set $\{n \in \mathbb{N}^q : \sum_i n_i \leq w \cdot (y - x) / \varepsilon\}$. For each such vector the remaining fibre is empty or a singleton by Theorem 22.1. Hence the complete fibre is finite. $\square$

These corollaries are restricted positive results, not consequences of a general finite-fold theorem. Their hypotheses are explicit linear-algebraic or progress conditions. In particular, the number of phases is fixed in the polynomial, and arbitrary branching between an unbounded number of phases is outside their scope.

22.4 A directly linear description of each support case

For a fixed support set of each phase, all active-action tests in the envelope become finitely many linear inequalities in the endpoints, boundary states, and multiplicities. Inactive counts are zero and active counts are positive. Thus these block relations have a finite union of such integer linear descriptions, with boundary states eliminable by the displacement equations. The polynomial construction is valuable because it supplies an explicit unique completion and a uniform degree bound, not because it proves previously unavailable decidability for these restricted relations.

23 Worked examples [01]

23.1 Eleven witnesses for an enormous shared-resource block

Consider three one-place Petri actions:

$$a : (c, d) = (2, 1), \quad b : (c, d) = (3, 1), \quad r : (c, d) = (1, 1). \quad (105)$$

The first two decrease the marking by one and two respectively; the third reads the shared retained token. By Proposition 5.3, all three commute. For a, b , both final production thresholds equal one; the read threshold is at most both retained production thresholds.

Put $L = n_a + 2n_b$. If at least one multiplicity is positive, the exact condition is

$$x \geq L + 1, \quad y = x - L. \quad (106)$$

If all multiplicities vanish, the condition is instead $y = x$, including $x = 0$. The empty-support case cannot be silently replaced by the positive-support inequality.

In this example the eleven witnesses are

$$z_a, z_b, z_r, \quad k_a, k_b, k_r, \quad L, G, \quad \alpha_a, \alpha_b, \alpha_r.$$

For $i \in \{a, b, r\}$ the support residuals are

$$z_i(z_i - 1), \quad n_i - z_i(1 + k_i), \quad (1 - z_i)k_i.$$

The remaining nine residuals are

$$\begin{aligned} &L - n_a - 2n_b, \quad G, \quad y + L - x - G, \\ &z_i(x - L - 1 - \alpha_i), \quad (1 - z_i)\alpha_i \quad (i = a, b, r). \end{aligned}$$

Their eighteen squares define one quartic. The generic construction retains the forced-zero gain witness G for a uniform interface; it could be eliminated in this particular example, so eleven is not asserted to be minimal.

Set

$$n_a = 10^{100}, \quad n_b = 10^{80}, \quad n_r = 10^{90}, \quad x = 10^{100} + 2 \cdot 10^{80} + 1, \quad y = 1.$$

The unique witness is $z_i = 1$, $k_i = n_i - 1$, $L = n_a + 2n_b$, $G = 0$, and all three lower slacks zero. These formulas evaluate directly, without traversing the astronomical number of events. The attached expanded polynomial has total degree four and 76 nonzero monomials before specializing the parameters. Its JSON file includes all eighteen residuals, the full expanded coefficients, and the exact natural certificate.

23.2 Many schedules, one trace, one polynomial root

For the same three actions, take the normal form

$$C_1 = \{a, b, r\}, \quad C_2 = \{a, r\}, \quad C_3 = \{a\}.$$

It has counts $(3, 1, 2)$. Since all distinct labels are independent, all

$$\frac{6!}{3!1!2!} = 60$$

words with these counts are one trace. Its height is three, not six, because repeated copies of a label remain dependent.

From $x = 9$, the boundary markings are 9, 6, 5, 4, so $y = 4$. The compiler with $m = 3$, $p = 1$, $H = 3$, $f = 0$, $e = 0$, and $d = 3$ produces

$$N_H = 32, \quad E_H = 48.$$

There is exactly one root for this trace. There may be other roots for other traces with the same endpoints and height bound; adding the three count equations isolates this particular trace. This qualification is essential: endpoint equality alone is not a specification of the history being counted.

23.3 Upper guards and an identity block

Let a be the increment on $0 \leq x \leq 4$ and let r be a read on $0 \leq x \leq 5$. They commute: the domain of either two-action order is $0 \leq x \leq 4$. For multiplicities n_a, n_r , the upper envelope includes $x \leq 5 - n_a$ whenever $n_a > 0$, and the same inequality from the read whenever it is active; the lower envelope is $x \geq 0$. Thus with three increments and any positive number of reads, the allowed starts are exactly 0, 1, 2, and the endpoint is $x + 3$.

If no increment occurs but the read does occur, the bound is $x \leq 5$. If neither occurs, there is no upper bound at all. The support variables in the quartic distinguish all three cases without an arbitrary branch-choice witness.

23.4 A diagnostic for spurious concurrency

Let a, b be distinct labels, each consuming one token and producing none. At $x = 1$, the naive Boolean layer conditions “both selected actions are individually enabled” would accept $\{a, b\}$. The exact layer envelope has loss two and retained margin zero, so rejects it. At $x = 2$, both sequential orders execute and the layer is valid. This is precisely the operational interpretation of Remark 5.2; the compiler does not confuse a clique with a feasibility certificate.

24 Saturation: nested normality only sees zero, one, and two [06]

Theorem 24.1 (Square saturation). *Every signature $S = (C, R, \alpha, \beta)$ satisfies*

$$S^{\odot 2} = \left(C \prod_{a \in A} (1 - R_a \beta_a), R, \alpha, \beta \right), \quad S^{\odot 3} = S^{\odot 2}. \quad (107)$$

Consequently $S^{\odot k} = S^{\odot 2}$ for every $k \geq 2$.

Proof. Apply (76) with both factors equal to S . Since the entries are bits, $\alpha_a^2 = \alpha_a$ and $R_a \vee (R_a \alpha_a) = R_a$. Disjointness gives $\alpha_a \beta_a + \beta_a = \beta_a$. The scalar coordinate is the one shown in (107). Multiplication by S once more changes only the scalar coordinate and multiplies it by the same Boolean factor $C \prod_a (1 - R_a \beta_a)$. That factor is idempotent, so nothing changes. Induction proves the assertion for every larger power. $\square$

There are two separate stabilization phenomena. A block’s final-memory map is already idempotent after one application. A second copy can nevertheless expose a forbidden pattern straddling the boundary between the copies. Once that possibility has been checked, a third copy cannot expose a new kind of failure. The signature calculation records both facts, including rejecting blocks.

Define $\text{cap}_2(k) = \min(k, 2)$ and apply it componentwise to parameter tuples.

Theorem 24.2 (Cap-at-two theorem for nested schemes). *For every expression E from Definition 2.16 and every parameter tuple $\mathbf{k} \in \mathbb{N}^r$,*

$$\text{sig}(\llbracket E \rrbracket_{\mathbf{k}}) = \text{sig}(\llbracket E \rrbracket_{\text{cap}_2(\mathbf{k})}). \quad (108)$$

In particular, the two expanded words are either both normal or both nonnormal. Parameters may be reused, and repetitions may be nested to any fixed finite depth.

Proof. Proceed by structural induction. The empty and letter cases do not involve parameters. For concatenation, the induction hypotheses give equal child signatures, and the product formula gives equal parent signatures. At a repetition node F^{k_j} , the induction hypothesis first identifies the signature S of the body under the two parameter assignments. The two parent signatures are then $S^{\odot k_j}$ and $S^{\odot \text{cap}_2(k_j)}$, which agree by Theorem 24.1, including the identity case $k_j = 0$. Reuse of a parameter causes no difficulty because the same componentwise capping is applied everywhere. Finally, acceptance from initial memory zero is exactly the scalar condition $C = 1$. $\square$

Corollary 24.3 (Finite parameter cells for normality). *For a fixed expression with r distinct parameters, its normality predicate is a union of some of the 3^r cells*

$$X_1 \times \cdots \times X_r, \quad X_j \in \{\{0\}, \{1\}, \{2, 3, 4, \dots\}\}.$$

It can be evaluated from an $O(qN)$ -size circuit over the three-way tests on parameters, without constructing an expanded word or enumerating all cells.

Proof. The theorem shows constancy on each cell. Its inductive proof supplies the circuit: one signature product per concatenation, and at a repetition choose the identity, the child signature, or its square according to the three-way parameter test. Each signature product has $O(q)$ gates. $\square$

Example 24.4 (Why the threshold two is sharp). Take $a < b$ and $(a, b) \in I$. The word ab is normal, but $(ab)^2 = abab$ is not: its middle ba can be swapped. Thus capping all positive powers at one would be false, even for one unnested repetition node. The theorem does not claim that normality is unchanged when zero is identified with one, either.

Remark 24.5 (What saturation does not preserve). The words a^2 and a^{1000} need not have the same resources, output, length, Parikh vector, or trace class. The signature is a homomorphic image of the free word monoid, not of the trace monoid: the independent words ab and ba can have different normality signatures. By contrast, the resource-summary map factors through the trace relation. Cap-at-two is a theorem about the canonicity test only, not a compression of the actual resource action to bounded powers.

25 A single-fold quartic compiler [06]

We now compile the two summaries together. The construction is elementary: it uses ordinary polynomial equations, nonnegative unknowns, and no exponentiation predicate, bounded universal quantifier, or invocation of MRDP.

25.1 Quadratic gates with forced values

A Boolean wire b is constrained by $b(b-1) = 0$. With Boolean inputs, the following equations uniquely determine their Boolean outputs:

$$t = xy \quad (\text{AND}), \quad t = x + y - xy \quad (\text{OR}), \quad t = 1 - x \quad (\text{NOT}). \quad (109)$$

One may add the output bit equation explicitly, although its value already follows from the inputs. Long conjunctions are implemented by a chain or a balanced tree of binary AND gates. Expressions such as $R_a \vee ((S_0)_a \alpha_a)$ must use an intermediate wire for the inner product; substituting it directly into the OR equation would create a cubic residual. Likewise the product defining C' in (76) is not left as a single high-degree residual.

To share a repetition switch with both branches of the compiler, introduce $z_0, z_1, z_2, h \in \mathbb{N}$ and impose

$$\begin{aligned} z_i(z_i - 1) &= 0 \quad (0 \leq i \leq 2), & z_0 + z_1 + z_2 &= 1, \\ k &= z_1 + 2z_2 + h, & (1 - z_2)h &= 0. \end{aligned} \quad (110)$$

Exactly one selector is one. If it is z_0 , then $h = 0$ and $k = 0$. If it is z_1 , then $h = 0$ and $k = 1$. If it is z_2 , then $k = 2 + h \geq 2$. Conversely, each k chooses exactly one case and a unique h . Thus the selector gadget has no surplus solutions.

For resources the active bit is $e = z_1 + z_2$, used in (28). For a signature, compute the child's square and then select, coordinate by coordinate, among the identity, child, and square:

$$s' = z_0 s_{\text{id}} + z_1 s + z_2 s_{\text{square}}. \quad (111)$$

The three terms are at most quadratic, since each s is a wire rather than an expanded polynomial. Exactly one term is active. Both the child and its square remain fully constrained in inactive cases.

25.2 The polynomial and its correctness contract

At each syntax node allocate its $2p$ resource coordinates. For an empty node they are zero; for a letter node they are the transition weights. At a concatenation node impose the minimum gadget coordinatewise and the two output equations following Lemma 7.2. At a repetition node impose the child minimum, the selector gadget, and the power output equations. For the canonical version, also allocate the signature at each node and the intermediate Boolean wires described above.

At the root impose, with a new slack vector $r \in \mathbb{N}^p$,

$$m = U_E + r, \quad n = V_E + r. \quad (112)$$

In the canonical version also impose $C_E = 1$. List all integer residuals as $r_1, \dots, r_s$. Every residual has total degree at most two in all free inputs and witnesses. Define

$$Q_E(m, n, \mathbf{k}, z) = \sum_{i=1}^s r_i(m, n, \mathbf{k}, z)^2. \quad (113)$$

This is one ordinary polynomial in $\mathbb{Z}[m, n, \mathbf{k}, z]$ of degree at most four. Its sum-of-squares presentation is simply a convenient exact way to store it; it is not an extension of polynomial syntax.

Theorem 25.1 (Compressed single-fold compilation). *For every fixed expression E , the construction above effectively produces polynomials Q_E^{res} and Q_E^{can} of degree at most four. For every fixed $m, n \in \mathbb{N}^p$ and $\mathbf{k} \in \mathbb{N}^r$,*

$$\#\{z : Q_E^{\text{res}}(m, n, \mathbf{k}, z) = 0\} = \mathbf{1}_{m \xrightarrow{\llbracket E \rrbracket_{\mathbf{k}}} n}, \quad (114)$$

$$\#\{z : Q_E^{\text{can}}(m, n, \mathbf{k}, z) = 0\} = \mathbf{1}_{m \xrightarrow{\llbracket E \rrbracket_{\mathbf{k}}} n \wedge \llbracket E \rrbracket_{\mathbf{k}} \text{ normal}}. \quad (115)$$

The resource-only compiler uses $O(pN)$ witnesses and equations. The canonical compiler uses $O((p+q)N)$ witnesses and equations. These bounds are independent of the expanded execution length and of the numerical parameter values.

Proof. All residuals are integer-valued. Therefore $Q_E = 0$ is equivalent to every residual being zero: a sum of nonnegative integer squares vanishes only when each square vanishes.

We first show that all internal witnesses are uniquely determined by the parameters. At an empty or letter node the summary coordinates are fixed constants. If the child summaries at a concatenation are fixed, Lemma 7.4 uniquely determines the cancellation variables, and the output equations uniquely determine the parent coordinates. At a repetition, the child minimum is likewise unique, (110) uniquely determines the selectors and remainder, and (28) uniquely determines the parent summary. Structural induction proves uniqueness everywhere in the resource branch, as well as agreement with the actual word summary by Lemma 7.2 and Theorem 7.5.

For the canonical branch, the base signatures are fixed. Each elementary Boolean gate has a unique value for fixed inputs; concatenation therefore computes exactly (76). The same argument computes the child square and the selected power signature, which agrees with the full word power by Theorem 24.1. All inactive branches are still computed, so zero or one repetitions do not release any unconstrained variables. This proves uniqueness of every signature and intermediate wire.

The only remaining witnesses are the root slacks. By Lemma 7.1, they exist exactly for an execution with the stated endpoints and are unique when they exist. In the canonical version, the additional equation $C_E = 1$ holds exactly when the word accepts from zero memory, hence exactly when it is normal. This proves both counting equalities, including nonexistence on rejected inputs.

For the size estimate, each resource node needs a constant number of vectors of dimension p and each signature node needs $O(q)$ wires. A repetition switch has constant size. Thus summing over N

syntax nodes gives the stated bounds. The use of intermediate wires ensures the quadratic residual bound, and (113) doubles the degree at most. $\square$

25.3 An explicit resource-only variable bound

The asymptotic estimate does not hide a dependence on the numerical repetition counts. A more precise bound is available for a streamlined resource-only implementation using the two-way switch (27). Let C be the number of concatenation nodes and R the number of repetition nodes. Allocate all $2p$ summary coordinates at every node, three minimum witnesses per coordinate at every internal node, two switch witnesses per repetition, and p endpoint slacks. This gives

$$2pN + 3p(C + R) + 2R + p \leq (5p + 2)N + p \quad (116)$$

witnesses. Some are constant or could be shared, but retaining them makes the inductive proof transparent. The supplied prototype intentionally uses the three-way switch in both modes and some redundant Boolean bit equations; its exact counts therefore need not equal this optimized resource-only count.

25.4 Certificate height and bit complexity

Let W be the largest transition weight, K the largest supplied repetition parameter, and h the repetition depth. Put $K = 0$ when no parameter occurs. The next bound concerns a syntax tree, not a recursively defined program.

Proposition 25.2 (Witness height). *For an accepted input to the compiler in Theorem 25.1, its unique witness can be chosen—and in the displayed construction is forced—with all coordinates at most*

$$H = \max \left(1, K, \|m\|_\infty, WN(1 + K)^h \right). \quad (117)$$

In particular, exponentially long expansions need not require exponentially many witness bits per coordinate.

Proof. For any subexpression, each letter occurrence in its syntax contributes at most a product of h repetition counts to its expanded length. Summing over at most N letter leaves gives length at most $N(1 + K)^h$ for every subexpression, including ones discarded by an outer zero repetition.

For a word of length L , its summary satisfies $U_w, V_w \leq WL$ coordinatewise. One way to see this is to start with the bound W at leaves and use (24), which never exceeds the sum of the child bounds; an equivalent direct bound follows from (23). Minimum witnesses and their residual differences are bounded by the corresponding child coordinates. The intermediate nonnegative terms in a power output are bounded by that output. Selector remainders are at most K , Boolean wires are at most one, and the endpoint slack is at most m . These bounds give (117). $\square$

Thus a safe per-coordinate binary-length bound is

$$O(1 + \log(1 + K) + \log(1 + \|m\|_\infty) + \log(1 + W) + \log(1 + N) + h \log(1 + K)).$$

The root expansion length alone would not provide a correct bound: a zero outer power can erase an enormous child, whose uniquely constrained internal certificate is still present. This is another reason to track all syntax nodes, rather than only the visible final word.

A directed acyclic expression graph can share child summaries and retain the $O((p + q)N)$ variable bound when N counts distinct graph nodes. The tree-height estimate must then be replaced by a bound on expansion of shared nodes. A coarse universal estimate is $2^N(1 + K)^N$ for their lengths, obtained by induction over an ordering of the acyclic graph. Its logarithm is still polynomial in N and the input bit lengths. The prototype in this package uses trees, not graph sharing.

25.5 Worked examples and a scope warning

Example 25.3 (A nested multiplication schedule). Let $+$ increment a counter and $-$ decrement it when positive. The scheme

$$E_{x,y,z} = ((++)^x)^y + + + (-)^z \quad (118)$$

executes from zero to zero exactly when

$$z = 2xy + 3.$$

Its expansion first makes $2xy + 3$ increments and then z decrements. Thus there is no intermediate enabling obstruction when the equality holds. For $x = 3, y = 4, z = 27$, the concrete word has length 54. For $x = 10^{100}, y = 10^{120}$ and $z = 2xy + 3$, the same fixed compiler verifies a word whose length $2z$ has 221 decimal digits, without expanding it. There is no independence between $+$ and $-$ for a single counter, so every word is normal for the resulting empty independence relation.

Example 25.4 (Shared-resource commutation). Suppose a consumes one token and b consumes two tokens from the same counter, producing none, and $a < b$. The global commutation criterion includes (a, b) : either order has summary $(3, 0)$. From marking 3, the words ab and ba are two executable representatives of one trace. The canonical compiler accepts the former and rejects the latter even though their resource certificates describe the same partial action.

Remark 25.5 (A canonicity filter is not a normalizer). For a restricted expression family, adding the canonicity condition can remove every feasible instance. If a fixed expression is ba with $a < b$ and $(a, b) \in I$, its normal representative ab need not occur anywhere in that expression family. Therefore (115) tests whether the *given expansion* is canonical; it does not preserve existential reachability for every restricted schedule language. The trace-class bijection in the next section avoids this problem by ranging over all words of the fixed length, a trace-closed universe.

26 Expressiveness and an exact nesting boundary [06]

The compiler gives short certificates *once the repetition counts are fixed*. The existence of suitable counts is a separate problem. We now show that sharing parameters between nested repetitions is enough to recover Diophantine universality even with exceptionally simple counter commands.

26.1 Flat schemes are Presburger-definable

Theorem 26.1 (Flat-scheme decidability). *For every fixed token system and every expression of repetition depth at most one, the relation*

$$\{(m, n, \mathbf{k}) : m \xrightarrow{\llbracket E \rrbracket_{\mathbf{k}}} n\}$$

is effectively definable in Presburger arithmetic. The same holds with the additional requirement that the expanded word is normal. Consequently, existence of parameters for fixed endpoints is decidable in this fragment.

Proof. A repetition body has no repetition nodes and hence has a constant resource summary (u, v) . In the zero case its power is $(0, 0)$; in the positive case (26) is affine in the repetition parameter, because c, α, β are constants. This case split is expressible with $k = 0$ or $k \geq 1$.

Concatenation combines summaries by addition and coordinatewise minimum. The graph of $c = \min(x, y)$ has the Presburger definition

$$(x \leq y \wedge c = x) \vee (y \leq x \wedge c = y).$$

Thus structural induction yields a Presburger formula for all summaries and endpoints. Reusing a parameter only reuses the same variable in affine expressions; it does not introduce a product of two variable quantities at a depth-one repetition node. Normality is also Presburger-definable, either by the finite cells of Corollary 24.3 or by the finite Boolean circuit and the tests $k = 0, k = 1, k \geq 2$. Existential quantification over parameters preserves Presburger definability, and Presburger arithmetic is decidable. $\square$

This proof deliberately does not use the quadratic equation $st = 0$ to define the minimum inside Presburger arithmetic. It replaces that gadget by a linear case distinction. The nonlinear certificate language and the simpler decision procedure for a restricted fragment are compatible but serve different purposes.

26.2 Separated one-counter schemes and polynomial degree

Consider one counter with commands $+$ and $-$. Call a scheme *separated* if it is a concatenation E_+E_- where E_+ uses only $+$ and E_- uses only $-$. Empty expressions are allowed. For a fixed parameter tuple, let $A(\mathbf{k})$ and $B(\mathbf{k})$ be their respective lengths.

Lemma 26.2 (Length polynomial). *For any expression using a single letter, its expansion length is a polynomial in its repetition parameters with nonnegative integer coefficients. Its total degree is at most the repetition depth.*

Proof. The length polynomials of ε and a letter are 0 and 1. Concatenation adds length polynomials, while repetition by k_j multiplies the body length by k_j . Induction proves both nonnegativity of coefficients and the degree bound, since repetition increases nesting depth by one and concatenation takes the maximum of child depths. $\square$

Theorem 26.3 (Solution-preserving degree–depth correspondence). *For each fixed $d \geq 0$, the following two problem presentations are effectively interconvertible, preserving the parameter solution set exactly:*

- (i) *one polynomial equation $P(\mathbf{k}) = 0$ over $\mathbb{N}$, with $P \in \mathbb{Z}[\mathbf{k}]$ and $\deg P \leq d$;*
- (ii) *a separated one-counter scheme of repetition depth at most d , required to execute from zero to zero.*

No extra existential parameters are needed in either direction.

Proof. A separated scheme executes first $A(\mathbf{k})$ increments and then $B(\mathbf{k})$ decrements. It executes from zero to zero if and only if $A(\mathbf{k}) = B(\mathbf{k})$: equality supplies enough tokens for every decrement and leaves none. By Lemma 26.2, $P = A - B$ has degree at most d and gives exactly the same parameter solutions.

Conversely split an integer polynomial coefficientwise as $P = P_+ - P_-$, where P_+, P_- have nonnegative coefficients. For a monomial $c \prod_j k_j^{e_j}$, begin with c copies of the appropriate sign command and place them under $\sum_j e_j$ nested repetitions, using each k_j exactly e_j times. Its length is the monomial and its repetition depth is the monomial degree. Concatenate the positive monomial blocks to form E_+ and the negative ones to form E_- . The resulting scheme has depth at most d and accepts exactly when $P = 0$. $\square$

The direct construction writes a coefficient c as c letter occurrences. It is an effective solution-set correspondence, not a claim of polynomial size in a binary encoding of coefficients. Constant blocks could instead be represented by shared straight-line expressions. We do not need that optimization for any undecidability or multiplicity conclusion.

At $d = 1$, this recovers a linear Diophantine fragment. At $d = 2$, the separated *one-counter* problem is exactly natural solvability of one quadratic equation; decision procedures for positive integral solutions of arbitrary quadratic equations, and more general linear-region restrictions, are due to Grunewald and Segal [24]. Translating variables by one interconverts positive and nonnegative domains. At $d = 3$, the theorem states an exact equivalence with cubic Diophantine solvability; we make no additional classification of that problem here. At $d = 4$, undecidability follows from the next degree-reduction lemma and MRDP.

26.3 Unique quadratic systems and quartic equations

Lemma 26.4 (Multiplicity-preserving degree reduction). *For every $P(x, y) \in \mathbb{Z}[x, y]$, one can effectively construct quadratic or linear polynomials $r_1(x, y, g), \dots, r_s(x, y, g)$ such that, for each fixed $(x, y) \in \mathbb{N}^{a+b}$, the system $r_i = 0$ has exactly one natural solution g if $P(x, y) = 0$, and no solution otherwise. Hence*

$$Q(x, y, g) = \sum_{i=1}^s r_i(x, y, g)^2$$

has degree at most four and its roots project bijectively to the roots of P .

Proof. Split $P = P_+ - P_-$ into nonnegative-coefficient polynomials. Evaluate each by a finite arithmetic circuit using only nonnegative integer constants, original input variables, addition, and multiplication. Introduce one new variable for each gate output. An addition gate gives $g - u - v = 0$ and a multiplication gate gives $g - uv = 0$, with u, v either previously available variables or constants. Finally equate the two output wires, or the corresponding constants when a circuit has no gates.

Topological induction on the circuit shows that every gate variable is uniquely forced to a nonnegative integer for any fixed original inputs. The final equation then holds exactly when $P_+ = P_-$. This proves the unique-lifting assertion for the system. A sum of its squares is zero precisely when every gate equation and the final equality are zero, and its degree is at most four. $\square$

Avoiding signed intermediate witnesses is important. Representing each signed integer as an arbitrary difference of two naturals would introduce infinitely many encodings. Splitting the original polynomial into two nonnegative circuits avoids that problem entirely. This is a standard arithmetic-circuit normal form, used here with its multiplicity contract made explicit.

Corollary 26.5 (Depth-four one-counter universality). *Every computably enumerable relation $R \subseteq \mathbb{N}^a$ has a representation*

$$x \in R \iff \exists k \in \mathbb{N}^b \ 0 \xrightarrow{\llbracket E \rrbracket_{(x,k)}} 0$$

by a fixed separated one-counter scheme E of repetition depth at most four. The scheme uses only increment and enabled decrement. Its parameters may be globally reused.

Proof. Apply MRDP to obtain a polynomial representation of R . Apply Lemma 26.4, and then Theorem 26.3 at $d = 4$. Treat the input coordinates x as prescribed repetition parameters and all the other coordinates as existential repetition parameters. $\square$

This is a normal form for *existential schedule parameterization*. It does not say that an ordinary one-counter machine with freely chosen steps can compute an arbitrary partial recursive function. The numerical relationships between globally shared loop counts carry the Diophantine expressiveness.

26.4 Depth two is already universal with several resources

For each counter j , let $+_j$ and $-_j$ increment or decrement that counter and leave all others unchanged. A vector-separated scheme consists of a positive block using only the $+_j$, followed by a negative block using only the $-_j$.

Theorem 26.6 (Quadratic-system substrate and sharp flat/nested boundary). *A finite system of polynomial equations $P_j(\mathbf{k}) = 0$ of degrees at most d can be represented, without new parameters, by a vector-separated scheme of depth at most d on one counter per equation, executing from $\mathbf{0}$ to $\mathbf{0}$. Conversely, every such scheme gives a system of polynomial equations of degrees at most its depth.*

In particular, every computably enumerable relation has a fixed vector-separated representation of depth at most two. Therefore parameter existence for depth-two schemes in this class is undecidable, whereas it is decidable for all depth-one token schemes by Theorem 26.1.

Proof. Write $P_j = A_j - B_j$ coefficientwise with nonnegative polynomials. In the positive phase, use the monomial construction in Theorem 26.3 to perform A_j increments of counter j . Do this for every j . In the negative phase, perform B_j decrements of counter j for every j . Counter j starts and ends at zero exactly when $A_j = B_j$. If all equalities hold, every negative phase is enabled. If the run ends at zero, its net increment in every coordinate is zero, so all equalities hold. The depth bound and the absence of extra parameters follow from the monomial construction. The converse takes the increment and decrement counts in each coordinate and applies the length-polynomial induction.

For a computably enumerable relation, MRDP supplies one polynomial. Replace it by the quadratic system of Lemma 26.4, without first summing its squares. The preceding construction gives a depth-two scheme, with the gate variables among its existential parameters. Taking a computably enumerable undecidable relation proves undecidability. Since a single universal relation has one fixed polynomial and one fixed quadratic system, even the number of counters and the schedule template can be fixed for that relation; only its input parameters vary. The depth-one decision theorem proves the sharp boundary in the stated multi-resource class. $\square$

Corollary 26.7 (Universality can coexist with an already canonical word). *The universal schemes in Theorem 26.6 can be chosen so that every expansion is lexicographically normal for the global commutation relation.*

Proof. In the positive phase, group all occurrences of $+_1$, then $+_2$, and so on. Group the negative phase by $-_1, -_2, \dots$ in the same way. Choose the total alphabet order

$$+_1 < \dots < +_p < -_1 < \dots < -_p.$$

Every expanded word is globally nondecreasing in that order. It is therefore lexicographically least even among all permutations with the same letter multiplicities, hence also among the smaller set of words in its trace class. Grouping does not change the coordinatewise positive and negative counts or the depth of any monomial block. $\square$

The nontrivial search thus does not come from interleaving ambiguity. It persists for sorted, phase-separated schedules whose commands merely change one counter by one.

Part IV

Polynomial trajectories

Source and scope. [write] This Part is manuscript 02, “Canonical Diophantine Certificates for Polynomial Trajectories”: its statement of the problem and of the restricted result (the first two subsections of its introduction), then its Sections 3–9 (the finite-difference sign certificate, the single-fold quartic theorem and its cost, Boolean guards, acceleration of polynomial trajectories, the computational substrates covered, nontermination certificates, and worked examples), and its conclusion. Its canonical signed coordinates and total circuits (its Section 2) are printed with the conventions, in Section 2.3; its remaining introduction is in Section 3; its other substrates are in Parts VII and VIII and its boundary results in Part IX.

Setting and hypotheses. [write] Coefficient parameters are *signed* integers, $\mathbf{c} \in \mathbb{Z}^{d+1}$, with d a nominal degree bound; existential coordinates range over $\mathbb{N}$. T is the horizon of the index range $\{0, \dots, T\}$, not a number of execution steps. The main theorem is a family P_d indexed by the degree bound, uniform in coefficients and horizon.

27 The problem addressed and the precise restricted result [02]

[write] *The first two subsections of manuscript 02’s introduction; the rest of it is in Section 3.3.*

27.1 Existence is not multiplicity preservation

A computational model can be represented arithmetically in several different senses. An existence-preserving representation says that a computation exists exactly when an equation has a solution. A counting-faithful representation must additionally explain which solutions correspond to which computations, and whether auxiliary arithmetic choices create extra solutions. Even a deterministic computation can acquire infinitely many Diophantine witnesses through a noncanonical trace encoding.

The distinction is central to Matiyasevich’s finite-fold and single-fold questions. Classical results give both polynomial Diophantine representations of enumerable sets and single-fold *exponential* Diophantine representations. Combining the two properties is a separate problem, not a consequence of the ordinary MRDP theorem [9]. The present paper does not assume that combination.

Our starting question is more specific:

Can a long, deterministic, polynomial trajectory with nonmonotone guards be replaced by a fixed-dimensional polynomial certificate that has exactly one witness when the claimed run is valid?

Here “fixed-dimensional” means independent of the numerical duration of the run; the dimension may depend on the fixed program and on fixed degree bounds. The answer proved below is affirmative.

27.2 The precise restricted result

For $d \in \mathbb{N}$, put

$$p_{\mathbf{c}}(t) = \sum_{j=0}^d c_j t^j, \quad \mathbf{c} \in \mathbb{Z}^{d+1}, \quad T \in \mathbb{N}.$$

Coefficients may vanish, so d is a *nominal* degree bound, not necessarily the actual degree. Define

$$\text{PolyPos}_d(\mathbf{c}, T) \iff p_{\mathbf{c}}(i) \geq 0 \text{ for every integer } 0 \leq i \leq T.$$

The main theorem constructs an effective family of polynomials P_d with

$$\text{PolyPos}_d(\mathbf{c}, T) \iff \exists \mathbf{w} \in \mathbb{N}^{M_d} P_d(\mathbf{c}, T, \mathbf{w}) = 0, \quad (119)$$

where $\deg P_d \leq 4$, $M_d = O((d+1)^3)$, and every true instance has *exactly one* complete witness $\mathbf{w}$.

The family is uniform in coefficients and horizon. It is not a single polynomial accepting an arbitrarily long coefficient list as one encoded input. Varying d changes the finite set of variables and the polynomial itself. This distinction prevents an unjustified leap from our theorem to arbitrary bounded arithmetic.

The main mathematical development proceeds in three stages. First, a finite tower of differences yields a unique short sign table. Second, a completely evaluated arithmetic and Boolean circuit checks this table, including its padding and all comparisons. Third, canonical signed-number and sign relations turn this circuit into one quartic equation. Only after uniqueness of the entire witness has been proved do we project away an execution time, and then only for a first-exit relation where that time is unique.

28 The finite-difference sign certificate [02]

28.1 Differences and sign runs

For $p \in \mathbb{Z}[t]$, define

$$\Delta p(t) = p(t+1) - p(t), \quad q_k = \Delta^k p \quad (0 \leq k \leq d).$$

Then $\deg q_k \leq d - k$, and q_d is constant. In ascending coefficient order, the difference transformation is explicit:

$$[t^j] \Delta p(t) = \sum_{r=j+1}^d \binom{r}{j} c_r. \quad (120)$$

Thus every coefficient of every q_k is a fixed integer linear combination of the input coefficients.

A *sign run* on $\{0, \dots, T\}$ is a maximal nonempty interval of consecutive integers on which the sign is constant. We encode the interval as $[a, b) \cap \mathbb{Z}$, with $0 \leq a < b \leq T+1$. The signs $-1, 0, 1$ are encoded by natural tags 0, 1, 2, respectively.

Lemma 28.1 (A bound on the number of runs). *A real polynomial of degree at most m has at most $2m+1$ sign runs on any nonempty finite interval of consecutive integers. The zero polynomial has one run.*

Proof. Suppose the polynomial is nonzero. Let Z be the number of distinct integer zeros in the interval. Let C be the number of edges between consecutive sampled integers at which the signs are strictly opposite, with neither value zero. The intermediate value theorem puts a noninteger real root in the open interval of each such edge. These roots are distinct from one another and from the Z integer roots. Therefore $C + Z \leq m$.

Every change of sign tag occurs either at one of these C edges or at an edge incident with an integer zero. There are at most $2Z$ edges of the second kind. Thus the number of tag changes is at most $C + 2Z \leq 2m$. There is one more run than changes. Consecutive zero values cause no difficulty: edges joining two zeros are not changes, and our bound may only overcount them. $\square$

28.2 Canonical padding

Set

$$L_k = 2(d - k) + 1 \quad (0 \leq k \leq d).$$

Level k has L_k slots, with cuts and tags

$$b_{k,0}, \dots, b_{k,L_k}, \quad \tau_{k,0}, \dots, \tau_{k,L_k-1}.$$

The outside cuts are fixed:

$$b_{k,0} = 0, \quad b_{k,L_k} = T + 1.$$

All other cuts and all tags are natural primary witness coordinates. The following finite conditions specify the permitted shape:

$$b_{k,j} \leq b_{k,j+1}, \quad 0 \leq \tau_{k,j} \leq 2, \quad (121)$$

$$b_{k,j} = b_{k,j+1} \implies (b_{k,j} = T + 1 \wedge \tau_{k,j} = 1), \quad (122)$$

$$b_{k,j} < b_{k,j+1} < b_{k,j+2} \implies \tau_{k,j} \neq \tau_{k,j+1}. \quad (123)$$

The last condition is imposed only when the displayed indices exist. A slot is *active* when its two cuts are different. Condition (122) forces all inactive slots to be terminal padding; their tag is the fixed zero-sign tag. Condition (123) prohibits splitting a constant-sign run into two active slots. Every genuine maximal-run table therefore has exactly one padded encoding.

The count of primary coordinates is exact:

$$\sum_{k=0}^d ((L_k - 1) + L_k) = \sum_{m=0}^d (4m + 1) = (d + 1)(2d + 1). \quad (124)$$

The $2(d + 1)$ outside cuts are not counted, because they are fixed expressions rather than witnesses.

28.3 The local endpoint condition

The top level has one slot. Require

$$\tau_{d,0} = 1 + \text{sgn}(q_d(0)). \quad (125)$$

Now let $k < d$. Consider any active parent slot $[a, b) = [b_{k,j}, b_{k,j+1})$ and active child slot $[c, e) = [b_{k+1,\ell}, b_{k+1,\ell+1})$. Define

$$u = \max(a, c), \quad v = \min(b - 1, e). \quad (126)$$

Whenever $u \leq v$, require

$$1 + \text{sgn}(q_k(u)) = \tau_{k,j} = 1 + \text{sgn}(q_k(v)). \quad (127)$$

The child right endpoint in (126) is e , not $e - 1$. This is intentional: information about Δq_k on $[c, e)$ makes q_k monotone on the integer interval $[c, e]$. Using $e - 1$ would also cover the domain if handled consistently, but (126) gives a convenient closed monotonicity interval and is the convention used throughout the implementation.

All pairs of slots are considered. An implication disables the endpoint assertion for an inactive slot or empty intersection. Its arithmetic computations are nevertheless total, as in Lemma 2.8. Maxima and minima are implemented by comparison bits and selection gates, so these conditions are a fixed quantifier-free polynomial formula for fixed d .

Lemma 28.2 (Endpoint propagation). *Assume every child tag at level $k + 1$ is correct. If a parent slot satisfies (127) for every active child slot, then its tag is the actual sign tag of q_k at every integer point in that parent slot.*

Proof. On an active child interval $[c, e]$, the difference $q_k(i+1) - q_k(i) = q_{k+1}(i)$ has a constant sign. Hence q_k is nondecreasing, nonincreasing, or constant on $[c, e] \cap \mathbb{Z}$. Its restriction to the intersection

$$[a, b-1] \cap [c, e] \cap \mathbb{Z} = [u, v] \cap \mathbb{Z}$$

is also monotone. If both endpoint values are positive, every intervening value is positive; the analogous statement holds for negative values. If both endpoint values are zero, monotonicity forces every intervening value to be zero. Thus the parent tag is correct throughout each nonempty intersection.

Every integer t in the parent slot belongs to an active child slot, because the child slots partition $\{0, \dots, T\}$. For that child, $t \in [c, e-1] \subseteq [c, e]$, so the relevant intersection is nonempty and contains t . This proves correctness at every point of the parent. $\square$

Theorem 28.3 (Existence and uniqueness of the sign tower). *For every $\mathbf{c} \in \mathbb{Z}^{d+1}$ and $T \in \mathbb{N}$, there is exactly one primary tuple satisfying (121)–(127). It is the canonically padded sequence of maximal sign-run tables of $q_0, \dots, q_d$ on $\{0, \dots, T\}$.*

Proof. For existence, take the actual maximal sign runs of each q_k . Lemma 28.1 provides at most L_k runs, so terminal padding is possible. The shape conditions hold by construction. The top difference is constant, so (125) holds. Each endpoint appearing in (127) is an integer point in the parent slot; its tag is therefore correct.

For soundness, start at the top level, whose unique active slot is correct by (125). Descend through $k = d-1, \dots, 0$ and apply Lemma 28.2. Every active slot at every level is then a constant-sign interval with its true tag.

Finally, the shape conditions imply that the active slots form an ordered partition of the entire domain, and adjacent active slots have different tags. Such a partition must be the maximal sign-run partition: a boundary inside a constant-sign run is prohibited, and a missing boundary at a sign change would make a slot nonconstant. Terminal empty slots and their tags are prescribed by (122). Consequently every coordinate is uniquely determined. $\square$

Remark 28.4 (Degenerate cases are part of the theorem). The proof includes $T = 0$, the zero polynomial, a nominal degree larger than the actual degree, and runs consisting of one integer zero. No genericity, simple-root, nonzero-leading-coefficient, or sufficiently-large-horizon assumption is used. A zero polynomial at any level has one active zero-sign slot followed by uniquely fixed padding.

29 The single-fold quartic theorem and its cost [02]

Theorem 29.1 (Canonical bounded polynomial positivity). *There is an effective family of integer polynomials P_d , indexed by $d \in \mathbb{N}$, and an absolute constant C such that*

$$\deg P_d \leq 4, \quad M_d \leq C(d+1)^3,$$

and (119) is a single-fold representation for every signed coefficient tuple and natural horizon. Exactly $(d+1)(2d+1)$ primary coordinates suffice before auxiliary arithmetic and Boolean wires are introduced. Both the number of quadratic residuals and a bounded-arity straight-line description length are $O((d+1)^3)$.

Proof. Take the finite sign-tower conditions and add

$$b_{0,j} < b_{0,j+1} \implies \tau_{0,j} \neq 0 \quad (0 \leq j < L_0). \quad (128)$$

By Theorem 28.3, the primary tuple is unique before these conditions are added. They hold precisely when $q_0 = p$ has no negative value in the domain. Apply Lemma 2.8 to the complete quantifier-free

checking formula. It gives one quartic equation, and its internal wires have a unique extension for every fixed primary tuple. Thus the complete existential tuple is unique whenever the positivity relation holds, and no tuple exists otherwise.

It remains to justify the cubic size bound, rather than merely asserting that the formula is finite. At a level where the parent degree bound is $m \geq 1$, there are $(2m + 1)(2m - 1) = 4m^2 - 1$ parent–child pairs. Their intersection and tag comparisons take boundedly many gates per pair. The potentially expensive operations are polynomial evaluations.

Do not evaluate q_k afresh at the selected endpoint of every pair. The possible left endpoints are parent starts and child starts; the possible right endpoints are parent stops minus one and child stops. There are $O(m)$ such candidate expressions. Evaluate q_k once at each candidate by Horner’s rule, using $O(m)$ arithmetic gates per evaluation, and cache the sign tags. A comparison then selects the appropriate precomputed endpoint tag in $O(1)$ gates. The total cost at this level is $O(m^2)$.

The difference coefficient tower can be constructed in $O((d + 1)^3)$ scalar arithmetic operations using (120). The shape tests use $O((d + 1)^2)$ gates. Summing the $O(m^2)$ level costs therefore gives $O((d + 1)^3)$ gates overall. The canonical circuit lemma introduces only a constant number of variables and residuals per gate. The exact primary count is (124). $\square$

29.1 An explicit specification, not only an existence proof

The preceding construction completely specifies P_d : unroll the stated slot conditions, evaluate the stated finite circuit, introduce the signed wire and sign equations, and sum the squared residuals. A factored sum-of-squares description is an exact description of an ordinary polynomial; expanding all monomials is unnecessary.

The accompanying compiler makes this description executable. Its exported JSON records every residual as a sparse polynomial with integer coefficients and named variables. The equation represented by the file is

$$P_d = Q_1^2 + \cdots + Q_N^2 = 0.$$

There is no exponentiation primitive, bounded quantifier, floor function, black-box Diophantine predicate, or implicit oracle left in this output. The only powers are the fixed squares and products appearing in the polynomial itself.

29.2 Generating the unique witness without scanning the horizon

Proposition 29.2 (Generation and verification complexity). *The sign tower can be generated using $O((d + 1)^3 \log(T + 2))$ exact arithmetic operations, and its canonical endpoint checker can be implemented using $O((d + 1)^3)$ such operations. With dense binary integer coefficients and binary T , generation and verification have polynomial bit complexity in*

$$d + 1, \quad \log(H + 2), \quad \log(T + 2), \quad H = \max_j |c_j|.$$

The arithmetic auxiliary witnesses also have polynomially bounded bit length in these quantities.

Proof. The constant top row is immediate. Suppose the child row has been constructed. On each child interval $[c, e)$, q_k is monotone. For a nondecreasing sequence, find the first index with value at least zero and the first index with value strictly greater than zero by binary search. These two indices split the interval into its negative, zero, and positive subintervals. Empty subintervals are discarded. For a nonincreasing sequence, apply the same procedure to $-q_k$ and reverse the sign labels. A constant sequence is included in the nondecreasing case. Merge adjacent subintervals with equal tags when passing from one child interval to the next.

There are $O(m)$ child intervals at a level of degree bound m . Each requires at most two binary searches, each with $O(\log(T + 2))$ evaluations. Horner evaluation costs $O(m)$ arithmetic operations. This gives the generation bound after summing over the levels. The verification bound follows from the cached endpoint implementation in the proof of Theorem 29.1.

For bit complexity, the explicit identity

$$\Delta^k p(t) = \sum_{\ell=0}^k (-1)^{k-\ell} \binom{k}{\ell} p(t + \ell)$$

shows that the coefficient heights of all differences have bit length $O(\log(H + 2) + d \log(d + 2))$. One can also obtain this bound by expanding $(t + \ell)^r$ and bounding its binomial coefficients. Evaluation at an integer of magnitude at most $T + 1$ adds $O(d \log(T + 2))$ bits. All candidate cuts lie in $[0, T + 1]$, signs and tags are bounded, and the sign-test slack is at most the magnitude of its argument. Polynomially many additions and multiplications on numbers with these polynomial bit bounds establish the claim using ordinary integer arithmetic. The same estimates apply to stored arithmetic outputs and comparison slacks. No assertion that unit-cost arithmetic is unit-cost *bit* computation is needed. $\square$

Remark 29.3 (Cubic size is an upper bound, not an optimum). The supplied compiler favors transparency and total evaluation over small constants. The theorem does not claim minimal witness count, optimal quartic sparsity, or an improved universal Diophantine equation. The specialized quartics should not be compared numerically with universal witness-count records as if they represented the same class of relations.

30 Boolean guards and empty execution prefixes [02]

Theorem 30.1 (Bounded Boolean polynomial guards). *Fix a Boolean formula Φ in signs of $r \geq 1$ polynomials $p_1(t), \dots, p_r(t)$, with respective fixed degree bounds $d_j \leq D$. The relation*

$$\forall i \in \{0, \dots, T\} \Phi(\text{sgn } p_1(i), \dots, \text{sgn } p_r(i))$$

has an effective single-fold quartic representation, uniform in all coefficients and in T . A witness and circuit size bound is

$$O\left(r(D + 1)^3 + r^2(D + 1)^2 + |\Phi|r(D + 1)\right).$$

A guard with no polynomial atoms is simply a constant Boolean guard and is handled directly.

Proof. Construct the unique difference sign tower for every p_j . Take the union of the starts of all active level-zero runs. It includes zero. Between two consecutive distinct starts in this union, every polynomial has constant sign, hence Φ has constant truth value.

It is unnecessary to existentially sort or deduplicate this union. For every active start of every polynomial, evaluate all r polynomials at that start and require Φ to be true there. Repeated starts merely repeat a true assertion and do not add choices. These tests are sufficient because every common sign cell begins at one of the tested starts; they are necessary because each tested start belongs to the domain.

There are $O(r(D + 1))$ candidate starts. Evaluating r polynomials of degree at most D at each uses $O(r^2(D + 1)^2)$ gates. Evaluating the Boolean formula at these starts uses $O(|\Phi|r(D + 1))$ gates. The sign towers contribute $O(r(D + 1)^3)$ gates. Every primary tower is unique and every extra evaluation is total and deterministic, so Lemma 2.8 gives the claimed single-fold quartic. $\square$

Strict comparisons, equations, disequalities, and arbitrary negations are all included. For example, a guard may be

$$(p_1 > 0 \wedge p_2 \neq 0) \vee (p_3 = 0 \wedge \neg(p_4 \geq 0)).$$

There is no monotonicity assumption on this entire guard or on p_j . Monotonicity is used only on the certified difference intervals.

30.1 Why a zero-length run requires explicit treatment

In execution semantics the usual condition is $\forall i < n \ G(i)$. For $n = 0$ this is true, even if $G(0)$ is false. Replacing it by a test on $[0, \max(n - 1, 0)]$ without an additional condition would therefore be wrong. Use instead the uniquely determined number

$$h = \max(n - 1, 0).$$

Always construct the full sign towers on $[0, h]$, but enable the final universal-guard assertions only when $n > 0$. The tables and their checking wires remain fully determined when $n = 0$; only the demand that the guard be true is disabled. The graph of this maximum is a quantifier-free piecewise-polynomial relation, and the circuit lemma supplies its unique arithmetic implementation. This method preserves both vacuity and single-foldness.

31 Accelerating polynomial trajectories [02]

Definition 31.1 (Polynomially iterable map). A fixed integer-state map $f : \mathbb{Z}^s \rightarrow \mathbb{Z}^s$ is *polynomially iterable* when there is a fixed vector $F(t, \mathbf{x}) \in \mathbb{Q}[t, \mathbf{x}]^s$ such that

$$F(0, \mathbf{x}) = \mathbf{x}, \quad F(t + 1, \mathbf{x}) = f(F(t, \mathbf{x}))$$

for every natural t and integer $\mathbf{x}$. In particular, $F(t, \mathbf{x}) = f^t(\mathbf{x})$ is integer-valued at these inputs. The trajectory formula, not just its existence, is supplied as part of the fixed program specification.

Let $G(\mathbf{x})$ be a fixed Boolean combination of polynomial comparisons with integer coefficients. The loop tests G *before* performing each simultaneous update. Define

$$\text{Run}_f^G(\mathbf{x}, n, \mathbf{y}) \iff \mathbf{y} = F(n, \mathbf{x}) \wedge \forall i < n \ G(F(i, \mathbf{x})), \quad (129)$$

$$\text{Exit}_f^G(\mathbf{x}, n, \mathbf{y}) \iff \text{Run}_f^G(\mathbf{x}, n, \mathbf{y}) \wedge \neg G(\mathbf{y}). \quad (130)$$

The run relation allows stopping while the guard is still true; the exit relation does not. This semantic distinction is essential when an existential execution time is introduced.

Theorem 31.2 (Single-fold polynomial acceleration). *For every fixed polynomially iterable f and fixed polynomial Boolean guard G , both (129) and (130) have effective single-fold representations by ordinary integer quartics with finitely many natural witnesses. Their witness dimensions are independent of n and of the values of the initial and final states.*

Proof. Substitute $F(t, \mathbf{x})$ into each polynomial atom of G . The result is a polynomial in t whose coefficients are rational polynomials in $\mathbf{x}$. Multiply each atom's polynomial difference by a fixed positive common denominator. This preserves its sign and gives an integer polynomial in $(t, \mathbf{x})$. Its degree in t is fixed by f and G , not by n or by the state values.

Compute the coefficient values from $\mathbf{x}$ using a fixed arithmetic circuit and canonical signed wires. Apply Theorem 30.1 to these polynomials, with the empty-prefix convention from Section 30. Add the

equations $\mathbf{y} = F(n, \mathbf{x})$ after clearing fixed denominators. These equations can be checked by the same canonical circuit lemma, so no polynomial degree increase survives the introduction of gate variables. For exit, add the Boolean test $\neg G(\mathbf{y})$.

Coefficient computations and endpoint computations are unique. The primary sign tables are unique. All their auxiliary wires are unique. Consequently the complete witness is unique for each valid $(\mathbf{x}, n, \mathbf{y})$. Every invalid triple fails at least one of the equivalent checking conditions. Finally sum the squared quadratic residuals. $\square$

Corollary 31.3 (Projecting the first-exit time). *The halting domain*

$$H_f^G(\mathbf{x}) \iff \exists n \in \mathbb{N} \exists \mathbf{y} \in \mathbb{Z}^s \text{Exit}_f^G(\mathbf{x}, n, \mathbf{y})$$

and the partial output graph

$$O_f^G(\mathbf{x}, \mathbf{y}) \iff \exists n \in \mathbb{N} \text{Exit}_f^G(\mathbf{x}, n, \mathbf{y})$$

have effective single-fold quartic representations with natural witnesses.

Proof. For a fixed initial state, there is at most one first-exit time. Indeed, if $n < m$ were two such times, the exit at n would give $\neg G(F(n, \mathbf{x}))$, while the valid prefix for m would give $G(F(n, \mathbf{x}))$. The final state is then uniquely $F(n, \mathbf{x})$. Thus existentially projecting these quantities cannot introduce additional choices. Represent every projected signed state coordinate by the normalized pair of Lemma 2.6; merely using an unrestricted difference of two naturals would be insufficient. The existing certificate for the unique exit triple has a unique extension. The added normalization equations are quadratic. $\square$

Remark 31.4 (Reachability is not automatically single-fold). Replacing Exit by Run in this corollary would be unjustified. For the identity update and the guard *true*, the same state is reachable at every duration. Even a perfect single-fold certificate for each fixed n then gives infinitely many witnesses after n is projected away. The theorem does not rule out a different single-fold representation of that trivial relation; it identifies why this particular projection argument would fail.

32 Computational substrates covered by the theorem [02]

32.1 Unipotent affine state transformations

Consider

$$f(\mathbf{x}) = A\mathbf{x} + \mathbf{b}, \quad A \in M_s(\mathbb{Z}), \quad \mathbf{b} \in \mathbb{Z}^s.$$

Introduce the augmented matrix

$$M = \begin{pmatrix} A & \mathbf{b} \\ 0 & 1 \end{pmatrix}, \quad \tilde{\mathbf{x}} = (\mathbf{x}, 1)^\top.$$

Then $M^t \tilde{\mathbf{x}}$ is the augmented state at time t .

Proposition 32.1 (Unipotent affine acceleration). *Suppose $(M - I)^r = 0$ for a fixed $r \geq 1$. Then*

$$M^t = \sum_{j=0}^{r-1} \binom{t}{j} (M - I)^j \quad (t \in \mathbb{N}). \quad (131)$$

Consequently this affine update is polynomially iterable and every fixed polynomial Boolean guard admits the single-fold execution, first-exit, and output representations above.

Proof. Expand $(I + (M - I))^t$ by the finite binomial theorem. All powers of $M - I$ of exponent at least r vanish. For a fixed j , the expression

$$\binom{t}{j} = \frac{t(t-1) \cdots (t-j+1)}{j!}$$

is a rational polynomial in t , with the usual value 1 for $j = 0$. For natural $t < j$ it is zero, so (131) also covers small t . Matrix multiplication by the initial augmented state gives the required polynomial trajectory. The identities are valid over integers after clearing fixed denominators. $\square$

This includes translations, counter transfers with nilpotent coupling, and accumulation chains. State coordinates may increase or decrease; nonnegativity requirements for a natural-counter interpretation can be inserted as guard atoms. There is no assumption that the full state or the guard value is monotone.

32.2 Unitriangular nonlinear updates

Consider simultaneous assignments

$$\begin{aligned} x'_1 &= x_1 + c, \\ x'_i &= x_i + R_i(x_1, \dots, x_{i-1}) \quad (2 \leq i \leq s), \end{aligned} \tag{132}$$

where $c \in \mathbb{Z}$ and every R_i is an integer polynomial. The word *simultaneous* means that all right-hand sides use the old state.

Lemma 32.2 (Discrete antidifferentiation). *For every $q(t) \in K[t]$ over a characteristic-zero field K , there is a unique polynomial $S(t) \in K[t]$ satisfying*

$$S(0) = 0, \quad S(t+1) - S(t) = q(t).$$

It has degree at most $\deg q + 1$ when $q \neq 0$.

Proof. The polynomials $\binom{t}{j}$, $j \geq 0$, form a basis of $K[t]$, since they have distinct degrees and nonzero leading coefficients. Write $q(t) = \sum_j \alpha_j \binom{t}{j}$ and set $S(t) = \sum_j \alpha_j \binom{t}{j+1}$. Pascal's identity proves the difference equation and $S(0) = 0$. If two such polynomials existed, their difference would have zero forward difference; a nonconstant polynomial in characteristic zero cannot have that property, because taking a difference lowers its degree by exactly one. Thus their difference is a constant, and its value at zero is zero. $\square$

Theorem 32.3 (Nonlinear unitriangular substrates). *Every map (132) is polynomially iterable. Therefore its exact guarded execution, first exit, halting domain, and partial output graph admit the single-fold quartic representations of Theorem 31.2 and Corollary 31.3.*

Proof. The first coordinate is $F_1(t, \mathbf{x}) = x_1 + ct$. Suppose the trajectories of the first $i-1$ coordinates have already been expressed as rational polynomials in $(t, \mathbf{x})$. Then

$$R_i(F_1(t, \mathbf{x}), \dots, F_{i-1}(t, \mathbf{x}))$$

is another such polynomial. Apply discrete antidifferentiation in the variable t , treating the initial coordinates as coefficients, and add x_i to obtain F_i . The resulting polynomial satisfies exactly the update recurrence and the initial condition. Induction on the integer time then shows that its values coincide with the actual integer computation, so no separate integer-valuedness assumption is needed. Continue through all coordinates. $\square$

Degree growth in this construction can be substantial. For example, if $\deg_t F_j \leq D_j$ for $j < i$, the degree in t of F_i is bounded by one plus the largest weighted degree $\sum_{j<i} \alpha_j D_j$ among monomials $x_1^{\alpha_1} \cdots x_{i-1}^{\alpha_{i-1}}$ of R_i , with the initial constant added. The resulting certificate is fixed-dimensional for a *fixed* program, but the theorem does not promise a small polynomial bound in the source-code length of an arbitrary nested triangular expression.

32.3 A certified eventually quasi-polynomial affine extension

Polynomial trajectories exclude sign-alternating linear modes. A fixed residue decomposition handles a wider class without introducing variable exponentiation.

Theorem 32.4 (Eventually quasi-polynomial affine acceleration). *Let M be the augmented integer matrix of an affine update. Suppose fixed integers $h \geq 0$, $p \geq 1$, and $r \geq 1$ are supplied and satisfy*

$$M^h(M^p - I)^r = 0. \quad (133)$$

For every fixed polynomial Boolean guard, exact guarded execution and first exit have single-fold quartic representations of fixed dimension. Their halting domains and partial output graphs are also single-fold.

Proof. For $0 \leq s < p$ and $k \in \mathbb{N}$, all the matrices involved commute, and the binomial expansion gives

$$M^{h+s+pk} = \sum_{j=0}^{r-1} \binom{k}{j} M^{h+s} (M^p - I)^j. \quad (134)$$

Terms of exponent at least r vanish by (133). Thus the trajectory is a polynomial in k on each of the finitely many residue classes after the fixed prefix of length h .

In a claimed run of length n , the prefix tests at times $i < h$ are required only when $i < n$. In residue class s , the relevant times are $h + s + pk < n$. Their number is the uniquely determined natural number

$$N_s = \left\lceil \frac{\max(n - h - s, 0)}{p} \right\rceil.$$

The graph of this expression has a canonical polynomial certificate: compute the positive part by a sign test, divide its sum with $p - 1$ by the fixed positive integer p , and impose the unique Euclidean remainder condition $0 \leq u < p$. Comparisons are converted using the circuit lemma. This determines N_s and all its auxiliary values uniquely, including when $N_s = 0$.

Substituting (134) into the guard gives a polynomial Boolean guard in k . Apply the bounded guard theorem on the prefix $0 \leq k < N_s$ for each s , with the zero-length convention already proved. The terminal state is selected from the fixed prefix or the unique residue decomposition of $n - h$. These are finite, deterministic piecewise-polynomial computations; every inactive branch is evaluated on uniquely prescribed arguments or explicitly padded by zero. Thus all auxiliaries are unique. The circuit lemma again reduces the finite combined system to a quartic. First exit and its projections follow by the same uniqueness argument as before. $\square$

Condition (133) is an exact integer matrix identity that can be checked directly. It is sufficient; this paper does not require an algorithm for finding optimal h, p, r or a spectral classification. Taking $p = 2$ accommodates appropriate alternating modes, while nilpotent transient modes can be absorbed into h . The period is fixed as part of the substrate certificate, not an unbounded existential choice.

32.4 Composition of deterministic phases

Proposition 32.5 (Single-fold composition of partial functions). *For fixed integer or natural tuple domains A, B, C , if the graphs of partial functions $f : A \rightarrow B$ and $g : B \rightarrow C$ have single-fold polynomial representations, then so does the graph of $g \circ f$. It can be presented by a quartic after introducing canonical circuit variables.*

Proof. Use the conjunction of the two graph representations and existentially quantify the intermediate value b . For fixed input a , functionality of f makes b unique if it exists. The two auxiliary tuples are then unique by hypothesis. Encode signed components of b canonically. Flatten the finite polynomial expressions into arithmetic gates and sum squared quadratic residuals. $\square$

A fixed sequence of terminating phases from our loop classes therefore inherits single-foldness. This is not closure under arbitrary recursive calls or a variable number of phases. A deterministic branch controlled by a polynomial test can also be handled, provided every inactive certificate tuple is explicitly normalized. Unrestricted nondeterministic choice requires a separate multiplicity analysis.

33 Canonical nontermination certificates and input bounds [02]

The exact-run theorem treats a supplied duration. For polynomial trajectories, a canonical stabilization bound also handles all future times without leaving ordinary polynomial arithmetic.

Lemma 33.1 (An elementary canonical root bound). *Let $q(t) = \sum_{j=0}^m a_j t^j$ be an integer polynomial, with zero leading coefficients permitted in the displayed list. Set*

$$B_q = 2 + \sum_{j=0}^m |a_j|.$$

If q is nonzero, it has no real root $t \geq B_q$. Its sign is constant on $[B_q, \infty)$. The zero polynomial has constant zero sign there.

Proof. Constant polynomials require no argument. Otherwise discard zero leading coefficients and let $a_m \neq 0, m \geq 1$. Since a_m is a nonzero integer, $|a_m| \geq 1$. Put $A = \max_{j < m} |a_j/a_m| \leq \sum_j |a_j|$. For a real $t > 1 + A$, the finite geometric sum gives

$$\left| \sum_{j < m} a_j t^j \right| \leq |a_m| A \frac{t^m - 1}{t - 1} < |a_m| t^m.$$

Thus the lower terms cannot cancel the leading term, and $q(t)$ has the sign of a_m . Every $t \geq B_q$ satisfies the strict inequality needed above. The extra constant 2 removes any endpoint ambiguity. $\square$

Theorem 33.2 (Specified-input nontermination is single-fold). *Let f, F, G be as in Theorem 31.2. The relation*

$$\text{Nonterm}_f^G(\mathbf{x}) \iff G(F(t, \mathbf{x})) \text{ holds for every } t \in \mathbb{N}$$

has an effective single-fold quartic representation. Moreover, an explicit canonical bound $B(\mathbf{x})$ can be computed such that every terminating input first exits at some time at most $B(\mathbf{x})$.

Proof. After clearing fixed positive denominators as before, write the polynomials of the guard along the trajectory as

$$q_j(t, \mathbf{x}) = \sum_{\ell=0}^{D_j} a_{j\ell}(\mathbf{x}) t^\ell, \quad a_{j\ell} \in \mathbb{Z}[\mathbf{x}].$$

Define the exact, not merely upper-bounded, number

$$B(\mathbf{x}) = 2 + \sum_{j,\ell} |a_{j\ell}(\mathbf{x})|. \quad (135)$$

For each specified input, Lemma 33.1 makes every atom's sign constant from time $B(\mathbf{x})$ onward. The Boolean combination G is therefore constant from that time onward as well. It follows that

$$\text{Nonterm}_f^G(\mathbf{x}) \iff \forall 0 \leq t \leq B(\mathbf{x}) \ G(F(t, \mathbf{x})). \quad (136)$$

Indeed, if the right-hand side holds, its truth at B extends to the entire tail; the converse is immediate. If the loop exits at all, some false guard value must consequently occur no later than B .

The coefficient values and their absolute values are computed uniquely using canonical signed pairs. Equation (135) determines B uniquely. Apply the bounded Boolean guard theorem to (136). Its primary tables and all auxiliary arithmetic values are unique, so the complete nontermination certificate is single-fold. Flattening the finite combined circuit gives a quartic. $\square$

Corollary 33.3 (Bit-efficient specified-input classification). *For a fixed polynomial trajectory and fixed polynomial Boolean guard, termination or nontermination from a binary-encoded integer initial state can be decided in polynomial time in that state's bit length. The corresponding canonical certificates have polynomial bit length. When the loop terminates, its first-exit time and output can be produced within the same polynomial-time bound.*

Proof. Let $H = \max(1, \|\mathbf{x}\|_\infty)$. Since the program and the coefficient polynomials are fixed, there are fixed constants C, K with $B(\mathbf{x}) \leq C(H + 1)^K$. Coefficient computation therefore uses polynomial bit time and yields $O(\log(H + 2))$ -bit values, with constants allowed to depend on the program.

Generate the fixed-degree sign towers on $[0, B]$ by Proposition 29.2. Their running time is polynomial in $\log(H + 2)$. Inspect the finitely many common sign-cell starts used in Theorem 30.1. If all guard values are true, the loop does not terminate. Otherwise the smallest false cell start is exactly the first exit time. Evaluate the fixed polynomial trajectory there to obtain the output. The same magnitude estimates bound the complete certificate's bit length. $\square$

This is an acceleration statement about binary arithmetic, not a bound on the literal number of loop iterations. A loop can run for a very large number of steps and still have an efficiently produced compressed certificate. The constants in the complexity bound depend on the fixed trajectory and its degrees; this corollary is not a polynomial bound for arbitrary programs presented as input.

33.1 Why this does not decide global termination

The quantifier over the initial state is crucial. For an arbitrary integer polynomial $Q(\mathbf{x})$, consider the identity update with guard $Q(\mathbf{x}) = 0$. There is a nonterminating initial state exactly when Q has an integer zero. With natural initial states, the same example expresses natural solvability. Hence a uniform procedure deciding whether *every* initial state terminates for all such supplied loops would decide a Hilbert-tenth solvability problem. Ordinary MRDP supplies the undecidability background [14].

Our theorem assigns a finite single-fold certificate to each specified initial state. It does not decide the existential assertion that some state has a nontermination certificate. That assertion is itself a Diophantine solvability problem. Thus the polynomial-time specified-input corollary is compatible with the broader termination distinctions in the triangular-loop literature [25, 26].

34 Worked examples [02]

34.1 A guard that fails only in the interior

Let

$$p(t) = (t - 2)(t - 4) = t^2 - 6t + 8, \quad T = 6.$$

Its values are

$$(p(0), \dots, p(6)) = (8, 3, 0, -1, 0, 3, 8).$$

Both outer endpoint values are positive, but bounded nonnegativity is false. The difference tower is

$$q_0 = t^2 - 6t + 8, \quad q_1 = 2t - 5, \quad q_2 = 2.$$

The actual active runs are shown in Table 3. Their tags are one plus the signs shown in the table.

Level	Active half-open sign intervals on $\{0, \dots, 6\}$
q_0	$[0, 2) : +, [2, 3) : 0, [3, 4) : -, [4, 5) : 0, [5, 7) : +$
q_1	$[0, 3) : -, [3, 7) : +$
q_2	$[0, 7) : +$

Table 3: A unique short certificate exposing a negative interior value.

The nominal level capacities are 5, 3, 1. Thus q_1 gets one inactive slot $[7, 7)$ with tag 1, while the other levels need no padding. The sign-table certificate is valid, but the additional positivity assertion rejects the negative level-zero run.

For the loop $x' = x + 1$ with guard $(x - 2)(x - 4) \geq 0$, started at $x = 0$, the first exit is $n = 3$, $x = 3$. A procedure checking only the guard at the start and at time 6 would incorrectly infer that the entire prefix was valid. The sign-table method does not make that inference.

34.2 A nonlinear accumulation chain

Consider the simultaneous update

$$x' = x + 1, \quad y' = y + x, \quad z' = z + y^2, \quad (137)$$

with initial state $(a, b, c) \in \mathbb{Z}^3$. Its exact trajectory is

$$\begin{aligned} x_t &= a + t, \\ y_t &= b + at + \binom{t}{2}, \\ z_t &= c + b^2t + (2ab + a^2)\binom{t}{2} \\ &\quad + (2a^2 + 2b + 4a + 1)\binom{t}{3} + (6a + 6)\binom{t}{4} + 6\binom{t}{5}. \end{aligned} \quad (138)$$

To verify the last formula, write $y_j = b + aj + \binom{j}{2}$ and use

$$\begin{aligned} j^2 &= \binom{j}{1} + 2\binom{j}{2}, \\ j\binom{j}{2} &= 2\binom{j}{2} + 3\binom{j}{3}, \\ \binom{j}{2}^2 &= \binom{j}{2} + 6\binom{j}{3} + 6\binom{j}{4}. \end{aligned}$$

Expanding y_j^2 in this basis gives the coefficients in (138) after summation with $\sum_{j=0}^{t-1} \binom{j}{k} = \binom{t}{k+1}$. This is a symbolic proof; the accompanying tests also compare the formula with 2,500 explicitly iterated states.

An example guard is

$$(z^2 - 3xy + \lambda \geq 0) \wedge (y \neq \mu),$$

where λ, μ are fixed parameters or additional unchanged input coordinates. Along the trajectory, the first atom has degree at most ten in t , and the second has degree at most two. The bounded Boolean theorem therefore supplies a fixed-dimensional certificate for any duration. The result does not depend on the first atom being monotone; it may pass through several sign intervals.

34.3 An enormous horizon with a small certificate

The test suite includes

$$A = 10^{50}, \quad T = 10^{100}, \quad p(t) = (t - A)^2.$$

The three difference levels have respectively three, two, and one active sign runs. Their total active run count is six. The supplied generator used 832 distinct exact polynomial evaluations to obtain this certificate; the horizon was not enumerated. The degree-two quartic compiler then verified its generated natural witness against every quadratic residual.

The numbers 10^{50} and 10^{100} are simply concrete test inputs, not new primitive operations in the output equation. The compiled polynomial is the same degree-two template used for small coefficients and horizons; only its free parameter assignment changes.

35 Manuscript 02: Conclusion [02]

A long computation need not be represented by a long explicit trace. For polynomial trajectories, the signs of a guard can be certified by a unique hierarchy of maximal sign intervals of successive finite differences. The hierarchy is finite because degree decreases; its correctness is local because differences control monotonicity; and its uniqueness is global because all runs are maximal and all padding is fixed.

Combined with total canonical arithmetic circuits, this yields an ordinary quartic equation with a unique natural witness and a dimension independent of the run duration. It applies to genuinely nonlinear unitriangular dynamics, not just monotone linear counters. First-exit semantics permits safe projection of time, and a canonical polynomial root bound yields unique specified-input nontermination certificates.

The construction gives a concrete addition to a research program in which ordinary Diophantine reachability is already established: a multiplicity-preserving accelerator with an executable polynomial output. Its exact scope, proof obligations, and exponential boundary are explicit.

Part V

Memory logs, RAM and graph evaluation

Source and scope. [write] This Part is manuscript 03, “Linear-Size Canonical Diophantine Certificates for Random-Access and Graph Computation”: its Sections 2–9 (canonical comparison and zero-test gadgets, a sorting network as a canonical polynomial circuit, the canonical random-access memory theorem, size and witness height, the linear-size compiler by exact large-base products, complete RAM executions, applications to counters, cellular automata, stacks, tapes and an immutable SK/SKI/Iota graph evaluator, and exact counting and first-halting probabilities), and its conclusion. Its conventions are in Section 2, its introduction in Section 3, its boundary theorem in Part IX.

Setting and hypotheses. [write] A log of length L records reads and writes of a random-access memory with natural addresses and values; N is the number of padded records and C the number of comparators of the sorting network. “Canonical certificate” means a single-fold representation (Section 2); “witness height” bounds the magnitude of witness coordinates. The radix packing of Section 40 is the device that Part VI uses for queue words (Lemma 45.1).

36 Canonical arithmetic with quadratic residuals [03]

36.1 Comparison in two equations

The elementary observation doing most of the work is a signed-gap identity.

Lemma 36.1 (Canonical comparison). *For every $x, y \in \mathbb{N}$, the system in $b, d \in \mathbb{N}$*

$$b(b-1) = 0, \tag{139}$$

$$y - x - (2b-1)d - b + 1 = 0 \tag{140}$$

has exactly one solution. It is

$$(b, d) = \begin{cases} (1, y-x), & x \leq y, \\ (0, x-y-1), & x > y. \end{cases} \tag{141}$$

Thus b is the indicator of $x \leq y$, with equality assigned to the first branch, and $0 \leq d \leq \max(x, y)$.

Proof. Equation (139) forces $b = 0$ or $b = 1$. If $b = 1$, (140) becomes $y - x = d$, possible precisely when $x \leq y$, and then $d = y - x$ is forced. If $b = 0$, it becomes $x - y = d + 1$, possible precisely when $x > y$, and then $d = x - y - 1$ is forced. The two cases are disjoint and exhaustive. The stated bound follows in either case. $\square$

The strict offset 1 is important. Without it, a tie could choose either branch. Equally importantly, there is only one slack coordinate: no unused branch slack survives as an arbitrary witness. A naive disjunctive encoding with two unnormalised slacks would generally have many auxiliary solutions.

Example 36.2. For $(x, y) = (3, 7)$, the unique witness is $(b, d) = (1, 4)$. For $(7, 3)$ it is $(0, 3)$, not $(0, 4)$, because the reverse branch uses a strict inequality. For $(3, 3)$ it is $(1, 0)$. The formulas include zero without a separate convention.

36.2 Zero tests and conditional choice

Putting $(x, y) = (\delta, 0)$ in Lemma 36.1 gives the following useful form.

Corollary 36.3 (Canonical zero test). *For $\delta \in \mathbb{N}$, the two equations*

$$z(z - 1) = 0, \quad \delta + (2z - 1)h + z - 1 = 0 \quad (142)$$

have the unique natural solution $(z, h) = (1, 0)$ when $\delta = 0$, and $(z, h) = (0, \delta - 1)$ when $\delta > 0$.

The nonnegativity assumption on δ is part of this corollary. When we use it on an address difference, that assumption will follow from the sorting proof, not from an unproved premise in the polynomial system.

For a Boolean b and natural inputs x, y , the residual

$$u - bx - (1 - b)y = 0 \quad (143)$$

uniquely specifies a natural output. Addition, multiplication, copying, and fixed constants are likewise represented by their defining equations. Boolean AND is multiplication, NOT is $1 - b$, and OR can be represented by $u - b - c + bc = 0$. The latter is nonnegative for Boolean inputs.

Natural truncated subtraction is also canonical without allowing a negative intermediate. Compare x with y using (b, d) , and set

$$u - (1 - b)(d + 1) = 0. \quad (144)$$

Then $u = \max(x - y, 0)$. For fixed $B \geq 2$, Euclidean division is specified by

$$x - Bq - r = 0, \quad r + h + 1 - B = 0, \quad (145)$$

with $q, r, h \in \mathbb{N}$. The second equation says $r < B$ and fixes h ; division with remainder proves uniqueness. No multiplicity is introduced by the bound.

36.3 Composing canonical gadgets

Lemma 36.4 (Acyclic composition). *Suppose a finite directed acyclic circuit is built from gadgets whose output and auxiliary coordinates are uniquely determined by their inputs. Use fresh coordinates for different gadgets and identify only prescribed connecting wires. Then the combined residual system has a unique assignment to every internal coordinate for each external input. Adding predicates on the final outputs either keeps that assignment or removes it; it never creates another.*

Proof. Order the gadgets topologically. The first gadget has fixed inputs, so its output and auxiliaries are forced. The same statement then holds for the next gadget, and induction forces the entire assignment. Conversely, computing the gadgets in that order constructs an assignment satisfying their residuals. Additional final predicates merely test the already determined outputs. $\square$

This elementary lemma is a stronger contract than closure of Diophantine relations under existential quantification. It retains the size of each fiber. It applies to acyclic certificate circuits, not to an arbitrary cyclic system of equations. In the RAM theorem below, temporal memory correctness supplies a separate induction for the apparent dependence of future states on read values.

37 Deterministic sorting as a canonical polynomial circuit [03]

37.1 One four-field compare-exchange

A record has four natural coordinates

$$X = (\alpha, \kappa, w, v), \quad (146)$$

where κ is the sorting key. Given two records X, Y , apply Lemma 36.1 to their keys, obtaining b, d . For each field $f \in \{\alpha, \kappa, w, v\}$ introduce outputs ℓ_f, u_f and require

$$\ell_f - bX_f - (1 - b)Y_f = 0, \quad (147)$$

$$u_f - X_f - Y_f + \ell_f = 0. \quad (148)$$

The first output record has the smaller key, with the left record first at a tie; the second contains the other record. Descending comparators simply interchange the two output ports. No additional variable is necessary for that change of wiring.

Proposition 37.1 (Comparator cost). *The four-field comparator introduces exactly ten natural variables and ten residual equations, all of degree at most two. Its complete auxiliary and output assignment is unique for every pair of input records. Its outputs are exactly those records, not merely records with the same keys.*

Proof. There are two variables and two equations for (b, d) , and eight output variables with eight equations for the fields. The key comparison forces (b, d) ; (147) forces each lower field, and (148) forces each upper field. In either Boolean case these are the two original records in the required order. $\square$

Consequently, a fixed sorting network with C comparators becomes a canonical arithmetic circuit with exactly $10C$ new coordinates and $10C$ residuals. Unlike existential permutation routing, there is no choice of internal switch settings once the input records are fixed.

37.2 A completely explicit network

For $N = 2^k$, use the iterative bitonic network whose complete generator is

```

for block in 2, 4, ..., N:
    for stride in block/2, block/4, ..., 1:
        for i in 0, ..., N-1:
            j = i xor stride
            if j > i:
                compare wires i and j
                ascending exactly when (i bitwise-and block) == 0

```

All operations in this generator act on fixed wire indices at compilation time. No bitwise arithmetic is needed inside the Diophantine equation.

For completeness, here is the mathematical sorting argument behind the network. A finite sequence is bitonic when some cyclic rotation is nondecreasing and then nonincreasing. Comparing positions separated by half the length, and sending the smaller entries to the first half, is a bitonic split. Both resulting halves are bitonic, and every entry of the first half is at most every entry of the second half.

To verify this split property rigorously, threshold all entries at an arbitrary value. A bitonic zero-one sequence has a single circular interval of ones. If that interval has length at most half the total length, no opposite pair contains two ones, so the lower half is all zero; the upper half has a circular interval

of ones. If the interval is longer, every opposite pair contains a one, so the upper half is all one and the lower half again has a circular interval of ones. This proves the split property for every threshold. Comparison commutes with thresholding. Any failure of separation or bitonicity in the original total order would appear at a threshold between offending values. Thus the split property holds in the original order as well.

Recursively splitting each half sorts a bitonic sequence. To sort an arbitrary sequence, sort one half ascending and the other descending, producing a bitonic sequence, and merge. The displayed iterative generator performs precisely these merges on successively doubled blocks, with the alternating block orientations needed for the next merge. Its final block is ascending.

Lemma 37.2 (Exact bitonic count). *For $N = 2^k$, the displayed network sorts arbitrary totally ordered keys and has*

$$C_N = \frac{N}{4}k(k+1) \quad (149)$$

comparators, with $C_1 = 0$.

Proof. Correctness was established above. At block size 2^r there are r stride layers, each containing $N/2$ disjoint comparisons. Therefore $C_N = (N/2) \sum_{r=1}^k r = Nk(k+1)/4$. $\square$

The proof also explains why duplicate keys cause no problem. The comparator's left-biased tie convention makes its internal variables unique, while sorting correctness only requires a nondecreasing output. In the memory application, timestamps will in fact make all keys distinct.

38 The canonical random-access memory theorem [03]

38.1 The semantic relation

A finite chronological log is

$$E = ((a_t, w_t, v_t))_{0 \leq t < L}, \quad a_t, v_t \in \mathbb{N}, \quad w_t \in \{0, 1\}. \quad (150)$$

The memory is initially zero at every address. A write event ($w_t = 1$) stores v_t at address a_t . A read event ($w_t = 0$) asserts that the current value at a_t is v_t and does not change memory. Write values are unrestricted natural numbers; reads before any write must return zero. Let $\text{Valid}_L(E)$ mean that all events obey this semantics. Non-Boolean write flags make the log invalid.

We now construct one polynomial for each L , treating the $3L$ event coordinates as free parameters. The polynomial's coefficients do not depend on the numerical values of those parameters.

38.2 Padding and chronological keys

Set

$$N = 2^{\lceil \log_2 \max(1, L) \rceil}, \quad B = N + 1. \quad (151)$$

For each genuine event introduce

$$\alpha_t = a_t + 1, \quad \kappa_t = B\alpha_t + t, \quad (152)$$

and the record $(\alpha_t, \kappa_t, w_t, v_t)$. The two definitions are enforced by linear residuals; also include $w_t(w_t - 1) = 0$. For each $L \leq t < N$, append the fixed dummy record

$$(\alpha_t, \kappa_t, w_t, v_t) = (0, t, 1, 0). \quad (153)$$

These records are constants, not additional existential choices. They are zero writes at a reserved shifted address. No genuine event has that shifted address, so padding cannot affect the original memory.

Lemma 38.1 (Key order). *The keys of all N records are distinct. Sorting by κ first sorts by shifted address α and, within an address, by original timestamp t . All dummy records precede all genuine records.*

Proof. All timestamps lie in $[0, N - 1]$. If $\alpha < \alpha'$, then

$$B\alpha + t \leq B\alpha + N - 1 < B(\alpha + 1) \leq B\alpha' + t'.$$

At equal addresses, the key comparison is the timestamp comparison. Distinct events have distinct timestamps, proving distinctness. Dummy addresses are zero and genuine shifted addresses are positive. $\square$

Apply any fixed sorting network on N wires using the four-field comparator. Write the sorted records as

$$(\alpha_j^*, \kappa_j^*, w_j^*, v_j^*) \quad (0 \leq j < N). \quad (154)$$

By the preceding lemmas, all these records and all internal sorting coordinates are uniquely determined by E .

38.3 Four equations per adjacent pair

For the first sorted record require

$$(1 - w_0^*)v_0^* = 0. \quad (155)$$

For each $1 \leq j < N$, the difference $\delta_j = \alpha_j^* - \alpha_{j-1}^*$ is nonnegative by Lemma 38.1. Introduce three natural variables z_j, h_j, u_j , and the following four residuals:

$$z_j(z_j - 1) = 0, \quad (156)$$

$$\alpha_j^* - \alpha_{j-1}^* + (2z_j - 1)h_j + z_j - 1 = 0, \quad (157)$$

$$u_j - z_j v_{j-1}^* = 0, \quad (158)$$

$$(1 - w_j^*)(v_j^* - u_j) = 0. \quad (159)$$

The first two are the canonical zero test of δ_j . Consequently $z_j = 1$ exactly when the addresses agree; h_j is the unique corresponding gap witness. The third equation supplies the previous value when the address is unchanged, and zero otherwise. The fourth checks a read but puts no condition on the value of a write.

Introducing u_j is necessary for our degree bound: writing $(1 - w_j^*)(v_j^* - z_j v_{j-1}^*) = 0$ directly would create a cubic residual and a sixth-degree square. The new coordinate is uniquely defined and therefore does not spoil parsimony.

Lemma 38.2 (Sorted memory criterion). *For sorted records arising from (152)–(153), (155)–(159) have a natural solution exactly when the original log is valid. When a solution exists, all their auxiliary coordinates are unique.*

Proof. Consider a maximal consecutive block of one shifted address. At its first record, the value supplied as the predecessor is zero: this follows either from (155), or from $z_j = 0$ and (158). Thus its first read must return zero, while its first write may set any value.

At every subsequent record of that block, $z_j = 1$ and $u_j = v_{j-1}^*$. A read is therefore required to agree with the preceding record's value; a write may replace it. Inductively, the value in every record is the memory contents just after that event. A read leaves that contents unchanged, and a write changes it to its stated value. Because timestamp order is preserved within the block, this is precisely the chronological memory semantics at that address.

Different addresses do not interact, and dummy zero writes are harmless. This proves equivalence with the original log. For uniqueness, the sorted addresses and values fix each z_j, h_j by Corollary 36.3, then fix u_j by (158). The read equations are only tests of those forced values. $\square$

38.4 The main theorem and its exact constants

Theorem 38.3 (Canonical quartic memory compiler). *Fix $L \geq 0$ and N as in (151). Let a sorting network on N wires have C comparators. One can explicitly construct*

$$P_L \in \mathbb{Z}[a_0, w_0, v_0, \dots, a_{L-1}, w_{L-1}, v_{L-1}, \boldsymbol{\eta}]$$

of degree at most four such that, for every natural event-parameter tuple E ,

$$\#\{\boldsymbol{\eta} \in \mathbb{N}^W : P_L(E, \boldsymbol{\eta}) = 0\} = \begin{cases} 1, & \text{Valid}_L(E), \\ 0, & \text{otherwise.} \end{cases} \quad (160)$$

The construction has exactly

$$W = 10C + 2L + 3(N - 1), \quad (161)$$

$$R = 10C + 3L + 4(N - 1) + 1 \quad (162)$$

auxiliary variables and residuals, before omission of identically zero residuals. It uses no exponentiation, divisibility predicate, bounded universal quantifier, or appeal to MRDP.

Proof. Form the input residuals in (152) and the write-bit residuals, the C comparator systems, and the final memory residuals. Let P_L be their sum of squares. Every residual is quadratic or linear, proving the degree bound.

If $P_L = 0$, every residual vanishes. Write flags are Boolean. The input coordinates force the shifted addresses and keys; the comparator lemma and network correctness force the sorted permutation of complete records. The sorted memory criterion then proves $\text{Valid}_L(E)$. This proves soundness.

If the log is valid, compute the shifted records, run the deterministic network, and choose every comparator and adjacent-pair variable by its explicit formula. All residuals vanish by Lemma 38.2, giving a root. This proves completeness. The same sequence of forcing arguments shows that any root must be the one just constructed, proving uniqueness.

There are $2L$ input auxiliaries and $3L$ input residuals. Each comparator contributes ten of each. Each of the $N - 1$ adjacent pairs contributes three auxiliaries and four residuals, while the first-record check contributes one residual and no auxiliary. Adding these quantities gives (161)–(162). $\square$

Remark 38.4 (The empty log). For $L = 0$, take $N = 1$. The sole record is the constant dummy record $(0, 0, 1, 0)$. There are no witnesses and just the identically zero first-record residual. The empty tuple is the unique witness in $\mathbb{N}^0$. Thus the theorem includes the empty computation without an exceptional existential parameter.

Remark 38.5 (Natural versus unrestricted integer witnesses). The theorem does not assert uniqueness, or even the same relation, when the witnesses are allowed to be arbitrary integers. A negative signed-gap witness would destroy the meaning of comparison. Replacing natural variables by sums of four integer squares restores existential representability, but generally creates many representations and destroys the stated bijection. A formal implementation must retain the natural-number domain explicitly.

39 Size, height, and the cost of canonical routing [03]

39.1 Explicit and asymptotic size

Substituting (149) into Theorem 38.3 gives

$$W_L = \frac{5N}{2}k(k+1) + 2L + 3(N-1), \quad (163)$$

$$R_L = \frac{5N}{2}k(k+1) + 3L + 4(N-1) + 1. \quad (164)$$

These are scalar-coordinate and scalar-equation counts. For $L \geq 1$ we have $L \leq N < 2L$ unless $L = 1$, where $N = L$. Thus the explicit construction is $O(L \log^2(L+2))$. For any sorting network with C comparators the simpler bounds $W_L \leq 10C + 5N$ and $R_L \leq 10C + 7N$ hold.

The theorem of Ajtai, Komlós, and Szemerédi supplies sorting networks of size $C = O(N \log N)$ [29]. Substituting those networks in the *same* canonical comparator construction proves the following statement.

Corollary 39.1. *There is an effectively constructible family of quartic natural-number polynomials representing valid length- L memory logs with exactly one auxiliary witness and $O(L \log(L+2))$ auxiliary coordinates and residuals.*

This is an asymptotic existence/construction result using a known sorting network theorem, not a practical improvement implemented in the companion code. The simple bitonic network has transparent constants and a short checker.

Every displayed memory residual has at most eight monomials after collecting terms. Therefore the expansion of P_L has at most $64R_L$ monomial *contributions*, and hence at most that many distinct nonzero monomials. The generous constant includes repeated or constant input wires. Expanding the sum of squares does not entail an exponential expression-size explosion. The residual coefficients have absolute value at most $2(N+1)$; a safe bound for the absolute coefficients of the expanded polynomial is $256R_L(N+1)^2$. Variable names or indices themselves require logarithmically many bits in a serialized description. None of these statements identifies a scalar count with the bit length of a complete file.

39.2 Witness height

Theorem 39.2 (Height bound). *For a valid log let*

$$A = \max(\{a_t : 0 \leq t < L\} \cup \{0\}), \quad V = \max(\{v_t : 0 \leq t < L\} \cup \{0\}).$$

Every coordinate of its unique auxiliary witness is at most

$$H = \max\{1, V, (N+1)(A+1) + N-1\}. \quad (165)$$

Consequently, if the addresses and values have at most b binary digits, every auxiliary coordinate has $b + O(\log(N+2))$ digits, with an absolute implied constant.

Proof. Shifted addresses are at most $A+1$ and keys at most $(N+1)(A+1) + N-1$. Every sorting output is a field of an input record, so it obeys the same bound, as do Boolean flags. The comparator gap is bounded by the larger input key by Lemma 36.1. In an adjacent-pair test, $\alpha_j^* - \alpha_{j-1}^* \leq A+1$; its gap-minus-one coordinate is at most A , and its previous-value coordinate is either zero or a log value. These cover all auxiliary coordinates. Taking binary lengths proves the last claim, also when the maximum input value is zero. $\square$

The witness can be generated by the very comparisons and substitutions used in the proof. In particular, its generation and checking are polynomial-time in the input log's bit length and the displayed certificate size. Address *magnitude* does not enter the number of coordinates; address *bit length* necessarily enters the cost of reading and manipulating a coordinate.

This distinction becomes essential when certificates describe arithmetic programs. The T -step program

$$x_0 = 2, \quad x_{t+1} = x_t^2$$

has $x_T = 2^{2^T}$. Its local quadratic description has $O(T)$ coordinates, but the last coordinate needs $2^T + 1$ binary digits. Thus a small arithmetic certificate, even a unique one, does not by itself supply an NP witness of polynomial bit length. Memory-log height bounds and instruction-level growth bounds must be stated separately.

39.3 What is optimal, and what is not

Proposition 39.3 (Comparison-network lower bound). *An oblivious comparison network that sorts arbitrary N distinct keys has at least $\lceil \log_2(N!) \rceil = \Omega(N \log N)$ comparators. In particular, the comparison order in Corollary 39.1 is optimal within this sorting-network architecture.*

Proof. Give the input positions every possible permutation of N distinct ranks. The sorting process must route those positions into $N!$ different output permutations. A sequence of C binary comparison outcomes determines at most 2^C routing patterns. Therefore $2^C \geq N!$. For example, $N! \geq (N/2)^{N/2}$ for $N \geq 2$, giving the asymptotic bound. The memory-key format does not evade this argument: choosing distinct addresses in an arbitrary order realizes every rank permutation of the keys. $\square$

This is not a lower bound for arbitrary Diophantine encodings. A different certificate architecture could avoid explicit comparison sorting. Nor does it say that Batchier's extra logarithmic factor is necessary.

40 A linear-size alternative: exact large-base products [03]

The comparison lower bound is not a lower bound on Diophantine certificates. We now remove the sorting network entirely. The key is to certify a proposed sorted list by one exact product identity, at a uniquely determined integer base large enough to prevent every possible carry collision. Unlike a random fingerprint over a finite field, this check has no soundness error. It exchanges small witness height for a linear number of scalar coordinates.

40.1 A deterministic multiset lemma

Lemma 40.1 (Carry-free product fingerprint). *Let $C \geq 2$, $L \geq 1$, $\beta = C^L + 1$, and let $c_1, \dots, c_L, d_1, \dots, d_L$ belong to $\{0, \dots, C-1\}$. Then*

$$\prod_{i=1}^L (\beta + c_i) = \prod_{i=1}^L (\beta + d_i) \tag{166}$$

holds if and only if the two lists have the same multiset of entries.

Proof. Form $f(X) = \prod_i (X + c_i)$ and $g(X) = \prod_i (X + d_i)$ in $\mathbb{Z}[X]$. Their coefficients are nonnegative, and the sum of the coefficients of f is

$$f(1) = \prod_i (1 + c_i) \leq C^L < \beta.$$

Thus every coefficient of f is a base- β digit; the same holds for g . Equality $f(\beta) = g(\beta)$ forces equality of all digits, by repeated Euclidean division by β . Hence $f = g$ as polynomials.

To recover the multiset, evaluate at $X = -c_1$. Some factor $-c_1 + d_j$ must be zero, since a product of integers is zero only if a factor is zero. Cancel $X + c_1$ from the two polynomial factorizations and proceed by induction. Cancellation is valid in the integral domain $\mathbb{Z}[X]$. This argument includes repeated entries and zero. Conversely, equal multisets plainly give equal products. $\square$

The carry bound, rather than any unproved collision-resistance assumption, is the entire reason this evaluation at one point is exact. Evaluating at a small fixed integer without the bound would not justify the conclusion. The elementary base-expansion argument is not itself claimed to be a new algebraic principle; the point is its fully canonical use in the memory compiler.

40.2 Packing complete records

For $L \geq 1$, use the genuine records

$$X_t = (\alpha_t, \kappa_t, w_t, v_t), \quad \alpha_t = a_t + 1, \quad \kappa_t = (L + 1)\alpha_t + t \quad (0 \leq t < L).$$

No padding is needed. Define uniquely

$$p_0 = 0, \quad p_{t+1} = p_t + \alpha_t + \kappa_t + w_t + v_t, \quad D = p_L + 2. \quad (167)$$

Each input field is strictly less than D . Introduce L proposed output records $X'_j = (\alpha'_j, \kappa'_j, w'_j, v'_j)$ as witness coordinates. For each of their four fields f add a natural slack s and the equation

$$f + s + 1 - D = 0. \quad (168)$$

This both asserts $f < D$ and fixes the bound witness uniquely.

The radix- D code of a record is

$$\rho(X) = \alpha + D\kappa + D^2w + D^3v. \quad (169)$$

For four digits in $[0, D - 1]$, the code belongs to $[0, D^4 - 1]$ and is injective in the complete tuple, by uniqueness of base- D expansion. Represent it with three quadratic gates, not the high-degree expression in (169):

$$h_1 - vD - w = 0, \quad h_2 - h_1D - \kappa = 0, \quad \rho - h_2D - \alpha = 0. \quad (170)$$

Do this for both input and proposed output records.

Introduce $D_2, C, q_1, \dots, q_L, \beta$ and impose

$$D_2 = D^2, \quad C = D_2^2, \quad q_0 = 1, \quad q_i = q_{i-1}C \quad (1 \leq i \leq L), \quad \beta = q_L + 1. \quad (171)$$

These quadratic equations force $C = D^4$ and $\beta = C^L + 1$ without putting exponentiation into the polynomial language. The chain length depends on the fixed parameter L , not an exponent variable. Binary powering could shorten the chain, but the displayed version gives particularly simple exact counts.

Let $A_0 = B_0 = 1$ be constants. The two product chains are

$$A_{i+1} = A_i(\beta + \rho(X_i)), \quad B_{i+1} = B_i(\beta + \rho(X'_i)) \quad (0 \leq i < L). \quad (172)$$

Impose $A_L - B_L = 0$. Every equation remains quadratic. Finally require the proposed keys to be nondecreasing by introducing one natural gap per adjacent pair:

$$\kappa'_j - \kappa'_{j-1} - g_j = 0 \quad (1 \leq j < L). \quad (173)$$

The product lemma and radix injectivity force the proposed records to be a permutation of the complete input records. The keys are distinct, so the nondecreasing order fixes that permutation uniquely. All gap variables are then uniquely determined.

Attach the first-record and adjacent-record memory equations from Section 38, now on these L sorted records. Their address differences are nonnegative by the same key-order argument, and the previous proof applies unchanged.

40.3 The linear compiler theorem

Theorem 40.2 (Linear-size canonical quartic memory). *For every $L \geq 0$ there is an explicit polynomial*

$$P_L^{\text{lin}} \in \mathbb{Z}[X_1, \dots, X_{3L}, Y_1, \dots, Y_{24L}]$$

of degree at most four such that each valid length- L memory log has exactly one natural witness, and each invalid log has none. It is the sum of squares of exactly $22L + 1$ quadratic residuals, each with at most eight monomials. Its expanded sparse polynomial therefore has $O(L)$ monomial contributions. The polynomial and its coefficients depend on L , not on the numerical log.

Proof. For $L = 0$, take no witnesses and the single zero residual. Suppose $L \geq 1$. The input definitions and write-bit constraints force precisely the declared records and Boolean write flags. The prefix chain uniquely fixes D , so the input fields satisfy the digit bounds. The explicit output bound equations force every proposed output field into the same digit range. The Horner and power chains fix all codes and β . The product chains and their final equality, by Lemma 40.1, force equality of code multisets. By radix injectivity this is equality of complete record multisets. Distinct keys and (173) therefore force the single correctly sorted output list. The sorted memory criterion proves soundness.

For a valid log, take that sorted list, use its uniquely determined digit slacks, and compute every chain as displayed. Multiset equality verifies the product identity, and memory validity verifies the final checks. This gives a root. Conversely, in any root the preceding argument forces the output list; all remaining coordinates are then forced by their defining equations, bound slacks, key gaps, and canonical memory tests. Thus there is exactly one root.

The complete accounting is as follows. “Output fields” are existential coordinates whose semantics are forced globally by the product and order checks; they do not need separate defining residuals.

Component	Witnesses	Residuals
Input shifted addresses, keys, write bits	$2L$	$3L$
Prefix sums and D	$L + 1$	$L + 1$
Proposed output fields	$4L$	0
Output digit-bound slacks	$4L$	$4L$
Input and output Horner chains	$6L$	$6L$
$D_2, C, q_1, \dots, q_L, \beta$	$L + 3$	$L + 3$
Two product chains and final equality	$2L$	$2L + 1$
Adjacent key gaps	$L - 1$	$L - 1$
Memory checks	$3(L - 1)$	$4(L - 1) + 1$
Total	$24L$	$22L + 1$

Every residual has degree at most two and a bounded number of terms. The sum-of-squares argument gives the degree and sparse-size claims. All generated constants are fixed small integers or functions of L such as timestamps and $L + 1$; the large base itself is a witness fixed by quadratic equations. $\square$

Theorem 40.3 (Height cost of linear size). *For a valid nonempty log, let D have the value in (167). Every witness of the linear compiler is at most*

$$H_{\text{lin}} = 2^L D^{4L^2}. \quad (174)$$

If the input addresses and values have at most b bits, each witness has $O(L^2(b + \log(L + 2)))$ bits. Generation and verification of the full certificate are polynomial-time in the bit length of the finite log and in L .

Proof. All fields, digit slacks, prefix sums, key gaps, and local memory coordinates are at most D . Horner coordinates are below $C = D^4$; the powers are at most C^L , and $\beta = C^L + 1$. Each factor in a product chain is at most $C^L + C \leq 2C^L$. Every prefix product is therefore at most $(2C^L)^L = 2^L D^{4L^2}$, which also dominates the other coordinates. Further, $D \leq 2 + 4LM$, where M is the largest input-record field, and $\log_2 M = O(b + \log(L + 2))$. Taking logarithms gives the bit-length bound. The chains have linear length, and all their integer operands have the polynomial bit lengths just bounded. Exact addition, multiplication, comparison, and division by fixed bases can thus generate and verify the witness in polynomial time. $\square$

There is a genuine tradeoff, not a free improvement in every resource:

Backend	Scalar witnesses	Maximum witness bit length
Explicit bitonic network	$O(L \log^2(L + 2))$	$O(b + \log(L + 2))$
Optimal sorting network	$O(L \log(L + 2))$	$O(b + \log(L + 2))$
Exact large-base products	$24L$	$O(L^2(b + \log(L + 2)))$

The linear construction is not a bounded-word online memory checker or a finite-field proof system, and makes no claim to evade lower bounds for those different models. It is an offline exact natural-number certificate whose large coordinates encode the entire multiset check without carries. For $L = 4$ the simple linear version has 96 witnesses versus the network's 77; for $L = 16$ it has 384 versus 877. Thus even the scalar improvement is asymptotic, not a claim of the smallest constant for every short trace.

Corollary 40.4 (Two-backend compilation principle). *Any composition argument using only the memory checker's existence, uniqueness, and decoding contracts may substitute the linear backend for the network backend. The semantic bijection is unchanged. Only the coordinate count and the height estimate change.*

Proof. Both theorems characterize the identical relation on the same chronological log, with a singleton fiber on every valid log. Replace the unique witness of one checker by the unique witness of the other, leaving the semantic log and all other coordinates unchanged. $\square$

41 From memory logs to complete RAM executions [03]

41.1 A precise local interface

A *normalized RAM* has a fixed finite tuple of natural registers and a finite control state. One time step has an explicit choice label c from a fixed finite alphabet (a singleton for a deterministic machine). It computes, from its current local state s and c , a request

$$(a, w, d) = \text{request}(s, c), \quad w \in \{0, 1\}.$$

The request is computed *before* the response is used. Here $d = 0$ on a read and d is the value to store on a write. Let r be the response, canonically zero on a write. The event submitted to the memory checker is

$$(a, w, v), \quad v = d + r, \quad (175)$$

and the next local state is $\text{update}(s, c, r)$. Include the residuals

$$(1 - w)d = 0, \quad wr = 0, \quad v - d - r = 0. \quad (176)$$

On a read the memory checker forces r to be the old value; on a write the local normalization forces $r = 0$. This convention removes an otherwise unused response variable on write steps.

The functions `request` and `update` must have canonical acyclic quadratic implementations, with a bounded number of coordinates per step for the fixed program. Addition, multiplication, comparison, truncated subtraction, Boolean logic, and division by a fixed positive base satisfy this requirement by Section 36. A fixed finite control table is compiled by equality tests and Boolean selection among these operations. Compute every branch using total operations and select the chosen outputs; then even an inactive branch has uniquely determined internal data. An alternative guarded encoding must explicitly zero or otherwise normalize inactive coordinates. Merely multiplying all its constraints by a branch bit would leave arbitrary witnesses.

A fixed finite tuple of possible branch labels can be validated by comparing c with its allowed labels and testing the resulting Boolean disjunction. Exactly one label is retained after halting. Conditional branches of a deterministic program are computed by comparisons, not represented by free choice labels.

We reserve physical address zero for a no-operation event $(0, 1, 0)$. All user addresses are shifted by one before they enter this physical interface. A local step not needing memory uses that event. This reserve is distinct from, and compatible with, the memory compiler's own shifted dummy records. A finitely supported initial memory is supplied by I prescribed initialization writes. Their addresses and values are fixed by the input, not arbitrary witness choices. The initial local state is likewise fixed by the input.

41.2 Halting without padding multiplicity

After the first halt, leave the local state unchanged, force the choice label to zero, set the response to zero, and submit the reserved no-operation write. Thus a run that halts before the horizon has one, not many, padded extensions. The accepting and rejecting halted states are absorbing. Acceptance by time T is tested on the final control state after T steps. A run already halted at time zero is included.

Theorem 41.1 (Parsimonious RAM compilation). *Fix a normalized RAM program as above and a horizon T . There is an effectively constructed integer polynomial $Q_{P,T}(x, Y)$ of degree at most four whose natural-number solutions Y , for each fixed well-formed input x , are in bijection with the labelled executions of P that accept by time T . The bijection is projection to the chronological local states and choice labels; its inverse is canonical certificate generation.*

If the initialization has I writes, the linear backend gives $O_P(T + I)$ coordinates and residuals. The bitonic construction uses $O_P((T + I) \log^2(T + I + 2))$ coordinates and quadratic residuals, counting the trace coordinates as witnesses. An optimal sorting network improves this to $O_P((T + I) \log(T + I + 2))$. In particular, a deterministic program has either no root or exactly one root at each fixed input and horizon.

Proof. Introduce local state, label, request, and response coordinates for each step. Add the canonical request and update circuits, normalization (176), the initial-state constraints, and the final acceptance constraint. Join the I initialization events and the T step events into one chronological log, and attach

the memory checker of Theorem 38.3. Use fresh internal coordinates for every circuit and for the checker. The sum of the squares of all residuals has degree at most four.

Consider any root. The local circuits describe valid instruction steps for the values supplied to them. The memory theorem proves that the entire event log obeys sequential zero-initialized memory semantics, including initialization. Start at the fixed initial state and proceed in chronological order. The choice label and current state determine the request before any read. On a read, sequential memory fixes the response uniquely; on a write, the response is zero by (176). The update circuit then fixes the next state. This proves that the projected data form a genuine labelled run, not a self-consistent but causally impossible circular solution. Its final control state is accepting.

Conversely, take any labelled accepting run. Extend it by the prescribed absorbing steps to length T . Its chronological events form a valid memory log. Assign every local auxiliary using its canonical circuit, then the unique memory witness. All residuals vanish.

Finally, fix the semantic run. Its local coordinates and event values are fixed. Lemma 36.4 fixes each local circuit's auxiliaries, and Theorem 38.3 fixes the memory auxiliaries. Therefore the constructed root is the only one projecting to that run. Distinct labelled runs project differently, proving the bijection. There are $O_P(T + I)$ local coordinates and residuals, in addition to the memory checker on $T + I$ events. The size estimates follow; substituting Theorem 40.2 gives the linear bound. A deterministic program has a unique length- T padded run, so its accepting root set has size zero or one. $\square$

The instruction interface is a hypothesis with a concrete implementation method, not an appeal to MRDP for arbitrary local predicates. For a family of programs the constants depend on their finite instruction circuits. A fixed interpreter can instead make program data part of the initialized memory, but one must count that data and its interpreted runtime.

Corollary 41.2 (Guarded resource restrictions). *Imposing additional Boolean tests on the uniquely decoded trace preserves parsimony. Imposing a coordinate bound $y \leq H$ using $y + z - H = 0$, $z \in \mathbb{N}$, also preserves it, because the new slack is forced.*

Proof. The tests can only remove decoded runs, and the bound slack is necessarily $z = H - y$. Neither can split a surviving root into several roots. $\square$

42 Applications to computational substrates [03]

42.1 Counter machines and bounded cellular computation

For a fixed counter machine, increment is linear and a zero/decrement branch uses the canonical zero test and truncated subtraction. Control is finite. There is no random-access memory consistency problem if all counters are explicit local registers. Unrolling T steps therefore yields $O_P(T)$ quadratic residuals and a parsimonious quartic equation. The memory theorem is valuable precisely when replacing a large register array by actual indirect addressing; it is not a necessary overhead for every substrate.

A finite-alphabet cellular automaton has a similarly direct description. Fix a finite space–time region, its boundary convention (for example a periodic finite lattice), and an initial configuration. Use one-hot Boolean coordinates for each cell state. Enforce their sum to be one. For each output symbol, express its local transition rule as the sum of the products of the appropriate input indicators. The neighborhood size and alphabet are fixed, so each product has a fixed-length chain of quadratic multiplication gates. The products and sums are unique on valid one-hot inputs. Temporal induction fixes the next layer. Thus a region with V cell updates has an $O(V)$ canonical quartic certificate. For a nondeterministic rule, retain an explicit permitted local choice label; solutions then count labelled space–time histories, not only distinct final configurations.

This construction does not represent an infinite configuration by one finite list without specifying a coding. Infinite-lattice claims require a finite causal cone or an explicit finite-support convention.

42.2 Stacks and Turing tapes

Let the stack alphabet have b symbols and set $B = b + 1$. Encode symbols by $1, \dots, b$, with zero representing the empty stack. Pushing digit d onto stack code n produces $Bn + d$. Popping a nonempty valid stack uses the unique quotient and remainder in (145); its remainder is the top symbol and its quotient the residual stack. A zero test handles the empty stack with a fixed blank-symbol convention. Nonzero digits avoid leading-zero ambiguity. When checking externally supplied stack codes, validate their finite symbol sequence or assume a well-formed initial encoding; a random natural number need not encode a valid stack in this scheme.

Two stacks and a scanned symbol describe a Turing tape: moving right pushes the old scanned symbol onto the left stack and pops the right stack, with blank on an empty stack; moving left is symmetric. Therefore a fixed Turing machine has a canonical bounded-horizon quartic description by the local-gadget method. After T steps only $O(T)$ symbols beyond the input can have been visited, so these radix codes have $O(T + n)$ digits for input length n . Alternatively, representing stacks and tape cells in RAM uses the memory theorem without large radix codes. These are two resource tradeoffs, not distinct proofs of computability equivalence.

42.3 An immutable graph evaluator for SK, SKI, and Iota

The following concrete substrate is close to ProveIt's combinatory-logic work, but its evaluation strategy must be specified. An immutable directed acyclic graph has leaf nodes S, K, I, ι and binary application nodes $\text{app}(p, q)$. A fresh application node points only to already allocated nodes. Number the initial nodes densely, including the four reserved leaf nodes, and allocate subsequent nodes at successive fresh identifiers. This is the input representation used in the bit bounds below. A runtime state consists of a focus pointer and a stack of argument pointers. The stack's first item is the next argument to apply.

If the focus is $\text{app}(p, q)$, push q and set the focus to p . For a leaf with sufficient arguments, perform one of the following actions, leaving any remaining argument stack unchanged:

$$\begin{aligned} I; [x, \dots] &\mapsto x; [\dots], \\ K; [x, y, \dots] &\mapsto x; [\dots], \\ S; [x, y, z, \dots] &\mapsto \text{app}(\text{app}(x, z), \text{app}(y, z)); [\dots], \\ \iota; [x, \dots] &\mapsto \text{app}(\text{app}(x, S), K); [\dots]. \end{aligned}$$

The S action allocates three application nodes, and the Iota action two. If there are too few arguments, the machine is in weak-head normal form and halts. There are no reductions inside arguments before they become a focus.

Proposition 42.1 (Unfolding and immutable sharing). *Unfold the finite acyclic heap into ordinary combinator terms and apply the focus term to the argument stack in order. An application-navigation step leaves this term unchanged. Each leaf action realizes exactly the corresponding head contraction of SKI/Iota. The evaluator is deterministic. If the initial heap has n nodes, after m navigation or leaf actions it has at most $n + 3m$ nodes and can be implemented by $O(n + m)$ fixed-size RAM microsteps, including initialization.*

Proof. Acyclicity gives an unambiguous recursively defined unfolding of each pointer. For navigation, unfolding $\text{app}(p, q)$ with residual arguments is precisely unfolding p with q pushed in front. The I, K, S, ι identities are exactly the displayed rules after replacing every pointer by its unfolding. In the S

case both occurrences of z refer to the same immutable node, but their unfoldings are identical copies of the same term. No update through one occurrence changes the other. Fresh nodes point to older nodes, preserving acyclicity and the meaning of every old pointer.

Node tags and two pointer fields occupy a fixed number of memory words. Push/pop and each leaf action require a bounded number of field accesses, register operations, and allocations. Distinct affine address ranges encode node fields and stack slots without collision. The stack length grows by at most one per action. Initialization is linear in n . The node count follows because no action allocates more than three nodes. Determinism follows from inspection of the focus tag and the available stack length. $\square$

Corollary 42.2 (Canonical graph-evaluation certificates). *For a well-formed size- n initial combinator DAG and a budget of m spine actions, reaching weak-head normal form within that budget has a quartic natural-number certificate with a unique witness and $O((n + m) \log^2(n + m + 2))$ coordinates. Using optimal sorting networks replaces the squared logarithm by one logarithm. For this pointer-only evaluator, the certificate coordinates have $O(\log(n + m + 2))$ bits. Alternatively, the linear backend uses $O(n + m)$ coordinates with $O((n + m)^2 \log(n + m + 2))$ bits per coordinate.*

Proof. Choose the fixed microstep implementation just described, with a bounded number of phases per spine action. Insert an action counter so that the budget is tested on completed spine actions, rather than accepting a partial contraction. Pad halted states canonically. Apply Theorem 41.1 with a constant-factor microstep horizon large enough for the initialization and m actions. Pointer identifiers, stack positions, control values, allocation counters, and addresses are bounded by $O(n + m + 1)$ when nodes are given dense identifiers. Its local canonical circuits have the same order of magnitude for intermediate values. Theorem 39.2 bounds the memory-checker coordinates by $O((n + m + 1)^2)$, which gives the claimed bit length. Substituting the linear backend and Theorem 40.3 gives the alternative bounds. $\square$

Here m counts navigation *and* contraction actions. It is not merely the number of source reductions. Well-formedness can be checked separately in linear-size canonical circuits by verifying tags and that each edge points to an earlier identifier. The certificate does not expand shared subterms: the acyclic family $D_0 = K$, $D_{j+1} = \text{app}(D_j, D_j)$ has only $j + 1$ graph nodes but 2^j leaves in its unfolded term. Pointer sharing can therefore be meaningful even before any reduction occurs.

ProveIt's existing development uses context-closed weak beta and combinator relations, while its cited Coq equivalence uses a different call-by-value lambda model [4]. The strategy above is a particular deterministic weak-head graph evaluator. The proof establishes its unfolding semantics; it does not silently identify those different evaluation relations, prove strong normalization, or complete the repository's outstanding exact-semantics bridge. Likewise, injectivity of a source compiler and forward simulation alone do not imply a bijection on complete target execution histories.

43 Exact counting and first-halting probabilities [03]

43.1 Counting executions, not certificate accidents

Let $\text{AccPaths}_{P,T}(x)$ be the finite set of labelled accepting runs of the normalized machine, with the unique halted padding included. The RAM theorem states the exact equality

$$\#\{Y \in \mathbb{N}^{s(P,T)} : Q_{P,T}(x, Y) = 0\} = \#\text{AccPaths}_{P,T}(x). \quad (177)$$

Finiteness follows from the finite branch alphabet and the fixed horizon, although the coordinates themselves may be large. This equality would generally fail for an existential encoding with arbitrary inactive slacks, arbitrary permutation routes, or unconstrained choices after halting.

The same idea yields a useful bounded counting statement. Define the *bounded quartic counting problem* to take a sparse integer polynomial of degree at most four and a binary-encoded bound for each natural variable, and to count its roots inside that box.

Proposition 43.1 (Bounded counting completeness). *Bounded quartic counting is #P-complete under parsimonious polynomial-time reductions. Hardness already holds for polynomials that are sums of squares of quadratic residuals with finite natural root sets.*

Proof. Membership is immediate from the definition of #P: guess the fixed binary representation of each bounded coordinate, reject out-of-range strings, and evaluate the sparse polynomial using exact integer arithmetic. The total number of guessed bits is the sum of the input bound lengths. Degree four ensures that evaluation and its intermediate bit lengths are polynomial in the instance size. Fixed-length binary representations with leading zeros represent each coordinate exactly once.

For hardness, let a #P function be the number of accepting labelled paths of a polynomial-time nondeterministic Turing machine. The stack encoding in Section 42, or its RAM implementation, gives a canonical quartic certificate for every path within its polynomial horizon. Pad after halting canonically. Stack codes have polynomially many bits, because only a polynomial number of fixed-alphabet symbols can occur; the local arithmetic and the memory height bound therefore give a polynomial-length binary bound for every certificate coordinate. The certificate's sparse description and these bounds are computable in polynomial time. Equation (177) proves that the number of bounded roots is precisely the required accepting-path count. The compiler already has finitely many roots for the fixed horizon, so one may choose a box containing all of them. $\square$

This proposition is a consequence of the parsimonious construction and the definition of #P, not a claim that bounded counting hardness is new. In particular, it does not place unrestricted quartic root counting in #P; unrestricted equations can have infinitely many roots.

43.2 Canonical stopping versus random padding

Suppose each live step consumes exactly one independent fair bit. At a halted state no bit is consumed; the recorded padding label is forced to zero. If an accepting path first halts after $\tau \leq T$ live steps, its probability is $2^{-\tau}$, not 2^{-T} . Merely counting canonically padded roots and dividing by 2^T would therefore be incorrect.

Let $e_t \in \{0, 1\}$ indicate that the machine is live at the start of step t . It is computed canonically from the control state. Attach coordinates $u_0, \dots, u_T$ and quadratic residuals

$$u_0 - 1 = 0, \quad u_{t+1} - u_t(2 - e_t) = 0 \quad (0 \leq t < T). \quad (178)$$

These coordinates do not change the number of solutions: their values are forced by the run.

Theorem 43.2 (Exact path weights). *For the extended quartic certificate, the first-halting accepting probability by horizon T is*

$$\Pr[\text{accept by } T] = 2^{-T} \sum_{Y: \tilde{Q}_{P,T}(x,Y)=0} u_T(Y). \quad (179)$$

Every accepting root whose first halt occurs at τ has $u_T = 2^{T-\tau}$. An input already accepting at time zero has one root of weight 2^T , contributing probability one.

Proof. Before the first halt, $e_t = 1$ and the multiplier in (178) is one. After the halt, $e_t = 0$ and the multiplier is two. There are $T - \tau$ such padding steps, so $u_T = 2^{T-\tau}$. The root's contribution to (179) is $2^{-\tau}$, the probability of its live choice sequence. Distinct first-halting labelled paths describe disjoint

cylinders in the space of infinite fair-bit streams: no already halted live path is a proper prefix of another live path. Summing their cylinder probabilities gives the probability of acceptance by T . $\square$

More generally, replace e_t by a canonical bit indicating whether a random bit is *consumed* at that step. The same proof uses the number of consumed bits, provided every consumed bit is retained in the labelled path, even when both choices lead to the same next state. Non-random steps have one label. The fixed common denominator remains 2^T when at most one bit is consumed per step. This formulation covers deterministic navigation between probabilistic instructions.

Example 43.3 (Stop on the first one). At each live step, toss a fair bit; accept and halt when it is one, and otherwise continue. There are exactly T accepting canonical paths by time T , namely $1, 01, \dots, 0^{T-1}1$. Their weights are $2^{T-1}, 2^{T-2}, \dots, 1$. Hence

$$\Pr[\text{accept by } T] = \frac{2^{T-1} + \dots + 1}{2^T} = 1 - 2^{-T}.$$

Counting the T roots without weights would instead produce $T/2^T$, illustrating the error caused by conflating canonical halted padding with random padding.

44 Manuscript 03: Conclusion [03]

The main constructive result is an exact linear-size Diophantine memory compiler: $24L$ natural witnesses, $22L + 1$ quadratic residuals, and one sparse quartic equation, with a singleton fiber for every valid log. The central nonlocal check is deterministic carry-free product equality. The complementary comparison-network compiler retains much smaller coordinate heights, exposing a useful quantitative distinction between arithmetic size and bit resources.

Both backends preserve the same semantic object. Their canonical fibers allow complete RAM executions to be compiled parsimoniously, support immutable graph evaluation without unfolding shared terms, and make exact path counting and first-halting probability formulas possible. Those statements are stronger than bare existential representability, but remain finite-horizon statements. The article gives complete mathematical proofs, executable memory compilers, and a precise boundary between what has been obtained and the further fixed-variable compression problem. Establishing literature-wide novelty and formalizing the new proofs are separate tasks, not facts inferred from the successful computations.

Part VI

Queues: tag systems and FIFO networks

Source and scope. [write] This Part is manuscript 07, “Causality Without Tableaux: Compact Diophantine Certificates for Queue Universality”, delivered in batch 61. It prints the manuscript’s little-endian word codes (its Section 2.3), then its Sections 4–10 in their own order: cyclic tag systems without intermediate queues (Theorem 46.1), a real computation and a self-starting fiction, the variable/degree/witness-height tradeoffs, general deletion-tag systems (Theorem 49.1), the exact dimension of polynomial word memory (Theorem 50.4), a quartic certificate from two-coordinate memory (Theorem 51.1), and FIFO networks and switched arithmetic dynamics (Theorem 52.2); it closes with the manuscript’s conclusion. Its first-failure product, on which every certificate here depends, is Theorem 8.1 in Part I; its conventions are in Section 2; its fixed-arity boundary section is in Part IX. The word-memory section is kept before the quartic certificate, as in the manuscript, because the quartic stores each queue in the two coordinates of Proposition 50.1.

Relation to the other Parts. [write] Manuscript 07 names manuscripts 04 and 06 as its companions and treats what they leave out: FIFO word content and the temporal availability of produced symbols. Manuscript 06 says that its counts cannot encode FIFO order and asks which queue restrictions can be added (Research question 71.33); this Part answers that question for bounded runs of fixed-read FIFO networks, while data-dependent read counts stay open. The bounded executions here have one witness each, as in Parts II and III, but on a different substrate: an ordered queue instead of a multiset of tokens, so none of the constructions below is a second route to Theorem 11.2. Where Part II spends $(2T + 1)(d + m)$ or more coordinates on per-step markings, the stream certificate spends $T + 1$ (degree $\max\{4, 2T\}$), the quartic $3T - 2$, the deletion-tag certificate $dT + 1$, and a fixed-read network $D + 1$. The radix word code is the device of Part V's linear-size compiler (Section 40).

Setting and hypotheses. [write] Queues are first-in first-out. Symbols lie in $\{0, \dots, B - 1\}$; the first written symbol is the head and the *least* significant digit. Formal lengths are signed integers and may become negative after an illegal step. The program, the input word, the terminal word and the horizon $T \geq 1$ are data of the construction, not unknowns; for $T = 0$ the certificate is a constant polynomial. In fixed-read networks the number of symbols read in each phase does not depend on the data. Inside this Part, p is the period of a cyclic-tag program or the number of queues, m_i, a_i, c_i are the length, value and $2^{m_i} - 1$ of an appendant, d is the deletion number, $K = (B - 1)!$, $J = ((B - 1)!)^C$, D is the total number of consumed symbols, G the guard with slack u , and H an upper bound on queue length (Section 2).

45 Little-endian words and faithful concatenation [07]

[write] Section 2.3 of manuscript 07; its Sections 2.1–2.2 are in Section 2 and its Section 3, the first-failure product Theorem 8.1 used below, is in Part I.

Let $B \geq 2$ and let words use digits $0, \dots, B - 1$. The first written symbol is the queue head. Define

$$\text{val}_B(a_0 \cdots a_{n-1}) = \sum_{i=0}^{n-1} a_i B^i, \quad \text{val}_B(\varepsilon) = 0.$$

When $B = 2$ the subscript is omitted. The first symbol is thus the *least* significant digit. Written terminal zero symbols still contribute to length.

Lemma 45.1 (Word-code identities). *For finite words u, v ,*

$$\text{val}_B(uv) = \text{val}_B(u) + B^{|u|} \text{val}_B(v).$$

For each fixed n , the value map is injective on words of length n . In particular, equality of length and value is equivalent to equality of words.

Proof. Split the defining sum for uv at position $|u|$. For injectivity, reduce an equality of values modulo B to recover the first digit, subtract that digit, divide by B , and repeat n times. The length condition is essential: $\varepsilon, \emptyset$, and $\emptyset\emptyset$ all have value zero. $\square$

A candidate execution will be represented by its sequence of consumed symbols. A global concatenation identity checks the content of this sequence. The main difficulty is to verify that each of those symbols existed *before* it was consumed.

46 Cyclic tag systems without intermediate queues [07]

46.1 Operational semantics

A cyclic-tag program is a nonempty finite list of binary words

$$\mathcal{A} = (A_0, A_1, \dots, A_{p-1}).$$

The phase at time i is $i \bmod p$, with initial phase zero. From a nonempty queue bq , where $b \in \{0, 1\}$, one step produces

$$qA_{i \bmod p} \quad \text{if } b = 1, \quad q \quad \text{if } b = 0.$$

The phase advances in both cases. There is no transition from an empty queue. Empty appendants are allowed. This is Cook's cyclic-tag semantics with the two symbols renamed 0 and 1 [31].

Fix an input word w , a terminal word v , and $T \geq 1$. Set

$$n = |w|, \quad x = \text{val}(w), \quad h = |v|, \quad y = \text{val}(v),$$

and abbreviate the phase-dependent program constants by

$$m_i = |A_{i \bmod p}|, \quad a_i = \text{val}(A_{i \bmod p}), \quad c_i = 2^{m_i} - 1.$$

These constants depend on i , but do not depend on the candidate run.

46.2 The complete polynomial

Introduce precisely $T + 1$ unknowns

$$b_0, \dots, b_{T-1}, u \in \mathbb{N}.$$

The b_i are intended to be the consumed bits. Define the following *polynomial expressions*, not new variables:

$$\ell_i = n - i + \sum_{j < i} m_j b_j, \tag{180}$$

$$D_i = 1 + c_i b_i, \quad R_i = \prod_{j < i} D_j, \tag{181}$$

$$L = \ell_T - h, \tag{182}$$

$$F = x + 2^n \sum_{i < T} a_i b_i R_i - \sum_{i < T} 2^i b_i - 2^T y, \tag{183}$$

$$G = \prod_{i < T} \ell_i. \tag{184}$$

The representing polynomial is

$$P_{\mathcal{A}, w, v, T}(\mathbf{b}, u) = \sum_{i < T} (b_i(b_i - 1))^2 + L^2 + F^2 + (G - u - 1)^2. \tag{185}$$

Every symbol on the right has been defined by integer polynomial operations. In particular, R_i uses products of affine factors, not an exponent whose exponent is an unknown.

Theorem 46.1 (Causal stream certificate). *For the fixed data above, natural zeros of (185) correspond bijectively to legal T -step executions of $\mathcal{A}$ from w to v . The solution records their consumed bits, and its last coordinate is*

$$u = \prod_{i < T} |w_i| - 1,$$

where w_i is the queue immediately before step i .

The polynomial has exactly $T + 1$ named variables, $T + 3$ residuals, degree at most $\max\{4, 2T\}$, and an $O(T)$ arithmetic circuit for a fixed program. Since cyclic-tag execution is deterministic, its natural zero set is either empty or a singleton. When $v = \varepsilon$, a zero exists exactly when first halting occurs at step T .

46.3 Proof: reconstructing the entire queue from one word identity

Proof. Suppose first that $P(\mathbf{b}, u) = 0$. All residuals vanish, so $b_i \in \{0, 1\}$. Write $A_i^{[b_i]}$ for $A_i \bmod p$ when $b_i = 1$ and for ε when $b_i = 0$. This notation denotes a choice of words, not a variable-exponent polynomial operation.

On these Boolean values,

$$D_i = 2^{m_i b_i}, \quad R_i = 2^{\sum_{j < i} m_j b_j}.$$

By Lemma 45.1, the right side of

$$b_0 b_1 \cdots b_{T-1} v = w A_0^{[b_0]} A_1^{[b_1]} \cdots A_{T-1}^{[b_{T-1}]} \quad (186)$$

has value $x + 2^n \sum_{i < T} a_i b_i R_i$. Its length is $n + \sum_{i < T} m_i b_i$. The left side has value $\sum_{i < T} 2^i b_i + 2^T y$ and length $T + h$. Consequently $F = 0$ and $L = 0$ prove the actual word equality (186), not merely a congruence.

It remains to show that this static equality describes a causal computation. The equation $G = u + 1$ makes G positive. Theorem 8.1, with initial length n , unit consumptions, and productions $m_i b_i \geq 0$, implies

$$\ell_i \geq 1 \quad (0 \leq i < T). \quad (187)$$

For each i , let

$$U_i = w A_0^{[b_0]} \cdots A_{i-1}^{[b_{i-1}]}.$$

It is a prefix of the right side of (186), and

$$|U_i| = n + \sum_{j < i} m_j b_j = i + \ell_i \geq i + 1.$$

Thus the first $i + 1$ symbols of the global word already occur in U_i , before appendant i is produced. In particular the symbol at position i is b_i and is already available.

Inductively, the actual queue before event i is the suffix of U_i obtained by deleting its first i symbols. The assertion holds at $i = 0$, where $U_0 = w$. It is nonempty by the preceding inequality, its head is b_i , and deleting that head and appending $A_i^{[b_i]}$ gives the stated suffix description for U_{i+1} . This proves every transition. After T steps, (186) says that the remaining suffix is exactly v .

Conversely, consider a legal T -step execution with consumed bits b_i . An induction on the operational semantics gives the same suffix description of its queues, and therefore (186). This implies $L = F = 0$. Its queue lengths are the values (180), all positive before the T steps. Set $u = G - 1 \in \mathbb{N}$. All residuals vanish.

For uniqueness, a fixed input and program determine the first consumed bit and the next queue. Repeating this observation determines the entire consumed sequence. The guard equation then

determines u . If $v = \varepsilon$, all earlier queues are nonempty while the last is empty, so T is exactly the first halting time.

Finally, ℓ_i is affine, R_i has degree at most i , and $b_i R_i$ has degree at most $i + 1$. Hence $\deg F \leq T$, $\deg L \leq 1$, and $\deg G \leq T - 1$ because $\ell_0 = n$ is constant. Combining with the Boolean residuals gives degree at most $\max\{4, 2T\}$. There are T Boolean residuals and three further residuals. Prefix recurrence $R_{i+1} = R_i D_i$, length recurrence, and running sums and products evaluate all expressions in $O(T)$ arithmetic operations, with constants from the fixed program. $\square$

46.4 What the theorem does not need

The proof does not need a guessed queue at each time, a guessed time-varying power of two, a division witness for each read, or a general-purpose bounded universal quantifier elimination. Once the consumed bits are fixed, all such quantities are polynomial expressions or are reconstructed at the word level.

The condition that v is fixed matters for the variable count. A terminal *word parameter* can be supplied as data to the compiler without adding witnesses. Treating its numerical code and length as arbitrary unknowns would require a separate validity interface and a different resource statement.

For $T = 0$, no step is taken. Use the constant polynomial 0 if $w = v$ and 1 otherwise, with no witness variables. The unique empty tuple is the zero of the former. The executable compilers deliberately restrict their interface to $T \geq 1$ and report this convention rather than manufacture negative variable counts.

47 Two examples: a real computation and a self-starting fiction [07]

47.1 A small polynomial with exactly one natural zero

Take the period-two program

$$\mathcal{A} = (10, \varepsilon), \quad w = 1, \quad v = \varepsilon, \quad T = 3.$$

Remember that the first written symbol is the head. The execution is

$$1 \longrightarrow 10 \longrightarrow 0 \longrightarrow \varepsilon.$$

It consumes $(b_0, b_1, b_2) = (1, 1, 0)$ and has pre-step lengths $(1, 2, 1)$. Therefore $G = 2$ and $u = 1$.

The general formulas simplify to

$$\begin{aligned} L &= -2 + 2b_0 + 2b_2, \\ F &= 1 + b_0 - 2b_1 - 2b_2 + 6b_0b_2, \\ G &= 2b_0(2b_0 - 1). \end{aligned}$$

Thus the completely explicit polynomial

$$\begin{aligned} P &= \sum_{i=0}^2 b_i^2 (b_i - 1)^2 + (-2 + 2b_0 + 2b_2)^2 \\ &\quad + (1 + b_0 - 2b_1 - 2b_2 + 6b_0b_2)^2 \\ &\quad + (2b_0(2b_0 - 1) - u - 1)^2 \end{aligned} \tag{188}$$

has the single natural zero $(1, 1, 0, 1)$. Here cancellation lowers the actual degree to four; the general theorem only promises degree at most six when $T = 3$.

For a direct check of uniqueness, $L = 0$ forces $b_0 + b_2 = 1$ after the Boolean conditions. If $b_0 = 0$, the guard is zero and its residual cannot vanish. Hence $b_0 = 1$, $b_2 = 0$, and $F = 0$ gives $b_1 = 1$. The guard then gives $u = 1$. This agrees with the semantic proof and with the exported exact-integer certificate.

47.2 Why global balance alone is false

Now take

$$\mathcal{A} = (\varepsilon, 1), \quad w = \mathbf{0}, \quad T = 2,$$

and propose the consumed word $\mathbf{0}1$. The global word equation says

$$\underbrace{\mathbf{0}1}_{\text{consumed}} = \underbrace{\mathbf{0}}_{\text{initial}} \underbrace{\varepsilon}_{\text{step 0}} \underbrace{1}_{\text{step 1}},$$

which is true. The length equation is also true. Algebraically these two residuals are $L = b_1 - 1$ and $F = -b_0$, so $(b_0, b_1) = (0, 1)$ satisfies both.

But the actual queue evolves as $\mathbf{0} \rightarrow \varepsilon$ and halts. The proposed second read tries to consume the very symbol that would be appended *as a consequence of that read*. This is not a legal cyclic-tag transition. Here $\ell_0 = 1$ and $\ell_1 = 0$, so $G = 0$ and no natural slack can solve $G = u + 1$.

This example is more than an endpoint bookkeeping issue. It exhibits a genuinely circular dependency in a globally consistent production/consumption equation. The product guard breaks precisely that circularity. In the finite test suite, 268 candidate bitstrings satisfy the raw content and length equations but are rejected by causality; this count is evidence for the implementation, while Theorem 46.1 supplies the proof for all horizons.

Remark 47.1 ([write] Relation to the project's tag-system notes). The suffix induction in the proof of Theorem 46.1, and the observation that a globally balanced proposed stream can read a symbol supplied by its own later appendant, also appear in exploratory notes of ProveIt's Hilbert's-tenth-problem project for deletion- β tag systems (`Computability/HilbertTenthProblem/Papers/1980/EXPLORATION_TAG_HALTING_WITHOUT_PREFIX_GUARDS.md`, Sections 2–3; `EXPLORATION_FINITE_UNIVERSAL_HISTORY_VERIFIERS.md`), which manuscript 07 did not consult. Those notes record the causal prefix inequalities that an exact execution requires, and prove that the eventual-halting language needs none of them: the sampled-symbol equations and the final length window already force a genuine halt at the first short prefix. There is no conflict. Theorem 46.1 certifies an exact T -step execution with a unique witness, and the guard is what buys the uniqueness: without it, the notes' own example ($\beta = 2$, $w = 00$, $h(0) = 0$, $h(1) = 100$) is accepted with both selectors 0 and 001. The notes' scalar FIFO identity $D = I + WA$ for a queue of constant length (`EXPLORATION_NATIVE_STREAM_RAW_QUEUE.md`, Section 1) is the global word identity of this Part in a case where the guard is automatic, since every queue length there equals the initial length. The project's Lean route for Jones's 1980 tag certificate stores each queue as a content and a length marker 3^ℓ (`Computability/HilbertTenthProblem/Lean/Diophantine/Paper1980/TagSystem.lean`, `TagHistory.lean`: `Jones1980.length_step'`, `Jones1980.content_step'`), the two-coordinate word memory of Proposition 50.1 in base three. These notes count the arithmetic operations of fixed-arity universal certificates; no result of this Part concerns operation counts.

48 Variable count, degree, and witness height [07]

48.1 The single guard witness has controlled bit height

Let $M = \max_j |A_j|$, put $g = \max\{0, M - 1\}$, and define

$$H = \max\{1, n + Tg\}. \quad (189)$$

During any legal T -step run, queue length can increase by at most g per step. Hence $1 \leq \ell_i \leq H$ for every $i < T$.

Proposition 48.1 (Witness size). *Every zero of the causal stream polynomial satisfies*

$$0 \leq u = G - 1 \leq H^T - 1.$$

The full witness has a binary description of length

$$O(T + T \log_2 \max\{2, H\}).$$

Proof. The T consumed-symbol coordinates are bits. Multiplying T positive queue lengths bounded by H gives $1 \leq G \leq H^T$. A nonnegative integer below H^T needs at most $1 + T \log_2 \max\{2, H\}$ bits under any usual convention for encoding zero. $\square$

The product can be enormous as an integer, but its bit length is only logarithmic in that magnitude. This is one reason not to confuse witness-coordinate count with witness-value bounds. The statement is an upper bound, not a proof that this witness height is best possible.

The coefficients 2^n and 2^T have $n + 1$ and $T + 1$ bits, respectively. If every power 2^i is stored as a separate literal, their total textual size is quadratic in T . An arithmetic circuit can instead generate these powers by repeated doubling. The $O(T)$ arithmetic statement therefore does not assert that every literal-based serialization is bit-linear, nor that multiplication of all intermediate values takes unit time.

48.2 Removing the slack by a larger polynomial guard

For the positive integer H of (189), define

$$Z_H(z) = \prod_{r=1}^H (z - r).$$

Its integer roots are exactly $1, \dots, H$. Keep the Boolean, length, and content residuals from Section 46, but replace $G - u - 1$ by T residuals $Z_H(\ell_i)$.

Theorem 48.2 (No-slack certificate). *The polynomial*

$$\tilde{P}(\mathbf{b}) = \sum_{i < T} (b_i(b_i - 1))^2 + L^2 + F^2 + \sum_{i < T} Z_H(\ell_i)^2$$

has natural zeros in bijection with exact T -step executions from w to v . It uses exactly T variables, $2T + 2$ residuals, has degree at most $2 \max\{2, T, H\}$, and admits an $O(T(1 + H))$ arithmetic circuit for a fixed program.

Proof. Every legal execution has all pre-step lengths in $\{1, \dots, H\}$, so it satisfies the added residuals. Conversely, a zero forces every ℓ_i into that positive range. The same global word identity and suffix induction used in Theorem 46.1 then reconstruct the legal execution. No auxiliary choice remains once its consumed bits are fixed.

Since ℓ_i is affine, $Z_H(\ell_i)$ has degree at most H . The other residual degree bounds were already established. Each bounded-root guard uses $O(H)$ operations, and there are T of them. $\square$

For an empty terminal word, $n > T$ is immediately impossible, because a step removes only one symbol and appends a nonnegative number. When $n \leq T$ and the program is fixed, $H \leq T \max\{1, M\}$. Thus the no-slack version still has degree $O(T)$ and a quadratic-size circuit. It improves the number of witnesses without preserving the linear guard circuit.

48.3 Why these are not universal optimality claims

For a fully specified bounded computation, an unconstrained compiler could simulate all T steps and output the constant polynomial 0 or 1. That would use no variables at all, but would not preserve the computation as a certificate and would tell us nothing about a structural reduction. Meaningful comparisons must specify which computational data remain visible in the solution and what the compiler is permitted to do.

Our constructions are symbolic: they are obtained from the program, input, endpoint, and horizon without needing to run the machine. The implementation optionally simulates the run *after constructing the formula*, solely to attach an example witness. The command-line option `-no-witness` disables this extra operation. The theorem's uniqueness is uniqueness of the complete listed witness, not merely uniqueness of the final machine state.

The no-slack construction also warns against claiming that one guard variable is logically unavoidable. It is not. What that variable buys is a particularly small arithmetic circuit, a simple proof, and a compact witness-height estimate. The exact tradeoff among low degree, nearly minimal trace coordinates, and linear circuit size remains a substantive design problem.

49 General deletion-tag systems [07]

49.1 Semantics and interpolation of the production table

Fix an alphabet $\{0, \dots, B-1\}$ with $B \geq 2$, a deletion number $d \geq 1$, and a word $A(a)$ for each alphabet symbol a . A step requires at least d symbols. It consumes the first d of them and appends $A(a)$, where a was the first consumed symbol. A queue of length less than d is halted. The construction applies to every such system; no universality assumption is required for an individual instance. The family of 2-tag systems with suitable finite alphabets is a standard universal substrate [31, 32].

We need polynomial table lookups without one-hot variables. For each $a \in \{0, \dots, B-1\}$ define

$$\delta_a(z) = \prod_{\substack{0 \leq b < B \\ b \neq a}} \frac{z - b}{a - b}.$$

For an alphabet digit z , this equals one when $z = a$ and zero otherwise. Let

$$M(z) = \sum_{a < B} |A(a)| \delta_a(z), \tag{190}$$

$$A(z) = \sum_{a < B} \text{val}_B(A(a)) \delta_a(z), \tag{191}$$

$$D(z) = \sum_{a < B} B^{|A(a)|} \delta_a(z). \tag{192}$$

Here $A(z)$ is a numerical interpolant; $A(a)$ in the operational semantics is a word. The argument or surrounding value notation distinguishes them. All three interpolants lie in $\mathbb{Q}[z]$ and have degree at most $B-1$.

49.2 The residuals and explicit integer clearing

Fix input w , terminal word v , and $T \geq 1$. Write $n = |w|$ and $x = \text{val}_B(w)$. The unknowns are

$$z_0, \dots, z_{dT-1}, u \in \mathbb{N},$$

where $z_0 \cdots z_{dT-1}$ is the full consumed word, including the $d - 1$ ignored symbols of each step. Put $h_i = z_{di}$ and define

$$E_B(z) = \prod_{a=0}^{B-1} (z - a), \quad (193)$$

$$\ell_i = n - di + \sum_{j < i} M(h_j), \quad R_i = \prod_{j < i} D(h_j), \quad (194)$$

$$L = \ell_T - |v|, \quad (195)$$

$$F = x + B^n \sum_{i < T} A(h_i) R_i - \sum_{j < dT} B^j z_j - B^{dT} \text{val}_B(v), \quad (196)$$

$$G = \prod_{i < T} (\ell_i)_{\underline{d}}. \quad (197)$$

These expressions are rational polynomials, but denominators can be cleared explicitly. Set $K = (B - 1)!$. The denominator of δ_a is, up to sign, $a!(B - 1 - a)!$, which divides K . Hence KM , KA , and KD have integer coefficients. It follows that the residuals

$$E_B(z_j) \ (j < dT), \quad KL, \quad K^T F, \quad K^{d(T-1)} (G - u - 1) \quad (198)$$

all lie in $\mathbb{Z}[\mathbf{z}, u]$. The last exponent suffices because $\ell_0 = n$ is constant, so only $T - 1$ falling factorials have a possible denominator. All clearing multipliers are nonzero integers. No new unknowns are introduced.

Theorem 49.1 (Deletion-tag stream certificate). *The sum of squares of (198) has natural zeros in bijection with exact T -step executions from w to v . It uses $dT + 1$ variables and $dT + 3$ residuals. Its degree is at most*

$$2 \max\{B, T(B - 1), d(T - 1)(B - 1), 1\}. \quad (199)$$

For fixed B, d , and production table, it has an $O(T)$ arithmetic circuit. Each successful instance has a unique witness. If $|v| < d$, it certifies first halting at exactly T .

Proof. The digit residuals force every z_j to be an alphabet symbol. On those symbols, the interpolants have their prescribed natural values. Clearing denominators does not change any zero condition.

The length and content equations, together with Lemma 45.1, give the word equality

$$z_0 \cdots z_{dT-1} v = w A(h_0) A(h_1) \cdots A(h_{T-1}). \quad (200)$$

The guard equation and Theorem 8.1, with consumption d at every step, imply $\ell_i \geq d$ for every $i < T$. Let $U_i = w A(h_0) \cdots A(h_{i-1})$. Its length is $di + \ell_i \geq d(i + 1)$. Thus the whole next consumed block $z_{di} \cdots z_{di+d-1}$ is already in U_i , not in an appendant produced by this or a later step. The suffix induction now deletes d symbols at a time. Its first symbol is h_i , so the selected appendant is correct. The terminal suffix is v .

Conversely, any actual execution satisfies (200), the length equation, the alphabet constraints, and the positive resource product. It therefore gives a zero with $u = G - 1$. Deterministic semantics fixes every consumed symbol, including the ignored ones, and the guard fixes the slack.

For the degree estimate, digit residuals have degree B , length has degree at most $B - 1$, and $A(h_i)R_i$ has degree at most $(i + 1)(B - 1)$. The nonconstant part of the guard has at most $d(T - 1)$ factors of degree at most $B - 1$. The term $-u - 1$ contributes degree at most one. Squaring gives (199). With fixed alphabet and deletion number, each table evaluation and falling factorial uses a fixed number of arithmetic operations; prefix products, lengths, and sums are accumulated in $O(T)$ operations. The clearing multipliers are constant coefficients with bit length linear in T for fixed B, d . $\square$

Integer-only circuit realization. Denominator clearing need not introduce division gates. Put $\widehat{M} = KM$, $\widehat{A} = KA$, $\widehat{D} = KD$, and

$$\lambda_i = K(n - di) + \sum_{j < i} \widehat{M}(h_j), \quad \Pi_i = \prod_{j < i} \widehat{D}(h_j).$$

Then the cleared content and guard residuals can be evaluated as

$$K^T F = K^T \left(x - \sum_{j < dT} B^j z_j - B^{dT} \text{val}_B(v) \right) + B^n \sum_{i < T} K^{T-i-1} \widehat{A}(h_i) \Pi_i,$$

$$K^{d(T-1)}(G - u - 1) = (n)_d \prod_{i=1}^{T-1} \prod_{r=0}^{d-1} (\lambda_i - Kr) - K^{d(T-1)}(u + 1).$$

All exponents in these expressions are fixed nonnegative integers. Precompute the powers of K by multiplication; every other gate is an integer addition, subtraction, or multiplication. This gives the asserted linear arithmetic size without using rational constants internally.

49.3 Halting tails are not necessarily empty

A deletion- d tag machine can halt with $1, \dots, d - 1$ symbols still present. Equating its terminal queue with ε would miss those computations. There are

$$N_{B,d} = 1 + B + \dots + B^{d-1}$$

possible halted words. The endpoint theorem handles each of them.

Corollary 49.2 (Unspecified halted endpoint). *For fixed B, d , program, input, and $T \geq 1$, first halting at exactly T has a witness-faithful polynomial with $dT + 1$ variables and degree at most $N_{B,d}$ times (199).*

Proof. For each word v of length less than d , let P_v be the sum-of-squares polynomial of Theorem 49.1, using the same names for all consumed-symbol variables and the slack. Use $\prod_{|v| < d} P_v$. An integer product vanishes exactly when one factor vanishes. Each factor certifies an actual execution to its specified endpoint. Determinism permits at most one such endpoint and one complete witness. The common guard does not create extra choices. The degree of the product is at most the sum of the degrees of its factors. $\square$

The product here is over a fixed finite collection determined by B, d , not over an unbounded family of words. Its size can be large as these parameters grow; the linear-in- T statement keeps them fixed.

49.4 A ternary example with a nonempty halted tail

Let $B = 3$, $d = 2$, and choose productions

$$A(0) = \varepsilon, \quad A(1) = 20, \quad A(2) = 0.$$

Starting from 10 , the execution is

$$10 \longrightarrow 20 \longrightarrow 0.$$

At $T = 2$ it halts with one symbol left. The consumed digits are $(1, 0, 2, 0)$; both pre-step lengths are two, so

$$G = (2)_2(2)_2 = 4, \quad u = 3.$$

The global identity is $10200 = 10200$. Its value is 19 in base three, with the stated little-endian convention. This illustrates why the ignored symbols and the terminal length must both be retained.

50 The exact dimension of polynomial word memory [07]

50.1 Two affine coordinates suffice

The stream certificate eliminates queue states globally. For a local encoding, a useful state representation is

$$\mathcal{E}_B(w) = (\text{val}_B(w), B^{|w|}) = (v, s) \in \mathbb{N}^2. \quad (201)$$

The second coordinate records scale, not length itself. Define

$$(v, s) \star (w, r) = (v + sw, sr). \quad (202)$$

Lemma 45.1 gives

$$\mathcal{E}_B(uv) = \mathcal{E}_B(u) \star \mathcal{E}_B(v).$$

The operation is associative, as is clear either by direct expansion or by identifying (v, s) with the matrix

$$\begin{pmatrix} s & v \\ 0 & 1 \end{pmatrix}.$$

The empty word has code $(0, 1)$, the identity element. Insertion of a fixed symbol at either end is affine:

$$L_a(v, s) = (Bv + a, Bs), \quad R_b(v, s) = (v + bs, Bs). \quad (203)$$

For every pair of symbols, $L_a R_b = R_b L_a$, reflecting the word identity $a(wb) = (aw)b$.

Proposition 50.1. *The map $\mathcal{E}_B$ is a faithful natural-number encoding of all finite words, with polynomial concatenation and affine insertion at both ends.*

Proof. The scale determines the length because $B^m = B^n$ implies $m = n$ for natural m, n . The value then determines the word by Lemma 45.1. The displayed formulas establish the operation claims. $\square$

The next result shows that two is the minimum number of natural coordinates for this insertion architecture. It does not assume that arbitrary-word concatenation is polynomial; two prepend maps and a single fixed append map already suffice for the obstruction.

50.2 An elementary centralizer lemma

Composition of polynomials is denoted by $\circ$. A polynomial commutes with Q when $F \circ Q = Q \circ F$.

Lemma 50.2 (Uniqueness at a fixed degree and leading coefficient). *Let $Q \in \mathbb{R}[X]$ have degree $d \geq 2$. Two nonconstant polynomials H_1, H_2 that commute with Q , have the same degree m , and have the same leading coefficient must be equal.*

Proof. Suppose $\Delta = H_1 - H_2$ is nonzero and has degree $k < m$. Let $c \neq 0$ be the leading coefficient of Q and $a \neq 0$ the common leading coefficient of H_1, H_2 . Subtract the two commutation identities to obtain

$$Q(H_1) - Q(H_2) = \Delta(Q).$$

The right side has degree dk . On the left, factor the difference of the leading powers of Q . Its highest term has nonzero coefficient

$$dc a^{d-1} \text{lc}(\Delta)$$

and degree $(d-1)m + k$. Lower powers of Q give smaller degrees. Thus equality would imply $(d-1)m + k = dk$, or $(d-1)(m-k) = 0$, a contradiction. $\square$

Lemma 50.3 (Positive-leading centralizers are commutative). *Let Q be a nonconstant real polynomial with positive leading coefficient, and assume $Q \neq X$. Any two nonconstant real polynomials F, G with positive leading coefficients that commute with Q also commute with one another.*

Proof. First suppose $\deg Q = d \geq 2$ and let $c > 0$ be its leading coefficient. If $\deg F = m$ and $a > 0$ is the leading coefficient of F , comparison in $F(Q) = Q(F)$ gives

$$ac^m = ca^d, \quad a^{d-1} = c^{m-1}.$$

The positive solution is $a = c^{(m-1)/(d-1)}$. Similarly, if $\deg G = n$, its leading coefficient is $c^{(n-1)/(d-1)}$.

Both $F \circ G$ and $G \circ F$ commute with Q . They have degree mn , and their leading coefficients both equal $c^{(mn-1)/(d-1)}$. Lemma 50.2 makes them equal.

Now suppose $Q(X) = cX + e$ with $c > 0$. If $c = 1$, then $e \neq 0$ because $Q \neq X$. Commutation gives

$$F(X + e) - F(X) = e.$$

For a nonconstant polynomial the left side has degree $\deg F - 1$, so F has degree one. Comparing its slope shows that $F(X) = X + f$ for a constant f . The same holds for G , and translations commute.

If $c \neq 1$, translate the fixed point $e/(1-c)$ to zero. In the translated coordinate, the commuting condition is $F(cX) = cF(X)$. For the coefficient of X^k , this requires $c^k = c$ whenever that coefficient is nonzero. Since $c > 0$ and $c \neq 1$, only $k = 1$ is possible, including the exclusion of the constant coefficient. Thus all such maps are scalar multiplications in the same translated coordinate, and they commute. $\square$

This argument is a small positive-real specialization of the subject of commuting polynomials. It is included in full rather than importing a classification theorem or presenting the general centralizer theory as new [33].

50.3 One natural coordinate cannot support both ends

Theorem 50.4 (One-coordinate polynomial obstruction). *There is no injective map*

$$e : \{0, 1\}^* \longrightarrow \mathbb{N}$$

and no real polynomials P_0, P_1, Q satisfying, for every finite binary word w ,

$$P_0(e(w)) = e(0w), \quad P_1(e(w)) = e(1w), \quad Q(e(w)) = e(w0). \quad (204)$$

In particular, a faithful natural-coordinate word representation with polynomial insertion of arbitrary binary symbols at both ends requires at least two coordinates. Proposition 50.1 attains that dimension with affine maps.

Proof. The image of e is an infinite subset of $\mathbb{N}$ and is therefore unbounded. Each of P_0, P_1, Q is nonconstant, because its outputs on that image include infinitely many distinct word codes. Each has positive leading coefficient: a nonconstant real polynomial with negative leading coefficient is eventually negative, contrary to its natural values on an unbounded subset of $\mathbb{N}$.

For $a \in \{0, 1\}$ and every w ,

$$(P_a \circ Q)(e(w)) = e(a(w0)) = e((aw)0) = (Q \circ P_a)(e(w)).$$

Polynomials agreeing on an infinite set are equal, so P_a commutes with Q as a polynomial identity. Moreover $Q \neq X$, since $Q(e(w)) = e(w)$ would imply $w0 = w$ by injectivity, which contradicts word length.

Lemma 50.3 now gives $P_0 \circ P_1 = P_1 \circ P_0$. Evaluating at $e(w)$ yields

$$e(01w) = e(10w).$$

The two words are distinct, contradicting injectivity. $\square$

[write] *Theorem 50.4 is consistent with manuscript 01's injective polynomial pairing $\pi(a, b) = (a + b)^2 + a$ (Lemma 56.1) and with manuscript 05's pairing (241): those are maps of two arguments used to code finite terms and lists, while the theorem concerns one-argument insertion maps on the code of a single word.*

50.4 The boundary is architectural, not computational

The theorem does not say that one integer cannot encode a word. For example,

$$e(w) = 2^{|w|} + \text{val}(w)$$

is a faithful sentinel encoding. Both prepends are polynomial:

$$e(aw) = 2e(w) + a.$$

Appending a zero, however, requires adding $2^{|w|}$, the leading power of two of the code. The theorem proves that no alternative scalar coding can make both prepends and this append simultaneously polynomial on all word codes.

The result does not exclude one-register computation with conditional operations, floor, division, prime-exponent tests, or auxiliary Diophantine witnesses. It does not apply to a finite set of word lengths, where arbitrary finite tables can be interpolated, or to a code whose operations depend on additional hidden control state. Nor does it claim a dimension bound for all possible continuous or rational encodings. Its conclusion is exactly about faithful natural-number word codes with fixed scalar polynomial insertion maps.

51 A quartic certificate from two-coordinate memory [07]

51.1 Encoding a single queue step

For binary words let $(v, s) = \mathcal{E}_2(w)$. A nonempty word has $s = 2^{|w|} \geq 2$. Write its head as $b \in \{0, 1\}$ and the value of its tail as q . Then

$$v = b + 2q.$$

At phase i , the update after the optional appendant is

$$v' = q + \frac{a_i b s}{2}, \quad s' = \frac{s(1 + (2^{m_i} - 1)b)}{2}. \quad (205)$$

The denominator two is exact on a valid nonempty input scale. The updated scale is $2^{|w|-1+m_i b}$, and the updated value is precisely the value of the tail followed by the selected appendant.

51.2 The full quadratic residual system

Fix w and $T \geq 1$, and require the endpoint to be empty. Substitute the constant endpoints

$$(v_0, s_0) = (\text{val}(w), 2^{|w|}), \quad (v_T, s_T) = (0, 1).$$

The natural unknowns are the $2(T - 1)$ internal coordinates

$$(v_i, s_i) \quad (1 \leq i < T)$$

and T quotients q_i for $0 \leq i < T$. In the following formulas,

$$b_i = v_i - 2q_i$$

is an abbreviation for an affine expression, not a new variable. For each $i < T$ use the three residuals

$$B_i = b_i(b_i - 1), \tag{206}$$

$$V_i = 2v_{i+1} - 2q_i - a_i b_i s_i, \tag{207}$$

$$S_i = 2s_{i+1} - s_i - (2^{m_i} - 1)b_i s_i. \tag{208}$$

Let $Q_{\mathcal{A}, w, T} = \sum_{i < T} (B_i^2 + V_i^2 + S_i^2)$.

Theorem 51.1 (Local quartic certificate). *The natural zero set of $Q_{\mathcal{A}, w, T}$ is a singleton if the cyclic-tag run from w first halts at time T , and is empty otherwise. The polynomial has $3T - 2$ variables, $3T$ quadratic residuals, degree at most four, and $O(T)$ arithmetic-circuit size for a fixed program.*

Proof. Every actual run first halting at T gives the displayed internal word codes and quotients $q_i = \lfloor v_i/2 \rfloor$. Equations (206)–(208) are just the head decomposition and the update (205) with denominators cleared. Thus it gives a zero.

Conversely, suppose there is a zero. We induct from the valid initial word code. Assume (v_i, s_i) is the code of an actual queue. Equation $B_i = 0$ forces the signed integer $b_i = v_i - 2q_i$ to be either zero or one. Since $q_i \in \mathbb{N}$, the decomposition $v_i = 2q_i + b_i$ uniquely determines both q_i and b_i .

If the queue is nonempty, its scale is even and at least two. The other residual equations uniquely force the actual next value and scale from (205); hence they again form a valid word code. Notice that this establishes scale validity by induction. There is no separate existential predicate for being a power of two.

If the queue were empty before a required step, then $v_i = 0$ and $s_i = 1$. The Boolean residual and nonnegativity of q_i force $q_i = b_i = 0$. The scale residual would become $2s_{i+1} = 1$, which has no natural solution. Therefore no zero can pass through an empty queue before step T .

Induction proves all T actual transitions, and the final constants $(0, 1)$ identify the empty word. The run consequently first halts at T . Determinism and quotient uniqueness fix every variable. The count is $2(T - 1) + T = 3T - 2$, and all displayed residuals are quadratic, proving the remaining bounds. $\square$

51.3 The small example in quartic coordinates

For the example $\mathcal{A} = (10, \varepsilon)$, $w = 1$, $T = 3$, the internal states are

$$(v_1, s_1) = (1, 4), \quad (v_2, s_2) = (0, 2),$$

and the quotients are $(q_0, q_1, q_2) = (0, 0, 0)$. Thus the seven-coordinate witness is

$$(v_1, s_1, v_2, s_2, q_0, q_1, q_2) = (1, 4, 0, 2, 0, 0, 0).$$

This is the witness in `data/07-queue-causality-quartic_example.json`. The stream construction uses only four coordinates for the same computation; the quartic construction guarantees degree four for every horizon instead of allowing degree to grow with T .

51.4 Comparing information content

A valid state of length at most $n + T \max\{0, M - 1\}$ has value and scale requiring $O(n + TM + 1)$ bits. The local witness therefore has the elementary upper bound $O(T(n + TM + 1))$ bits. The stream witness instead stores the consumed bits and a product whose bit length is controlled by Proposition 48.1.

These are different certificates for the same semantic object. One maintains a low-degree local recurrence by carrying explicit memory coordinates. The other eliminates those coordinates, retains one global content equation, and pays for the elimination in polynomial degree. The no-slack variant makes the independence of degree, circuit size, and variable count even more visible.

52 FIFO networks and switched arithmetic dynamics [07]

52.1 Causality for several channels

The causal reconstruction argument is not inherently a one-queue argument. Consider p FIFO queues. At event i , a candidate trace consumes the word C_{ij} from queue j and appends the word A_{ij} to it. All reads occur before that event's writes. Local rules, if present, must separately validate the proposed input/output words. Let W_j, V_j be the initial and final queues, and write

$$\ell_{ij} = |W_j| + \sum_{k < i} (|A_{kj}| - |C_{kj}|), \quad G = \prod_{i < T} \prod_{j < p} (\ell_{ij})_{|C_{ij}|}.$$

Theorem 52.1 (FIFO network reconstruction). *The candidate words form a legal FIFO execution between the specified endpoints if and only if $G > 0$ and, for each queue j ,*

$$C_{0j}C_{1j} \cdots C_{T-1,j}V_j = W_jA_{0j}A_{1j} \cdots A_{T-1,j}. \quad (209)$$

Here “legal” refers to queue availability and the stated consumption/production words; satisfaction of any additional local control rule is a separate premise.

Proof. Any legal execution satisfies the word identities by following each queue's FIFO order, and its resource product is positive by Theorem 8.1.

Conversely, the same resource theorem makes every queue long enough for the prescribed consumption at each event. Before event i , the already produced prefix on queue j is $W_jA_{0j} \cdots A_{i-1,j}$. Its length is at least the number of symbols previously consumed on that queue plus $|C_{ij}|$. The corresponding next block in the global identity (209) therefore lies in this already produced prefix and equals C_{ij} . Simultaneously induct on all queues: the current queue is the suffix of its produced prefix after deleting all previous consumptions. Removing the next block and appending the designated output preserves that description. The final suffixes are exactly V_j . $\square$

52.2 A trace-symbol compiler for fixed-read networks

A useful concrete class of networks has a fixed finite cyclic control schedule. Each phase specifies a consumption vector $(c_{ij})_{j < p}$, independent of the data. A finite table maps the tuple of consumed symbols to one output word on each queue. Such a table may branch arbitrarily on the consumed tuple, but the number of symbols read in that phase is fixed. A phase is disabled if some queue is too short.

Fix T , all endpoints, and the program. Let

$$D = \sum_{i < T} \sum_{j < p} c_{ij}, \quad C = \max_{i < T} \sum_{j < p} c_{ij}, \quad h = C(B - 1).$$

For $T \geq 1$, introduce one alphabet-digit variable for each of the D consumed symbols, and one natural slack u . Tensor-product Lagrange interpolation of each finite output table gives its output length, output value, and output scale as polynomials of degree at most h in that event's digits. Give each queue its length and value equation from (209); append prefixes are accumulated by multiplying their scale interpolants. Use the single global resource residual $G - u - 1$.

Theorem 52.2 (Fixed-read network certificate). *The resulting residuals, after fixed integer denominator clearing, give a witness-faithful polynomial with $D + 1$ variables, $D + 2p + 1$ residuals, and degree at most*

$$2 \max\{B, Th, hD, 1\}.$$

For a fixed network program with a fixed number of queues, it has $O(T)$ arithmetic-circuit size. Its natural zeros correspond bijectively to exact T -event executions between the fixed endpoints.

Proof. The D digit residuals force the local table arguments to be alphabet symbols. Table evaluation then returns actual nonnegative lengths and actual finite words. The two residuals per queue imply (209), by equal length and equal radix value. The resource residual makes the global guard positive, so Theorem 52.1 reconstructs the execution. Conversely a legal execution provides all its consumed digits and the unique slack $G - 1$. The fixed schedule and deterministic tables determine every consumed word from the endpoints' initial queues, hence the complete witness is unique.

Each digit residual has degree B . An output value times the product of preceding output scales has degree at most Th . Formal queue lengths have degree at most h . The product guard has D linear-in-length factors and therefore degree at most hD . Including u and squaring gives the stated bound.

For explicit clearing, put $J = ((B - 1)!)^C$. Every tensor-product table interpolant has denominator dividing J . Multiply each length equation by J and each content equation by J^T . Writing $\lambda_{ij} = J\ell_{ij}$, the cleared guard is

$$\prod_{i < T} \prod_{j < p} \prod_{r < c_{ij}} (\lambda_{ij} - Jr) - J^D(u + 1).$$

It is an integer polynomial. Weighted integer prefix products give integer-only content circuits exactly as in Section 49. With the program fixed, table sizes, p , C , and B are fixed, while $D = O(T)$; the arithmetic size follows. $\square$

This class contains cyclic tag systems as the case of one queue, one symbol read per phase, and the two-entry append/skip table. It therefore includes universal computation, but the theorem does not claim that every individual network is universal. It also does not cover arbitrary data-dependent read counts or hidden zero tests without additional residuals. The general network theorem is proved here; the shipped executable compilers instantiate the one-queue cyclic-tag and deletion-tag cases, not the full network-table interface.

52.3 A two-track arithmetic substrate

On valid binary word codes, (205) becomes the following periodically switched affine rule:

$$(v, s) \mapsto \begin{cases} (v/2, s/2), & v \text{ even,} \\ ((v - 1 + a_i s)/2, 2^{m_i - 1} s), & v \text{ odd.} \end{cases} \quad (210)$$

The state domain is

$$\mathcal{W} = \{(v, 2^n) : n \in \mathbb{N}, 0 \leq v < 2^n\},$$

and the process stops at $s = 1$. When $m_i = 0$, the second multiplier is $1/2$, a rational coefficient, not an omitted case.

Corollary 52.3 (Conjugate arithmetic computation). *The map $\mathcal{E}_2$ is a bijection from binary words to $\mathcal{W}$ and conjugates cyclic-tag transitions to (210), including halting. Consequently this family of two-track, parity-switched rational-affine processes inherits cyclic-tag universality and the exact certificates of Theorems 46.1 and 51.1.*

Proof. Each integer $0 \leq v < 2^n$ has a unique length- n base-two digit expansion, including terminal zeros. For a nonempty word, its head is the parity of v , and (205) is precisely the stated branch formula. Scale one corresponds only to the empty word. This establishes a step-for-step bijection, not merely a simulation with unspecified overhead. $\square$

The switching condition is essential. This is not a claim that iteration of one everywhere-polynomial scalar function is universal. Nor does the two-coordinate lower bound apply to arbitrary parity-switched scalar dynamics.

52.4 What transports to spatial computation

Cook’s construction connects cyclic tags to glider interactions and then to Rule 110 [31]. A certified simulation can transport a statement about an encoded cyclic-tag run to the corresponding spatial computation. But the transport has to preserve the chosen event and endpoint semantics. A tag step is not automatically one cellular-automaton time step, and an encoded infinite periodic background is not a finite queue state.

Thus our displayed counts measure tag events or fixed-read network events. They do not establish a direct $T + 1$ -variable polynomial for every arbitrary T -step Rule 110 space-time diagram. A direct glider or cellular-automaton compiler with certified clocks, boundary conditions, and observation conventions is a separate research target. This distinction keeps the unconventional-substrate connection concrete without conflating different notions of time and input.

[write] *The bounded cellular-automaton constructions of the other manuscripts are finite space–time tableaux with a fixed number of cells: manuscript 05’s finite cellular automata and manuscript 04’s cellular automata in a finite box, manuscript 01’s circuit cylinders and manuscript 02’s Boolean spacetime substrates (all in Part VIII), and manuscript 03’s bounded cellular computation (Section 42). The “without tableaux” of manuscript 07’s title contrasts its queue certificates with such constructions; as just said, it does not claim a direct polynomial for arbitrary Rule 110 diagrams.*

53 Manuscript 07: Conclusion [07]

A queue computation has two logically different requirements: its consumed word must agree with what the computation produces, and each consumed symbol must already be present when it is read. A global radix identity addresses the first requirement. A first-failure product addresses the second without quantifying intermediate queues.

Together they give an exact $T + 1$ -variable cyclic-tag certificate, a $dT + 1$ -variable deletion-tag certificate, and a consumed-symbol-plus-one certificate for fixed-read FIFO networks. The no-slack and quartic alternatives show how circuit size, degree, and variable count can be traded without losing witness uniqueness. The polynomial-memory obstruction independently explains why two-coordinate local word memory cannot simply be replaced by one scalar with polynomial insertion at both ends.

These results answer concrete structural questions about Diophantine compilation. Their next useful tests are formalization, sharper resource bounds, and verified transport to genuinely spatial substrates. Their bounded uniqueness is a precise starting point for fixed-arity research, not a substitute for the additional argument that research requires.

Part VII

FRACTRAN, SKI and term rewriting

Source and scope. [write] Manuscript 04's Sections 6–7 give a linear-size, unique-witness certificate for bounded FRACTRAN runs with first-applicable-fraction priority (Theorem 54.2) and a context-local certificate for SKI reduction along a prescribed schedule (Theorem 55.2), extended to finite first-order rewriting (Corollary 55.6). The last section of the Part collects three shorter treatments: manuscript 02's local single-fold FRACTRAN step interface, manuscript 01's local polynomial adapter for combinatory logic and its warning about forward simulations, and manuscript 05's bounded word rewriting and explicit term graphs.

Relations. [write] For FRACTRAN, manuscript 04's certificate covers a complete bounded run; manuscript 02's interface certifies one step and the terminal condition, and is printed after it as the local counterpart. For combinatory logic the three constructions are different and all are kept: manuscript 04 encodes whole subterms by integer codes and follows a fixed schedule, manuscript 01 gives a local head-rule adapter with the injective pairing of Lemma 56.1, and manuscript 03's immutable graph evaluator is in Part V (Section 42.3). The repository's combinatory-logic development distinguishes source syntax, runtime syntax and evaluation strategy ([4]); none of these constructions is a formal bridge to it.

Setting and hypotheses. [write] Manuscript 04: an ordered list of r fractions in lowest terms and a horizon of T steps; SKI terms coded by $\text{App}(\ell, r)$ with leaves S, K, I coded 0, 1, 2, a fixed schedule σ of total context depth H . Manuscript 01: the same pairing $\pi(a, b) = (a + b)^2 + a$ with $\text{App}(a, b) = 3 + \pi(a, b)$, validity of the input term as a domain promise. Manuscript 05: a fixed alphabet, substring rules, horizon T and an explicit word-length bound B .

54 A linear-size, unique-witness FRACTRAN priority certificate [04]

54.1 Semantics and the issue with naive disjunction

A FRACTRAN program is a finite ordered list of positive fractions

$$\frac{a_1}{b_1}, \dots, \frac{a_r}{b_r}, \quad r \geq 1,$$

which we first put in lowest terms. At a positive integer n , choose the least j for which $b_j \mid n$ and replace n by $a_j n / b_j$. If there is no such j , execution halts. Reduction to lowest terms is essential: applicability of an unreduced fraction a/b is $b \mid an$, not generally $b \mid n$.

Diophantine treatment of this language is established prior work [30, 13]. Our question is the exact cost of a bounded, priority-correct, unique-witness certificate. An existential disjunction over all applicable fractions would lose priority. Giving every inactive branch an unrestricted remainder witness would also lose uniqueness. The following circuit handles both issues with a linear, rather than all-pairs, priority scan.

54.2 Canonical division and zero detection

Fix a positive denominator b . Introduce $q, \rho, s, u, z \in \mathbb{N}$ and impose

$$n = bq + \rho, \quad (211)$$

$$\rho + s = b - 1, \quad (212)$$

$$z(z - 1) = 0, \quad (213)$$

$$z\rho = 0, \quad (214)$$

$$\rho = (1 - z)(u + 1), \quad (215)$$

$$zu = 0. \quad (216)$$

Lemma 54.1 (Single-valued divisibility gadget). *For every $n \in \mathbb{N}$ and $b \geq 1$, equations (211)–(216) have exactly one natural solution. It has $q = \lfloor n/b \rfloor$, $\rho = n \bmod b$, $s = b - 1 - \rho$, and*

$$z = \mathbf{1}_{b|n}, \quad u = \begin{cases} 0, & \rho = 0, \\ \rho - 1, & \rho > 0. \end{cases}$$

Proof. The first two equations imply $0 \leq \rho < b$, so the quotient and remainder theorem forces q, ρ , and then s . If $\rho = 0$, the fifth equation makes $z = 0$ impossible, since it would say $0 = u + 1$. Hence $z = 1$, and the last equation forces $u = 0$. If $\rho > 0$, the fourth equation forces $z = 0$, and the fifth forces $u = \rho - 1$. These values satisfy all six equations. The argument also covers $b = 1$, where the remainder is always zero. $\square$

Equation (216) is not cosmetic. Without it, the divisible case would permit every natural u , producing infinitely many encodings of the same division.

54.3 A cumulative least-applicable-index circuit

At time t , run the preceding gadget for every denominator b_j , obtaining $q_{t,j}, \rho_{t,j}, s_{t,j}, u_{t,j}, z_{t,j}$. Add natural variables $h_{t,0}, \dots, h_{t,r}$ and $x_{t,1}, \dots, x_{t,r}$ satisfying

$$h_{t,0} = 1, \quad (217)$$

$$h_{t,j} = h_{t,j-1}(1 - z_{t,j}), \quad (218)$$

$$x_{t,j} = h_{t,j-1}z_{t,j}, \quad (219)$$

$$\sum_{j=1}^r x_{t,j} = 1, \quad (220)$$

$$n_{t+1} = \sum_{j=1}^r a_j q_{t,j} x_{t,j}. \quad (221)$$

The prefix variables say that no applicable index has yet occurred. No separate Boolean equations for h or x are required: induction forces their values to be bits.

Theorem 54.2 (Exact FRACTRAN compiler). *Fix a reduced positive r -fraction program, positive integers n_0^{in} and n^{out} , and $T \in \mathbb{N}$. There is an explicitly given polynomial of degree at most four with at most one natural zero; it has a zero if and only if the program executes exactly T steps from n_0^{in} to n^{out} . The construction uses exactly*

$$V_{\text{frac}} = T(7r + 2) + 1, \quad (222)$$

$$E_{\text{frac}} = T(8r + 3) + 2 \quad (223)$$

variables and residuals. With $A = \max_j a_j$ and $B = \max_j b_j$, every solution coordinate is at most

$$\max\{1, A^T n_0^{\text{in}}, B\}. \quad (224)$$

Proof. Use configurations $n_0, \dots, n_T$ and endpoint equations $n_0 = n_0^{\text{in}}, n_T = n^{\text{out}}$. At every $t < T$ impose the division gadgets and (217)–(221). Lemma 54.1 forces all the division data and all z bits from n_t . The prefix recurrence then forces $h_{t,j} = 1$ precisely when none of $1, \dots, j$ is applicable. Consequently $x_{t,j} = 1$ precisely at the first applicable index, if one exists; all other selectors are zero. Equation (220) rules out a halted step. The output equation is therefore exactly the FRACTRAN update.

Starting from the fixed positive input, this proves by induction that any solution is the actual run and all its auxiliaries are uniquely determined. In particular, each selected quotient is positive and every configuration remains positive. Conversely, an actual run supplies the uniquely determined division data and prefix bits and satisfies all equations. The final endpoint test is exact.

There are $T + 1$ configuration variables. Each step adds $5r$ division variables, $r + 1$ prefix variables, and r selectors, giving $(T + 1) + T(7r + 1)$ variables. Each index contributes six division equations and two prefix/selection equations; each step adds an initial prefix, a nonhalt test, and an output equation. The two endpoint equations give (223). All residuals have degree at most two, including the output products $q_{t,j}x_{t,j}$.

Finally, each update satisfies $n_{t+1} \leq A n_t$, so $n_t \leq A^t n_0^{\text{in}}$. Quotients are at most n_t , remainders and their slacks are below B , and all flags are bits. This proves (224). Sum the residual squares to obtain the polynomial. $\square$

The theorem uses “exactly T steps” to specify the length, not to require halting at time T . The next construction adds that requirement. For $T = 0$, the basic certificate has one variable and two equations and asserts equality of the positive endpoints.

54.4 First-halting certificates

For each denominator b_j , add three natural variables Q_j, U_j, V_j and two equations

$$n_T = b_j Q_j + U_j + 1, \quad (225)$$

$$U_j + V_j = b_j - 2. \quad (226)$$

They assert that division by b_j has a strictly positive remainder. If $b_j = 1$, the second equation has no natural solution, as it should: an integer fraction is always applicable.

Corollary 54.3 (Unique first-halting witness). *Adding (225)–(226) to the exact- T certificate gives a polynomial with a unique natural zero precisely when the first halt occurs at step T with the prescribed output. This adds $3r$ variables and $2r$ residuals, without increasing the degree bound.*

Proof. For $b_j \geq 2$, the equations force the remainder $U_j + 1$ to lie in $\{1, \dots, b_j - 1\}$, so quotient and remainder are unique, as are U_j, V_j . They are solvable precisely when $b_j \nmid n_T$. All denominators must fail. Earlier times were explicitly required to have an applicable fraction by (220), proving first halting rather than mere terminality of an unrelated state. $\square$

Example 54.4 (Priority and a complete halt). For the program $(3/2, 5/3)$ on input 8, the exact run is

$$8 \longrightarrow 12 \longrightarrow 18 \longrightarrow 27 \longrightarrow 45 \longrightarrow 75 \longrightarrow 125.$$

At 12 and 18, both denominators divide the current integer, but the first fraction must be chosen. At 125 neither is applicable. With $r = 2$ and $T = 6$, the basic certificate uses 97 variables and 116 residuals; the first-halting extension uses 103 variables and 120 residuals. Its largest witness coordinate is 125. The exact example is supplied as `fractran_first_halt.json`.

The height estimate gives witness bit lengths

$$O(1 + \log(n_0^{\text{in}} + 1) + T \log(A + 1) + \log(B + 1)).$$

Thus the bounded certificate has polynomial bit-size in the input, the program, and a unary horizon. This quantitative statement is stronger than a variable count alone. It does not provide a bound on the first halting time as a computable function of an arbitrary program and input.

55 Context-local Diophantine compilation of SKI rewriting [04]

55.1 A total natural coding of finite terms

Let SKI terms be the finite binary application trees with leaves S, K, I , under the usual context-closed rules

$$Ix \rightarrow x, \quad Kxy \rightarrow x, \quad Sxyz \rightarrow xz(yz). \quad (227)$$

Application associates to the left. We fix a concrete syntax; semantic equivalence of terms is not used as equality of their codes.

Define

$$\begin{aligned} \lceil S \rceil &= 0, & \lceil K \rceil &= 1, & \lceil I \rceil &= 2, \\ \text{App}(\ell, r) &= 3 + \frac{(\ell + r)(\ell + r + 1)}{2} + r. \end{aligned} \quad (228)$$

For natural n, ℓ, r , let the application residual be

$$C(n, \ell, r) = 2n - 6 - (\ell + r)(\ell + r + 1) - 2r. \quad (229)$$

It has degree two and integer coefficients.

Lemma 55.1 (Unique natural tree decoding). *Every natural number is the code of exactly one finite SKI term. For $n \geq 3$, there is exactly one pair $(\ell, r) \in \mathbb{N}^2$ with $C(n, \ell, r) = 0$, and both children are smaller than n . For $n < 3$, no such pair exists.*

Proof. Cantor's pairing function $\pi(\ell, r) = \frac{1}{2}(\ell + r)(\ell + r + 1) + r$ enumerates the diagonals $\ell + r = k$. On diagonal k , it takes the consecutive values $k(k + 1)/2, \dots, k(k + 1)/2 + k$, in increasing r . The next diagonal starts at the following integer. Thus it is a bijection $\mathbb{N}^2 \rightarrow \mathbb{N}$.

Equation $C = 0$ is exactly $n = 3 + \pi(\ell, r)$, giving the claimed unique pair and excluding $n < 3$. If $k = \ell + r$, then $\pi(\ell, r) \geq k$ for $k \geq 1$, and the assertion is also immediate for $k = 0$. Hence $\ell, r < n$. Strong induction on n now decodes a finite tree, with the three smaller codes interpreted as the leaves. The same induction and pairing uniqueness prove uniqueness. $\square$

There is no separate well-formedness witness. Moreover, $\text{App}(\ell, r) \geq \ell + r + 3$ shows inductively that the decoded number of tree nodes is at most $n + 1$. This is an effective bound, not a claim that decoding is polynomial in the *binary* length of n .

55.2 Prescribed schedules and context equations

A reduction schedule is a finite list

$$\sigma = ((R_0, \gamma_0), \dots, (R_{T-1}, \gamma_{T-1})), \quad R_t \in \{S, K, I\}, \quad \gamma_t \in \{L, R\}^*.$$

The path γ_t selects the redex in the term at time t . The schedule is *external data*, not an existentially chosen variable-length path. Define its total context depth

$$H = \sum_{t < T} |\gamma_t|,$$

and let n_I, n_K, n_S count its three rule types, so $n_I + n_K + n_S = T$.

Use global configuration variables $c_0, \dots, c_T$. During one descent through a context, suppose u, v are the old and new whole-context codes. For a left descent introduce the old selected child d , the sibling s , and the new selected child d' , and impose

$$C(u, d, s) = 0, \quad C(v, d', s) = 0. \quad (230)$$

Continue the descent with the pair (d, d') . For a right descent use

$$C(u, s, d) = 0, \quad C(v, s, d') = 0. \quad (231)$$

Each context level therefore contributes three variables and two residuals. Sharing the sibling variable forces the off-path subtree to be unchanged, without inspecting its interior.

At the selected root, use the following equations, with u, v now denoting old and new redex codes:

$$I : \quad C(u, 2, x) = 0, \quad v - x = 0; \quad (232)$$

$$K : \quad C(u, h, y) = 0, \quad C(h, 1, x) = 0, \quad v - x = 0; \quad (233)$$

$$S : \quad C(u, h, z) = 0, \quad C(h, g, y) = 0, \quad C(g, 0, x) = 0, \\ C(p, x, z) = 0, \quad C(q, y, z) = 0, \quad C(v, p, q) = 0. \quad (234)$$

The respective fresh-variable counts are 1, 3, and 7; their residual counts are 2, 3, and 6. Reusing the same argument code twice in the S rule represents actual syntactic duplication, not an obligation to create two independently encoded copies.

Theorem 55.2 (Context-local scheduled-reduction compiler). *Fix source and target codes $a, b \in \mathbb{N}$ and an external schedule σ . The context equations, root gadgets, and endpoint equations $c_0 = a, c_T = b$ have exactly one natural solution if the schedule is valid and transforms a into b , and none otherwise. Their sum of squares has degree at most four. The construction uses exactly*

$$V_{\text{SKI}} = T + 1 + 3H + n_I + 3n_K + 7n_S, \quad (235)$$

$$E_{\text{SKI}} = 2 + 2H + 2n_I + 3n_K + 6n_S. \quad (236)$$

In particular, $V_{\text{SKI}} \leq 8T + 3H + 1$ and $E_{\text{SKI}} \leq 6T + 2H + 2$, independently of the number of nodes inside untouched subtrees.

Proof. Fix a source code for one step. At each level of the scheduled descent, Lemma 55.1 forces the old selected child and its sibling. If that level is a leaf, the corresponding application equation is impossible. At the selected root, the old-side equations force the argument codes and test the required leaf I, K , or S . Thus an incorrect redex kind also makes the system impossible.

For a correct redex, the root output is uniquely determined: it is x for I or K , and $\text{App}(\text{App}(x, z), \text{App}(y, z))$ for S . All intermediate root variables are forced. Going back up the path, each new-side context equation uniquely forms an application with the already determined new child and old sibling. Hence the whole successor v and all context auxiliaries are unique and equal to the scheduled contraction. This is a two-way argument: a valid contraction supplies exactly those values and satisfies all equations.

Induct on the steps beginning at $c_0 = a$. A solution is precisely the prescribed valid reduction; its final code must equal b . Conversely, that reduction supplies its unique full witness. The counts

follow by adding $T + 1$ configurations, three variables and two equations per context level, the listed root-gadget costs, and two endpoint equations. Every residual has degree at most two, proving the quartic statement. $\square$

Example 55.3 (A two-step root computation). The term $(SKI)I$ has code 635. The root schedule $(S, \varepsilon), (K, \varepsilon)$ gives

$$(SKI)I \longrightarrow (KI)(II) \longrightarrow I.$$

Here $T = 2, H = 0, n_S = n_K = 1$, so $V = 13$ and $E = 11$. The certificate `ski_SKII.json` has a unique solution and maximum coordinate 635.

Example 55.4 (A nonempty context). The term $I(IK)$ has code 91. Contracting an I at path R and then at the root gives $I(IK) \rightarrow IK \rightarrow K$. Thus $T = 2, H = 1, n_I = 2$, giving $V = 8$ and $E = 8$. The exact file is `ski_context.json`.

55.3 The unavoidable arithmetic-height qualification

A small number of integer coordinates does not guarantee that those integers have small binary representations. The next bound makes the qualification explicit.

Proposition 55.5 (A coarse height bound). *For a valid scheduled reduction with source code a , let $g = H + 3n_S$. Every witness coordinate is at most a number M satisfying*

$$\log_2(M + 1) \leq 2^g (\log_2(a + 1) + 4) - 4. \quad (237)$$

Thus the construction controls the number of coordinates linearly, but its general bit-height guarantee can be exponential in the amount of rebuilt context.

Proof. Decomposition into subterms never increases the current maximum code. An I or K contraction creates no new application at its root. An S contraction creates three, and each context level creates one on reconstruction, so at most g new applications are formed in temporal order.

For children at most M , formula (228) gives

$$\text{App}(\ell, r) + 1 \leq 16(M + 1)^2.$$

This deliberately loose inequality follows directly from $\ell + r \leq 2M$. If $L = \log_2(M + 1)$, one new application increases the running maximum to a value whose L is at most $2L + 4$. Starting from a and iterating that recurrence at most g times gives (237). All witnesses are old subterms, reconstructed subterms, or configurations, so they are covered. $\square$

Accordingly, Theorem 55.2 is not a polynomial-time verification theorem in a measure that charges only for schedule length while treating large integer values as free. The code-based locality is genuine, but it trades structural expansion for potentially large arithmetic values. Better tree or DAG encodings are a concrete research target, not something supplied by the present pairing construction.

55.4 Extension to finite first-order rewriting

Corollary 55.6 (Fixed-schedule finite-pattern rewriting). *Consider a finite first-order term signature with a disjoint, injective natural constructor coding, each constructor having a fixed-size unique-extension quadratic certificate. Fix finitely many rewrite rules in which every variable on the right occurs on the left, and fix a schedule of rule names and context paths. Then the scheduled reduction admits a unique-extension quadratic residual certificate whose size is linear in the total selected-context size plus the total sizes of the selected rule patterns and constructed right sides.*

Proof. Decompose the source along the fixed context and then along the selected left pattern. Constructor injectivity makes all subterm codes unique. Repeated occurrences of a pattern variable use one shared code, thereby testing their equality; incompatible occurrences make the system unsatisfiable. Every right-side variable has now been determined. Construct the right side using the unique-extension constructor gadgets and rebuild the context. This is the same one-step uniqueness argument as in Theorem 55.2; iteration gives the result. Summing constant costs per visited pattern, constructor, and context node gives the stated bound. $\square$

For a finite signature, such constructor codings can be obtained by tagged pairing, with fixed-arity tuples built from a fixed number of pairings. The corollary concerns ordinary first-order substitution at a fixed occurrence. It does not silently extend to binding syntax with capture-avoiding substitution, or identify different evaluation strategies of the lambda calculus.

56 Local FRACTRAN and combinatory-logic interfaces, and bounded rewriting [02, 01, 05]

[write] *Three subsections from the substrate surveys of manuscripts 02 (its Section 10.2), 01 (its Sections 9.3–9.4) and 05 (its Section 9.4). Manuscript 02’s local FRACTRAN interface certifies one step and the terminal condition; the complete bounded run with first-applicable priority is manuscript 04’s Theorem 54.2 above.*

56.1 A local single-fold FRACTRAN interface

For completeness, a second substrate illustrates how to avoid local multiplicity before confronting the difficult global problem. Fix a FRACTRAN list

$$\frac{p_1}{q_1}, \dots, \frac{p_m}{q_m}, \quad p_i, q_i > 0.$$

At positive integer state n , the first fraction producing an integer is applied. For every i , introduce the *unique* Euclidean quotient and remainder

$$p_i n = q_i a_i + r_i, \quad 0 \leq r_i < q_i.$$

All quotients and remainders are evaluated, not just those before or at the selected fraction. The applicable index j , if any, is characterized by $r_j = 0$ and $r_i \neq 0$ for $i < j$; the next state is a_j . The terminal condition is $r_i \neq 0$ for every i .

These finite arithmetic conditions have unique quotient–remainder witnesses and a unique selected index. The circuit construction therefore gives single-fold local step and terminal-state certificates. This does not imply a single-fold certificate for an unbounded FRACTRAN run. FRACTRAN’s role as a useful intermediate model in a full mechanized MRDP proof is established by Larchey-Wendling and Forster [14]; our discussion here isolates the distinct local uniqueness issue.

56.2 A local polynomial adapter for combinatory logic

The repository’s combinatory-logic development makes careful distinctions between source syntax, runtime syntax, and evaluation strategy. In particular, its SKI-to-Iota simulation is not declared reduction-reflecting or fully abstract, and its exact Coq equivalence uses a weak-call-by-value lambda model distinct from the scoped contextual one [4]. A new Diophantine adapter must not erase those distinctions.

There is nevertheless a useful elementary local construction. Define

$$\pi(a, b) = (a + b)^2 + a, \quad \text{App}(a, b) = 3 + \pi(a, b). \quad (238)$$

Give the three SKI leaves codes 0, 1, 2. Codes of valid terms are generated inductively by these leaves and App.

Lemma 56.1 (Injective polynomial application coding). *The map $\pi : \mathbb{N}^2 \rightarrow \mathbb{N}$ is injective. Moreover, $\text{App}(a, b) > \max(a, b)$, so validity of an alleged finite-term code is decidable by finite recursion on smaller codes.*

Proof. For $s = a + b$, the value lies in the interval $[s^2, s^2 + s]$. Consecutive such intervals are disjoint because $(s + 1)^2 > s^2 + s$. Thus the value determines s , then determines $a = \pi(a, b) - s^2$, and then $b = s - a$. This proves injectivity. Since $s^2 \geq s$ for natural s , the application code is greater than both children. To test validity, search finitely for a pair of children with the given application code and recursively test the unique pair if it exists; both recursive arguments are smaller. $\square$

For the head rule $Sxyz \rightarrow xz(yz)$, fixed valid input code u and proposed output code v can be related by the six quadratic equations

$$\begin{array}{lll} t_1 = \text{App}(0, x), & t_2 = \text{App}(t_1, y), & u = \text{App}(t_2, z), \\ l = \text{App}(x, z), & r = \text{App}(y, z), & v = \text{App}(l, r). \end{array}$$

All seven natural witnesses x, y, z, t_1, t_2, l, r are unique when they exist, because successive decompositions of the valid input determine the three arguments and then all constructions are functional. Their squared residuals give a quartic local rule. The same observation applies to the K and I head contractions.

Two restrictions must remain visible. First, the construction as stated has *validity of the input term as a domain promise*. Applied indiscriminately to arbitrary natural codes, it could decompose a numeral with invalid subterms. Adding an arbitrary MRDP representation of validity need not preserve unique fibres. Second, contextual reduction needs a redex position and a surrounding context, and arbitrary-length reduction needs an additional history mechanism. The local rule therefore gives a concrete starting component, not a completed canonical representation of unrestricted SKI or Iota evaluation.

56.3 What cannot be transported from a forward simulation alone

Suppose a compiler maps source steps into nonempty target traces. This proves preservation of source execution, but by itself it does not prove that every target accepting trace is the image of a source trace. Nor does it prove that one source trace has a unique target realization. A parsimonious transport requires a decoding or reflection theorem and control of administrative target steps.

For ProveIt, this means that the existing operational embeddings are valuable reusable components, but a fibre-preserving Diophantine bridge must state the evaluation relation it preserves and prove the missing reflection and canonicalization obligations. The present guarded-translation construction avoids that ambiguity by defining its substrate and its represented histories explicitly.

56.4 Bounded word rewriting and explicit term graphs

Fix a finite alphabet, a finite family of substring replacement rules, a horizon T , and an explicit maximum word length B . Store the word in B cells with a canonical blank suffix and a uniquely encoded length. At each step choose a rule and a position using a fixed one-hot code, check the matched substring and

size bound, and compute the uniquely padded next word. This is a finite deterministic verifier of the candidate choice history. Its circuit can be constructed by finite comparisons, multiplexers, and shifts of the B cells. By Theorem 57.1 it has a quadratic witness-preserving arithmetic encoding.

The objects counted here are *labelled derivations*, including the chosen rule and position, not merely their endpoint words. Distinct rules with identical replacements count separately unless the object definition first quotients them. For length-changing rules, the numerical positions of later redexes move; declaring two static position labels independent without accounting for that movement is unsound.

An analogous finite verifier is available for a specified bounded term-graph representation with deterministic allocation conventions. For tree representations of duplicating combinators, a bound on the number of reduction steps need not be a comparable bound on tree size. A size budget or a fully specified sharing representation is therefore essential. This article does not claim a linear-size arithmetic encoding of arbitrary tree expansion merely from a linear step bound.

Part VIII

Other substrates

Source and scope. [write] Five manuscripts close with a survey of further computational substrates reached by their methods. This Part prints manuscript 05's (Boolean verifiers, counter and register programs with zero tests, finite cellular automata, exact time and padding), manuscript 04's (counter machines, stacks, Boolean circuits and cellular automata in a finite box, a compositional statement), manuscript 01's (counter machines with finite control, finite arithmetic and Boolean circuits), manuscript 02's (fixed-horizon counter machines, finite Boolean spacetime substrates) and manuscript 06's (what transfers without a further theorem, an abstract compiler contract, and why doubling, stacks and rewriting need more work). The rewriting subsections of these surveys are in Part VII, and their boundary subsections in Part IX.

Overlaps. [write] Boolean circuits are treated by four manuscripts; manuscript 05's Theorem 57.1 (affine equalities, convex quadratic, bounded real relaxation) is the strongest statement, and the circuit encodings of manuscripts 01, 02 and 04 are unique-extension encodings of the same kind with quadratic gate equations, hence quartic rather than quadratic polynomials. Counter machines appear in manuscripts 01, 02, 03 (Part V), 04 and 05; manuscript 06 is the main treatment of counter programs (Parts I–III), and the others are cross-referenced. Bounded cellular automata are compiled as finite space–time tableaux by manuscripts 01, 02, 03 (Section 42), 04 and 05; manuscript 07 does not compile Rule 110 diagrams directly and explains why its queue certificates do not transport to a $(T + 1)$ -variable space–time polynomial (the end of Section 52).

57 Other computational substrates [05]

The preceding theorem is most informative when the arithmetic front end is reusable. We give precise extensions and state which parts retain the Petri-net commutation theorem. In every case, size and time bounds refer to an explicitly unrolled finite computation, not a fixed-dimensional universal equation.

57.1 A general Boolean verifier compilation lemma

Theorem 57.1 (Unique extension of a finite verifier). *Let C be a finite deterministic Boolean circuit whose input strings encode candidate computational objects, with exactly one input encoding for each*

object. There is an effectively constructed system of affine integer equalities in natural variables, of size linear in the number of inputs and gates, whose full natural solutions are in bijection with the inputs accepted by C . Its sum-of-squares polynomial is convex quadratic, and its nonnegative real solution set is bounded.

Proof. For each input bit p , introduce its complement $\bar{p}$ and impose $p + \bar{p} = 1$. This gives exactly one complement for each possible bit. Process gates topologically. A NOT gate is $p + y = 1$. For an AND gate, use (2), with output u . For an OR gate, first compute $u = p \wedge q$ by that gadget and impose $y + u = p + q$. Constants have equations $y = 0$ or $y = 1$. Finally set the acceptance output to one.

On natural assignments the gate values are forced Boolean by induction, and all slacks are unique by Lemma 2.3. Thus every accepted input has one full extension and every full solution decodes an accepted input. Over the reals the input bits are in $[0, 1]$. The AND bounds keep the output in $[0, 1]$, the NOT equation does the same, and the lower bound $u \geq p + q - 1$ makes the OR output at most one. Its nonnegativity is part of its domain. All auxiliary slacks are bounded as in Lemma 2.3. Hence the real feasible set is bounded. Scalarization proves the polynomial assertion. $\square$

The unique input-encoding assumption must not be skipped. Padded records with unconstrained unused bits, multiple syntactic descriptions of the same object, or ignored nondeterministic choices would all change the solution count. The lemma is a compiler for a specified verifier and specified coding convention, not a theorem that arbitrary encodings are automatically parsimonious.

57.2 Counter and register programs, including zero tests

A finite counter program with natural registers and finite control can be unrolled using the raw variables of Section 10. Increment and decrement are consumption–production updates. To enforce a control edge with source $s(a)$ and target $t(a)$, add a control variable q_i and equations

$$q_i = \sum_a s(a) e_{i,a}, \quad q_{i+1} = \sum_a t(a) e_{i,a},$$

with prescribed initial and final control values. Equivalent one-hot control places can be used instead.

For a transition a carrying a zero guard on register p , add

$$e_{i,a} x_{i,p} = 0. \tag{239}$$

This imposes the guard exactly on the selected branch. There is no auxiliary witness in the inactive branch, so uniqueness is unchanged. Nonzero/decrement guards are already expressed by consumption of a token. Thus a bounded counter-program execution has a direct parsimonious quartic sum-of-squares encoding, with finite control and every guard explicit.

For fixed numerical initial registers and a fixed horizon, the same extension can be made quadratic. Let $B_p = M_p + T \max_a v_{a,p}$, known from the raw induction. Replace (239) by the affine equation

$$x_{i,p} + s_{i,a,p} = B_p (1 - e_{i,a}), \quad s_{i,a,p} \in \mathbb{N}. \tag{240}$$

When $e_{i,a} = 1$, it forces $x_{i,p} = 0$. Otherwise it only enforces the already valid bound $x_{i,p} \leq B_p$, and its slack is unique. The coefficient B_p is a fixed integer for this instance. When the initial registers are free parameters, $B_p e_{i,a}$ is no longer jointly affine; the uniform-parameter presentation is then quartic unless another representation is supplied.

[write] *Manuscript 05 continues here with its exact commutation theorem for interval guards, printed in Part I as Theorem 4.5 (Section 4.3), and its corollary for guarded programs, printed in Part II as Corollary 11.5 (Section 11.5).*

57.3 Finite cellular automata

Consider a one-dimensional deterministic cellular automaton on a ring of W cells, alphabet size q , radius r , and fixed local rule. Give each cell a one-hot alphabet encoding, initially constrained by a sum equal to one. The local rule can be compiled by testing each of its q^{2r+1} neighbourhood patterns with Boolean conjunctions and collecting patterns that produce each output symbol. Thus one time layer has a deterministic Boolean circuit of size

$$O(W(r+1)q^{2r+1}),$$

and T layers have T times this size. Apply Theorem 57.1, treating the initial one-hot rows as the candidate data and their sums as affine constraints. Every valid initial configuration has exactly one space–time history and exactly one arithmetic extension. Constraints on the final pattern select its preimages without introducing extra encodings.

The same construction works on a finite interval with explicitly specified boundary conditions. For an infinite quiescent-background automaton, one must first specify and justify a sufficient finite space–time region; the ring theorem alone does not supply that justification. Also, synchronous local updates are not a sequence of freely swappable asynchronous transitions. A schedule quotient for an asynchronous automaton needs its own partial-map semantics.

[write] *Manuscript 05's subsection on bounded word rewriting and explicit term graphs, which stands here in the manuscript, is printed in Part VII (Section 56).*

57.4 Exact time, first halting, and padding

All previous constructions use an exact horizon. For a deterministic program that halts in $t \leq T$ steps, a unique length- T record can be obtained by a distinguished stopped mode with a single mandatory padding action and frozen data. The first visit to stopped mode is the halting event; every later position is forced. The native time is then a uniquely determined observable.

Allowing arbitrary stuttering at arbitrary positions is different. It introduces multiple records of the same computation and can change the interaction with the commutation relation. Unused bits in a padded record must also be forced. A compiler for “at most T ” needs a canonical padding convention as part of its correctness specification, not just an existential choice of a shorter length.

58 Other computational substrates through the same interface [04]

The three main compilers illustrate separate difficulties: concurrent equivalence, priority, and context locality. The unique-extension viewpoint also gives a disciplined route to further substrates. The following constructions are elementary applications of Section 2.5, not claims of newly discovered universality.

58.1 Counter machines and stacks

An increment of a natural register has equation $R' = R + 1$. A nonzero decrement has $R = R' + 1$, with $R' \in \mathbb{N}$ supplying the guard. A zero branch has $R = 0$ and the appropriate unchanged-register equations. Finite control and labeled instruction choices can be encoded by one-hot selectors and Lemma 2.14. A fixed horizon gives one solution per valid labeled instruction trace. In a deterministic program the guards, rather than an arbitrary disjunction, must force the chosen branch.

For a stack over digits $1, \dots, B-1$, use zero for the empty stack and encode the top digit as the least significant digit. A push of digit d and its inverse pop are

$$S' = BS + d, \quad S = BS' + d,$$

respectively. The selected digit is part of the labeled operation; it can alternatively be selected from the finite alphabet by a canonical circuit. Emptiness is $S = 0$. Starting with a valid stack, these equations preserve the no-zero-digit representation and hence the intended stack semantics. For a run of T pushes and pops, $S + 1 \leq B^T(S_0 + 1)$, so its code has $O(T \log B + \log(S_0 + 1))$ bits. Several stacks and finite control can be handled simultaneously.

A tape machine encoded by stacks additionally needs a convention for the current cell and implicit blank tails. Without that convention, distinct finite blank extensions would be distinct arithmetic objects representing the same tape. This is precisely the type of representational multiplicity that the compiler interface requires one to resolve explicitly.

58.2 Boolean circuits and cellular automata in a finite box

For Boolean inputs, AND and NOT gates have unique-extension equations

$$z = xy, \quad z + x = 1.$$

Output Booleanity follows from the input Booleanity. Thus an acyclic circuit has one gate assignment per input, and every gate residual is quadratic. A finite-alphabet cellular automaton in a fixed finite box can be represented by a Boolean encoding of each cell and a fixed local update circuit. With L cells, T rounds, and C gates per local update, a direct synchronous tableau uses $O(LTC)$ gate variables, together with the cell variables; all are bits. Boundary conditions and invalid finite-alphabet bit patterns must be specified, not inferred from an informal diagram.

Asynchronous local systems require more care. A proposed independence relation must guarantee that swapping actions preserves enabling and outcome. Disjoint complete read/write footprints are a sufficient condition. Disjoint write sets alone are not: an action may read a location changed by the other. Once a sound swap property is proved and the finite-action guards and updates have unique-extension certificates, the normal-form monitor of Section 14 applies independently of the particular state representation.

58.3 A compositional statement

Proposition 58.1 (Trace-faithful composition). *Suppose a finitely labeled transition system has unique-extension polynomial certificates for each labeled step, and a fixed independence relation with the enabled-swap property. Suppose further that a length- T operational tableau has no duplicate state encodings. Then adjoining the monitor equations yields a certificate whose solutions are in bijection with the independent-swap classes of the length- T endpoint traces. Quadraticization and the sum-of-squares construction preserve that bijection.*

Proof. Use canonical branch activation to form the labeled-step tableau, ensuring that every labeled trace has exactly one auxiliary extension. Append the deterministic flag tableau for its action word. By the normal-form theorem, precisely the least words survive. The enabled-swap property makes the endpoint language trace-closed, so every class still has its least word. Unique-extension quadraticization adds no multiplicity. This is the proof of Theorem 15.2 with the specific token equations replaced by the assumed step certificates. $\square$

The hypotheses are substantive. In particular, state-coding injectivity and canonical inactive variables are not automatic consequences of a simulation theorem or of ordinary Diophantine definability.

59 Other computational substrates and safe adapters [01]

59.1 Counter machines with finite control and zero tests

The guarded-translation model can describe a fixed finite counter program directly. Add a program-counter coordinate q to the nonnegative registers. An instruction with source label i and target label j imposes the exact guard $q = i$ and displacement $j - i$ on that coordinate. An increment of register t adds $+1$ to that register with lower guard zero. A positive decrement branch has lower guard one and displacement -1 ; its zero branch uses the exact guard zero and displacement zero. Other registers have zero displacement and unrestricted nonnegative guards.

For the program-counter coordinate, $\ell = i$ and $v = j - i$ satisfy (9), because $j \geq 0$. For a decrement, its lower guard similarly guarantees natural output. Consequently the general envelope and history theorems apply to the resulting finite instruction alphabet.

This adapter is operational: the action labels retain which branch was taken, and each step agrees exactly with the specified counter instruction. It does not assert that all instructions commute. In a sequential program most useful adjacent instructions will be dependent, and the causal height may equal the ordinary execution length. Commuting-block acceleration applies only where the exact pair test certifies it.

To use first-halt semantics, the halt state has no outgoing action. Adding a free halt self-loop would create many finite histories after the same successful computation; neither determinism nor the earlier result would remove those genuinely different words. The classical use of counter or register machines in Diophantine arithmetization supplies the context for this adapter, not a new universality proof [8, 14].

59.2 Finite arithmetic and Boolean circuits

A second elementary adapter concerns deterministic acyclic circuits. A gate equation such as $t = u + v$, $t = uv$, or a Boolean NAND equation $t = 1 - uv$ has degree at most two. Boolean wires can be constrained by $t(t - 1) = 0$. In a topological order the input values determine every wire value uniquely; hence the sum of squared gate residuals has degree at most four and a unique witness for the entire circuit evaluation.

More generally, any acyclic list of quadratic functional constraints with uniquely determined natural outputs can be lifted this way. The statement is proved by induction in the gate order, not by assuming that low degree alone forces uniqueness. A multiplication equation can have many factorizations if its two inputs are not already fixed.

This familiar circuit observation covers finite Boolean state-transition circuits and finite cellular-automaton cylinders with specified boundary conditions. It is a bounded circuit construction, not an unbounded cellular-automaton single-fold theorem. Boundary values, update order, and the exact cells represented must be part of the input specification. Unspecified boundaries would be additional semantic choices, not harmless auxiliary witnesses.

[write] *The remaining two subsections of this section of manuscript 01, a local polynomial adapter for combinatory logic and what cannot be transported from a forward simulation, are printed in Part VII (Section 56).*

60 Other substrates and the limit of the construction [02]

60.1 Fixed-horizon counter-machine certificates

A fixed finite counter program has configurations consisting of a finite control label and a fixed tuple of natural counters. Increment, decrement with zero test, and control transfer are described by finite Boolean combinations of polynomial equations and comparisons. The control and zero-test branches are mutually exclusive for a deterministic program.

By Lemma 2.8, each one-step relation has a single-fold arithmetic certificate when its endpoint configurations are fixed. For an *externally fixed* horizon T , introduce the $T - 1$ intermediate configurations and the one-step certificates. The number of variables is $O(T)$ for a fixed program. A deterministic initial configuration determines every intermediate configuration uniquely, so this unrolled system is single-fold whenever the claimed run exists. For a halting certificate, earlier configurations must also be required not to be halting configurations, unless the chosen semantics already prevents transitions from them.

This is a family of polynomials whose arity grows with T . It is not a fixed polynomial with T as a free parameter. The substantive gain in Theorem 31.2 is precisely the elimination of that duration-dependent arity for a restricted, nontrivial class of updates.

[write] *Manuscript 02's local single-fold FRACTRAN interface, the second subsection of this section, is printed in Part VII (Section 56); its last three subsections, on the exponentiation boundary, universal first-halting computations and witness heights, are in Part IX (Section 64).*

60.2 Finite Boolean spacetime substrates

A finite Boolean circuit has unique internal wire values once its inputs are fixed. The same is true of a finite spacetime diagram obtained by unrolling a deterministic cellular automaton with a fixed local rule, specified boundary data, and an externally fixed time window. Boolean gate equations turn such a diagram into a single-fold quartic system. The witness count scales with the number of explicitly represented cells and time steps.

Again, this elementary fact does not provide a fixed-dimensional polynomial certificate for an unbounded computation. It clarifies a useful design principle across substrates: local single-foldness is often straightforward, whereas unique compression of variable-length execution is the real obstacle.

61 Other computational substrates: transfers and limits [06]

61.1 What transfers without any additional theorem

The same resource compiler applies to a fixed finite multiset-rewriting system: a rule consumes the multiset u_a and produces v_a . A discrete chemical reaction network with stoichiometric integer updates is an instance of these firing semantics. The claim concerns exact discrete firings and token availability, not continuous concentrations, reaction rates, or stochastic kinetics.

Finite control can be included by adding one place for each control state. A transition consumes the token of its source control state and produces the token of its target state. On markings with exactly one control token, this reproduces the finite-state transition condition, while the remaining places hold the original resources. The summary theorem then holds for the expanded Petri system; semantic equivalence on all expanded markings is stronger than one restricted to the one-control-token invariant. Accordingly the globally computed independence relation can be conservative for the intended invariant, but it is sound.

Bounded channels containing indistinguishable messages can also be expressed by resource counts, with a complementary capacity place when needed. A FIFO channel over distinct symbols is different:

its state includes order, not just multiplicity. The present proof does not encode that order by pretending that each symbol count is sufficient.

61.2 A reusable abstract compiler contract

The resource argument suggests the following transfer principle.

Proposition 61.1 (Local unique summaries imply global unique compilation). *Suppose a class of word actions has finite-dimensional natural summaries with: constant summaries for letters and the empty word; exact composition and parameter-power relations given by fixed-size quadratic systems having a unique output and unique local auxiliaries for each valid input summary; and an endpoint-action relation with unique local witnesses. Assume the relations preserve validity of summaries. Then every fixed parametrized repetition expression in these actions has a single-fold quartic representation of its endpoint relation, with a number of witnesses linear in its syntax size, up to the fixed local summary dimensions.*

Proof. Allocate all child summaries and local auxiliaries as in Theorem 25.1. Structural induction on the syntax forces each valid summary and every local auxiliary. The endpoint contract gives the final zero-or-one solution condition. Every residual is quadratic, so its sum of squares is quartic. Summing the fixed local allocations proves the size bound. $\square$

This is a useful design contract, not a theorem that arbitrary machine models satisfy its hypotheses. The difficult clause is usually the unique polynomial representation of arbitrary powers, or the existence of a finite-dimensional exact summary in the first place. If local residuals have degree at most D instead, the same argument yields degree at most $2D$.

61.3 Why doubling, stacks, and rewriting require more work

If a primitive command doubles a counter, then its k th power applied to one produces 2^k . A generalization of our arbitrary-power interface to that command would in particular provide the corresponding witness-controlled polynomial specification of exponentiation. Our translation-only resource algebra avoids this issue; multiplication in (28) multiplies an externally supplied repetition count by a summary component, not the runtime counter value by itself at each iteration. These are different operations.

For stacks, a word of pushes and pops has cancellation information akin to a required prefix and a residual suffix, but these summaries are words of unbounded length, not fixed tuples of ordinary counters. Encoding the words as integers then introduces base powers, length constraints, and canonical coding questions. A proof of plain Diophantineness via MRDP would not preserve single-foldness automatically.

Lambda terms and SKI or Iota reductions introduce another issue: sharing and copying of syntax. A selected finite reduction sequence can be checked by an interpreter, but an exact compressed summary of arbitrary iterated rewriting need not have the affine principal-domain structure used here. Such models are natural continuation targets for ProveIt, but they are not covered by the current compiler. A reliable extension should state its own summary and witness contracts before claiming transfer of our bounds.

Part IX

The boundary of fixed-arity compression

Source and scope. [write] Every construction of Parts I–VIII is either a family of polynomials indexed by a horizon, a height, a length or a schedule, or a fixed-arity polynomial for a restricted relation. Every manuscript ends by explaining why this does not yield one fixed-arity single-fold or finite-fold polynomial for unbounded universal computation. This Part prints manuscript 05’s boundary section first (base), then those of manuscripts 01, 02, 03, 04, 06 and 07.

Merged statements. [write] Four manuscripts prove that a single-fold (finite-fold) polynomial for universal halting exists exactly when every computably enumerable set has a single-fold (finite-fold) Diophantine representation: 05, 01, 03 and 04, with manuscript 05 adding a decidable trace-verifier form, manuscript 04 a degree-four form and manuscript 01 relations of any finite arity; manuscript 02 argues the first-halting form informally. The statement is printed once, as Theorem 62.1, in manuscript 05’s section; each other manuscript’s proof stays in its own section, after a note giving its letters. Five manuscripts prove that no total computable function bounds a witness of a universal (or any undecidable) representation; that is printed once, as Proposition 62.2, in the most general form (manuscript 02’s), with the proofs in place. Manuscript 06’s finite-fold substrate equivalence (Theorem 67.1) and its parameter-bound obstruction (Proposition 67.2) are different statements about separated one-counter schemes and are kept whole. Manuscript 07 proves neither theorem; it separates horizon-indexed from fixed-arity representations and remarks that its witness bound gives no computable bound on the unknown halting time.

What is and is not claimed. [write] The equivalence is a reduction, not a resolution: nothing in this report proves or refutes the existence of a single-fold or finite-fold universal polynomial. The single-fold *exponential* Diophantine representation of every enumerable set (Jones–Matiyasevich 1984) is formalized in ProveIt (Remark 64.2); the ordinary polynomial version is the open problem.

62 Fixed-dimensional compression and the finite-fold boundary [05]

The constructions above are a family of equations P_T , with a number of unknowns growing with T . A single polynomial with a fixed number of unknowns can represent reachability by MRDP or the repository’s exact-iteration machinery, but preservation of witness multiplicity requires a separate argument. We make the precise logical strength of one natural proposed compression explicit.

62.1 A decidable verifier with one successful history

Fix a deterministic universal machine U , a canonical encoding of its programs and inputs, and a canonical encoding of finite configuration lists. Define

$$\text{Trace}_U(e, x, h)$$

to mean that h codes the complete execution of program e on input x , beginning in its specified initial configuration and ending at the *first* halting configuration. There is no padding and no irrelevant annotation. A run halting immediately has the singleton list containing its initial configuration.

This relation can be chosen primitive recursive. For specificity, a useful polynomial pairing injection is

$$\pi(a, b) = (a + b)^2 + a. \quad (241)$$

It is injective: for $s = a + b$, its values lie in $[s^2, s^2 + s]$, and these intervals for distinct s are disjoint. Within an interval, a determines b . Encode the empty list as zero and a nonempty list as

$$\text{cons}(a, h) = 1 + \pi(a, h).$$

The tail is strictly smaller than the code. Decoding is by bounded search, rejecting numbers that are not valid codes, and checking the whole list requires only bounded iteration. Choose a standard machine encoding with primitive-recursive initial-configuration, one-step, and halting tests. The verifier checks all adjacent pairs and verifies that no earlier configuration halted. It is therefore primitive recursive.

Determinism and the absence of padding imply

$$\text{for each } (e, x), \quad \#\{h : \text{Trace}_U(e, x, h)\} \leq 1. \quad (242)$$

Existence of this unique history is equivalent to $U(e, x)$ halting. The history can be enormous; decidability of a proposed finite history says nothing about deciding whether some such history exists.

62.2 An exact equivalence theorem

Theorem 62.1 (Universal halting and the single-fold and finite-fold problems; manuscripts 05, 01, 03 and 04). *Fix a universal programming system and let $K(e, x)$ mean that program e halts on input x . The following statements are equivalent.*

- (a) *Every computably enumerable subset of $\mathbb{N}$ has a finite-fold polynomial Diophantine representation over $\mathbb{N}$.*
- (a') *Every computably enumerable relation of finite arity over $\mathbb{N}$ has a finite-fold polynomial Diophantine representation over $\mathbb{N}$.*
- (b) *There is one integer polynomial $H(e, x, z)$, with a fixed finite tuple z , representing universal halting $K(e, x)$ and having finitely many z -solutions for every fixed (e, x) .*
- (b') *There is such a polynomial H of degree at most four.*
- (c) *There is one integer polynomial $V(e, x, h, y)$, with a fixed finite tuple y , representing the decidable relation Trace_U of Section 62, with finitely many y -solutions for every fixed (e, x, h) .*

The statements remain equivalent when “finite-fold” and “finitely many” are replaced throughout by “single-fold” and “at most one”; in the single-fold form of (b), the natural zero set of $H(e, x, \cdot)$ is a singleton when $K(e, x)$ holds and empty otherwise.

Sources, letters and proofs. [write] Manuscript 05 states $(a) \Leftrightarrow (b) \Leftrightarrow (c)$ for a deterministic universal machine U with the canonical first-halting histories defined above; its proof follows. Manuscript 01 proves $(a') \Leftrightarrow (b)$ for a universal interpreter of register programs with first-halt semantics; its (i) is (a') and its (ii) is (b) (proof in Section 63). Manuscript 03 proves $(a) \Leftrightarrow (b)$ for a universal deterministic interpreter of an acceptable enumeration; its (i) is (a) and its (ii) is (b) (proof in Section 65). Manuscript 04 proves $(a) \Leftrightarrow (b) \Leftrightarrow (b')$ for an effective universal programming system with halting relation $H(e, x)$; its (i), (ii), (iii) are (a) , (b) , (b') (proof in Section 66). The implications $(a') \Rightarrow (a)$ and $(b') \Rightarrow (b)$ are immediate, and $(b) \Rightarrow (a')$ is manuscript 01's argument; with the cycles proved by manuscripts 05 and 04, all five statements are equivalent. Manuscript 02 argues the first-halting form informally (Section 64.2), and manuscript 06 proves an equivalence for its own substrate (Theorem 67.1). The theorem is a reduction: it does not say whether the statements are true. The formalized exponential counterpart is recorded in Remark 64.2.

Proof of $(a) \Leftrightarrow (b) \Leftrightarrow (c)$, both versions (manuscript 05). $(a) \Rightarrow (c)$. The set of triple codes

$$S = \{\pi(e, \pi(x, h)) : \text{Trace}_U(e, x, h)\}$$

is computably enumerable, indeed decidable using the chosen bounded decoding. By (a), there is a polynomial $Q(n, y)$ with finite fibres representing S . Set

$$V(e, x, h, y) = Q(\pi(e, \pi(x, h)), y).$$

This is an integer polynomial because the code map is polynomial. Its natural y -fibre at a fixed triple is exactly the corresponding Q -fibre, so it has the required finiteness and represents exactly the trace relation.

$(c) \Rightarrow (b)$. Existentially quantify h as one more witness and use V itself as the halting polynomial. For fixed (e, x) , a nonhalting computation has no possible h , and a halting computation has exactly one possible h by (242). Above this unique value there are only finitely many y by (c). Hence the full (h, y) -fibre is finite and is nonempty exactly for halting inputs.

$(b) \Rightarrow (a)$. For a computably enumerable set A , choose a program e_A that halts on x exactly when $x \in A$. Specializing the program parameter gives the polynomial $H(e_A, x, z)$ with finite fibres representing A .

Every step also preserves “at most one”: polynomial substitution does not add witnesses, specialization does not add witnesses, and in the middle implication there is at most one admissible h as well as at most one admissible y . This proves the single-fold version. $\square$

This is a useful obstruction to an overambitious compiler contract. A fixed polynomial with finite fibres for a perfectly decidable first-halting trace verifier is already as strong as the general finite-fold conjecture. Calling the input a “verified finite history” does not make the required *uniform arithmetic representation* harmless. The verifier is fixed while its history parameter is unbounded, and projecting that parameter yields universal halting.

The theorem is an equivalence and a design boundary, not a proof that such a polynomial cannot exist. The finite-fold and single-fold questions are the classical stronger representation problems discussed in [9, 10]. Nothing in our bounded family contradicts a positive resolution of either question. It simply does not furnish that resolution.

62.3 Why obvious compression arguments lose multiplicity

A common informal argument says: encode a long execution in one integer, represent the digit tests Diophantinely, and combine all equations. This can correctly prove *existence*. It does not establish that a fixed history code has only one, or only finitely many, auxiliary arithmetic encodings. A representation of an exponentiation or sequence-decoding relation may have uncontrolled fibres. Each such relation needs its own finite-fold or single-fold theorem before this property can be composed.

Likewise, choosing “the least witness” at the semantic level is not a polynomial construction. Its minimality condition must itself be represented, and that representation can introduce further witnesses. The normal-form selection in Section 9 avoids this particular problem because it is an explicit finite-state recurrence unrolled to a fixed length; the number of variables is allowed to grow.

Proposition 62.2 (No computable witness-height bound; manuscripts 02, 01, 03, 04 and 05). *Let $A \subseteq \mathbb{N}$ be undecidable and suppose*

$$x \in A \iff \exists w \in \mathbb{N}^m P(x, w) = 0.$$

There is no total computable function $B(x)$ such that every positive instance has some solution with all coordinates at most $B(x)$. In particular, let $P(e, x, z)$ be any fixed polynomial, with a fixed finite witness

arity, representing universal halting, with no assumption on the multiplicity of its witnesses. There is no total computable function $B(e, x)$ with the following property: whenever $U(e, x)$ halts, there is a zero of $P(e, x, \cdot)$ whose every coordinate is at most $B(e, x)$.

Sources. [write] The first statement is manuscript 02’s, for an arbitrary undecidable set; the second is the form stated by manuscripts 05 (whose proof follows), 01, 03 and 04, whose proofs are printed in their own sections below. The same finite-box search proves both. Manuscript 06 proves the corresponding statement for the parameters of a separated one-counter scheme (Proposition 67.2), and manuscript 07 records the point as a remark: its witness bound depends on T and supplies no computable bound on the unknown halting time (Section 68). As manuscripts 01, 02, 03, 04 and 05 each observe, finite-foldness alone gives no computable bound either, and the explicit bounds of the bounded constructions pose no conflict, since the horizon, height or log is given with the instance.

Proof of Proposition 62.2 for universal halting (manuscript 05). Such a function would decide halting. Compute $B(e, x)$, enumerate the finite box $\{0, \dots, B(e, x)\}^{\dim z}$, and evaluate P exactly at each tuple. A zero exists in the box exactly when the machine halts, by the assumed representation and bound. This contradicts the undecidability of universal halting. $\square$

Even a bound on the *least available* witness would be impossible in that sense; it is not necessary to bound every zero. Finite-foldness does not provide an effective bound of this kind. In contrast, the explicit bounds of Theorem 11.2 depend on the given horizon and therefore pose no conflict.

63 The boundary of unbounded canonicalization [01]

63.1 Infinitely many histories can be semantically unavoidable

Proposition 63.1 (An infinite history fibre). *There is a two-action one-counter system with infinitely many distinct finite trace classes from a fixed state to itself. Therefore an unbounded representation in bijection with all its finite histories cannot be finite-fold at that pair of endpoints.*

Proof. Use an increment a enabled at every natural marking and a decrement b enabled at positive markings. They are dependent by Proposition 5.3. Starting at zero, each word $(ab)^k$ executes and returns to zero. The words for distinct k have different lengths and hence belong to different trace classes. A bijection with all these classes requires infinitely many natural roots at endpoints $(0, 0)$. $\square$

This does not disprove finite-fold representability of the endpoint relation. In this example an endpoint relation can have a very simple existential description. It proves only that “preserve every finite history” and “have finitely many roots per endpoint” are sometimes incompatible specifications.

63.2 Semantic uniqueness does not canonicalize a coding base

The next observation isolates a different source of multiplicity. It is an elementary warning about one coding architecture, not a no-go theorem for all possible codes.

Proposition 63.2 (CRT base multiplicity). *Fix a nonempty finite list $a_0, \dots, a_{t-1} \in \mathbb{N}$. There are infinitely many distinct pairs (B, A_B) which encode that same list by residues modulo*

$$m_i = 1 + (i + 1)B,$$

even if A_B is required to be the least nonnegative simultaneous CRT representative.

Proof. Choose any sufficiently large positive multiple B of $t!$ so that every $a_i < m_i$. For $i < j$, a common divisor g of m_i, m_j divides $(j - i)B$. But $\gcd(g, B) = 1$, since g divides $1 + (i + 1)B$. Thus g divides $j - i$. Every positive divisor of $j - i$ divides $t!$ and hence B , so $g = 1$. The moduli are pairwise coprime.

The Chinese remainder theorem therefore gives a unique A_B with $0 \leq A_B < \prod_i m_i$ and $A_B \equiv a_i \pmod{m_i}$ for all i . Infinitely many choices of B satisfy the stated conditions. They give distinct pairs regardless of possible coincidences among the A_B values. $\square$

A canonical least residue fixes A relative to B , not B itself. The example explains why extracting a unique run from a trace-coding equation is weaker than proving uniqueness of the complete arithmetic certificate. Our finite-height compiler avoids coding bases altogether; the accelerator avoids storing a list of repeated states.

63.3 A sharp universal test for a single-fold compiler

Let U be a fixed deterministic universal interpreter for register programs, with first-halt semantics. Let

$$K(e, x) \iff U \text{ with program code } e \text{ and input } x \text{ eventually halts.}$$

We use only the defining universal property: every computably enumerable relation has a recognizer simulated by some fixed program code. This is standard computability infrastructure, also underpinning the machine-to-Diophantine routes cited earlier [14, 8].

Theorem of manuscript 01: “Universal halting characterizes single-fold MRDP”. [write] Manuscript 01 states here, for the relation K just defined, that (i) every computably enumerable relation of finite arity over $\mathbb{N}$ has a single-fold ordinary Diophantine representation if and only if (ii) one ordinary integer polynomial $P(e, x; w)$ with a fixed finite witness tuple has, at each (e, x) , a singleton natural zero set if $K(e, x)$ and an empty one otherwise; and the same with “finite-fold” and a nonempty finite zero set. This is (a') $\Leftrightarrow$ (b) of Theorem 62.1, where the statement is printed once; manuscript 01’s proof follows.

Proof of (a') $\Leftrightarrow$ (b) of Theorem 62.1, both versions (manuscript 01). The halting relation K is computably enumerable, so (i) gives (ii).

Conversely, let $R \subseteq \mathbb{N}^k$ be computably enumerable. For $k \geq 1$, iterating the polynomial injection π of Lemma 56.1 gives a polynomial injection $C_k : \mathbb{N}^k \rightarrow \mathbb{N}$ with a computable inverse on its image. A program can decode a valid encoded tuple and simulate a recognizer for R ; on an invalid code it can diverge. Let e_R be its fixed code. Then

$$R(a_1, \dots, a_k) \iff K(e_R, C_k(a_1, \dots, a_k)) \iff \exists w P(e_R, C_k(a_1, \dots, a_k); w) = 0.$$

Substituting a constant and a polynomial into an integer polynomial gives an integer polynomial. No new existential variable is introduced, so the entire fibre is exactly the fibre of P at that substituted pair. A singleton remains a singleton and a finite set remains finite. Nullary relations are handled by a fixed dummy input. This proves the reverse implication in both versions. $\square$

The theorem identifies the precise strength of a hoped-for universal fixed-arity canonical halting compiler. It would resolve the single-fold principle discussed by Matiyasevich, or its finite-fold version, rather than being a routine corollary of an ordinary exact-iteration interface [9, 12]. The explicit results of this report establish neither premise (ii). They instead provide fixed-arity compression for commuting blocks and a parsimonious family at bounded causal height.

63.4 Witness bounds are a separate obstruction

Proposition of manuscript 01: “No computable global witness-height bound”. [write] For any integer polynomial $P(e, x; w)$ representing the halting relation K , this is the universal-halting case of Proposition 62.2; manuscript 01’s proof follows.

Proof of Proposition 62.2 for universal halting (manuscript 01). If such B existed, compute it and test P on the finite box $\{0, \dots, B(e, x)\}^{\dim w}$. A root would certify halting, and exhaustion without a root would certify nonhalting, contradicting undecidability. The argument uses only existence representation; it does not assume unique or finite fibres. $\square$

There is no conflict with Proposition 20.3: that proposition concerns a fixed bounded-height instance family, not a representation of unrestricted halting. Equally, finite-foldness by itself would not provide a computable bound on where its finitely many witnesses lie.

64 The exponentiation boundary and universal first halting [02]

[write] *The last three subsections of manuscript 02’s section “Other substrates and the limit of the construction” (Section 60, Part VIII).*

64.1 The exponentiation boundary

The polynomial trajectory hypothesis cannot simply be dropped. Even the single assignment

$$y' = 2y, \quad y_0 = 1$$

has the exact-time graph $y = 2^n$. This trajectory is exponential, not polynomial in n .

Proposition 64.1 (A precise implication of a stronger accelerator). *If a fixed-dimensional single-fold ordinary polynomial accelerator covered this doubling loop uniformly in n , then the graph $y = 2^n$ would have a single-fold ordinary polynomial representation. Combined with the classical single-fold exponential normal form, this would yield single-fold ordinary polynomial representations of all enumerable sets.*

Proof. The first assertion follows by specializing the accelerator’s initial state to 1 and its guard to *true*. For the second, use the classical single-fold normal form in which exponentiation is isolated in one relation $v = 2^u$ [10, p. 588]. Substitute the hypothetical single-fold polynomial graph certificate for that relation. The original outer tuple is unique, and the new graph certificate has a unique extension for its fixed (u, v) . Thus the combined tuple remains unique. Combine the resulting finite polynomial equations by a sum of squares, or first lower their degrees by canonical arithmetic wires. $\square$

The same issue arises for variable-base multiplication $y' = ay$, which contains doubling as a specialization. The proposition is an implication, not a claimed impossibility theorem. It identifies the strength of an extension that the present paper has *not* proved. The literature on finite-fold representations explicitly distinguishes ordinary polynomial and single-fold exponential results [9].

Remark 64.2 ([write] The formalized exponential counterpart). The classical input used here — every recursively enumerable set has a single-fold unary exponential Diophantine representation (Jones–Matiyasevich 1984) — is formalized in ProveIt as `JM1984.RM.re_sfu` and `JM1984.RM.re_single_equation` (`Computability/HilbertTenthProblem/Lean/Diophantine/Paper1984/DPR.lean`), with the single-fold predicate `JM1984.Exp.SFU`. The reductions in this report are not formalized.

64.2 Universal first-halting computations

A related conditional statement is useful when evaluating proposed “substrate-independent” encodings. Suppose one has a single-fold polynomial representation of the input–time–output relation for the *first halt* of a fixed universal deterministic machine. For each input program and data, there is at most one first-halting time and output. Projecting those quantities, canonically encoding any signed components, and specializing the program therefore gives a single-fold polynomial representation of its halting set. Universality then gives such a representation for every enumerable set.

Replacing single-fold by finite-fold gives the corresponding finite-fold implication: there is at most one first-halting computation, and finitely many auxiliary witnesses for it. Thus a universal finite-fold first-halt compiler would settle a genuinely general representation question, rather than merely repackage ordinary MRDP. Our accelerator is deliberately restricted to supplied polynomial or fixed-residue polynomial trajectories.

64.3 A height bound cannot be silently added to universality

Proposition of manuscript 02: “Witness-height obstruction”. [write] Manuscript 02’s statement, for an arbitrary undecidable set $A \subseteq \mathbb{N}$ represented by $P(x, \mathbf{w})$, is the first statement of Proposition 62.2, printed there once; its proof follows.

Proof of Proposition 62.2 (manuscript 02). Compute $B(x)$ and check the finitely many tuples in $\{0, \dots, B(x)\}^m$. A found zero certifies membership; if no zero is found, the assumed bound certifies nonmembership. This would decide A . $\square$

Finite-foldness alone does not give such a computable height bound. Our height and bit-complexity bounds are possible because we treat a decidable specified-input class, not an arbitrary universal halting predicate.

65 The exact boundary of uniform single-fold compression [03]

65.1 The quantifiers cannot be interchanged

The construction proved so far has the form

$$\forall L \exists P_L \in \mathbb{Z}[X_1, \dots, X_{3L}, Y_1, \dots, Y_{WL}] \quad \text{with a canonical fiber at every fixed log.} \quad (243)$$

The number of variables grows with L . For a fixed program we similarly have one polynomial per horizon. This is not the assertion that one polynomial with one fixed finite list of witnesses recognizes all halting computations while preserving unique fibers.

For a set $A \subseteq \mathbb{N}$, a *single-fold Diophantine representation* is a polynomial $F(x, y_1, \dots, y_s)$ such that $x \in A$ precisely when there is a natural root in y , and there is at most one such tuple for every x . A *finite-fold representation* requires finitely many such tuples instead of at most one. The general single-fold/finite-fold compression questions are not settled by MRDP or by determinism of the represented computation [9]. In particular, introducing a code for a unique trace does not prove that the auxiliary variables used to decode and check that code are unique.

Theorem of manuscript 03: “A universal substrate is the exact bottleneck”. [write] Manuscript 03 fixes a universal deterministic interpreter $U(e, x)$ for an acceptable enumeration of partial computations and states that (i) every computably enumerable subset of $\mathbb{N}$ has a single-fold natural-number Diophantine

representation if and only if (ii) one integer polynomial $F(e, x, Y_1, \dots, Y_s)$, with fixed s , has natural roots in Y that are unique when $U(e, x)$ halts and nonexistent otherwise; and the same with “finite-fold/finitely many”. This is (a) $\Leftrightarrow$ (b) of Theorem 62.1, where the statement is printed once; manuscript 03’s proof follows.

Proof of (a) $\Leftrightarrow$ (b) of Theorem 62.1, both versions (manuscript 03). For (i) $\Rightarrow$ (ii), encode pairs by the injective polynomial

$$\pi(e, x) = (e + x)^2 + e.$$

Its injectivity follows because for $n = e + x$ its possible values form the interval $[n^2, n^2 + n]$, and these intervals for successive n are disjoint; within one interval the value fixes e , then x . The set

$$K_\pi = \{\pi(e, x) : U(e, x) \text{ halts}\}$$

is computably enumerable. Apply (i) to obtain a single-fold polynomial $G(z, Y)$ for that set and substitute $z = \pi(e, x)$. Injectivity ensures that the result recognizes exactly the desired halting relation and preserves each witness fiber.

For (ii) $\Rightarrow$ (i), let A be computably enumerable. By universality there is a fixed program index e_A such that $U(e_A, x)$ halts exactly when $x \in A$. Specializing $e = e_A$ in F gives a single-fold polynomial for A . Both operations preserve finiteness of fibers as well, proving the finite-fold version. $\square$

The theorem is an elementary reduction exposing where an ambitious extension would have to make genuinely new progress. It is not a solution of either conjecture. The source determinism used in Theorem 41.1 eliminates multiple bounded executions, but says nothing by itself about the auxiliary fibers of a fixed-variable arithmetization of arbitrary run length.

65.2 An unavoidable witness-height obstruction

Proposition of manuscript 03: “No computable global witness bound”. [write] For any fixed polynomial $F(e, x, Y)$ representing the universal halting relation, without any assumption on multiplicity, and a candidate bound $h(e, x)$, this is the universal-halting case of Proposition 62.2; manuscript 03’s proof follows.

Proof of Proposition 62.2 for universal halting (manuscript 03). Given (e, x) , compute $h(e, x)$ and test the finitely many natural tuples in $[0, h(e, x)]^s$. Polynomial evaluation is decidable. By the supposed property, the search finds a root exactly when $U(e, x)$ halts, deciding the universal halting problem. This is impossible by the usual diagonal argument: a decider for halting would allow a program that halts precisely when the decider predicts that the same program on its own code does not halt. $\square$

There is no conflict with Theorem 39.2. That theorem takes a fixed finite log as input, including the values already appearing in it. The machine theorem fixes a horizon. Neither supplies a total computable bound on the first halting time of an arbitrary universal computation. Even finite-fold representability, by itself, would not make the maximum root computable.

A concrete formalization target is therefore a hierarchy of interfaces: canonical local gadgets; canonical finite logs; canonical horizon-indexed machine runs; ordinary fixed-variable Diophantine reachability by MRDP; and, separately, the presently unproved finite-fold or single-fold upgrade of that last interface. An implementation should not erase these distinctions behind a common “Diophantine” label.

66 Unbounded computation and the fixed-arity frontier [04]

66.1 What MRDP gives for the present substrates

For any fixed effective substrate, the relation

$$\text{Reach}(a, b) \iff \text{some finite valid execution transforms } a \text{ into } b$$

is computably enumerable: enumerate finite candidate histories and check each one. For SKI, every natural code is a finite term, and both redex checking and a finite contextual step are effective. For FRACTRAN and Petri nets the finite checks are explicit arithmetic procedures. MRDP therefore supplies a fixed finite-witness polynomial for each such reachability relation; program or net descriptions can also be included as natural inputs to a uniform interpreter.

This implication is classical, and the ProveIt exact-iteration interface is another route when the one-step relation has already been connected to its Diophantine API [3]. Neither route automatically preserves the one-to-one correspondences constructed here.

66.2 The difference between a family and one polynomial

Definition 66.1 (Finite-fold and single-fold). A fixed polynomial $P(\mathbf{a}, \mathbf{y}) \in \mathbb{Z}[\mathbf{a}, \mathbf{y}]$ represents a relation R if

$$R(\mathbf{a}) \iff \exists \mathbf{y} \in \mathbb{N}^m P(\mathbf{a}, \mathbf{y}) = 0.$$

It is finite-fold if the set of such $\mathbf{y}$ is finite for every fixed $\mathbf{a}$, and single-fold if that set has at most one element. The number m is fixed independently of $\mathbf{a}$.

The present FRACTRAN and scheduled-SKI polynomials are single-fold *for each fixed horizon or schedule*. But their number of variables grows with that external data. They do not produce one fixed polynomial $P(T, a, b, \mathbf{y})$ with fixed m and the same uniqueness guarantee. The Petri-net polynomial intentionally permits multiple solutions, one for each distinct class, while giving exactly one solution per class.

The fixed-arity requirement is the essential frontier. Single-fold specification and its connection with eliminating exponentiation are discussed as substantive research questions by Cantone, Cuzziol, and Omodeo [10]. Nothing in the bounded constructions here supplies that missing elimination or settles the general finite-fold or single-fold conjectures.

66.3 Three tempting but invalid shortcuts

First, a deterministic machine has only one run, but a number-theoretic representation of that run may have many witnesses. The omitted equation $zu = 0$ in the divisibility gadget gives a concrete example: the semantic branch remains unique, while the arithmetic fiber becomes infinite.

Second, representing a finite list by a variable base often introduces many choices of sufficiently large base and padding. One may still prove existential equivalence, but a uniqueness argument must specify and enforce a canonical base, length, and padding. Merely saying “choose the least encoding” is not a Diophantine construction: minimality introduces a statement excluding all smaller candidates, whose compilation and fiber behavior must themselves be justified.

Third, a forward operational simulation is not a reduction-reflecting equivalence. Extra target reductions can create arithmetic witnesses not corresponding to source computations. This matters for compiler chains involving different lambda-calculus evaluation strategies, or the distinction between a pure Iota source and a runtime syntax containing auxiliary combinators. The inspected repository

explicitly distinguishes several of these semantics [4]; our scheduled local theorem does not bypass those obligations.

66.4 A precise universal reduction of the remaining problem

Theorem of manuscript 04: “Universal-halting scope equivalence”. [write] Manuscript 04 fixes an effective universal programming system with halting relation $H(e, x)$ and states, for either choice of “finite-fold” or “single-fold”, the equivalence of (i) every computably enumerable subset of $\mathbb{N}$ has a representation of that kind; (ii) $H(e, x)$ has one by one fixed polynomial; (iii) $H(e, x)$ has one by a fixed polynomial of degree at most four. These are (a), (b) and (b’) of Theorem 62.1, where the statement is printed once; manuscript 04’s proof follows.

Proof of (a) $\Leftrightarrow$ (b) $\Leftrightarrow$ (b’) of Theorem 62.1 (manuscript 04). Assume (i). Encode pairs by the Cantor pairing function π from Section 55. The set $\{\pi(e, x) : H(e, x)\}$ is computably enumerable, so it has the assumed representation. Adjoin a variable z and the equation

$$2z - (e + x)(e + x + 1) - 2x = 0.$$

For fixed e, x , this determines exactly one natural z . Combining this equation with the representation of the paired set preserves its fiber cardinality and gives (ii).

Assume (ii). For a computably enumerable set S , choose the index e_S of a program halting exactly on S . Substitute the fixed integer e_S for the program coordinate. The remaining polynomial represents S and has exactly the same fibers, proving (i).

Finally, (ii) implies (iii) by unique-extension quadratization and summing squares, while (iii) trivially implies (ii). $\square$

This is a scope theorem, not a resolution. It identifies why a successful fixed-arity witness-faithful compiler for a universal substrate would be much stronger than any of the horizon-indexed families above. Restricting degree to four does not remove the main obstruction once an arbitrary finite number of variables is allowed.

66.5 No computable global witness-height bound for halting

Theorem of manuscript 04: “Witness-height obstruction”. [write] For $P(e, x, y)$ of fixed finite witness arity representing the halting relation of a universal programming system, this is the universal-halting case of Proposition 62.2 (printed there as a proposition); manuscript 04’s proof follows.

Proof of Proposition 62.2 for universal halting (manuscript 04). If such B existed, compute it on (e, x) and enumerate the finite box $\{0, \dots, B(e, x)\}^m$. Evaluate the integer polynomial on every tuple. A zero would certify halting; if none existed, the assumed bound would certify nonhalting. This would decide the universal halting problem, a contradiction. $\square$

Notice that the hypothesis asks only for *one* small witness per positive instance, not for a bound on every witness. Finite-foldness does not imply such a computable bound: a finite set can have an extremely large maximum with no computably uniform control. Conversely, the bounded-horizon height estimates in this article are entirely compatible with the obstruction because the horizon is supplied as part of the compiled instance, and no computable universal bound on halting horizons is provided.

67 The exact finite-fold obstruction [06]

Single-foldness for fixed schedule parameters is strong enough to prevent compiler-generated redundancy, but it cannot settle the general finite-fold problem. The preceding normal forms turn this limitation into an exact logical equivalence rather than a vague warning.

Theorem 67.1 (Finite-fold substrate equivalence). *For a relation $R \subseteq \mathbb{N}^a$, the following are equivalent:*

- (i) *R has a finite-fold Diophantine representation over $\mathbb{N}$.*
- (ii) *There is a fixed separated one-counter scheme E of depth at most four, with input parameters x and witness parameters k , such that*

$$x \in R \iff \exists k \in \mathbb{N}^b \ 0 \xrightarrow{\llbracket E \rrbracket_{(x,k)}} 0,$$

and for each fixed x there are only finitely many successful parameter tuples k .

The same equivalence holds with “finite-fold” replaced by “single-fold” and “finitely many” replaced by “at most one.”

Proof. From (i), apply Lemma 26.4 to a witnessing polynomial. For each original witness tuple there is exactly one tuple of circuit-gate values, so finite or singleton fibers are preserved. Apply Theorem 26.3 to the resulting quartic. It preserves its parameter solution set with no additional existential coordinates. This gives (ii) with exactly the same fiber cardinalities.

Conversely, a separated depth-four scheme has acceptance polynomial $A(x, k) - B(x, k)$ of degree at most four by Theorem 26.3. Its roots are exactly the successful parameter tuples, with no coding variables to eliminate. The assumed fiber cardinality therefore gives precisely the required finite-fold or single-fold Diophantine representation. $\square$

The result is a normal-form equivalence, not a proof that every computably enumerable relation satisfies these conditions. The general finite-fold program and the role of single-fold specifications are discussed in [9, 10]. We make no claim to settle either conjecture. Instead, the theorem places their full difficulty inside a very small substrate: one nonnegative counter, a block of increments followed by a block of decrements, and four levels of globally shared repetition.

There is an analogous formulation using vector-separated depth-two schemes. Use the quadratic-system version of Lemma 26.4 in the forward direction; in the reverse direction, sum the squares of the coordinate balance equations. The same parameter tuples are preserved. Thus moving from one counter to several trades nesting depth for simultaneous equations, not for an automatic improvement in witness multiplicity.

67.1 An exact accounting identity

For fixed endpoints and input parameters, split the schedule parameters into a prescribed tuple x and a quantified tuple k . The compressed compiler satisfies, with cardinalities understood in $\mathbb{N} \cup \{\infty\}$,

$$\#\{(k, z) : Q_E(m, n, x, k, z) = 0\} = \#\{k : \llbracket E \rrbracket_{(x,k)} \text{ meets the compiler's acceptance condition}\}. \quad (244)$$

This follows by summing the zero-or-one fiber equality of Theorem 25.1. There is no extra factor from arithmetic witnesses. But the right side can still be infinite, and it counts parameter tuples, not distinct words or traces. This is the exact point at which finite-fold information must enter from somewhere other than the local compiler.

67.2 No computable bound for a universal parameter search

Proposition 67.2 (Parameter-bound obstruction). *Let E be a fixed separated scheme whose existential parameter acceptance set is an undecidable computably enumerable set of inputs. There is no total computable function $B(x)$ such that every accepted input x has a successful witness parameter tuple all of whose coordinates are at most $B(x)$.*

Proof. If such a function existed, compute $B(x)$ and test the finitely many witness tuples in its box. For a separated scheme each test is an equality between explicit integer polynomial values, hence decidable. An accepted input would have a successful tuple in the box, and a rejected input would not. This would decide the chosen undecidable set, a contradiction. $\square$

The explicit height bound (117) does not conflict with this proposition. It bounds *auxiliary coordinates after the schedule parameters are supplied*; it does not bound a successful choice of those parameters from the original input. Even hypothetical single-fold representations would not automatically provide a computable bound on their unique witness.

68 The fixed-arity and finite-fold boundary [07]

68.1 Three quantifiers that must not be interchanged

For a fixed program and input, Theorem 46.1 has the form

for each $T \geq 1$, there is a polynomial P_T in $T + 1$ variables.

Unbounded halting is the existence of some successful horizon. This is not the same as one polynomial of fixed arity with T as an ordinary input coordinate. Writing $\prod_{i < T} \ell_i$ when T is an unknown does not perform that conversion; it merely writes a variable-length expression.

A fixed polynomial representation of a set $S \subseteq \mathbb{N}$ has the form

$$n \in S \iff \exists \mathbf{y} \in \mathbb{N}^k P(n, \mathbf{y}) = 0,$$

where both P and k are chosen before n . It is finite-fold if each input has finitely many witnesses, and single-fold if each input has at most one. Research on obtaining finite-fold representations goes substantially beyond constructing a unique finite trace; Cantone, Cuzziol, and Omodeo give a concrete sufficient-condition approach to that problem [11].

The constructions here solve the horizon-indexed witness problem. They do not provide the missing fixed-arity compression with controlled fibers. This is a statement of their exact logical strength, not a reason to abandon the stronger problem.

68.2 What MRDP supplies and what uniqueness still requires

An effective code for a finite queue computation has an effectively checkable transition relation. The set of successful codes, and the projection to halting inputs, can therefore be handled by the existing MRDP interface [5]. That yields a fixed finite polynomial representation. It does not, by its stated interface alone, assert unique or finitely many auxiliary solutions.

A compression may introduce arithmetic witnesses for coding a list, extracting digits, evaluating powers, or eliminating bounded quantifiers. Even when the machine has one exact halting trace, these new witnesses must be separately controlled. Replacing a variable-length trace by one natural code does not prove that its polynomial validity predicate has unique witnesses.

Our treatment of first halting does remove one possible source of multiplicity: a halted run cannot be padded with fictitious empty-queue steps. The product guard and, independently, the local parity-of-scale obstruction both enforce that fact. This is useful but does not settle the multiplicity of a later fixed-arity arithmetization.

68.3 No contradiction with bounded simulation

All formulas are effectively constructible, and for a specified finite T the actual deterministic run can also be simulated. Their purpose is not to outperform bounded simulation on this elementary decision problem. Their purpose is to preserve the run in a small, explicit algebraic witness and to support composition, formal verification, and comparison of computational substrates.

The witness bound depends on T . It supplies no computable bound on the unknown halting time from the input alone. Likewise, a circuit of size $O(T)$ is not necessarily small in the binary encoding length $\log T$. These distinctions are necessary when moving from exact finite certificates to unbounded computational questions.

69 Implementation and validation

[write] Every manuscript ships Python programs that import only the standard library, together with recorded outputs. Each subsection below is one manuscript's own account of what it implemented, what it tested and what the tests do not establish. The programs and data are shipped in `code/` and `data/` with the manuscript's prefix (`01-causal-traces-`, `02-poly-trajectories-`, `03-linear-memory-`, `04-witness-faithful-`, `05-trace-polytopes-`, `06-counter-schedules-`, `07-queue-causality-`); in the prose below the shipped names are printed. The displayed command lines are the manuscripts' own and use the delivered layout (for example `code/verify.py`, `examples/`, `tests/`), because the programs import one another and write their outputs by those names; the README gives rerun recipes that copy the shipped files into that layout, and warns which programs overwrite recorded outputs. The manuscripts' instructions for rebuilding their own PDFs do not apply to this report, whose build is described in the README. None of the finite checks is a proof; each manuscript says so, and its theorems rest on the proofs printed in Parts I–IX.

69.1 Manuscript 05: Implementation and reproducible verification

69.1.1 Reference implementation

The companion package contains a Python implementation using only the standard library. It emits sparse residual polynomials and regards the final polynomial as their squared sum. Every natural assignment is checked using exact integer arithmetic. The one fractional-relaxation example uses exact rational numbers, not numerical tolerances.

The primary entry point is

```
compile_certificate(net, initial, final, horizon,
                  form="quadratic", independence=None,
                  upper=None)
```

Here `form` can be `raw`, `quartic`, or `quadratic`. With no supplied independence relation the compiler computes the maximal sound relation. A supplied relation must be symmetric and contained in that maximal relation. The optional upper-guard matrix uses `None` for infinity and finite natural entries for upper bounds. One-step empty domains are rejected rather than silently retained; removing such transitions is a straightforward preprocessing step.

The returned certificate has methods for constructing the unique witness of a valid word, checking a whole natural assignment, decoding a zero, exporting the residual system as JSON, and expanding the polynomial for small instances. The code does not invoke an integer-programming solver or pretend to solve general Diophantine equations. It is a compiler and exact checker.

The source also implements the graph-realization construction of Theorem 4.4, the flag recognizer, a literal guarded-run simulator, and the circuit-to-safe-net reduction used in Theorem 13.1. The cellular-automaton and rewriting front ends in Section 57 are mathematical constructions; complete standalone translators for those front ends are not included.

69.1.2 What was tested

The full run of `python code/verify.py` produced PASS. The machine-readable record is `results/verification.json` [write] (shipped as `data/05-trace-polytopes-verification.json`; the program is `code/05-trace-polytopes-verify.py`). The following are the exact finite scopes of the checks.

Check	Scope and result
Petri demand criterion	256 scalar transition pairs, arc weights 0–3; 2,048 starting-marking comparisons.
Trace normal forms	All 75 independence graphs on ordered alphabets of sizes 1–4; all words of length at most 6: 358,509 words and 124,494 classes.
Boolean memory family	All 254 subset prefixes for $k = 1, \dots, 7$, including distinguishing one-letter continuations.
Core certificates	286 constructed full witnesses in the three presentations; 576 single-coordinate perturbations rejected.
Exhaustive zero search	All $2^{16} = 65,536$ assignments in the full forced-height box of a tiny quadratic instance; exactly its two predicted zeros.
Real relaxation	Exact rational witness for Example 11.4, together with absence of an enabled integer step.
Circuit reduction	AND, OR, XOR, and a repeated-input example; accepting-run counts 1, 3, 2, 1, with safety and distinct Parikh vectors checked.
Ballot formula	160 endpoint–horizon instances, with horizons through 7.
Interval guards	729 scalar transition pairs and 6,561 marking comparisons; 45 bounded execution instances and 216 constructed witnesses.

The normal-form oracle is independent of the flag recurrence: it constructs the graph of legal adjacent swaps, computes its connected components with a disjoint-set data structure, and compares accepted words with the actual minimum of each component. The guarded commutation check compares literal two-step execution domains against the closed-form box criterion. The ballot formula provides a separate combinatorial count for the producer–idle–consumer examples.

For the core three-transition net, the computed endpoint examples are:

M	N	T	Raw runs	Classes	Quadratic (V_2, R_2)
0	0	0	1	1	(4, 5)
0	0	4	9	4	(96, 77)
2	2	6	140	28	(142, 113)
1	2	5	44	13	(119, 95)

A finite test does not prove the universal theorems. In particular, changing one coordinate of a constructed witness is not an exhaustive uniqueness proof, and the full box search concerns only its explicitly stated tiny instance. Universal correctness and uniqueness are supplied by the preceding

mathematical arguments. The tests serve as independent checks of indexing, signs, degree accounting, boundary cases, and the executable realization of those arguments.

69.1.3 Reproduction and polynomial format

From the package directory [write] (in the delivered layout; see the README, which also explains why the shipped code/05-trace-polytopes-build.py must not be run in the report directory), run

```
python code/verify.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The verification program should be run without Python’s optimization flag, since its test assertions are part of the checks. The ordinary compiler’s witness validation is an explicit runtime check. Python 3.10 or later is required; no third-party Python package or network access is needed for verification. A standard TeX installation with the packages listed in the source is needed to rebuild the PDF.

Each exported JSON residual is a list of coefficient–monomial pairs. A monomial is a list of variable names, repeated when a power is present. The represented polynomial is the sum of the squares of the residuals, not the sum of the residuals. The domain field explicitly says “nonnegative integers.” A separate small text file contains a fully expanded quadratic polynomial as a format sanity check.

69.2 Manuscript 01: Implementation, validation, and repository integration

69.2.1 The reference compiler

The accompanying code/01-causal-traces-causal_diophantine.py uses only the Python standard library. It contains an exact integer operational semantics, the composition-box test, the envelope, Cartier–Foata normalization, sparse integer polynomials, and four compiler entry points:

Entry point	Mathematical contract
compile_accelerator	Theorem 21.1; all serializations, also some under active commutation.
compile_history	Theorem 20.1; one root for each bounded-height executable trace.
compile_blocks	Theorem 22.1; fixed ordered commuting phases, optionally existential counts.
compile_word_power	Proposition 22.2; arbitrary repetition of one fixed, possibly noncommuting word.

A separate code/01-causal-traces-verify_exports.py checker reads the exported JSON without importing the compiler. It validates both natural certificates and compares the expanded polynomial with the residual-square expression on fifty seeded assignments per file; these comparisons are finite checks, not a symbolic identity proof.

The matching `complete_*` functions compute the canonical witness when a given semantic object is valid. They do not search for arbitrary polynomial roots. A `System` can check an assignment, evaluate the sum of squares, expand it, or export all coefficients and an optional verified certificate to JSON. The exporter refuses to attach an invalid certificate.

Natural-number domains are part of the specification. Polynomial evaluation uses ordinary signed integer arithmetic, and all numerical work is exact. The implementation deliberately uses no floating-point feasibility test, relaxed solver, modular substitute for integer equality, or fixed-width machine counter.

69.2.2 What was actually tested

Run

```
python code/verify.py
```

from a machine with Python 3.10 or later. The recorded run used Python 3.13.5 and pseudorandom seed 20260930. The tests are reproducible and fail by raising an exception; they write their exact counts and examples to `data/` and `examples/`. [write] (Shipped as `data/01-causal-traces-*`; the command runs `code/01-causal-traces-verify.py` in a copy with the delivered layout, as the README describes.)

Finite check	Count
Pairs of scalar guarded actions, box test versus direct execution	5,184
Petri pairs, closed sign classification versus box criterion	1,296
State-multiset instances for the exact envelope	9,600
Individual serialized executions within those instances	102,768
Words checked against adjacent-swap connected components	8,744
Boolean three-layer matrices across all eight independence graphs	4,096
Accepted canonical matrices in that enumeration	443
Random finite-history instances	2,400
Compiler arity/equation-count instances	200
Changed auxiliary coordinates correctly rejected	43,804
Tiny history assignments exhaustively tested	1,041
Tiny accelerator assignments exhaustively tested	6,561
Fixed commuting-phase composition instances	512
Serializations of the worked six-event trace	60
Fixed-word power instances, including empty macro domains	16,380

Table 4: Exact finite implementation checks. Counts are not a claim of formal verification.

The domain-box tests use lower guards from zero through three, every displacement from the negative lower guard through two, and four upper-bound choices; direct execution is tested from zero through 25. The Petri classification uses every consumption and production value from zero through five. Random envelope instances have one through three coordinates, one through four action labels, multiplicities at most two, and total event count at most six. Every multiset serialization in each selected instance is executed independently of the envelope formula.

For the normal form, all eight independence graphs on three labels and every word of length at most six are checked. An independent union-find calculation of adjacent-swap components is compared with the computed layers. The tests also enumerate all 2^9 Boolean three-layer matrices for each graph, using read-only actions so that every chosen independence subrelation is operationally valid.

The mutation checks change auxiliary witnesses while holding the semantic layer bits fixed. A different layer matrix could represent a different valid trace, so it would be incorrect to demand rejection of every change to a layer bit. In the accelerator, all witness coordinates are auxiliary relative to fixed (x, y, n) and are tested accordingly.

The tiny exhaustive natural-root examples are a single read action at endpoint one with height bounds zero, one, and two. Their root counts are respectively one, two, and three. Every possible witness lies

in the tested Boolean box for these examples, as follows directly from the residual formulas. A single decrement accelerator is similarly exhaustively checked for $x, y, n \leq 2$, with witness values zero through two. These finite tests exercise root multiplicities, not only production of successful certificates.

For fixed-word powers, every word of length at most five over a guarded increment, a decrement, and a zero test is checked at nine initial markings and five repetition counts. This includes impossible loop bodies and the zero-repetition exception.

The large worked accelerator is also evaluated as an expanded polynomial: its 76-monomial quartic evaluates to exactly zero at the supplied eleven-witness assignment. The 32-witness history example passes all 48 residuals. Full results, including the test ranges encoded in the script, accompany the paper.

69.2.3 What the tests do not establish

The tests are not a proof for all dimensions, counts, guards, or height bounds; those claims rest on the mathematical arguments above. They do not constitute a Lean kernel check, an independent source-level verification of the Python implementation, or a proof that the arity bounds are optimal. This report did not rebuild the ProveIt repository or rerun its published axiom audits. Repository facts are taken from the pinned source and documentation inspected for this study.

69.3 Manuscript 02: Executable artifacts and validation

69.3.1 What is implemented

The supplementary files implement the core bounded polynomial theorem [write] (shipped as code /02-poly-trajectories-*.py and data/02-poly-trajectories-*.json under the delivered names below):

certificates.py

Exact integer forward differences, binary-search generation of the canonical sign tower, terminal padding, and an independent local endpoint checker. A brute-force sign-run routine is used only in small regression tests, never by the main generator or checker.

quartic_compiler.py

A reference compiler for the fixed-degree sign-table conditions and the optional nonnegativity requirement. It exports every quadratic residual, all variable names, and a sample complete assignment. Its symbolic structure depends only on the nominal degree and the chosen final assertion, not on coefficient values or the horizon.

run_tests.py and test_report.json

A deterministic regression suite and its actual output, with seed 20260930. The output includes operation counts, finite uniqueness tests, and compiler structure fingerprints.

quartic_example.json

The exact degree-two quartic template in sum-of-squares form, together with a satisfying assignment for $p(t) = (t - 2)^2$, $T = 6$.

verify_export.py

A separate sparse-polynomial evaluator that reads the exported JSON, checks natural witness domains and quadratic residual degrees, and calculates every residual without importing the compiler.

The higher-level Boolean-guard, arbitrary trajectory, and quasi-polynomial front ends are proved mathematically but are not packaged as automatic source-language compilers in this release. This distinction matters: working core code is not the same as a complete program-analysis tool.

69.3.2 Tests actually performed

The included run produced the following results, with all assertions passing:

Check	Count
Random integer polynomials, nominal degrees 0–8	1,620
Additional explicit edge cases	8
Degree-two coefficient triples in $\{-1, 0, 1\}^3$	27
Candidate sign towers exhaustively checked on $[0, 2]$	59,049
Incorrect individual tag mutations rejected	68
Compiled polynomial instances, degrees 0–4	50
Negative positivity instances among those compiled	33
Nonlinear closed-form state comparisons	2,500

For each of the 27 small coefficient triples, all possible sign strings at the first two levels and all three constant top tags were considered. The checker accepted exactly one tower, equal to the generated tower. This tests uniqueness in a finite domain; the general uniqueness proof is Theorem 28.3.

For each compiled instance, the checker confirmed that every exported residual has degree at most two, every sample existential coordinate is natural, and the squared-sum equation vanishes exactly on the positive instances among the tested assignments. Ten differently instantiated examples at each nominal degree produced identical symbolic polynomial structures. False instances fail the required positivity residuals even though their full sign tables remain valid.

69.3.3 Concrete compiler size

Table 5 reports the unoptimized reference implementation. It is not a lower bound or a record claim.

d	Primary coordinates	Auxiliary coordinates	Total witnesses	Residuals
0	1	24	25	42
1	6	202	208	312
2	15	834	849	1218
3	28	2192	2220	3132
4	45	4540	4585	6414

Table 5: Actual variable and residual counts for the supplied compiler. The final equation is the sum of the squares of all residuals.

These constants are intentionally conservative. Repeated comparisons, zero-test slack variables, and straightforward Boolean wiring create many auxiliaries. Eliminating a redundant gate is safe only when it preserves both logical equivalence and uniqueness of the remaining witness.

69.3.4 Validation limits

The numerical tests do not prove soundness on all integer assignments, completeness for arbitrary horizons, or historical novelty. Those are separate questions: soundness and completeness are addressed

by the mathematical proofs, and novelty requires further literature comparison. No Lean or Coq formalization of the new construction was compiled here. The ProveIt repository was inspected as a source, not independently rebuilt.

The independent JSON checker certifies an exported *assignment*'s arithmetic validity. It is not a solver and does not verify that the compiler correctly represents the intended relation for all inputs. That higher-level correspondence is exactly the theorem a future proof-assistant implementation should establish.

69.4 Manuscript 03: A hand-checkable example and executable validation

69.4.1 Four events

Consider the zero-initialized memory log

t	a_t	w_t	v_t	Interpretation
0	7	0	0	Read an untouched address.
1	2	1	9	Write 9 at address 2.
2	7	1	4	Write 4 at address 7.
3	2	0	9	Read back 9 at address 2.

Here $N = 4$ and $B = 5$. The chronological keys are 40, 16, 42, 18. The sorted records (α, κ, w, v) are

$$(3, 16, 1, 9), \quad (3, 18, 0, 9), \quad (8, 40, 0, 0), \quad (8, 42, 1, 4).$$

The adjacent address differences are 0, 5, 0. Their zero-test pairs (z, h) are (1, 0), (0, 4), (1, 0), and their previous-value coordinates are 9, 0, 0. The first sorted record is a write. The next two records are reads returning 9 and 0 respectively; the final write can store any natural value. All memory checks pass.

There are $C_4 = 6$ comparators, $W_4 = 77$ auxiliaries, and $R_4 = 85$ residuals, besides the 12 free event parameters. The supplied export contains the actual expanded polynomial: degree four, with 587 distinct nonzero monomials. Changing the final read value from 9 to 8 makes the appropriate read residual nonzero. Because the preceding circuit has a unique assignment, no alternate routing or slack assignment can repair that invalid log.

69.4.2 What the implementation does

The file `code/03-linear-memory-canonical_memory.py` is a dependency-free Python implementation of the memory theorem, using exact integers and sparse polynomials. Its representation stores each monomial as a sorted tuple of variable indices. It rejects residuals of degree greater than two. The compiler creates symbolic event coordinates, so its polynomial structure is determined by L , not by the particular numerical log used to generate the witness.

The JSON export includes the natural-domain convention, every parameter and witness name, the generated assignment, and every quadratic residual. With `-expanded`, it also exports the actual sum-of-squares polynomial. Canonical values are serialized as decimal strings to avoid accidental loss of precision in consumers that treat JSON numbers as floating point. Polynomial coefficients in this backend are small ordinary JSON integers. The independent semantic checker uses a dictionary as memory and compares reads with its current contents.

Both memory backends are implemented in this archive; `code/03-linear-memory-linear_memory.py` provides the large-base alternative. The RAM, graph, counting, and probability results are mathematical constructions proved in the preceding sections; a complete machine-language frontend is

not supplied. The probability test checks the stated weight formula, not an implemented probabilistic RAM compiler. No new Lean or Rocq proof artifact is claimed.

69.4.3 Executed checks

The complete test driver is `code/03-linear-memory-test_certificates.py`; its recorded results are in `data/03-linear-memory-test_results.json`. The following tests were actually executed in this environment:

Test family	Result
Comparison and zero tests	Exhaustive candidate witnesses for 81 ordered input pairs and 21 zero-test inputs; exactly the stated witnesses survived.
Sorting networks	105,883 cases: every permutation at widths 1, 2, 4, and 8, and all binary inputs at width 16.
Small memory logs	All 585 logs of lengths 0 through 3 over addresses, write flags, and values in $\{0, 1\}$: 259 valid and 326 invalid, in exact agreement with the independent semantic checker.
Large integers	150 generated valid logs, with addresses up to 160 bits and values up to 200 bits; 143 deliberately corrupted read logs were rejected.
Auxiliary corruption	All 273 single-coordinate increments of the auxiliary witness for a six-event valid log were rejected.
Expansion	The four-event quartic has 587 monomials and degree four; 20 additional assignments confirmed equality with the residual sum of squares.
Path weights	For each horizon 1 through 12, the stop-on-first-one weight sum was exactly $2^T - 1$.

The six-event test has $N = 8$, 24 comparators, 273 witnesses, and 287 residuals, in agreement with the exact formulas. All recorded tests passed.

Finite testing is not a proof that arbitrary natural assignments have no unexpected roots. That conclusion rests on the forcing argument in Theorem 38.3. The tests instead check correspondence between the written compiler and the equations, exercise edge cases, and make regressions observable. The mutation test in particular is not presented as an exhaustive search over all possible witness tuples.

69.4.4 Reproduction

From the archive's root, run [write] (the delivered layout; see the README)

```
python3 code/test_certificates.py
python3 code/canonical_memory.py artifacts/example/events.json \
  artifacts/rebuilt_example --expanded
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The test script writes its result file relative to its own location. The Makefile supplies equivalent `test`, `example`, and `pdf` targets. The Python files use only the standard library; Python 3.10 or later is required for their type-annotation syntax. The article uses standard pdfL^AT_EX packages. The included PDF is already compiled. [write] (The PDF is not shipped in ProveIt, and the shipped Makefile, `code/03-linear-memory-Makefile`, names delivered paths; see the README.)

69.5 Manuscript 04: Executable artifacts and exact validation

69.5.1 Contents and mathematical interpretation

The package contains `code/diophantine_compiler.py` [write] (shipped as `code/04-witness-faithful-diophantine_compiler.py`, with the test driver `code/04-witness-faithful-run_tests.py`, the results `data/04-witness-faithful-results.json` and the four examples `data/04-witness-faithful-*.json`), an exact sparse-integer-polynomial implementation; `tests/run_tests.py`; a generated `tests/results.json`; and four JSON examples. Each example lists all natural variables, all polynomial residuals as integer coefficient/monomial data, and a complete witness assignment. The JSON convention is

`combination = sum of squared residuals.`

Thus the residual list defines one explicit ordinary polynomial without printing an unnecessarily expanded quartic. No oracle, numerical tolerance, floating-point root test, or general Diophantine solver is used. The separate `code/verify_certificate.py` verifier does not import the compiler; it independently parses and evaluates the exported JSON residuals and checks the supplied witness and degree bound.

From the package directory [write] (in the delivered layout; see the README), run

```
python tests/run_tests.py
```

Python 3.10 or later is sufficient; only its standard library is used. To rebuild this article, run `pdflatex article.tex` twice, or `latexmk -pdf article.tex`. The bibliography is included in the source, so a separate bibliography tool is unnecessary.

69.5.2 What was checked

The normal-form check enumerates every symmetric irreflexive independence relation on alphabets of sizes one through four and all words of lengths zero through five. It constructs commutation classes independently using union-find on adjacent allowed swaps, then compares the class minima with the monitor. This checks 75 graphs, 90,404 words, and 39,093 classes. The lower-bound family is checked through $k = 7$, including all 254 prefixes in those instances and their distinguishing one-letter suffixes.

For Petri nets, 24 seeded small nets with three places and three transitions are tested on 2,904 candidate words. The test evaluates 1,417 enabled-run certificates, checks that zero energy coincides with normality, verifies swap closure, and checks exact variable and residual counts. A separate smallest example exhausts 729 natural assignments in a finite test box and finds one zero.

For FRACTRAN, the tests enumerate 132 one- or two-fraction programs whose positive reduced numerators and denominators are at most four, use inputs 1 through 12, and horizons zero through three. Together with explicit normalization and denominator-one cases, they evaluate 4,914 valid certificates and check their sizes and degree bounds. These checks test the constructive direction and bookkeeping; they do not exhaust all possible false assignments to the larger systems.

For SKI, 1,997 application codes are inverted and rebuilt, with both children checked to be smaller. The root rules and several context paths are exercised in 1,875 valid certificates. The proof of Theorem 55.2, rather than this finite test, establishes completeness and absence of arbitrarily large spurious witnesses.

Example	Variables	Residuals	Largest coordinate	Rejected mutations
Petri fork/join	56	67	1	69
FRACTRAN first halt	103	120	125	161
SKI root schedule	13	11	635	26
SKI contextual schedule	8	8	91	16

Every listed witness has exactly zero sum-of-squares value. The mutation column counts all permissible one-coordinate increments and decrements by one, each rejected by a nonzero residual. These local mutation checks are useful for catching disconnected variables but are not a proof of global uniqueness.

69.5.3 Validation status and reproducibility limitations

The reported suite completed successfully in the working environment. Its mathematical checks are deterministic apart from the seeded net generator, whose seed is fixed in the source; the elapsed-time field in the JSON report is not a mathematical invariant. The implementation mirrors the displayed residual systems and does not implement a fixed-arity MRDP elimination.

No new Lean or Rocq/Coq theorem was compiled as part of this package, and the repository's existing build was not rerun. The article supplies conventional proofs, executable constructions, and finite exact checks. A proof-assistant version of the new witness-level theorems remains a proposed next artifact, not an accomplished part of the present result.

69.6 Manuscript 06: Executable artifacts and verification

The accompanying implementation uses Python's exact integers and the standard library. It is an executable reference model of the displayed constructions, not a verified extraction from a proof assistant. Its sparse polynomials store integer coefficients and monomials explicitly. A certificate file stores the quadratic residual list and declares the represented polynomial to be their sum of squares. This gives an exact ordinary polynomial without forcing an unhelpful expansion of every square.

69.6.1 What the files implement

The module `code/compiler.py` [write] (shipped as `code/06-counter-schedules-compiler.py`) defines the expression syntax, transition semantics, resource product and powers, the globally sound commutation relation, the Boolean signatures, and both versions of the compiler. `Builder.construct` computes the deterministic auxiliary assignment from free inputs. It does not make a rejected instance satisfiable: the returned assignment has nonzero sum-of-squares value when endpoint or normality constraints fail. Witness-construction recipes involving a minimum or a case test are *not* extra symbols in the polynomial. The exported residuals are the sole algebraic constraints, and the proofs above establish that they force the recipe values on every root.

The module `code/substrates.py` [write] (shipped as `code/06-counter-schedules-substrates.py`) implements coefficientwise sign splitting, nonnegative arithmetic-circuit lifting, and the separated-schedule frontend. It compiles a polynomial directly to one counter at its original degree, compiles its quadratic circuit system to several counters at depth two, and compiles the sum-of-squares quartic to one counter at depth four. Its current coefficient encoding is unary, as specified in the proof of Theorem 26.3.

The bounded-length construction of Theorem 18.1 is proved in the article but is not a separate polynomial-export frontend in the supplied prototype. The tests do independently enumerate bounded trace classes to check the normality component. No claim of complete implementation coverage should be inferred from the scope of the mathematical results.

69.6.2 Finite checks and their limits

The main suite exhaustively checks small resource pairs and powers, every independence graph on a three-letter alphabet, and every word of length at most six over that alphabet. For the trace tests, connected components under actual adjacent independent swaps provide an independent source of true lexicographic representatives. The suite also checks signature composition, power saturation, and 160 seeded expression instances. Both resource-only and canonical compiled systems are tested on valid and invalid endpoints. A focused catalyst test catches the zero-power error discussed above.

Mutation tests change one auxiliary coordinate of a valid certificate and verify rejection. These are useful implementation checks, but they are not a proof that no coordinated multi-variable mutation could satisfy the system. The structural uniqueness proof in Theorem 25.1 supplies that theorem. Similarly, random expression tests do not establish cap-at-two for all expressions; Theorem 24.2 does.

Verification group	Checks passed
Exact resource product, powers, and global commutation	8,140
Trace cross-sections, memory signatures, composition, saturation	39,032
Seeded compressed evaluation and cap-at-two instances	480
Compiler acceptance, degree checks, invalid inputs, zero repetition	1,180
Valid mutation baselines and rejected one-coordinate mutations	569
A certificate for an execution with a 221-digit length	1
Polynomial/circuit/separated-substrate frontend checks	320
Total	49,722

Table 6: Executed finite checks. The main suite contains 49,402 checks; the substrate frontend adds 320. A check is a tested identity or predicate, not an independent mathematical theorem.

The substrate test uses

$$P(x, y, z) = xyz + 2z - x^2 - 3$$

on the grid $\{0, 1, 2, 3\}^3$. Its nonnegative circuit introduces six uniquely forced gate parameters and seven quadratic equations. The corresponding multi-counter scheme has seven counters and depth at most two. Summing the seven residual squares gives a quartic and hence a one-counter scheme of depth at most four. The test compares all three presentations with direct polynomial evaluation and checks rejection after changing a gate value.

For the multiplication schedule in Example 25.3, the resource-only prototype allocates 79 witnesses and 86 equations. Its canonical version allocates 309 witnesses and 547 equations, even though the independence relation is empty and could be optimized away. These deliberately transparent counts are not claimed to be minimal. The large-number test uses the same canonical compiled expression; its longest witness has 221 decimal digits.

69.6.3 Reproduction

From the package directory [write] (in the delivered layout; see the README), run:

```
python code/test_compiler.py
python code/substrates.py
python code/compiler.py --export examples/multiplication_certificate.json
```

The first two commands regenerate the reports in `verification/` [write] (shipped as `data/06-counter-schedules-*.json`). The random seed in the main suite is 20260930. The compiler has no network requirements or third-party Python dependencies. Timing fields are machine-dependent; the reported counts and mathematical comparisons are the reproducible content.

Rebuild the article with:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

Two or more direct `pdflatex` passes can be used instead. The bibliography is contained in this source, so no external bibliography database is needed. The PDF uses standard installed TeX fonts; font files are not included in the package.

69.7 Manuscript 07: Executable artifacts and exact validation

69.7.1 Contents and reproduction

The package includes `code/queue_certificates.py` [write] (shipped, like every file named here, with the prefix `07-queue-causality-` in `code/` or `data/`), with three cyclic-tag modes and the general deletion-tag compiler; `code/compile_tag.py`, a JSON-specification front end; and `code/verify_certificate.py`, an independent evaluator that does not import the compiler. All use only the Python standard library. Python 3.10 or later is sufficient.

From the package directory [write] (in the delivered layout; see the README), run

```
python tests/run_tests.py
python code/verify_certificate.py examples/stream_example.json
python code/verify_certificate.py examples/ternary_tag_example.json
```

To construct the small cyclic-tag example without simulating it, run

```
python code/queue_certificates.py --appendants 10 "" \
  --initial 1 --time 3 --mode stream --no-witness \
  --output examples/symbolic_only.json
```

The empty quoted argument is the second appendant. The general tag front end reads arrays of integer digits, so it is not limited to single-character decimal alphabets:

```
python code/compile_tag.py --spec examples/ternary_tag_spec.json \
  --output examples/ternary_tag_example.json
```

Compile the article with two runs of `pdflatex article.tex`. Its bibliography is included in this source file.

69.7.2 The certificate format

Each positive example stores the variable list, topologically ordered arithmetic nodes, residual-root indices, metadata, a conservative degree upper bound, and a complete natural witness. Constants and witness values are serialized as decimal strings to avoid fixed-width integer limits in other software. The convention `combination = sum_of_squares` defines one polynomial from the residual list.

The independent verifier validates the node operations and references, evaluates exact integers, checks the degree bound, and evaluates every residual. It has no floating-point tolerance and calls no Diophantine solver. It proves computationally that the supplied witness is a zero of the exported polynomial. Semantic equivalence and uniqueness rely on the mathematical proofs, not on that single evaluation.

The circuit's degree analysis is deliberately conservative: it may not detect cancellation or multiplication by zero. Thus `stream_example.json` reports degree at most six, while the simplified explicit polynomial (188) visibly has degree four. This is not a discrepancy in the represented equation.

69.7.3 What was checked

The reproducible results are stored in `tests/results.json`. The principal counts are:

Check	Cases
Cyclic-tag candidate consumed bitstrings	92,610
Raw word-balance solutions rejected by causality	268
Exact halting instances in that enumeration	370
Positive cyclic-tag certificates checked in two modes	740
First-failure resource-product checks	5,916
General deletion-tag candidate consumed words	194,940
No-slack cyclic-tag candidate bitstrings	840
Natural witness assignments exhausted for quartics	79,631
Word-monoid concatenation pairs	3,969
Integer table-interpolation evaluations	423
General integer-polynomial tag compiler instances	120
Candidate words evaluated in those compiled polynomials	4,200

The cyclic-tag enumeration uses all 49 period-two programs whose appendants have length at most two, all 15 inputs of length at most three, horizons one through six, and every candidate consumed bitstring. It compares the word-and-guard formulas with a direct string simulator. Every positive instance is compiled in stream and quartic modes; single-coordinate witness increments are rejected.

The resource tests include all scalar three-event histories with initial resource at most three and consumptions and productions at most two, followed by 3,000 seeded multi-resource cases. They explicitly allow the formal history to become negative after failure.

The general-tag checks compare the formulas with direct two-symbol deletion, including nonempty halted tails, over exhaustive small binary instances and seeded ternary instances. Separate compiler checks use alphabets of sizes two through four, deletion numbers one and two, and horizons one and two. They evaluate the cleared *integer-polynomial circuits*, not merely the interpolation tables. They also verify that disabling witness attachment changes no polynomial nodes or residuals.

The quartic tests exhaust actual finite boxes of natural assignments, not only generated semantic witnesses. Nevertheless no finite box proves global uniqueness. That is why Theorem 51.1 proves that every zero propagates a valid word code and cannot pass through an empty queue. The lower bound on polynomial memory dimension is likewise a mathematical proof, not an inference from the finite word-monoid test.

69.7.4 Limitations of the validation

The tests do not certify every implementation behavior for arbitrarily large integers, replace formal proof of the compiler, or establish historical novelty. They do provide independently reproducible checks of the stated formulas, exact resource counts in tested instances, symbolic table clearing, and several easily missed edge cases. The full fixed-read network compiler and a Lean development are proposed extensions, not unreported completed artifacts.

70 Formalization targets

[write] No theorem of this report is formalized in Lean or Rocq, and no manuscript ships a proof-assistant file; the subsections below are the manuscripts' proposals, and every module name in them is proposed, not asserted to exist. The report sits beside ProveIt's Lean project `Computability/HilbertTenthProblem`, which proves MRDP in both directions. The manuscripts use its existence

interfaces `Diophantine.boundedForall_dioph`, `Diophantine.exactIter_dioph` and `Diophantine.existsExactIter_dioph` (`Lean/Diophantine/Common/DiophantineTrace.lean`) and `Diophantine.mrdp` as context only: those theorems assert that representations exist and say nothing about how many witnesses a representation has. The project also formalizes the single-fold unary exponential representation of Jones and Matiyasevich (Remark 64.2), and the queue content and length-marker identities of its tag-system route (Remark 47.1). Placement beside a formal development confers no formal status on anything here.

70.1 Manuscript 05: A formalization route that preserves the stronger specification

The new proofs are not currently Lean-checked. A useful formalization should preserve the fibre information rather than immediately discarding it through an existential predicate. The repository's existing `Dioph` interfaces can remain unchanged; the stronger results can imply them as corollaries.

70.1.1 Separate semantic and arithmetic layers

The semantic layer should define a finite labelled family of partial maps, exact finite runs, a proved sound independence relation, and its generated word equivalence. The arithmetic layer should define a fixed finite set of variables and residual polynomials. A decoder reads the selected word from a natural zero. A witness constructor produces all variables from a canonical run. The central obligations are

$$\text{decode}(\text{encode}(w)) = w, \quad \text{encode}(\text{decode}(z)) = z.$$

The second identity is what excludes free arithmetic witnesses. Proving only the first would establish completeness but not injectivity of the representation.

An appropriate result object is an equivalence between a finite execution-class type and a subtype of natural assignments satisfying the polynomial equation. Cardinality and weighted-enumerator statements then follow from that equivalence rather than from separate ad hoc proofs. The quotient can be replaced by the subtype of normal runs, using the unique-normal-form theorem to build an equivalence with classes.

70.1.2 Suggested proof decomposition

A practical decomposition has six modules, whose names below are proposed rather than asserted to exist.

PetriDemand. Formalize Theorem 4.1, including both implications and the empty-coordinate case. Keep the distinction between integer vector updates and natural state domains explicit. Prove equality of partial functions, not merely equality of results when both are defined.

IntervalGuardCommutation. Represent upper endpoints by a type with an infinity element. Prove the composed-box formula, canonicalize empty boxes, and derive Theorem 4.5. Equality of nonempty boxes must include unbounded-coordinate cases.

TraceNormalForm. Prove the forbidden-factor characterization and the flag invariant. Establish uniqueness of the lexicographically least representative using finite word classes. Prove that the recognizer's natural recurrence forces Boolean values by induction, rather than adding unnecessary Boolean polynomial constraints.

UniqueAffineExtension. Formalize unique slack, the affine AND gadget, and deterministic circuit extension. A reusable combinator should compose witness equivalences over fixed inputs. Sum-of-squares scalarization should be stated over an ordered ring, then specialized to integer polynomials evaluated at natural assignments.

CanonicalTraceCertificate. Define the variable index type as a tagged finite union of the arrays in the construction. Prove decoding, existence, and coordinatewise uniqueness. This layer also carries the exact variable and residual counts, including $T = 0$ and $d = 0$.

TracePolytope. Reuse the affine system over the reals, prove each coordinate bound, and package boundedness and convexity. Relate the natural assignment subtype to lattice points without changing domains by an uncontrolled integer encoding.

70.1.3 Integration with the current repository

The existing `exactIter_dioph` theorem can provide the unbounded existence-level consequence once a substrate's one-step relation is Diophantine [3]. It should not be renamed or reinterpreted as a multiplicity theorem. Instead, add a clearly separate bounded compiler interface carrying unique extension, and document precisely what is forgotten when projecting to `Dioph`.

The most valuable first milestone is the raw affine certificate plus unique extension; it is small, reusable, and independent of trace-monoid infrastructure. The second is the exact flag recognizer, and the third is the combined bijection. Only after those should cardinality and the full real-polytope statements be layered on. No project axiom or placeholder proof is needed in the mathematical design; an actual implementation must still pass the repository's normal kernel and transitive-axiom checks.

70.2 Manuscript 01: A proposed Lean integration boundary

A useful formalization should keep the new proof obligations separate from existing ordinary-Diophantine closure. One possible layering is:

1. Define box-guarded partial translations, word execution, and equality of partial maps. Prove Proposition 5.1 and the natural-state invariant without importing the MRDP machinery.
2. Prove the envelope theorem using counts of positive and negative displacements in prefixes. State the theorem over arbitrary finite action and coordinate types; prove the Petri formula by simplification.
3. Introduce a representation structure containing a polynomial, a semantic decoder, and an equivalence between its natural zero set and the specified semantic type. Ordinary Diophantineness is then a projection, not the entire contract.
4. Formalize conditional slack canonicalization and the support gadget. Assemble the accelerator polynomial with finite indexed residual families and a sum-of-squares lemma.
5. Develop the dependence-poset normal form and its padded layer characterization, then prove the full-root bijection of the history compiler. Only after that should an adapter export ordinary reachability through the existing Diophantine interfaces.

The key extra structure is the *equivalence of fibres*. A theorem that a decoder is surjective onto semantic histories proves completeness but not uniqueness. Injectivity of that decoder on the complete zero set must also be retained. No uncompiled Lean skeleton is presented as a completed formalization in this package.

70.3 Manuscript 02: Integration with ProveIt and further research

70.3.1 An interface worth adding

A useful extension of the repository would distinguish three levels of representation explicitly. An ordinary Diophantine interface stores an existence equivalence. A finite-fold interface additionally proves finite fibers. A single-fold interface additionally proves subsingleton fibers. Conversions should retain these properties in their types or theorem statements rather than discard them in an undifferentiated existence predicate.

For the present construction, the clean decomposition is a theorem about unique sign tables, a theorem about canonical circuit evaluation, and a composition theorem relating the compiled quartic to the checking formula. The ordinary existence result can then be exported to the current Diophantine interfaces without making the stronger witness theorem depend on a noncanonical general MRDP backend.

The key invariants for a formal implementation are: ordered cuts cover the entire integer domain; empty slots form a terminal suffix; their tags are fixed; adjacent active tags differ; every parent point is covered by a child monotonicity interval; all internal arithmetic computations have a unique normal form. These invariants are small enough to isolate into reusable modules, but no claim of completed formalization is made here.

70.4 Manuscript 03: A proposed ProveIt formalization architecture

The repository already exposes ordinary Diophantine closure interfaces, including `Diophantine.boundedForall_dioph`, `Diophantine.exactIter_dioph`, and `Diophantine.existsExactIter_dioph`, according to the inspected status register [2]. These are useful downstream interfaces, but their existential statements should not be treated as uniqueness theorems. A new compiler should retain the stronger specification until an application explicitly chooses to forget it.

A natural representation is a finite list of polynomials over $\mathbb{Z}$, evaluated on natural-number assignments cast into $\mathbb{Z}$. A semantic certificate object should record a decoder from satisfying assignments to source objects and a canonical encoder in the reverse direction, with both inverse laws on their specified domains. Counting is then a consequence of an equivalence of types or finite sets, rather than a separate combinatorial assertion.

The following module names are *proposals*, not files claimed to exist in the inspected repository:

Proposed module	Main proof obligation
<code>CanonicalQuadratic</code>	Natural-domain residual semantics, sum-of-squares combination, fresh-variable composition, and fiber equivalences.
<code>CanonicalCompare</code>	The two-equation signed-gap theorem, zero tests, unique bound slacks, multiplexing, and exact gadget counts.
<code>CanonicalSorting</code>	Record-preserving comparators, fixed network semantics, the bitonic generator, and its closed count formula.
<code>ExactProductFingerprint</code>	Base-expansion coefficient recovery, multiset equality from product identity, and injective record packing.
<code>CanonicalMemory</code>	Both backend theorems, chronological semantics, exact counts, and distinct height bounds.
<code>CanonicalRAM</code>	The request-before-response interface, temporal soundness induction, normalization of inactive instructions, and halted padding.
<code>CanonicalGraphRuntime</code>	Immutable heap unfolding, spine evaluation, microstep accounting, and the explicitly stated strategy correspondence.
<code>CertificateAudit</code>	Emitted-polynomial reflection, checked examples, domain conventions, and transitive assumption reports.

The linear backend offers an especially short conceptual trusted boundary. Its nonlocal step is the multiset lemma, reducible to elementary base expansion and polynomial cancellation; it requires no asymptotically optimal sorting network formalization. The low-height backend remains valuable when later applications need tight bit bounds.

An untrusted compiler may emit a residual syntax tree, but the kernel should check that the emitted object is the polynomial claimed by the compiler correctness theorem. Testing the value at one supplied witness checks only that instance. The complete theorem must quantify over every natural assignment to prove that there are no spurious roots, and over every source execution to prove that none are lost. A degree or variable-count audit should inspect the actual emitted object, not just a separately maintained formula.

A sensible order of work is to formalize the common residual semantics, then the product and comparison lemmas independently, then the two memory backends, and finally one small RAM instruction language. The combinator frontend should come after the precise runtime strategy and heap invariants have been fixed. Existing ProveIt MRDP interfaces can then export an ordinary Diophantine reachability statement, with an explicit note when that export forgets the canonical-fiber contract.

The inspected status register lists further work on the 1984 paper’s specific NP characterizations (53) and (54) [2]. The bounded-counting corollary here is not a formal proof of those numbered statements or a replacement for their required bounded-exponentiation theory. It should be integrated as a distinct result unless an exact implication is separately proved.

70.5 Manuscript 04: How these results extend the inspected ProveIt development

The mathematical extension point is a *witness-level* interface. An ordinary Diophantine theorem records an existential equivalence. A stronger reusable interface would record a semantic object, a decoder from every zero, a canonical encoder, and proofs that the encoder and decoder are inverse. Section 2.5 shows how circuit compilation can preserve that interface, and the three substrate constructions instantiate it in substantially different ways.

The repository snapshot used for comparison is

e8bb0931d67f80d9fce87a8cddb0f661ff19f956.

The inspection was focused on its Hilbert-tenth and combinatory-logic branches; it was not a full audit of every repository file. Its existing exact-iteration development already proves the existential closure we need [3]. The present paper should therefore be integrated as a refinement tracking fibers, not as a duplicate theorem that finite reachability is Diophantine.

There are three natural mathematical modules. The first is an injective-state-coding and unique-extension layer, including canonical slacks and circuit wires. The second is a semantic quotient layer: an independence relation, enabled-swap proof, normal-form monitor, and class-to-zero bijection. The third is a resource layer measuring variables, residuals, degree, coefficient bit lengths, and witness heights separately. A verified composition theorem should carry all three layers rather than only a proposition about nonempty solution sets.

The numerical bounds in this article do not compete with the repository's universal polynomial pairs or its optimized straight-line universal certificates [1]. Here the horizon and often the schedule are external compilation data, whereas a universal equation encodes unbounded computation with fixed arity. Comparing these counts without recording that distinction would be misleading.

No new repository proof-assistant build is reported in this package. The executable artifacts are standard-library Python implementations of the explicitly displayed bounded compilers. A separate source-audit note records the inspected paths and the claims relied upon, without treating repository documentation as an independent verification of the new results.

70.6 Manuscript 06: Proof dependencies and precise completion status

Result	Essential ingredients	Status in this package
Exact resource summaries	Partial translations, coordinatewise minimum, word induction	Full proof; implemented and tested
Global swap criterion	Exact two-letter resource products	Full proof; implemented and tested
Compact normality signature	Classical forbidden factors; Boolean affine coordinate maps	Full proof; implemented and tested
Cap-at-two saturation	Signature product; $\alpha_a \beta_a = 0$	Full proof; implemented and tested
Compressed quartic compiler	Unique local gates, syntax induction, sum of squares	Full proof; implemented and tested
Bounded trace-class bijection	One-hot words, deterministic memory, trace-invariant resource action	Full proof; separate export frontend not implemented
Flat-scheme decidability	Affine positive-power formula; Presburger case splitting	Full reduction proof; decision frontend not implemented
Degree–depth correspondence	Length polynomials; signed monomial splitting	Full proof; frontend tested
Depth-two multi-counter universality	MRDP; unique arithmetic-circuit lift	Full reduction proof using classical MRDP; frontend tested on a sample
Finite-fold equivalence	Unique quartic lift; exact separated length balance	Full equivalence proof; no solution of the general conjectures

Result	Essential ingredients	Status in this package
Formal verification	A proof-assistant implementation of all new definitions and compiler semantics	Proposed; not supplied or claimed

The only deep external theorem used for the universality conclusions is MRDP. The main compressed compiler, its cap-at-two theorem, and the bounded trace-counting construction do not depend on it. The quadratic one-counter decidability observation cites a separate classical decision theorem; it is not used to prove the flat/nested multi-counter boundary.

70.7 Manuscript 07: Integration with ProveIt and the formal trust boundary

70.7.1 What was inspected

The repository context was pinned at commit

725d2ebb6909fe11a13a92354c0f47367a3cbbf5

The inspected Hilbert-tenth-problem README and MRDP guide describe `Diophantine.mrdp`, `Diophantine.mrdp_iff`, and the finite-polynomial and trace interfaces [5, 1]. The guide reports an axiom-audited development and distinguishes the current forward proof from the older shared trace route. Those are repository reports, not build results independently reproduced for this package.

No theorem about the queues in this article is inferred from an unrelated research claim elsewhere in the repository. The explicit queue and word-memory proofs are independent mathematical arguments. Existing MRDP infrastructure is relevant to eventual fixed-arity representability, not needed for the bounded constructions themselves.

70.7.2 A proposed formalization sequence

A manageable formalization should separate word semantics from polynomial syntax. First define the cyclic-tag step as a partial function on finite Boolean lists with a phase parameter, stopping on the empty list. Prove the radix concatenation identity and fixed-length injectivity, then prove the suffix reconstruction theorem at the word level. These proofs need no Diophantine machinery.

Next formalize Theorem 8.1 over integer-valued formal resource histories. The invariant before the first disabled event is nonnegativity. This is the crucial point where natural-number truncation must not enter the formal recurrence. The first disabled event supplies a specific zero factor in the finite product.

Only then define the representing polynomial in

$$\text{MvPolynomial } (\text{Fin } (T+1)) \mathbb{Z}.$$

The first T coordinates are consumed symbols and the last is the guard slack. Prove that evaluating each symbolic residual gives its semantic integer expression; combine this with the sum-of-squares zero criterion. Soundness, completeness, and uniqueness should be separate theorem endpoints, followed by degree and variable counts. The quartic compiler should have its own endpoint because its preservation-of-valid-scale argument is different.

Finally, a reified arithmetic-DAG checker can connect exported data to these polynomial definitions. JSON parsing and Python generation should remain outside the trusted boundary. The checker must prove that accepted nodes define the intended polynomial and that the displayed witness evaluates to zero. Checking one witness alone must not be presented as a proof that no other witness exists.

70.7.3 What is shipped now

The package supplies human-readable proofs and tested Python compilers, including integer-only finite-table interpolation for deletion-tag systems. It supplies neither a Lean proof file nor an assertion that the new theorems have passed a kernel audit. The file `notes/lean_integration.md` [write] (shipped as `07-queue-causality-lean_integration.md`) records proposed modules and obligations, so a later formalization can begin without confusing an interface plan with completed code.

71 Research questions

[write] This section prints every research question of the seven manuscripts: ten from manuscript 01 (its Section 12), ten from manuscript 02 (its Section 12.2), ten from manuscript 03 (its Section 13), ten from manuscript 04 (its Section 11), twelve from manuscript 05 (its Section 13), twelve from manuscript 06 (its Section 12), and ten from manuscript 07 (its Section 13): seventy-four questions in all. Manuscripts 03, 04 and 06 posed their questions under subsection headings; those headings are the titles here, and 06’s question numbers are kept in the titles. Where several manuscripts ask the same question, the questions are merged into one question environment whose title names every source; inside it each source’s question text is printed verbatim under *Source NN*, and each source’s follow-up paragraphs are printed verbatim after the environment under the same heading. Seven merges reduce the seventy-four questions to fifty environments. Listing a question here does not claim that it is answered, by this report or elsewhere. Two questions are partly answered inside this report, and a [write] note at each says how far: manuscript 01’s question on sharp degree, arity and domain comparisons (Research question 71.16), by the degree-two certificate of Part II (manuscript 05); and manuscript 06’s Question 9 on new storage monoids (Research question 71.33), for bounded runs of fixed-read FIFO networks, by Part VI (manuscript 07). Manuscript 02’s closing paragraph on what would constitute a stronger result is printed as the last subsection.

71.1 How the manuscripts frame their questions

Source 01. The following questions separate tractable extensions of the present construction from the genuinely global compression problem. They are proposed investigations, not claims of solutions.

Source 03. The following questions separate tractable implementation work, quantitative mathematics, and the much stronger fixed-variable frontier. They are proposals from this investigation; calling them questions does not assert that each is unanswered in all existing literature.

Source 04. The following questions range from bounded construction problems to genuinely stronger fixed-arity goals. They are proposed research targets, not all asserted to be previously named open problems. Each is stated so that a success criterion and a likely failure mode are visible.

Source 05. The following agenda separates finite, testable development tasks from the genuinely stronger uniform arithmetic questions. None of the questions is claimed to have a new answer merely because it is listed here.

Source 06. The following questions are proposed continuations, not a claim that every one is a newly identified open problem in the literature. They are organized to separate tasks that build directly on the proved compiler from those that confront the finite-fold frontier.

Source 07. The questions in this section are unresolved by this article. Except for the explicitly identified finite-fold topic, they are proposed construction and optimization problems, not assertions that a literature-wide open-problem status has been established.

Source 07, closing paragraph of its question section. The most immediate projects are the first-failure and FIFO formalizations, a complete fixed-read network compiler, and optimization of the quartic variable count. They have precise interfaces and can be validated independently. The fixed-arity problem is a separate, more demanding line of investigation, for which the present explicit unique traces identify exactly what information a successful compression must preserve.

[write] Manuscript 02 printed its ten questions under the heading “Research questions with concrete targets” without an introductory paragraph.

71.2 Verified compilers and fibre-preserving interfaces

Research question 71.1 (A verified single-fold compiler interface; sources 01, 02, 03, 04, 05, 06, 07). *Source 01 (Formalizing the exact resource envelope)*. Can ProveIt obtain a reusable finite-type theorem for box-guarded translations whose specialization automatically produces Petri and finite-control counter adapters? A useful first milestone is a kernel-checked equivalence between all multiset serializations and (17)–(18), including empty support, finite upper bounds, and empty composition domains. The elementary finite combinatorics make this a focused formalization target.

Source 02 (A verified single-fold compiler). Formalize the total sign gadget, signed normalization, and the endpoint propagation theorem, then extract a compiler whose output includes a proof of both equivalence and unique extension. A useful first milestone is the complete degree-two template, including zero polynomials and $T = 0$, rather than a generic compiler with unproved boundary cases.

Source 03 (What is the right verified compiler interface?). Build an actual Lean polynomial emitter whose contract contains soundness, completeness, inverse decoding, degree, sparsity, and height bounds. The two backends provide a test of whether the abstraction separates semantics from resource properties cleanly. A useful first milestone is formal verification of both four-event exports in this archive, followed by the symbolic theorem for arbitrary L ; finite examples alone are not the endpoint.

Source 04 (Canonicity-preserving arithmetic library design). Develop a library of polynomial gadgets certified not only by existential equivalences but by explicit solution-set bijections, together with exact cost and height annotations.

Source 05 (Kernel-checked fibre preservation). Can the full equivalence of Theorem 11.2, including every slack coordinate, be formalized as a reusable compiler theorem in ProveIt? A concrete first target is to reproduce the raw, quartic, and quadratic variable counts with a finite-indexed polynomial definition and an executable witness constructor.

Source 06, Question 10 (A small verified compiler and a multiplicity audit). Formalize the resource pair action, the minimum and selector gadgets, the Boolean signature product, and the structural single-fold theorem as a compact independent layer. The decisive endpoint should state a bijection between the specified semantic witnesses and polynomial roots, not merely an implication from a generated certificate to acceptance. A subsequent integration with ProveIt's existing Diophantine interfaces could preserve that stronger contract. The audit should explicitly check natural versus integer subtraction, zero-length traces, inactive branches, and every projection that turns a parameter into a witness.

Source 07 (Formal arithmetic-DAG compilation). Implement the proposed ProveIt modules for first-failure products, word reconstruction, and certificate evaluation. The first deliverable should be the full equivalence and uniqueness theorem for the cyclic-tag stream compiler, not just a theorem that a supplied example evaluates to zero. Degree-bound reflection can follow independently.

Source 04. Priority selection, canonical remainder, fixed-pattern matching, context reconstruction, and unique-extension quadratization are concrete initial modules. The important regression tests are inactive-variable freedom, integer-versus-natural subtraction, duplicate state encodings, and mismatched semantics between compiler stages. Formalizing the Petri-net theorem is an especially contained first milestone because its state and monitor equations are only quadratic.

Source 05. The decisive acceptance test is the identity $\text{encode} \circ \text{decode} = \text{id}$ on the full solution subtype. A proof only of solvability equivalence is not sufficient for the counting applications.

Research question 71.2 (Which normal forms preserve canonical fibers?; source 03). Investigate small coefficient sets, bounded variable occurrence, and more restrictive quadratic gate types. Introducing a uniquely defined copy of a coordinate preserves parsimony, but replacing natural coordinates by ordinary integer sum-of-squares representations generally does not. Identify exactly which normal-form

transformations preserve existence only, finite fibers, or singleton fibers. This could become a reusable theorem library independent of memory checking.

Research question 71.3 (Substrate interfaces with explicit witness contracts; source 02). Build counter-machine and FRACTRAN libraries whose one-step arithmetic certificates are single-fold and whose composition interfaces state exactly which property survives. Use the polynomial accelerator only when a checked trajectory certificate is available; otherwise fall back to an ordinary existence-preserving backend. The resulting mixed system would make the boundary between established local uniqueness and unresolved unbounded compression visible to users.

71.3 Commutation, independence and canonical representatives

Research question 71.4 (State-dependent independence and commutation beyond a fixed relation; sources 01, 04, 05, 06, 07). *Source 01 (Dynamic rather than global independence)*. Can state-dependent commuting diamonds support a canonical arithmetic trace quotient? Global partial-map equality gives a congruence for free. A rule that swaps actions only when both orders happen to work at the current state may lose a simple global normal form. A successful extension needs explicit coherence conditions and a proof of canonicity, not merely local diamond checks. Small counterexamples should be sought before designing the polynomial layer.

Source 04 (Canonical certificates for dynamically independent actions). Can the quadratic monitor be generalized to a sound state-dependent independence relation while retaining one solution per chosen execution-equivalence class and a polynomial-size bounded certificate?

Source 04 (Beyond disjoint supports). Find effectively checkable families of transition pairs with overlapping supports that satisfy the enabled-swap property for all reachable markings, and derive more aggressive trace quotients without losing witness-faithfulness.

Source 05 (State-sensitive commutation). Can the static relation be replaced by a state-dependent diamond test while retaining a unique polynomial-size normal-form verifier? What precise consistency conditions replace the trace-monoid forbidden-factor language?

Source 06, Question 5 (Invariant-sensitive and state-dependent swaps). Replace equality of partial actions on all markings by equality on a prescribed invariant, such as exactly one control token. This could admit more commutations than (29). Which classes of invariants permit an effective exact test, and when do the resulting swaps support a sound canonical cross-section? If the permitted swap depends on the current marking, a fixed trace monoid may no longer be the right object; confluence and uniqueness then require new proofs, not just a larger pair relation.

Source 07 (Concurrent traces rather than schedules). For communicating FIFO processes, can solutions represent equivalence classes of independent event schedules while preserving channel order? A swap can change which symbols an event observes, so a static independence relation is not automatically sufficient. A semantic commutation criterion should be proved before importing schedule-normalization techniques.

Source 04 (Canonical certificates for dynamically independent actions). The current proof relies on a fixed trace congruence. If independence changes with state, a swap can change later independence tests and equivalence need not be described by a fixed graph. A meaningful first target is a finite-state system with a certified local diamond property, together with a canonicalization theorem that explicitly proves class preservation. Simply evaluating a different I at every row of (57) is not justified by Section 14.

Source 04 (Beyond disjoint supports). For example, shared read-only resources can permit commutation beyond literal support disjointness. The right hypothesis is preservation of both enabling and outcome, not merely commutativity of the net incidence vectors. A certificate for an invariant region of markings may justify additional swaps. Any added pair must preserve the complete endpoint language used in Theorem 15.2.

Source 05. A local diamond at one marking is insufficient to justify arbitrary sequences of swaps. A satisfactory answer must show invariance of enabledness across the whole generated equivalence and uniqueness of the chosen representative, not just local endpoint equality.

[write] The hypothesis every part of this question asks to relax is the one fixed in Theorem 11.2: a single symmetric irreflexive relation $I \subseteq I_{\max}$, where $I_{\max}$ of (8) contains exactly the pairs whose two-step partial actions agree at every marking. Under that hypothesis a swap is sound in every context; the six texts above ask what replaces it when commutation is allowed to depend on the current state, on a prescribed invariant, or, for FIFO channels, on the symbols an event observes.

Research question 71.5 (Structure of independence graphs; source 05). Which graph parameters permit smaller flag systems or tractable exact class counting? Candidates include neighbourhood inclusion, modular decomposition, and widths of a graph encoding the residual interactions.

A useful project would report both coefficient sparsity and actual enumeration behaviour on a library of graphs. Theorem 4.4 rules out assuming that Petri-net semantic commutation graphs belong to a narrow graph class in general.

Research question 71.6 (Minimal signatures and algebraic structure; source 06, Question 1). For a given independence graph and alphabet order, what is the smallest exact block signature for normality? Our upper bound of $2 \cdot 6^q$ possible records is often redundant, especially when the scalar acceptance bit is zero. Determine the reachable signature submonoid, compare it with the syntactic monoid of the normal-form language, and characterize when every reachable signature is already idempotent. The general exponent two is sharp, but its dependence on graph structure can be finer. A useful outcome would be a smaller circuit representation together with a proof that it preserves the $S^2 = S^3$ law.

Research question 71.7 (Compressed normalization rather than testing; source 06, Question 2). Given a parametrized repetition expression, can one produce a comparably small expression for its lexicographic normal word, uniformly in the parameters? The present result only tests whether the original expansion is normal. In the fully commuting case, sorting is controlled by letter counts, but general dependencies retain nontrivial ordering information. The key issues are the output representation, whether new parameter arithmetic is allowed, and whether polynomial-size output can always exist. A useful first target is bounded dependence width or a fixed small alphabet.

Research question 71.8 (Trace-injective parameterizations; source 06, Question 3). Which repetition templates have at most one parameter tuple per trace, or finitely many tuples per trace? The example $(a^{k_1})^{k_2}$ shows why empty powers and factorization can destroy injectivity. Can redundant parameters be removed in a useful decidable fragment while retaining a compact expression and a witness-preserving compiler? A classification for flat schemes would connect the Presburger execution relation with trace equality, but execution equality alone is not a sufficient substitute for the latter.

Research question 71.9 (Unique acceleration of repeated causal patterns; source 01). The accelerator compresses one commuting multiset, and Proposition 22.2 already handles a prescribed fixed word repeated an arbitrary number of times. Can a general causal trace be assigned a unique compressed segmentation into such patterns? The unresolved step is to infer the decomposition without multiple period or boundary choices representing the same trace. A useful restricted target is a class with a canonical primitive-period factorization, together with a proof that the arithmetic witnesses preserve that factorization uniquely.

Research question 71.10 (Canonicity inside restricted schedule families; source 06, Question 4). Find effective syntactic conditions ensuring that a schedule family contains the normal representative of every trace it contains. Under such a trace-closure condition, the canonical compiler would preserve existential reachability within the family, avoiding the failure in Remark 25.5. One may alternatively seek a certified expansion of the family that is trace-closed but has controlled size. The cap-at-two theorem makes testing a proposed representative cheap; it does not by itself construct a family containing that representative.

71.4 Counting, weights and relaxations

Research question 71.11 (Counting trace classes, histories and certificates; sources 01, 03, 04, 05, 06). *Source 01 (Counting and generating functions)*. For a finite invariant marking region, can the canonical-layer transition system yield useful exact rational generating functions for trace counts, weighted by event number or cost? A finite-state construction is plausible, but extracting small formulas and comparing different valid independence relations remain worthwhile tasks. The quartic roots give a direct arithmetic interpretation of those counts and a way to test candidate recurrences on finite instances.

Source 03 (Can concurrency retain meaningful counting semantics?). A sequential log gives one total order of accesses. For a concurrent execution, different legal linearizations may describe the same partially ordered history. An existential timestamp certificate would count linearizations, not necessarily semantic histories. Develop canonical linearization conventions or explicitly weighted quotient semantics, distinguishing sequential consistency from weaker memory models. The local memory checker is a backend, not a solution to those semantic choices.

Source 04 (Counting directly from structured residual systems). For which structural restrictions on a net or independence graph can the quartic certificates be counted or sampled efficiently without expanding the entire reachable-state automaton?

Source 05 (Counting algorithms under explicit parameters). Which parameters yield fixed-parameter or output-sensitive algorithms for counting the lattice points of these particular trace polytopes? Parameters should be stated precisely: fixed horizon, small active transition set, bounded residual-graph width, or a restricted control architecture give different problems.

Source 06, Question 11 (Counting beyond bounded words). Extend the bounded trace-class bijection to compressed families with a known trace-injective parameterization, and then to weighted trace counting. A weight assigned to an execution must be invariant under the chosen commutations before it can be interpreted as a trace weight. The main obstruction for unbounded families is not the local quartic encoding; it is controlling the number of parameterizations of a behavior and the witnesses introduced by any unbounded coding mechanism.

Source 04. Potential parameters include the interaction-graph treewidth, the size of conflict sets, and the number of shared resource places. Theorem 16.2 rules out a general polynomial-time exact algorithm under standard complexity expectations, but does not rule out parameterized algorithms. The natural comparison point is current trace counting and sampling for regular languages [21]; input representation and trace closure must remain explicit.

Source 05. An empirical benchmark should compare direct canonical-word generation, lattice-point methods, and dynamic programming. The #P-completeness result gives a worst-case boundary, not a prohibition on useful structured algorithms.

Research question 71.12 (Weight-preserving quotients; source 05). For which state-dependent costs, probabilities, or resource observations does a commutation quotient preserve the intended observable? Can the necessary observation data be augmented without destroying a useful independence relation?

Label-additive weights are fully handled by Corollary 12.1. Peak space, scheduler probabilities, and costs depending on the current marking require additional analysis. The correct target may be a weighted sum over linear extensions rather than one unit per class.

Research question 71.13 (What extends to weighted, probabilistic, or algebraic computation?; source 03). The fair-bit formula handles exact positive path weights with canonical stopped padding. Extend the construction to bounded-denominator rational transition probabilities and track a uniquely defined common denominator. For quantum circuits, path counting alone is insufficient because amplitudes can

cancel; one must specify exact algebraic coefficient representations and certify the amplitude arithmetic and measurement rule. The distinction between counting paths and summing amplitudes is fundamental.

Research question 71.14 (Nondeterminism and path multiplicities; source 02). For a finite family of polynomial phases, can a certificate have one solution per distinct execution, rather than one solution per endpoint? Canonical encodings of branch sequences and phase boundaries are needed, especially when two phase descriptions overlap. A first tractable case is a fixed acyclic control graph with locally deterministic phases and explicitly labeled nondeterministic edges.

Research question 71.15 (Integral or ideal relaxations; source 05). For which subclasses of nets and independence relations is the canonical trace polytope integral? Can one characterize subclasses admitting a polynomial-size ideal extension, meaning an extension whose real projection is the convex hull of the relevant discrete encodings?

Example 11.4 supplies a small explicit obstruction to unrestricted integrality. Total unimodularity of a suitable residual matrix would be a sufficient condition, but the canonicalization constraints need not preserve it. The general counting hardness theorem also prevents treating a compact formulation as a general counting algorithm.

71.5 Size, degree and height of certificates

Research question 71.16 (Sharp degree, arity, domain and variable-count comparisons; sources 01, 07). *Source 01 (Sharp degree, arity, and domain comparisons)*. How much of the quartic construction is essential? Totals, forced-zero gains, and some blocker products can be eliminated in special cases. A systematic optimization pass should preserve the complete-root bijection, rather than only satisfiability. Extending the result from natural witnesses to unrestricted integer witnesses is also not automatic: a four-squares conversion can introduce many witnesses for one natural value. A separate canonical integer-domain encoding is needed for a parsimonious statement over $\mathbb{Z}$.

Source 07 (Nearly trace-minimal quartics). Can a structural cyclic-tag compiler retain the actual T consumed bits, use only $T + O(1)$ total natural variables, have degree at most four, and have polynomial, preferably linear, arithmetic-circuit size? The present stream construction achieves the variable count with growing degree; the local construction achieves quartic degree with $3T - 2$ coordinates.

Source 07. A useful first milestone is to improve the coefficient of T in the quartic construction while retaining complete witness uniqueness. Any lower-bound attempt must exclude the trivial compiler that first simulates the bounded run and outputs a constant.

[write] Manuscript 05's convex quadratic certificate (Theorem 11.2, degree two, Part II) answers part of manuscript 01's question: for length- T trace classes of Petri nets and of interval-guarded translations (Corollary 11.5), degree four is not needed. The height-bounded quartic of Theorem 20.1 and the integer-domain part remain open, and so does manuscript 07's variable-count question for cyclic-tag compilers, which the degree-two certificate does not address.

Research question 71.17 (Smaller canonical predecessor circuits; source 01). Can the $(H - 1)d$ blocker witnesses be reduced by sharing common subproducts among the dependence neighborhoods $D(a)$? The objective is a smaller *functional* Boolean circuit, so every introduced gate remains uniquely determined. Graph classes with repeated neighborhoods, bounded treewidth, or a small neighborhood basis offer concrete cases. The benchmark should report witness count, residual count, and expanded polynomial size separately.

Research question 71.18 (Smaller uniquely extended affine systems; source 05). Can the five local auxiliary variables per time-label pair in (45)–(49) be reduced uniformly while retaining a convex quadratic sum of affine squares, bounded real relaxation, and unique natural extension?

The displayed constants are not optimality claims. Eliminating variables symbolically, sharing complementary quantities, or exploiting implication relations among flags may help. The automaton lower bound concerns Boolean memory and does not obstruct such arithmetic improvements by itself.

Research question 71.19 (Witness and coefficient optimization; source 06, Question 8). How far can the explicit variable bounds be reduced while preserving unique lifting and manageable coefficient heights? Potential improvements include constant propagation, shared exponent selectors, common-subexpression elimination, suppression of determined bit equations, and direct compilation of the square-saturation formula rather than a full signature square. The relevant objective is a tuple of costs: witness count, residual count, degree, sparsity, coefficient bit length, and verification depth. Optimizing only an expanded monomial count can make the actual certificate worse. Comparisons with universal-equation operation counts must use the same charging conventions and witness domains.

Research question 71.20 (Reduce the cubic arithmetic overhead; source 02). Can the full certificate be reduced from $O(d^3)$ to $O(d^2)$ natural coordinates while retaining a quartic equation and a polynomial-time witness generator? The geometric coordinates are already quadratic. A promising target is a canonical merge of neighboring sign partitions, avoiding all parent–child pairs, together with shared multipoint polynomial evaluation. Both the merge and its arithmetic intermediates must have unique encodings; an arbitrary permutation witness would not do.

Research question 71.21 (Quantitative lower bounds; source 02). What lower bounds can be proved for canonical sign-table certificates under a specified gate model? A bound on our particular interval layout would not be a lower bound on all Diophantine representations. Separating geometric information, arithmetic circuit size, and bit height is necessary before any optimality statement is meaningful.

Research question 71.22 (Certificate height and practical checking; source 02). Derive tighter coefficient-sensitive bounds for every intermediate wire, not only asymptotic bit bounds. These can guide bounded exact search, serialization, and independently checked resource estimates. Compare the canonical root cutoff with a minimal stabilization point; a smaller cutoff is useful only when its certification does not cost more than the saved sign-table work.

Research question 71.23 (Can the size–height tradeoff be improved substantially?; source 03). The linear backend has $O(L)$ scalar witnesses but $O(L^2(b + \log L))$ maximum bit length, whereas the comparison backend has near-linear scalar size and essentially input-sized witnesses. Is there a canonical $O(L)$ -coordinate memory certificate with $O(L(b + \log L))$, or even $O(b + \log L)$, bits per coordinate? A first concrete target is an exact multiset certificate that avoids a base of size D^{4L} while introducing no choice of primes, hash challenges, or noncanonical factorization witnesses. The comparison lower bound does not settle this question.

Research question 71.24 (What is the best explicit scalar constant?; source 03). The count $24L$ deliberately uses a simple length- L power chain. Binary powering reduces the power stage to $O(\log L)$ coordinates. Eliminating redundant output fields, optimizing record packing, and sharing certified subexpressions may improve the leading constant further. Every proposed optimization must preserve injectivity of the packed record and uniqueness of all discarded or shared witnesses. Publish the exact generated residuals and count proofs, not only an operation-count estimate.

Research question 71.25 (Can one minimize total witness bit volume?; source 03). Maximum coordinate height is not the only meaningful metric. Balanced product trees can reduce the sum of intermediate bit lengths while retaining linear many product nodes. A systematic comparison of sequential products, balanced products, carry-free block packing, and sorting networks could yield an explicit Pareto frontier for scalar count, total certificate bits, arithmetic work, and parallel depth. The current implementation uses sequential products for a transparent exact count; it is not claimed optimal in those other metrics.

Research question 71.26 (Can canonical routing be made practical at optimal comparison order?; source 03). The AKS theorem supplies the asymptotic bound but not a small practical backend here. A routing-based memory checker may have fewer constraints, yet often admits multiple internal routes for one permutation. Can those routes be canonically normalized with low overhead and a short proof? A deterministic routing algorithm is not enough unless its own computation is certified without reintroducing the original overhead or unbounded auxiliary choices.

Research question 71.27 (Locality with polynomial bit-height; source 04). Can a canonical DAG or balanced-tree coding retain context-local certificate size and also provide polynomial witness bit lengths in the size of a succinct input representation and the performed updates?

The pairing code pays for repeated constructor nesting by repeated squaring. A DAG can share repeated subterms, but arbitrary node numbering introduces enormous witness multiplicity. One needs canonical numbering or a quotient theorem for node-renaming equivalence, plus unique proofs of pointer validity and acyclicity. A first useful milestone is a linear-height representation of purely applicative terms with a canonical topological order.

Research question 71.28 (Sharp arithmetic-memory tradeoffs; source 04). Can the r flags in the monitor be replaced by fewer natural variables without hiding exponential bit-height, high degree, or a costly digit-extraction gadget?

Theorem 14.3 gives a streaming bit lower bound, not a variable lower bound. Packing all flags into one integer is easy at the data level but makes bit access a new arithmetic obligation. A useful theorem would state a tradeoff among witness arity, witness height, residual degree, and circuit description size, with an explicit computational model rather than an informal “compression” claim.

Research question 71.29 (A linear-size zero-slack guard; source 07). Can the no-slack construction be made linear-size in T for a fixed program, while retaining degree $O(T)$ and precisely the consumed-symbol coordinates? The bounded-root guard uses quadratic size in the relevant emptying regime. The question concerns circuit structure, not existence of some polynomial with the correct finite zero set.

Research question 71.30 (Smaller uniquely determined guard values; source 07). Can a guard with similarly small circuitry replace the product of all queue lengths by a substantially smaller canonical natural witness? A useful objective is a bound linear in the trace length plus the logarithm of the maximum queue length, rather than their product, without introducing many separate slacks.

71.6 Richer actions, storage and trajectories

Research question 71.31 (Beyond translations; source 01). Which reset, affine, stack, or indirect-addressing primitives admit exact fixed-arity unique-witness acceleration? The present extremal-prefix argument works because displacement sums are independent of the current value. Reset and multiplication destroy that property. One concrete target is diagonal affine actions on invariant regions where products and guards have a controlled closed form; an answer must separately account for witnesses used to represent any exponentiation.

Research question 71.32 (Reset, copy, and richer guarded updates; source 05). Can Theorem 4.5 be extended to partial affine maps with reset or copy operations, with a similarly explicit exact commutation criterion?

Unlike translations, different affine expressions can agree on a lower-dimensional guarded domain without having equal coefficient matrices. Any exact criterion must account for those domain equalities as well as for empty compositions. A bounded compiler can still use finite Boolean verification, but that does not supply the desired symbolic all-state criterion.

Research question 71.33 (New storage monoids and genuine exponential growth; source 06, Question 9). Identify further action algebras satisfying the local unique-summary contract of Proposition 61.1. Promising structured cases include finite-state products, selected queue restrictions, and stack fragments with bounded unmatched contexts. The essential test is whether arbitrary powers retain an exact finite summary with controlled polynomial witnesses. Affine multiplication of a runtime counter and unbounded string coding should be treated as separate exponential-growth problems, not added as innocuous gates.

[write] The queue case is partly answered by Part VI (manuscript 07): Theorem 52.2 gives a witness-faithful polynomial whose natural zeros are in bijection with the exact T -event executions of a fixed-read FIFO network program. That is a bounded-run certificate for a fixed schedule; it does not supply the exact finite summary of arbitrary powers that the contract of Proposition 61.1 asks for. Networks whose read counts depend on data or on emptiness tests stay open (Research question 71.34).

Research question 71.34 (Data-dependent read counts and zero tests; source 07). Extend Theorem 52.2 from fixed phase-dependent read counts to finite-control networks whose consumption sizes depend on data or on emptiness tests. Specify exact witness counts and ensure that local branch selection and global availability do not admit circular justifications. The fixed-read theorem gives a controlled starting point rather than a completed compiler for every actor language.

Research question 71.35 (Rational and piecewise-polynomial memory; source 07). Which extensions of the scalar insertion architecture invalidate Theorem 50.4? Study rational insertion maps with poles excluded from the code image, or a bounded number of polynomial branches. The corresponding centralizer restrictions and the natural-number image condition both matter. A classification should separate genuine scalar memory from hidden finite-state coordinates.

Research question 71.36 (Automatic trajectory discovery; source 02). Develop a front end recognizing polynomially iterable updates and producing the trajectory identities as checked certificates. For affine maps, a nilpotence witness or an identity of the form $M^h(M^P - I)^r = 0$ is an immediate target. For nonlinear maps, triangular coordinate changes and verified discrete antidifferentiation could enlarge the recognized class. The synthesis procedure may be incomplete while its accepted certificates remain sound.

Research question 71.37 (Variable periods and variable degrees; source 02). What uniformity survives when the degree bound or residue period is part of the numerical input rather than fixed externally? A family of polynomials with growing arity does not answer this question. Any proposed single fixed polynomial must account for the encoding and checking of variable-length coefficient data without silently using an ordinary MRDP representation that loses single-foldness.

Research question 71.38 (Nonlinear guards on fixed-residue trajectories; source 02). Implement the residue-class front end of Theorem 32.4, with canonical Euclidean division and empty-class normalization. Extend its nontermination theorem using one canonical root bound per residue class. This is a concrete development target within the proved framework, not a claim about arbitrary exponential recurrences.

Research question 71.39 (Parameter sharing, depth, and resource tradeoffs; source 06, Question 6). Determine which restrictions on global parameter reuse restore decidability at nesting depth two. The universality proof reuses numerical variables across several monomial blocks and circuit equations. Restrictions such as one syntactic occurrence per parameter, bounded occurrence count, or an acyclic parameter-incidence pattern may separate tractable subclasses. Independently, determine quantitative tradeoffs between the number of resources and repetition depth. One counter at depth four and finitely many counters at depth two suffice; exact small-resource bounds would require an explicit, audited construction rather than a qualitative MRDP invocation.

Research question 71.40 (The separated cubic interface; source 06, Question 7). Use Theorem 26.3 as an exact interface for studying separated one-counter schemes of depth three. Every theorem about this fragment translates to a theorem about natural solutions of one cubic polynomial, and conversely. Useful restricted targets include bounded numbers of parameters, sparse support, prescribed sign patterns, or limited parameter reuse. The correspondence prevents one from mistaking an unproved general cubic decision assertion for a routine consequence of counter simplicity.

Research question 71.41 (Unbounded but structurally restricted finite-foldness; source 01). What larger classes of fixed-skeleton counter programs admit endpoint-level finite-fold certificates? Corollaries 22.3 and 22.4 give simple sufficient conditions. Potential extensions use guard-induced count bounds even when the displacement matrix has a nonnegative kernel, or unique solutions of a lower-triangular count system when full column rank fails. The target should be a checkable structural criterion with a proved fibre bound.

71.7 Rewriting, graph evaluation and comparisons across substrates

Research question 71.42 (Bridges between combinatory or graph evaluation and certificates; sources 01, 03, 04). *Source 01 (A fully specified SKI/Iota bridge)*. Can the local quadratic application-code equations be assembled into a parsimonious representation of a chosen bounded evaluation semantics? The immediate milestones are a unique-witness term-validity checker under a stated size bound, a canonical redex-position encoding, and an evaluation-strategy-specific reflection theorem. The repository's distinction between scoped contextual lambda reduction and weak call-by-value semantics must remain explicit.

Source 03 (How should graph strategies connect to the existing calculi?). Establish the exact relation between the immutable spine evaluator and the repository's contextual combinator reductions, then between the particular lambda semantics already present in its separate developments. Which source observables are preserved and reflected? Which administrative steps should be counted, erased, or quotiented? For probabilistic or nondeterministic evaluators, a quotient on histories can change path multiplicity and therefore requires a measure-preserving argument, not just a reachability simulation.

Source 04 (Existential SKI schedules without path duplication). For horizon T and a maximum context depth D , construct a certificate with $\text{poly}(T, D)$ local variables whose solutions are in bijection with valid scheduled SKI reductions, while the redex paths themselves are chosen existentially.

Source 04. The fixed-schedule theorem is not this result. A direct disjunction over all paths of length at most D has exponentially many branches. A prospective construction should choose one direction bit per active depth, enforce one canonical stopping point, and force all unused suffix variables to zero. It must also guarantee that a shorter path does not acquire many padded encodings. An even stronger version would choose a canonical reduction strategy, rather than count arbitrary schedules.

Research question 71.43 (Faithful graph-rewriting front ends; source 05). Can bounded term-graph or interaction-net histories be compiled with a canonical node-allocation discipline and a proved commutation quotient that respects shared substructure?

The important distinction is between counting graph-rewrite histories, tree interpretations, and observable normal forms. Allocation symmetries and accidental free names can destroy parsimony even when the underlying reduction relation is well understood.

Research question 71.44 (Cost-preserving compilation between substrates; source 04). Build translations among counter machines, FRACTRAN, and a deterministic combinatory evaluator that preserve a canonical execution object and give explicit arithmetic-certificate overheads.

Forward simulation is not enough. The target must not introduce multiple admissible administrative schedules for one source step, unless those schedules are canonically quotiented. A useful statement would identify a unique target macrotrace for each source step, prove reflection at designated boundaries, and bound variables, time expansion, and bit-height. This would make cross-substrate comparisons quantitative and semantically trustworthy.

Research question 71.45 (Spatial certificate transport with explicit clocks; source 07). Build a formally checked bridge from a concrete cyclic-tag program to its glider or Rule 110 realization, recording exact macro-step boundaries, periodic background conventions, and observable halting events. Then compare a transported queue certificate with a direct space-time polynomial, including bit costs as well as variable counts.

Research question 71.46 (Quantitative universal compilation; source 07). Compare universal simulation routes by the final arithmetic resource vector: number of consumed symbols, polynomial degree, circuit size, coefficient bit size, and witness bit height. Simulation time alone does not determine that vector. A useful experiment is to compile the same family of small algorithms through both deletion-tag and cyclic-tag routes, with formal clock correspondence.

[write] This question compares horizon-indexed compilation routes by their resource vectors; it asks nothing about fixed arity or fibres, so it is printed here and not merged into Research question 71.48.

Research question 71.47 (Representation-size versus operation-count optimization; source 05). Can ProveIt's compact universal-equation and straight-line-certificate work be given a semantic interface that records which optimizations preserve arithmetic fibres, and which preserve only solvability?

Operation counts for fixed universal certificates and variable counts for a time-unrolled compiler measure different resources. A useful comparative study should report both without conflating them, and should test whether elimination or relation-combining steps introduce multiple witnesses.

[write] This question stays a question. It concerns ProveIt's operation-counted straight-line certificates (the Hilbert's-tenth-problem project's 75-operation universal certificate [1]), which measure a different resource from the witness counts of this report, and no work in that project is opened here.

71.8 The fixed-arity frontier

Research question 71.48 (Fixed-arity compression with finite or single fibres; sources 01, 02, 03, 04, 06, 07). *Source 01 (The universal compression problem)*. Can a fixed-arity ordinary polynomial represent first-halt computations of a universal interpreter with finite, or single, witness fibres? Theorem 62.1 identifies this as the full finite-fold or single-fold MRDP principle, not an omitted routine assembly lemma. A useful contribution would isolate a genuinely canonical unbounded coding primitive and prove that every helper variable has controlled multiplicity. Merely choosing one semantic run or its least numerical code does not discharge that obligation.

Source 02 (The first genuinely exponential extension). Identify a meaningful subclass of exponential-polynomial trajectories that admits finite-fold or single-fold ordinary polynomial certificates without assuming such a representation for 2^n . Any extension including the doubling graph already has the strength established in Proposition 64.1. A conditional theorem that exposes this dependency is useful; hiding it inside an exponentiation predicate is not.

Source 03 (Can one make genuine progress on fixed-variable compression?). Theorem 62.1 identifies the strongest frontier precisely. A single-fold polynomial for one universal halting relation would settle the general single-fold question; a finite-fold one would settle its finite-fold counterpart. Investigate restricted trace families whose coding and all decoding relations have independently proved finite fibers. Any attempted extension to universal traces must explicitly discharge the new uniqueness or

finiteness obligations. The fact that the source machine is deterministic is not such a proof, and no total computable global root bound can be supplied by Proposition 62.2.

Source 04 (The universal fixed-arity challenge). Can one preserve the first-halting uniqueness of Corollary 54.3 while eliminating the horizon-dependent number of variables for a universal interpreter?

Source 06, Question 12 (Finite-foldness in its minimal substrate). Study finite or singleton successful-parameter fibers directly in the separated normal forms of Theorem 67.1. The theorem shows that an unrestricted positive result would have the full force of the corresponding Diophantine conjecture. More focused questions concern special computably enumerable relations or special polynomial support patterns whose schedule fibers can be controlled constructively. An answer must constrain the *parameter tuples*; uniqueness of the auxiliary arithmetic certificate is already supplied here and is not the missing step.

Source 07 (Fixed-arity compression with controlled fibers). Can one compress these canonical exact-halting certificates into a fixed number of Diophantine coordinates while preserving finite or unique fibers? An approach must identify and control every auxiliary introduced by word coding and bounded verification. This is the link to the published finite-fold research program [11], not a consequence already contained in the present theorems.

Source 04. Theorem 62.1 explains the strength of a positive answer. This is the genuinely global compression problem, rather than a consequence of the existing deterministic gadgets. Any proposal should identify the exact number-theoretic lemma controlling the fibers of trace encoding, bounded universal elimination, or exponentiation replacement. The witness-height obstruction also shows that no computably uniform bound on positive witnesses can accompany a universal solution.

[write] The source theorems cited in this question are merged in Part IX as Theorem 62.1 (the universal-halting equivalence) and Proposition 62.2 (no computable witness-height bound).

Research question 71.49 (Restricted unbounded classes with fixed-arity finite-fold or single-fold compression; sources 04, 05). *Source 04 (Finite-fold compression for restricted loop patterns).* Identify nontrivial unbounded program classes whose canonical bounded tableaux can be compressed into one fixed-arity finite-fold polynomial, with an explicit proof controlling every auxiliary fiber.

Source 05 (Restricted fixed-dimensional compression). Which explicitly described nonuniversal families admit fixed-dimensional single-fold arithmetic representations of their histories? Can a theorem state a sufficient structural hypothesis on the transition system together with the exact arithmetic primitives needed for compression?

Source 04. A tractable starting point is flat control flow with a fixed number of loops and affine or multiplicative updates. The challenge is not merely to summarize a loop existentially, but to show that loop counts, intermediate summaries, and all number-theoretic auxiliaries admit finitely many choices. One should state precisely which restrictions prevent the construction from immediately covering a universal halting relation.

Source 05. A computable time bound alone is not a proof that its graph, or the associated sequence coding, has a single-fold polynomial representation. A successful restricted result must audit those arithmetic fibres. Theorem 62.1 provides a sharp warning whenever the family contains canonical universal first-halting histories.

Research question 71.50 (Finite-fold arithmetic primitives; source 05). Can one establish finite-fold or single-fold polynomial representations for the sequence or growth relations actually used in a uniform history compressor, rather than for an unspecified black-box decoder?

This connects directly to the arithmetic questions in [9, 10]. The computational front end developed here isolates what such a result would need to preserve. It neither supplies nor assumes the missing number-theoretic theorem.

71.9 What would constitute a stronger result (manuscript 02)

A successful optimization of the present compiler, or its formal verification, would strengthen a restricted theorem already proved here. A genuine extension to universal first-halting computations, or to a single-fold polynomial graph of 2^n , would cross a different boundary and have general representability consequences. Keeping those two kinds of progress separate prevents a modest but useful construction from being mistaken for a resolution of the finite-fold program.

72 Conclusions of manuscripts 05, 01, 04 and 06

[write] *The conclusions of manuscripts 02, 03 and 07 close Parts IV, V and VI.*

72.1 Manuscript 05: Conclusion

The existence of a Diophantine representation and the preservation of a computation's witnesses are different mathematical requirements. For bounded executions, a fully explicit solution to the stronger requirement is available: compute exact partial-map commutations, select lexicographic representatives with a small Boolean state, and express the resulting verifier by affine equations with unique natural slacks. The final polynomial is convex quadratic, while its natural zeros still carry the full discrete computational content.

The resulting theorem package does more than certify reachability. It preserves class counts and label-additive weights, gives a bounded rational polytope with explicit coordinate bounds, handles interval-guarded counter programs, and explains both an automaton-size obstruction and a counting-complexity obstruction. The elementary ballot family and the exhaustive checks provide independent reference cases for an eventual formal implementation.

At the same time, a growing family of bounded certificates is not one fixed-dimensional universal equation. The exact equivalence in Theorem 62.1 identifies where a proposed uniform finite-fold compressor reaches the full strength of the classical finite-fold problem. This separation makes the construction useful without overstating it: the bounded theorem is proved and executable, and the stronger arithmetic frontier is stated precisely enough to guide further research.

72.2 Manuscript 01: Conclusion

For box-guarded translations, the exact resources needed for all serializations of a repeated block admit a surprisingly compact description: total coordinatewise losses plus the largest retained lower margin, and dually the smallest upper margin minus total gains. Global commutation turns this universal condition into exact existential acceleration. The resulting ordinary quartic has a unique complete natural witness and constant arity with respect to the repetition counts.

Combining that resource calculation with canonical causal layers yields an explicit quartic whose roots correspond one-for-one to executable trace classes of bounded height. This preserves a stronger semantic object than reachability alone, and it exposes the otherwise easy-to-miss roles of inactive slacks, padding, and auxiliary circuit variables. Fixed block programs provide additional unbounded-repetition cases, with rank and progress criteria for single-fold or finite-fold endpoint fibres.

The general unbounded problem remains sharply separated. Infinitely many real histories cannot all be encoded by finitely many roots at one endpoint pair, and unique semantic execution does not by itself canonicalize arithmetic sequence codes. A universal fixed-arity single-fold halting polynomial would settle the corresponding full MRDP principle. The present constructions provide explicit, testable

components on the tractable side of that boundary, together with a proof-oriented route for extending ProveIt.

72.3 Manuscript 04: Conclusion

The informative unit of Diophantine compilation is not merely a yes/no equivalence. It is a controlled fiber: the collection of arithmetic witnesses attached to a computational object. Canonical slacks handle guards, prefix circuits handle priority, symbolic flags remove redundant schedules, and local constructor equations handle rewriting without inspecting untouched subterms.

The resulting theorems give explicit quartic certificates with exact variable and residual counts for three different substrates. For concurrent Petri executions, the zeros represent trace classes rather than their many linearizations. For FRACSTRAN, every bounded deterministic run has one arithmetic witness, including its first-applicable-fraction proof. For fixed SKI schedules, a certificate follows the changed context rather than the full tree. These guarantees are compatible with substantial complexity and with large witness heights.

The unbounded question remains sharply separated: fixed arity with finite or unique fibers is not supplied by finite-horizon determinism. The practical research contribution is therefore both constructive and diagnostic—a set of proved, executable witness-faithful compilers, and a precise statement of what additional compression would have to accomplish.

72.4 Manuscript 06: Conclusion

There is a substantial middle ground between a black-box MRDP invocation and an explicitly expanded computation trace. For token systems, exact resource summaries collapse nested repetition into syntax-sized arithmetic. For lexicographic trace normality, a compact Boolean signature collapses every power to the three cases zero, one, and at least two. These two compressions are compatible with unique quadratic witnesses and hence with a single quartic polynomial.

The resulting guarantees are precise. A fixed schedule with fixed parameters has one auxiliary certificate. All words of a fixed length admit a separate polynomial whose roots count executable trace classes exactly. Neither fact implies that arbitrary existential schedule parameters have finite fibers. Indeed, separated increment/decrement schedules already contain the full Diophantine finite-fold question, while their flat fragment remains Presburger-definable. This combination of constructive positive results and sharp boundary statements provides a concrete extension of ProveIt's Diophantine program without attributing to the compiler a solution of problems it does not solve.

Appendices

[write] *The appendices of manuscripts 05, 01, 02, 03, 04 and 06, in that order, followed by this report's provenance record. Manuscript 04's first appendix and manuscript 06's second appendix are printed in Section 69 and Section 70; manuscript 07 has no appendix.*

A Manuscript 05: A minimal end-to-end example

The following program can be run from the package directory. Python indices start at zero, whereas the mathematical exposition numbers labels and places from one. The three ordered labels remain $a < b < c$.

```

import sys
sys.path.insert(0, "code")
from trace_polytope import Net, compile_certificate

net = Net(pre=((0,), (0,), (1,)),
          post=((1,), (0,), (0,)),
          names=("a", "b", "c"))
cert = compile_certificate(net, (1,), (1,), 3)

# b,c,a is the canonical representative of c,a,b.
word = (1, 2, 0)
z = cert.witness(word)
assert cert.zero(z)
assert cert.energy(z) == 0
assert cert.decode(z) == word
cert.export("certificate.json")
print(len(cert.variables), len(cert.residuals)) # 73, 59

# Make b a zero-test instead of an unconditional no-op.
# The compiler recomputes the guarded commutation relation.
guarded = compile_certificate(
    net, (0,), (0,), 3,
    upper=((None,), (0,), (None,)))
z2 = guarded.witness((0, 2, 1)) # increment, decrement, zero-test
assert guarded.zero(z2)

```

The exported equation is exact even without monomial expansion. To expand a small instance, use `cert.expanded_polynomial()`; for large instances, retaining the sum-of-squares circuit is preferable. Expansion is not needed to evaluate the equation or to verify a proposed zero.

B Manuscript 05: Edge cases and specification checklist

Zero horizon. There are no selectors, remainders, or step-local auxiliaries. The endpoint equations require $M = N$, and the flags are forced to zero. There is exactly one empty execution class when the endpoints agree and no solution otherwise.

No places. Markings have the unique empty tuple as value. The net has only labelled identity transitions. The selector and normal-form constraints still distinguish the intended words and their classes. The full-box test uses this legitimate degenerate case to obtain a genuinely exhaustive small search.

Repeated effects and repeated circuit inputs. Two labels with equal transition data remain distinct actions and can commute. Circuit gates that mention one predecessor twice must read one token once; otherwise a supposedly 1-safe construction would demand two tokens and fail.

Empty interval compositions. Both orders may have empty domains. They then agree as partial maps, even when their numerical composed-box endpoints differ. The guarded commutation routine canonicalizes an empty box before comparing it. Empty *one-step* domains are excluded in the stated input convention.

Inactive guards and branches. An inactive upper guard has the bound B_p , not an unconstrained witness. An unused branch of a logical disjunction must not leave arbitrary variables. Every variable listed in the exported certificate has a unique semantic value once the canonical word is fixed.

The domain is part of the theorem. The identity $\sum_a e_a = 1$ forces a one-hot selector over $\mathbb{N}$, but not over $\mathbb{Z}$ or $\mathbb{R}$. Real solutions are deliberately retained only as a relaxation. Natural nonnegativity must not be erased when transferring the polynomial to another system.

Representation versus decision. The program constructs and evaluates equations. It does not enumerate all natural zeros of a large certificate. The explicit height theorem makes the set finite, but the associated search and count can still be computationally hard.

C Manuscript 05: Source and verification provenance

The repository sources inspected for the baseline were its Hilbert-tenth-problem README, formalization status register, and the actual `DiophantineTrace.lean` interface. The latter was retrieved at the pinned commit listed in Section 1; its Git blob identifier was

08e5796b22fc85ef0cd97255dba8b7527960160b.

Repository citations below use immutable commit URLs. The literature was checked through the authors’ papers, publisher pages, or the ACL Anthology. The bibliography is selective: it documents the reused conceptual background and the finite-fold boundary, not a claim to an exhaustive priority survey.

The package’s `README.md` describes its contents and limitations. Its `SHA256SUMS.txt` records the distributed files, including the article source, compiled PDF, code, and test receipts. The verification receipt reports the precise finite scopes; it should not be read as a formal proof certificate. [write] In ProveIt the delivered `README.md` is replaced by this report’s README, the checksum ledger was verified (19 of 19 entries) and retired, and the PDF is not shipped (Appendix K).

D Manuscript 01: Reproducibility and source-audit notes

The package contains the self-contained `article.tex`, its compiled PDF, the reference compiler and test runner, machine-readable verification output, and two expanded polynomial examples. To rebuild the document from a clean directory, run `pdflatex article.tex` three times to stabilize the contents and cross-references. The bibliography is included directly; no external bibliography database or source checkout is required.

The JSON polynomial format lists each monomial as an integer coefficient and a list of variable names, with repetition recording powers. The declared single polynomial is the sum of the squares of the listed residuals. Exported examples also contain its expanded form. Parameters and witnesses are separately listed, and every assignment entry is an exact integer. These conventions make the files inspectable without a computer algebra system.

The repository audit was limited to relevant live files at the pinned commit: the Hilbert-tenth README, its formalization status register, the actual exact-iteration interface, and the combinatory-logic README. A full-repository novelty audit was not performed. The code and mathematical proofs in this report were not committed to, or used to modify, the user’s repository. Literature was checked through the primary sources below; no conclusion of world-first priority is drawn from the search. [write] In ProveIt the package’s `article.tex` and PDF are not shipped separately (they survive in the

arrival commit, Appendix K); its programs and data are now committed as files of this report under the names listed in the README, and no other repository file was changed.

E Manuscript 02: The finite checking formula in one place

For readers implementing a separate compiler, the following is a complete specification of the primary checking relation. The parameters are d fixed, $\mathbf{c} \in \mathbb{Z}^{d+1}$, and $T \in \mathbb{N}$. Compute the coefficient tower by (120). At level k , use $L_k = 2(d - k) + 1$ slots, fixed cuts $b_{k,0} = 0$, $b_{k,L_k} = T + 1$, and natural primary coordinates for all remaining cuts and all tags.

Impose ordered cuts, tags in $\{0, 1, 2\}$, terminal zero-tag padding, and different tags in adjacent active slots. Impose $\tau_{d,0} = 1 + \text{sgn } q_d(0)$. For every $k < d$, parent slot j , and child slot ℓ , let

$$\begin{aligned} a &= b_{k,j}, & b &= b_{k,j+1}, & c &= b_{k+1,\ell}, & e &= b_{k+1,\ell+1}, \\ u &= \max(a, c), & v &= \min(b - 1, e), & E &= (a < b) \wedge (c < e) \wedge (u \leq v). \end{aligned}$$

Require both implications

$$E \implies 1 + \text{sgn } q_k(u) = \tau_{k,j}, \quad E \implies 1 + \text{sgn } q_k(v) = \tau_{k,j}.$$

For a positivity certificate, also require $(b_{0,j} < b_{0,j+1}) \implies \tau_{0,j} \neq 0$ for every j . Omitting only these last assertions gives the unique sign-table certificate for *every* polynomial, including those negative somewhere in the interval.

Every comparison is evaluated, even when E is false. To preserve the cubic bound, evaluate and cache q_k at all candidate expressions $a, c, b - 1, e$ before selecting their sign tags. Use canonical sign bits for the selections and the intersection comparison. The signed values $b - 1$ are ordinary integer expressions; truncated natural subtraction must not be substituted into the checking formula.

F Manuscript 02: Reproduction and the exported polynomial format

The code requires Python 3.10 or later and uses only its standard library. Run the following commands from the extracted directory [write] (the delivered layout; see the README for a copy of the shipped files in that layout):

```
python run_tests.py
python certificates.py --coefficients 8 -6 1 --horizon 6
python quartic_compiler.py --coefficients 4 -4 1 --horizon 6 \
  --output quartic_example.json
python verify_export.py quartic_example.json
```

The coefficient order is ascending: $8 -6 1$ means $8 - 6t + t^2$. The optional compiler switch `--sign-table-only` omits the final positivity requirements while retaining all sign-table conditions.

An exported residual consists of terms with an integer coefficient and a list of variable names. For example,

```
{"coefficient": -2, "variables": ["b0_1", "w17"]}
```

means $-2b_{0,1}w_{17}$. The empty variable list denotes a constant. Repeated names denote powers; each residual has total degree at most two. The exact polynomial is the sum of the squares of the listed residuals. Free coefficient parameters are signed, the free horizon is natural, and all primary and auxiliary witness coordinates are natural.

The file includes a sample assignment but is not specialized to it. Changing an input coefficient in that assignment does not change the polynomial; it normally requires regenerating the unique witness. The structure fingerprints in the test report confirm equality of the exported symbolic template across the tested parameter assignments at fixed nominal degree.

Compile this article with

```
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

No external bibliography processor or shell escape is required.

G Manuscript 03: Additional validation of the linear backend

The independent driver code/`03-linear-memory-test_linear_memory.py` runs both algebraic and operational checks. It verifies collision-free product fingerprints for 451 bounded multisets, including zeros and repetitions. It repeats all 585 small logs, checks six non-Boolean write flags, and tests 40 valid larger-integer logs together with 37 corrupted reads. All agree with the independent chronological memory semantics. A further adversarial test replaces one sorted write record by a duplicate and recomputes every dependent output-side coordinate. All local bounds, packing, ordering, and memory checks still pass; exactly the final product-equality residual rejects the forged multiset.

For the four-event example, the linear backend emits exactly 96 witnesses and 89 residuals. Its expanded quartic has 499 nonzero monomials. The largest witness has 471 bits, compared with six bits for the bitonic backend on the same log; here $D = 164$. All 96 single-coordinate increments of the witness are rejected. Twenty additional assignments verify the expanded-polynomial identity, and compiling two different logs of the same length produces identical symbolic variable lists and residual polynomials.

These checks are recorded in `data/03-linear-memory-linear_test_results.json`; the complete polynomial and assignment are in `data/03-linear-memory-linear_example-*`.

Run

```
python3 code/test_linear_memory.py
python3 code/linear_memory.py artifacts/example/events.json \
    artifacts/rebuilt_linear_example --expanded
```

As with the comparison backend, the proof of uniqueness for arbitrary inputs is mathematical, not inferred from the bounded test corpus.

H Manuscript 03: Claim and dependency ledger

Claim	Dependencies	Status in this archive
Canonical comparison	Integer algebra and nonnegative witnesses	Proved; implemented; tested.
Low-height memory	Comparison, bitonic sorting, local memory criterion	Proved; implemented; tested.
Optimal network size	External AKS sorting-network theorem	Proved conditional on that established theorem; not implemented.

Claim	Dependencies	Status in this archive
Carry-free product lemma	Base expansion and cancellation in $\mathbb{Z}[X]$	Proved; finite cases tested.
Linear-size memory	Product lemma, radix packing, local memory criterion	Proved; implemented; tested.
Parsimonious RAM	Canonical local instruction interface and either memory backend	Proved; complete frontend not implemented.
Immutable graph evaluator	Acyclic unfolding and normalized RAM implementation	Mathematical construction and proof; no frontend implementation.
Counting and weights	RAM bijection and elementary counting/probability	Proved; weight example tested.
Universal compression equivalence	Universal computation and specialization	Proved elementary equivalence; neither conjecture solved.
Lean certification of this work	Proposed modules in Section 70.4	Not performed.
Literature-wide novelty	Additional independent specialist review	Not established.

I Manuscript 04: Notation and resource-count reference

Symbol	Meaning
$\mathbb{N}, \mathbb{Z}$	Natural witness domain including zero; integer polynomial coefficient and evaluation domain.
E, V	Number of displayed residuals; number of natural witness variables. Neither is claimed minimal.
I	Fixed symmetric irreflexive independence relation on the labeled action alphabet.
$\text{NF}_I(w)$	Lexicographically least word in the class generated by independent adjacent swaps.
p, r, T	Petri places, labeled actions, and externally fixed horizon; in FRACTRAN, r is the number of ordered reduced fractions.
U, V (net)	Consumption and production matrices. The production matrix V is unrelated to the scalar variable count when its indices are displayed.
m, x, e, f	Petri marking, action-selector, enabling-slack, and normal-form-flag variables.
q, ρ, s, u, z	Canonical quotient, remainder, bound slack, positive-remainder slack, and divisibility bit.
h, x (priority)	No-earlier-applicable-index bit and first-applicable selector.
$\text{App}(\ell, r)$	Natural code for binary application; leaves S, K, I have codes 0, 1, 2.
σ, H, n_I, n_K, n_S	Fixed SKI reduction schedule, its total context depth, and counts of each root rule.
P_T vs. $P(T, \cdot)$	A horizon-indexed polynomial family may have growing arity; one polynomial with horizon as an input has fixed arity.

J Manuscript 06: A completely displayed local power polynomial

To make the ordinary-polynomial nature of the encoding visible without any compiler machinery, consider one resource-summary pair (u, v) , an exponent k , and proposed powered coordinates (U, V) . Introduce $c, \alpha, \beta, e, \ell \in \mathbb{N}$ and set

$$\begin{aligned} \mathcal{P} = & (c + \alpha - u)^2 + (c + \beta - v)^2 + (\alpha\beta)^2 + (e(e - 1))^2 \\ & + (k - e - \ell)^2 + ((1 - e)\ell)^2 + (U - ec - k\alpha)^2 + (V - ec - k\beta)^2. \end{aligned} \quad (245)$$

This is a polynomial of total degree at most four in $u, v, k, U, V, c, \alpha, \beta, e, \ell$. For fixed u, v, k , its natural solutions in all seven remaining coordinates $(U, V, c, \alpha, \beta, e, \ell)$ form a singleton. The first three residuals force $c = \min(u, v)$, $\alpha = u - c$, $\beta = v - c$. The next three force $e = \mathbf{1}_{k>0}$ and $\ell = k - e$. The final two give exactly the powered summary. Thus the uniqueness claim includes both the result and its internal witnesses.

For $u = v = 1$ and $k = 0$, the forced values are

$$c = 1, \quad \alpha = \beta = e = \ell = 0, \quad U = V = 0.$$

The internal minimum remains one, but the powered action is the identity. For the same pair and any $k > 0$, the forced values are $e = 1$, $\ell = k - 1$, $U = V = 1$. This small example shows simultaneously why inactive internal witnesses must remain fixed and why their values must not be confused with the resources required by a zero power.

K Provenance of this report

[write] This report was built from seven manuscripts. Six arrived in batch 60 of ProveIt’s incoming reports (arrival commits 725d2ebb6 and c1f56f842) and were placed by commit 7498484af; the seventh arrived alone in batch 61 (b998f70c6) and was placed by 3bf66a8bc as the report’s seventh member, before the report was written. Each manuscript’s source and delivery README survive only in its arrival commit, except manuscript 05’s source, which is the base of this article; no manuscript PDF is shipped.

No.	Archive (batch, manuscript)	Arrival	Pin	Placed	Contribution (Parts)
01	Diophantine_Causal_Computation.zip (60, 01)	725d2ebb6	e8bb0931d	7498484af	box test and resource envelope (I); causal-height quartic (II); accelerator, block programs, word powers, examples (III); substrate adapters (VII, VIII); boundary (IX)
02	canonical_diophantine_certificates.zip (60, 02)	725d2ebb6	e8bb0931d	7498484af	signed coordinates and total circuits (conventions); polynomial trajectories (IV); FRACTRAN step (VII); substrates (VIII); exponentiation boundary (IX)
03	ProveIt_Linear_Size_Diophantine_Certificates.zip (60, 03)	725d2ebb6	e8bb0931d	7498484af	memory logs, RAM, graph evaluation, counting (V); boundary (IX)
04	Witness_Faithful_Diophantine_Compilation.zip (60, 04)	c1f56f842	e8bb0931d	7498484af	monitor, Petri quartic, counting (II, second route); FRACTRAN and SKI (VII); substrates (VIII); boundary (IX)
05	canonical_trace_polytopes.zip (60, 05), base	c1f56f842	e8bb0931d	7498484af	introduction; commutation (I); trace polytopes, costs, counting (II); rewriting (VII); substrates (VIII); boundary (IX)
06	diophantine_counter_programs.zip (60, 06)	c1f56f842	e8bb0931d	7498484af	resource summaries (I); signatures and trace bijection (II, second route); saturation, compiler, nesting boundary (III); substrates (VIII); finite-fold substrate equivalence (IX)
07	Causal_Diophantine_Queue_Compilation.zip (61, 01)	b998f70c6	725d2ebb6	3bf66a8bc	first-failure product (I); queues, tag systems, word memory, FIFO networks (VI); boundary (IX)

Where the merge had to choose. [write]

- *Base.* Manuscript 05, whose trace-class theorem needs the weakest hypothesis (any sound $I \subseteq I_{\max}$, with interval guards) and gives the strongest conclusion (a convex quadratic whose real zero set is a rational polytope with exactly the natural zeros as lattice points). Its text supplies the title page layout, the preamble, the introduction and the first place of every shared result.
- *Printed once.* The universal-halting equivalence (manuscripts 05, 01, 03, 04) is Theorem 62.1; the absence of a computable witness-height bound (02, 01, 03, 04, 05) is Proposition 62.2. The labels of the four equivalence theorems and the five height statements (four propositions and manuscript 04’s theorem) that they replace point to the merged statements, and each manuscript’s proof is printed in its own section after a note mapping its letters.
- *Second routes.* Manuscripts 04’s and 06’s trace-class bijections (Theorems 15.2 and 18.1), manuscript 01’s box test (Proposition 5.1) and manuscript 06’s commutation criterion (Theorem 7.7) are kept with their own statements, counts and proofs, marked in their titles;

manuscript 04’s monitor, lower-bound family and counting theorem, and the Parikh refinements of manuscripts 01, 04 and 06, likewise point to manuscript 05’s versions. Manuscript 01’s height-bounded compiler is a different theorem and is kept whole.

- *Moved text.* Manuscript 05’s interval-guard theorem and guarded-program corollary were moved from its section on other substrates to Parts I and II, and its rewriting subsection to Part VII; the substrate surveys of manuscripts 01, 02 and 05 are split between Parts VII, VIII and IX; manuscript 02’s first two introductory subsections open Part IV and its Section 2 stands in the conventions; manuscript 07’s word-code subsection opens Part VI and its first-failure section closes Part I. Every move is announced by a note at both ends.
- *Back matter.* Implementation and validation, formalization proposals and conclusions are collected by manuscript (the conclusions of the single-manuscript Parts IV–VI close those Parts). The research questions of all seven manuscripts (74) are merged into 50 questions; each merged question names its sources and prints each source’s text (Section 71).
- *Bibliography.* One list; duplicates merged, each item naming the manuscripts that cite it. Manuscript 07 cites a different Cantone–Cuzziol–Omodeo article ([11]) from the one cited by manuscripts 02, 04, 05 and 06 ([10]); they are kept apart. Manuscript 07’s two unpublished “companion” items are manuscripts 04 and 06 themselves; its sentence citing them now names them.
- *Corrections.* Manuscript 06 cited Jones–Matiyasevich for ProveIt’s documentation of MRDP and its interfaces; it now cites the repository. Manuscript 04’s hard-coded section numbers (“Section 2”, “Section 3”, “Section 7”) now point to its sections here. Sentences in which manuscripts 01 and 05 describe their own delivered packages carry notes on what is and is not shipped.
- *Additions.* The conventions (Section 2), the Part openings, the remarks on the formalized exponential counterpart (Remark 64.2) and on the project’s tag-system notes (Remark 47.1), and every other paragraph marked [write].

Labels. [write] Every label carries the prefix `cdc:`. Manuscript 05’s labels keep their names after it (`cdc:thm:main`); the other manuscripts’ labels carry a second prefix: 01 `cdc:ct:`, 02 `cdc:pt:`, 03 `cdc:mem:`, 04 `cdc:wf:`, 06 `cdc:cs:`, 07 `cdc:qc:`. Labels written in the merge use `cdc:conv:`, `cdc:bd:`, `cdc:part:`, `cdc:sec:`, `cdc:q:`, or a manuscript’s prefix for a section it did not label (`cdc:wf:sec:interfaces`).

References

[write] One bibliography for the seven manuscripts. An item cited by several manuscripts is printed once; the manuscripts citing it are listed at its end, and a detail given by only one of them is kept and attributed. Repository items are pinned at `e8bb0931d67f80d9fce87a8cddb0f661ff19f956` (manuscripts 01–06) or `725d2ebb6909fe11a13a92354c0f47367a3cbbf5` (manuscript 07); the cited files of `Computability/HilbertTenthProblem` are identical at the two commits. Manuscript 07’s two “companion” items were manuscripts 04 and 06 of this report and are not repeated.

- [1] V. Reshetnikov (and contributors). *ProveIt: Hilbert’s tenth problem* — project README (manuscripts 01, 03, 04, 07), with its corrected Jones articles and Diophantine developments (04), and, as one item, README and formalization status register (02). `Computability/HilbertTenthProblem/README.md`, inspected September 30, 2026.
<https://github.com/VladimirReshetnikov/ProveIt/blob/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/HilbertTenthProblem/README.md>; at manuscript 07’s pin <https://github.com/VladimirReshetnikov/ProveIt/blob/725d2ebb6909fe11a13a92354c0f47367a3cbbf5/Computability/HilbertTenthProblem/README.md>; the development’s tree as cited by manuscript 02: <https://github.com/VladimirReshetnikov/ProveIt/tree/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/HilbertTenthProblem>. Manuscript 02: the repository supplies corrected editions and source developments; inspection here does not constitute an independent build audit. Cited by manuscripts 01, 02, 03, 04, 07.
- [2] V. Reshetnikov (and contributors). *ProveIt: Hilbert’s tenth problem, formalization status*. `Computability/HilbertTenthProblem/Lean/STATUS.md`, inspected September 30, 2026.
<https://github.com/VladimirReshetnikov/ProveIt/blob/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/HilbertTenthProblem/Lean/STATUS.md>. Cited by manuscripts 03, 05.
- [3] V. Reshetnikov (and contributors). *ProveIt: Diophantine bounded universals and finite traces*. `Computability/HilbertTenthProblem/Lean/Diophantine/Common/DiophantineTrace.lean`, inspected September 30, 2026. <https://github.com/VladimirReshetnikov/ProveIt/blob/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/HilbertTenthProblem/Lean/Diophantine/Common/DiophantineTrace.lean>. Cited by manuscripts 01, 02, 04, 05.
- [4] V. Reshetnikov. *ProveIt: Combinatory logic — SK, SKI, and Iota universality*. `Computability/CombinatoryLogic/README.md`, inspected September 30, 2026.
<https://github.com/VladimirReshetnikov/ProveIt/blob/e8bb0931d67f80d9fce87a8cddb0f661ff19f956/Computability/CombinatoryLogic/README.md>. Cited by manuscripts 01, 03, 04.
- [5] V. Reshetnikov, ProveIt repository. *MRDP: recursively enumerable if and only if Diophantine*. `Computability/HilbertTenthProblem/Lean/MRDP.md`, inspected at commit `725d2ebb6909fe11a13a92354c0f47367a3cbbf5`, September 30, 2026.
<https://github.com/VladimirReshetnikov/ProveIt/blob/725d2ebb6909fe11a13a92354c0f47367a3cbbf5/Computability/HilbertTenthProblem/Lean/MRDP.md>. Cited by manuscript 07.
- [6] V. Reshetnikov. *ProveIt*, public research repository, inspected September 30, 2026. Repository tree at commit `e8bb0931d67f`; relevant documents: `Computability/HilbertTenthProblem/README.md` and `Computability/HilbertTenthProblem/Lean/STATUS.md`. [Pinned Diophantine development](#). The repository descriptions were inspected; its proofs were not rebuilt for manuscript 06. Cited by manuscript 06.
- [7] Yu. V. Matiyasevich. Enumerable sets are Diophantine. *Soviet Mathematics Doklady* **11** (1970), 354–358. (Manuscript 04 gives the pages as 354–357.) Cited by manuscripts 04, 06.
- [8] J. P. Jones and Yu. V. Matiyasevich. Register machine proof of the theorem on exponential Diophantine representation of enumerable sets. *Journal of Symbolic Logic* **49**(3) (1984), 818–829. doi:[10.2307/2274135](https://doi.org/10.2307/2274135). Bibliographic details, a corrected edition and its formalization status are catalogued in [1] (manuscripts 01 and 06). Cited by manuscripts 01, 03, 04, 05, 06.

- [9] Yu. V. Matiyasevich. Towards finite-fold Diophantine representations. *Journal of Mathematical Sciences* **171** (2010), 745–752. doi:10.1007/s10958-010-0179-4. Manuscript 03: this reference documents the finite-fold/single-fold frontier; its universal-substrate equivalence is proved directly (Theorem 62.1). Cited by manuscripts 01, 02, 03, 05, 06.
- [10] D. Cantone, L. Cuzziol, and E. G. Omodeo. On Diophantine singlefold specifications. *Le Matematiche* **79**, no. 2 (2024), 585–620. Published December 28, 2024. doi:10.4418/2024.79.2.18. Journal page: <https://lematematiche.dmi.unict.it/index.php/lematematiche/article/view/2703>. Author-deposited text: <https://arts.units.it/bitstream/11368/3101478/1/CC024.pdf>. Cited by manuscripts 02, 04, 05, 06.
- [11] D. Cantone, L. Cuzziol, and E. G. Omodeo. *Six equations in search of a finite-fold-ness proof*. 2023; version 3, September 30, 2024. <https://arxiv.org/abs/2303.02208v3>. A different article from [10]. Cited by manuscript 07.
- [12] D. Cantone, A. Casagrande, F. Fabris, and E. Omodeo. Does every recursively enumerable set admit a finite-fold Diophantine representation? *CEUR Workshop Proceedings* **2396**, paper 11 (2019). <https://ceur-ws.org/Vol-2396/paper11.pdf>. Cited by manuscript 01.
- [13] D. Larchey-Wendling and Y. Forster. Hilbert’s tenth problem in Coq. In *FSCD 2019*, LIPIcs **131** (2019), 27:1–27:20. doi:10.4230/LIPIcs.FSCD.2019.27. Cited by manuscripts 04, 07.
- [14] D. Larchey-Wendling and Y. Forster. Hilbert’s tenth problem in Coq (extended version). *Logical Methods in Computer Science* **18**(1), article 35 (2022). doi:10.46298/lmcs-18(1:35)2022. Author version: <https://arxiv.org/abs/2003.04604> (version 5: <https://arxiv.org/abs/2003.04604v5>). Cited by manuscripts 01, 02.
- [15] M. Carneiro. *A Lean formalization of Matiyasevič’s Theorem*. arXiv:1802.01795, 2018. <https://arxiv.org/abs/1802.01795>. Cited by manuscripts 03, 07.
- [16] P. Cartier and D. Foata. *Problèmes combinatoires de commutation et réarrangements*. Lecture Notes in Mathematics **85**, Springer, 1969. Cited in manuscript 01 for the classical partially commutative monoid and normal-form framework; see also [17]. Cited by manuscript 01.
- [17] C. Krattenthaler. *The theory of heaps and the Cartier–Foata monoid*, author-hosted exposition. <https://www.mat.univie.ac.at/~kratt/artikel/heaps.pdf>. Cited by manuscript 01.
- [18] F. Barthélemy. Using Mazurkiewicz trace languages for partition-based morphology. In *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics* (2007), 928–935. See the lexicographic normal-form characterization in Section 2.2, p. 930. <https://aclanthology.org/P07-1117/>. Cited by manuscript 05.
- [19] A. V. Anisimov and D. E. Knuth. *Inhomogeneous sorting*. International Journal of Computer & Information Sciences **8** (1979), 255–260. doi:10.1007/BF00993053. Cited by manuscript 06.
- [20] M. Lohrey, F. Stober, and A. Weiß. The power word problem in graph products. *Theory of Computing Systems* **68** (2024), 403–464. doi:10.1007/s00224-024-10173-z. Cited by manuscript 04.
- [21] A. de Colnet, K. S. Meel, and U. Mathur. Counting and sampling traces in regular languages. *Proceedings of the ACM on Programming Languages*, POPL 2026. doi:10.1145/3776723. Preprint arXiv:2512.00314. Cited by manuscript 04.
- [22] L. G. Valiant. The complexity of enumeration and reliability problems. *SIAM Journal on Computing* **8**(3) (1979), 410–421. <https://doi.org/10.1137/0208032>. Cited by manuscript 05.
- [23] G. Zetsche. *The Emptiness Problem for Valence Automata over Graph Monoids*. arXiv:1710.07528 (2017). <https://arxiv.org/abs/1710.07528>. Cited by manuscript 06.
- [24] F. Grunewald and D. Segal. *On the integer solutions of quadratic equations*. Journal für die reine und angewandte Mathematik **569** (2004), 13–45. doi:10.1515/crll.2004.023. Cited by manuscript 06.

- [25] M. Hark, F. Frohn, and J. Giesl. *Termination of Triangular Polynomial Loops*. arXiv:1910.11588, version 7, February 13, 2024. <https://arxiv.org/abs/1910.11588v7>. Cited by manuscript 02.
- [26] N. Lommen, F. Meyer, and J. Giesl. *Automatic Complexity Analysis of Integer Programs via Triangular Weakly Non-Linear Loops*. arXiv:2205.08869, version 3, November 15, 2024. <https://arxiv.org/abs/2205.08869v3>. Cited by manuscript 02.
- [27] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza. SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge. In *Advances in Cryptology — CRYPTO 2013, Part II*, LNCS 8043, Springer, 2013, 90–108. https://doi.org/10.1007/978-3-642-40084-1_6. Extended version, Cryptology ePrint Archive 2013/507: <https://eprint.iacr.org/2013/507>. The extended version’s memory-consistency discussion is the relevant prior art. Cited by manuscript 03.
- [28] K. E. Batcher. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, AFIPS **32**, 307–314. <https://doi.org/10.1145/1468075.1468121>. Cited by manuscript 03.
- [29] M. Ajtai, J. Komlós, and E. Szemerédi. An $O(n \log n)$ sorting network. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, 1983, 1–9. <https://doi.org/10.1145/800061.808726>. Cited by manuscript 03.
- [30] J. H. Conway. FRACSTRAN: A simple universal programming language for arithmetic. In *Open Problems in Communication and Computation*, Springer, New York, 1987, pp. 4–26. Cited by manuscript 04.
- [31] M. Cook. *Universality in Elementary Cellular Automata*. *Complex Systems* **15**(1) (2004), 1–40. [doi:10.25088/ComplexSystems.15.1.1](https://doi.org/10.25088/ComplexSystems.15.1.1). The cyclic-tag definition and simulation are in Sections 2.1–2.2. Cited by manuscript 07.
- [32] D. Woods and T. Neary. *On the time complexity of 2-tag systems and small universal Turing machines*. *Proceedings of FOCS 2006*. [doi:10.1109/FOCS.2006.58](https://doi.org/10.1109/FOCS.2006.58); <https://arxiv.org/abs/cs/0612089>. Cited by manuscript 07.
- [33] P. Atela and J. Hu. *Commuting polynomials and polynomials with same Julia set*. 1995. <https://arxiv.org/abs/math/9504210>. Cited for the broader centralizer context, not as a dependency of the elementary proof in manuscript 07. Cited by manuscript 07.